∞ MMLA: How Memory Lets the Past Shape the Future

Technical Report of MMLA

Junyi Zou[1,*] • **Avrova Donz**[1,2,*]

[1]MMLA-org [2]Communication University of China (CUC), No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

[*]Equal contribution

Email: zoujunyi@zjydiary.cn · avrovadonz@icloud.com

ABSTRACT

Proposal. Extending a context window is not the same as learning from a stream. MMLA formalizes a bounded resident memory between transient context and slow weight updates. A completed local segment is eventized; for each event, a neural constructor proposes semantic content and a trusted assembler produces a complete versioned row; deployment either commits that row atomically or returns NULL. Realized futures may price actions during training, while deployment remains causal and future-blind.

Validated components. Controlled-language lifecycle execution is exact on 300/300 held-out records for each of three seeds. Calibrated selection with full-archive fallback improves over a weak budget-matched dense baseline by 5.5–16.6 F1 and over BM25 by 4.0–6.2 F1; the original Llama budget execution is retained as failed, while the corrected Qwen packer satisfies the stated per-record caps. Typed anchor–filler transport is exact on 240/240 controlled held-out records per seed while three same-checkpoint controls obtain no whole-record successes.

Current blocker. The integration loop is incomplete. External, raw-hidden, dense-row, structured-span, and checkpoint-native interfaces fail their preregistered semantic qualifications. PM-I3 now shows that the matched failure already occurs on fitted support: direct candidate reconstruction is 40/73,728, and learned content is 0/73,728 both with learned routing and when the target row is supplied by a routing oracle. Supplying oracle content while retaining learned routing reaches only 18,544/73,728, exactly the learned row-routing count; only the complete mechanical oracle is perfect. The current latent-row interface therefore fails before fresh composition can be the primary explanation. Predictive overwrite remains closed, and the next matched study moves to canonical and dual-view rows.

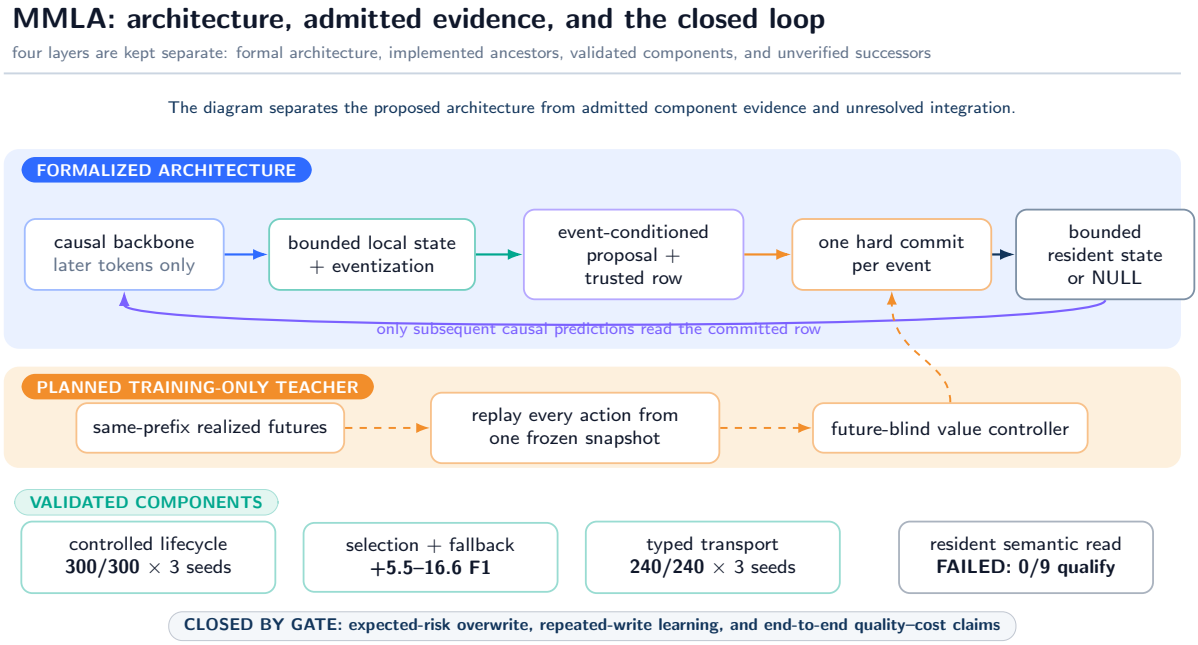

Figure 1: **Proposal, evidence, and present boundary in one view.** Solid arrows show the deployed causal path; the orange branch exists only during training. Admitted component evidence and the matched semantic-interface failure are distinct from the complete predictive-overwrite loop, which has not yet been validated.

Reading guide. This complete companion report uses the same event-level, trusted-assembly, and hard-commit architecture as the submitted arXiv v3 replacement, arXiv:2606.28876. It preserves the full derivations, experiment history, registered negative results, audits, and appendices. V3-R22 adds the accepted PM-I3 fit/support diagnostic: it does not alter PM-I2, qualify an interface, or open predictive overwrite. The public project page is https://github.com/MMLA-org/mmla-memory.

Report Map: Evidence Record and Formal Branches

This candidate is derived directly from the complete V3-R22 Technical Report. The evidence-bearing record below remains the authoritative body of this document: its validated components, registered negative results, PM-I2 same-checkpoint failure, and PM-I3 fitted-support fault localization are not promoted or reinterpreted by the additive theory layer. The five integrated branches are formal and protocol modules only. R25 imports the complete sealed R02 successors for Reasoning-Time Training, Atomic Memory Rows, Predictive Memory Admission, MMLA–RTT dual state, and Completed-Segment Bidirectional Consolidation. This editorial integration adds no experiment, oracle gap, learned admission policy, qualified semantic interface, or scientific acceptance claim.

Layer	Role in the complete successor	Status at this checkpoint
V3-R22 evidence record	Complete implemented, validated, failed, planned, audit, and reproducibility record retained below	Present and unchanged in scientific substance
Reasoning-Time Training	Typed within-problem policy-update witness and causal qualification branch	R02 successor integrated; Reviewer pending
Atomic Memory Rows	Typed reads, authority and commit receipts, ABA-safe migration, and layered invariants	R02 successor integrated; Reviewer pending
Predictive Memory Admission	Grouped futures, expected-risk oracle, uncertainty, and stop-rule branch	R02 successor integrated; Reviewer pending
MMLA–RTT dual state	Type-, privilege-, lifetime-, rollback-, and ledger-distinct policy and memory composition	R02 successor integrated; Reviewer pending
Completed-segment bidirectional consolidation	Post-closure timing, isolation, ownership, event recursion, and service-bound branch	R02 successor integrated; Reviewer pending

Table 1: Additive report map. Formal theory branches do not transfer empirical status across branches and do not bypass the frozen semantic-interface gate.

The integration order is serialized. Each module has a deterministic label, citation, and command prefix. The report uses one title page and one unified bibliography; repeated focus-paper front matter is excluded. Scientific acceptance remains solely Reviewer-owned.

1 Introduction

We retain the project name MMLA from the earlier *Memory-Managed Long-Context Attention* formulation. The present architecture is mixer-agnostic and is not restricted to attention alone.

Efficient long-context modeling has progressed from the Transformer [100] through linear attention, structured state spaces, recurrent and convolutional hybrids, and sparse attention [50, 36, 98, 75, 34, 76, 110, 115, 114, 22]. These methods reduce the per-token state size or the number of attended positions. They do not, by themselves, answer a different question: when should an observation change what the model will carry forward?

A million-token prompt can still contain mutually inconsistent versions of the same fact, preserve an obsolete value, or force the model to repeatedly process material whose relevance was impossible to know when it arrived. Conversely, a small persistent state can be useful if it supports reliable revision, deletion, abstention, and later reuse. The central object of this report is therefore not an unlimited context window. It is a bounded state that learns to retain the parts of the past that improve future behavior.

The difficult decision is the write, not the read. Immediate salience is a poor proxy for future utility: an ordinary sentence may become important only after later events, while a vivid sentence may never be used again. Training data, however, contains the future that inference lacks. This suggests a division of labor. Real subsequent trajectories can price alternative memory actions; a deployable controller must predict their *conditional expected* cost from completed history alone. Predictive information and information-bottleneck principles supply the target: compress the past while retaining information about the future [99, 8]. The expectation is essential. A policy fitted to the action preferred by one realized continuation need not minimize risk over the future distribution faced at deployment.

We call the resulting construction *predictive memory consolidation*. The model first predicts causally. After a local event, sentence, or chunk has fully arrived, a small bidirectional module may reinterpret that completed segment without changing any output already emitted. It eventizes the segment into a bounded ordered list. For each event, a target-conditioned neural proposal

and trusted assembler construct a complete candidate row; a controller chooses one resident target to overwrite or chooses NULL. Insert, update, consolidating update, eviction, deletion, protection, and stale rejection are lifecycle interpretations of this per-event operator. Merging two authoritative resident rows requires two ordered events or a future bounded transaction. The observation horizon and the mixture of local attention, recurrent state, and memory read are deployment choices rather than architectural constants; neither a fixed 1:3:9 ratio nor a fixed 50-turn delay defines the model.

Earlier versions of this report treated lifecycle, fallback, and relational compilation as the endpoint. We now treat them as evidence-bearing components of a broader memory architecture. Controlled lifecycle text exercises write–manage–read–fallback–generate behavior. Held-out multi-hop QA measures the point at which a query-independent writer should abstain. A third study isolates role binding when content and position are unreliable. The new formulation connects these pieces through future-valued consolidation while preserving every earlier result and failure.

Contributions.

1. We specify three bounded state scales—causal token state, completed-segment workspace, and persistent rows—plus slower model parameters. Its only authoritative transition is one complete-row overwrite or NULL per ordered event (§2).
2. We define overwrite selection as conditional action-value estimation. Grouped continuations with identical observed history provide counterfactual branch risks; a causal value predictor sees only completed history and chooses the lowest-risk feasible action. We state the no-leakage interface, information-theoretic objective, stability limits, and system boundary explicitly.
3. We separate representation qualification from predictive overwrite learning. Three frozen-checkpoint semantic-interface studies are registered negatives. PM-I3 then reuses the immutable final checkpoints without optimizer updates and localizes the latest failure to fit/support: a routing oracle does not repair learned content, while the complete oracle remains a mechanical ceiling. The conditional expected-risk intervention remains unopened.
4. Three completed studies provide component evidence: bounded lifecycle execution (§4), calibrated selection and fallback (§5–6), and same-checkpoint typed relational binding (§7).

We do not claim that predictive consolidation is already better than immediate writing, that its information-theoretic form proves intelligence, or that a biological system uses this implementation. The first two studies use frozen language models, and the relational study trains a compact synthetic compiler. A full claim first requires a semantically usable current-state interface and then a corrected same-checkpoint expected-risk intervention; compute-matched pretraining is conditional on both results.

Claim–evidence map: one vocabulary for every layer

status describes the strongest justified statement; it does not transfer upward to the full architecture

CLAIM	ADMITTED BASIS	STATUS
Causal completed-segment boundary	equations + interface contract; no deploy future	FORMALIZED
Controlled lifecycle executor	version, lock, tombstone, eviction implementation	IMPLEMENTED
Lifecycle and typed binding	300/300 and 240/240 per seed under controls	VALIDATED
Resident-row semantic read	PM-I2: 0/9 qualify; PM-I3: fit content fails under route oracle	FAILED
Expected-risk overwrite controller	PM-I3 fit/support terminal; overwrite oracle unopened	CLOSED BY GATE
Dual-view authoritative row	recommended vNext contract motivated by negatives	PLANNED
Full-system quality–cost claim	no repeated-write or full-system crossover test	PLANNED

Three qualification failures plus PM-I3 fault localization close overwrite without implying that the full MMLA program has failed.

Figure 2: **Claim–evidence matrix.** Status belongs to the strongest justified statement, not to the architecture as a whole. "Implemented" denotes a mechanical ancestor; "validated" requires evidence admitted under the frozen protocol.

Claim	Accepted basis	Boundary	Status
Causal boundary	Causal factorization and completed-segment interface	Formal contract only	**FORMALIZED**
Controlled lifecycle	Version, lock, tombstone, eviction executor	Mechanical ancestor	**IMPLEMENTED**
Lifecycle and typed binding	300/300 and 240/240 per seed under controls	Controlled grammars	**VALIDATED**
Resident semantic read	PM-S1, PM-I1, PM-I2; PM-I3 fit/support diagnostic	Learned content fails under route oracle	**FAILED**
Expected-risk overwrite	Semantic prerequisite fails before fresh composition	No learned policy result	**CLOSED BY GATE**
End-to-end cost advantage	No repeated-write or quality–cost crossover	Asymptotic motivation only	**PLANNED**

Table 2: One status vocabulary separates the formal architecture, implemented ancestors, validated components, failed interfaces, planned work, and work closed by gate.

2 Predictive Memory Consolidation

2.1 Three state scales and a slower parameter scale

Let $x_1, x_2, \ldots$ be a token stream. The model maintains a causal hidden state h_t for immediate prediction, a completed local workspace B_n for one consolidation boundary, and exactly S resident row cells. The authoritative resident state is

$$M_n = (r_{n,1}, \ldots, r_{n,S}) \in \mathcal{R}_B^S, \qquad \mathcal{R}_B = \mathcal{I}_{B_i} \times \mathcal{C}_{L_c} \times \mathcal{V}_d \times \mathcal{K}_{d_k} \times \mathcal{U}_{B_m}, \tag{1}$$

where $\mathcal{I}_{B_i}$ is a fixed-width identity field, $\mathcal{C}_{L_c}$ is a padded canonical encoding over one fixed alphabet Σ with at most L_c symbols, $\mathcal{V}_d$ and $\mathcal{K}_{d_k}$ are fixed-width neural payload and routing-key spaces, and $\mathcal{U}_{B_m}$ is a fixed-width metadata record of at most B_m bits. Concretely,

$$B_c = \lceil \log_2(L_c + 1) \rceil + L_c \lceil \log_2(|\Sigma| + 1) \rceil, \qquad B_v = d\, p_v, \qquad B_k = d_k\, p_k,$$

for fixed payload/key precisions p_v, p_k; padding and the stored length make the canonical serialization unique. Aliases have a fixed per-row count and length within $B_c + B_m$; provenance and rollback fields contain bounded identifiers or digests, never an append-only history. The resident table is therefore bounded by

$$\mathrm{bits}(M_n) \le S\big(B_i + B_c + B_v + B_k + B_m\big) =: B_{\text{resident}}, \tag{2}$$

independently of stream length. A full provenance document, superseded canonical value, rollback snapshot, retrieval corpus, or audit log belongs to a separately charged external ledger A_n, not to M_n. If that ledger grows, only resident state—not the total system—is bounded. Model parameters are a slower fourth time scale: online adaptation changes M_n, whereas conventional training changes the weights.

A block may expose local attention, a recurrent or linear state, and bounded memory read. Their ratio is not fixed. A budget-aware mixture is

$$\begin{aligned} y_t &= \sum_{j \in \{\text{local,linear,memory}\}} w_{t,j} y_{t,j}, \\ w_t &= \mathrm{softmax}\, r_\theta(h_t, M_{n(t)-1}, b_t), \end{aligned} \tag{3}$$

where b_t describes the available compute budget. An implementation may evaluate only the selected branches or place memory blocks sparsely. The invariant is bounded, editable state, not a particular layer ratio.

For a reproducible first end-to-end falsification, we nevertheless fix a proposed *MMLA-R0* reference: 12 mixer blocks in four repetitions of [local attention, recurrent/linear state, memory read], a 2,048-token local window, 256-token completed segments, $S = 256$ resident rows, at most four ordered events per segment, and at most one hard commit per event. The archive is disabled during semantic-interface qualification. Neural computation uses BF16 with FP32 normalization and controller reductions. This is a reference test configuration, not an empirically optimized product or a universal mixer ratio.

2.2 Local bidirectional consolidation without future leakage

Let the nth completed segment occupy $(b_{n-1}, b_n]$. Token prediction remains autoregressive,

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}, M_{n(t)-1}). \tag{4}$$

Only after x_{b_n} has been processed does the consolidator form

$$\begin{aligned} z_n &= C_\phi\big(h_{b_{n-1}+1:b_n}, M_{n-1}\big), \\ M_n &\twoheadrightarrow p_\theta(x_t \mid x_{<t}) \quad (t \le b_n). \end{aligned} \tag{5}$$

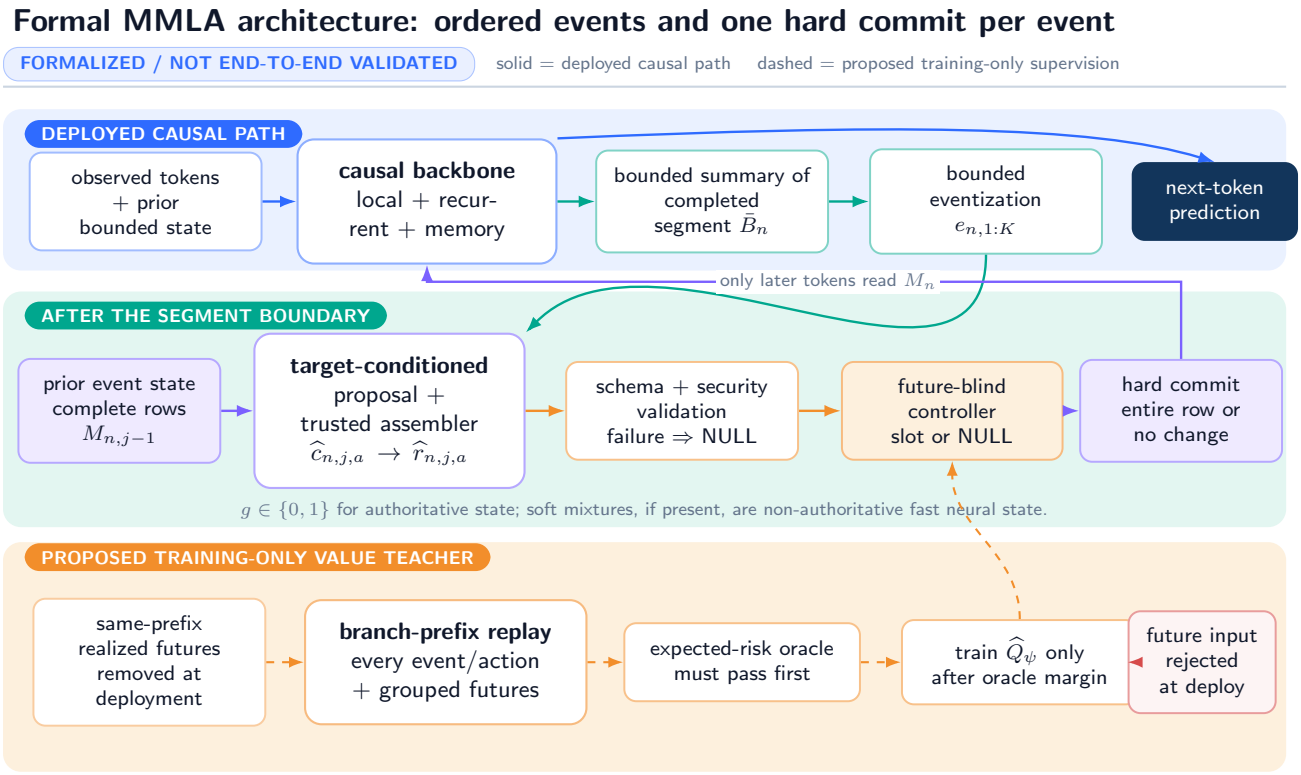

Figure 3: **Predictive memory consolidation.** The language model predicts causally and reads only prior persistent state. After a local segment is complete, a bounded bidirectional consolidator may reinterpret that observed segment and emit ordered events. At training time, several realized continuations with the same prefix estimate the conditional risk of every registered event-level action. A causal value model regresses those risks. At deployment the continuation branches are absent; one complete row is overwritten or NULL leaves state unchanged per event, and only later tokens read the resulting segment state.

Thus C_ϕ may use bidirectional attention *inside an already observed segment*; its output can affect only later tokens. This is local reinterpretation, not inference-time access to an unobserved future. The segment may be a fixed chunk in the first experiment and a learned stopping interval later. A maximum horizon is a safety and cost bound, not a claim that consolidation should always wait a fixed number of turns. Equivalently, the deployed consolidator must satisfy

$$z_n \perp X_{>b_n} \mid (X_{\le b_n}, M_{n-1}), \tag{6}$$

while training-only losses may use $X_{>b_n}$ as a target. This distinction is structural: the forward API contains no future-bearing argument, and the updated state is unavailable to logits at or before the boundary. "Local bidirectionality" therefore means retrospective encoding of observations already received, not bidirectional generation and not a hidden full-sequence look-ahead.

The consolidator next eventizes the bounded segment summary into an ordered list,

$$E_\eta(z_n) = (e_{n,1}, \ldots, e_{n,K_n}), \qquad K_n \le K_{\max}. \tag{7}$$

A segment containing several facts is decomposed before one-row actions are considered. Bounded multi-row atomic transactions are deferred; they are not smuggled into a single overwrite.

Within a completed segment, let $M_{n,0} = M_{n-1}$ and process the ordered events sequentially. The state after event j is $M_{n,j}$, and only the completed segment state $M_n = M_{n,K_n}$ becomes available to later tokens.

2.3 One atomic overwrite per event

For event $e_{n,j}$ and target $a \in \{1, \ldots, S\}$, a neural constructor proposes semantic content,

$$\widehat{c}_{n,j,a} = G_\phi(z_n, r_{n,j-1,a}, M_{n,j-1}, e_{n,j}, a). \tag{8}$$

The proposal contains bounded canonical value and aliases, neural payload, typed routing key, uncertainty, and proposed operation or tombstone intent. A trusted assembler then forms the complete row,

$$\widehat{r}_{n,j,a} = \text{AssembleTrusted}(\widehat{c}_{n,j,a}, \text{tenant, ACL, old protection, next version, provenance, rollback lineage}). \tag{9}$$

Tenant, ACL/security scope, monotone version, protection inheritance, source receipt, rollback lineage, and deletion verification are system-owned fields, not free neural outputs. Their resident representation is fixed-width: receipts and rollback lineage are

bounded identifiers into A_n, not embedded histories. Aliases live inside the bounded capsule or in a deterministic derived index; they never form an independently mutable authoritative map. Schema, size, security, and version checks run before action selection. An oversized or invalid candidate fails to NULL. The version counter is an unsigned B_u-bit member of the metadata budget. A write is feasible only when $u_{\text{old}} < 2^{B_u} - 1$ and the trusted assembler sets $u_{\text{new}} = u_{\text{old}} + 1$; exhaustion rejects the write (or requires an explicit external re-key/migration protocol) and never wraps, saturates, or reuses a prior (ι, u) pair.

We recommend a dual-view vNext contract: canonical content makes a row auditable and reproducible, while a synchronized neural payload supports differentiable routing and readout. The two views are not merely co-located. For row identity ι and version u, let c be the canonical view and (v, k) the neural payload and routing key. A deterministic canonicalizer Can and a versioned encoder $E_{\omega,u}$ define the consistency receipt

$$\rho(r) = \Big(\iota, u, H(\text{Can}(c)), H(E_{\omega,u}(\text{Can}(c))), H(v\|k), H(\omega)\Big). \tag{10}$$

A candidate is committable only if its identity/version/security fields agree and the fixed-width serialized neural bytes satisfy

$$\text{Ser}_{v,k}\big(E_{\omega,u}(\text{Can}(c))\big) = \text{Ser}_{v,k}(v, k). \tag{11}$$

The hashes in ρ are audit accelerators and tamper-evident receipts, not a substitute for the byte-equality check; a digest collision therefore cannot make unequal resident views valid. Canonical reconstruction and fresh-query semantic gates must also pass under the frozen versioned reader, and the receipt is included in the same atomic commit. The fixed-width $H(\omega)$ identifies an external immutable encoder artifact; encoder weights are not copied into the row. Any later neural re-encoding is a new row version, never an in-place drift. On load, recovery, export, and before use as teacher state, the system rechecks byte equality and ρ; failure quarantines the row and makes its action infeasible rather than choosing which view to trust. A deterministic derived index is rebuilt only from validated rows and is never authoritative. This closes the formal consistency invariant; whether a learnable encoder can satisfy its semantic gates remains unverified. The three negative semantic-interface studies motivate this contract; they do not validate it.

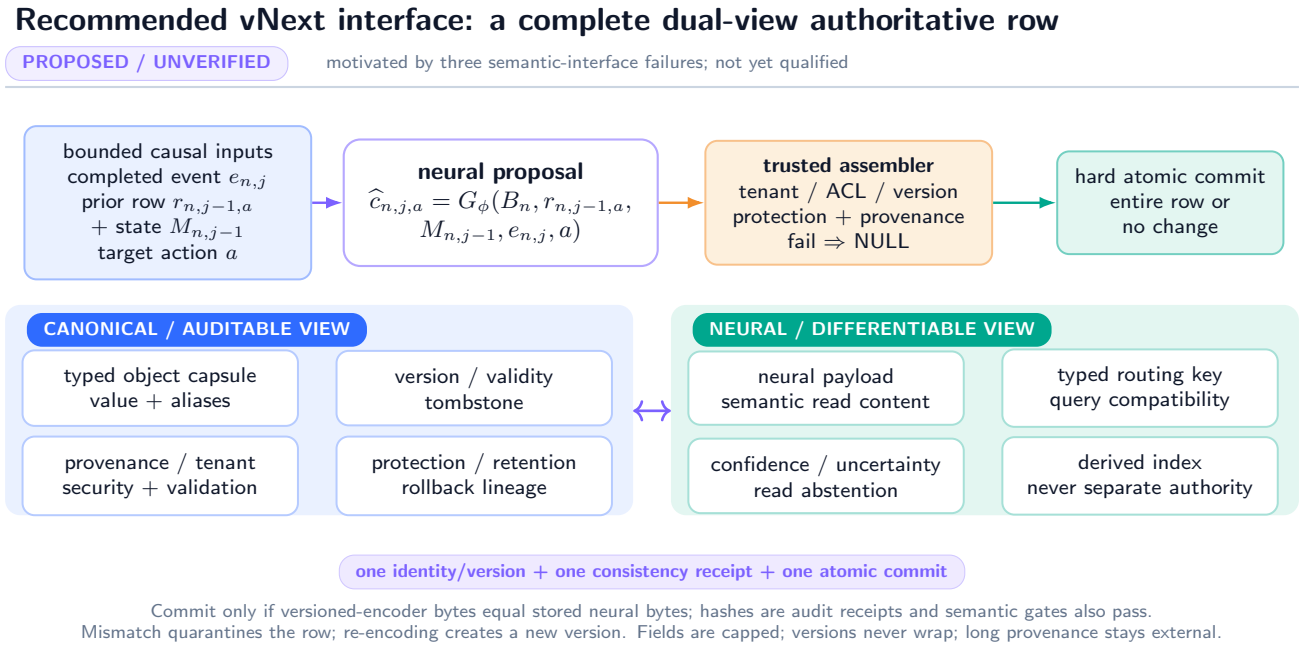

Figure 4: **Proposed complete-row contract.** Canonical and neural views share one identity, validation boundary, version, and security scope. This vNext interface is explicitly unverified.

For each event j, define the feasible set only after every bounded candidate has been assembled and validated,

$$\mathcal{A}_{n,j} = \{\varnothing\} \cup \left\{a \in \{1, \ldots, S\} : \widehat{r}_{n,j,a} \in \mathcal{R}_B,\ \text{Valid}(\widehat{r}_{n,j,a}) = 1\right\}. \tag{12}$$

The controller chooses $a_{n,j} \in \mathcal{A}_{n,j}$ and a hard authoritative gate $g_{n,j} \in \{0, 1\}$, with $g_{n,j} = 0$ exactly when $a_{n,j} = \varnothing$. The state transition is

$$M_{n,j} = \begin{cases} M_{n,j-1}, & g_{n,j} = 0 \quad \text{(NULL)}, \\ \text{Replace}(M_{n,j-1}, a_{n,j}, \widehat{r}_{n,j,a_{n,j}}), & g_{n,j} = 1, \end{cases} \qquad M_n = M_{n,K_n}. \tag{13}$$

Insert, update, consolidating update, eviction, and deletion are lifecycle interpretations of replacing one complete row. Merging two authoritative resident rows requires two ordered events or a future bounded transaction, because changing one target cannot silently invalidate another row. A tombstone is itself a committed, versioned row; NULL changes nothing. Protection removes forbidden actions. A soft mixture, if useful, belongs only to non-authoritative fast neural state and never to the versioned fact table.

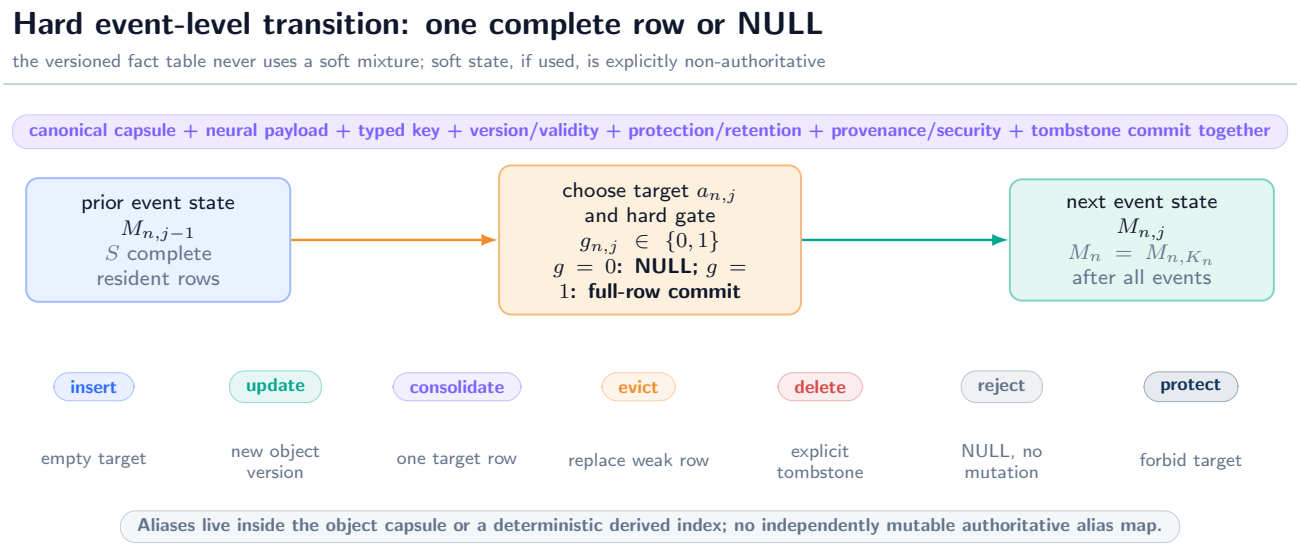

Figure 5: **One physical transition, several lifecycle meanings.** The selected target, complete replacement row, and NULL option define the authoritative operation.

If neural payloads satisfy $\|v(r)\|_2 \le B$, a committed payload change obeys

$$\|v(\widehat{r}_{n,j,a_{n,j}}) - v(r_{n,j-1,a_{n,j}})\|_2 \le 2B. \tag{14}$$

This is a one-step payload bound, not a convergence theorem. A poor policy can still oscillate or erase useful state. Permanent recovery of every historical version is impossible under fixed resident capacity unless versions are summarized or moved to a separately charged archive.

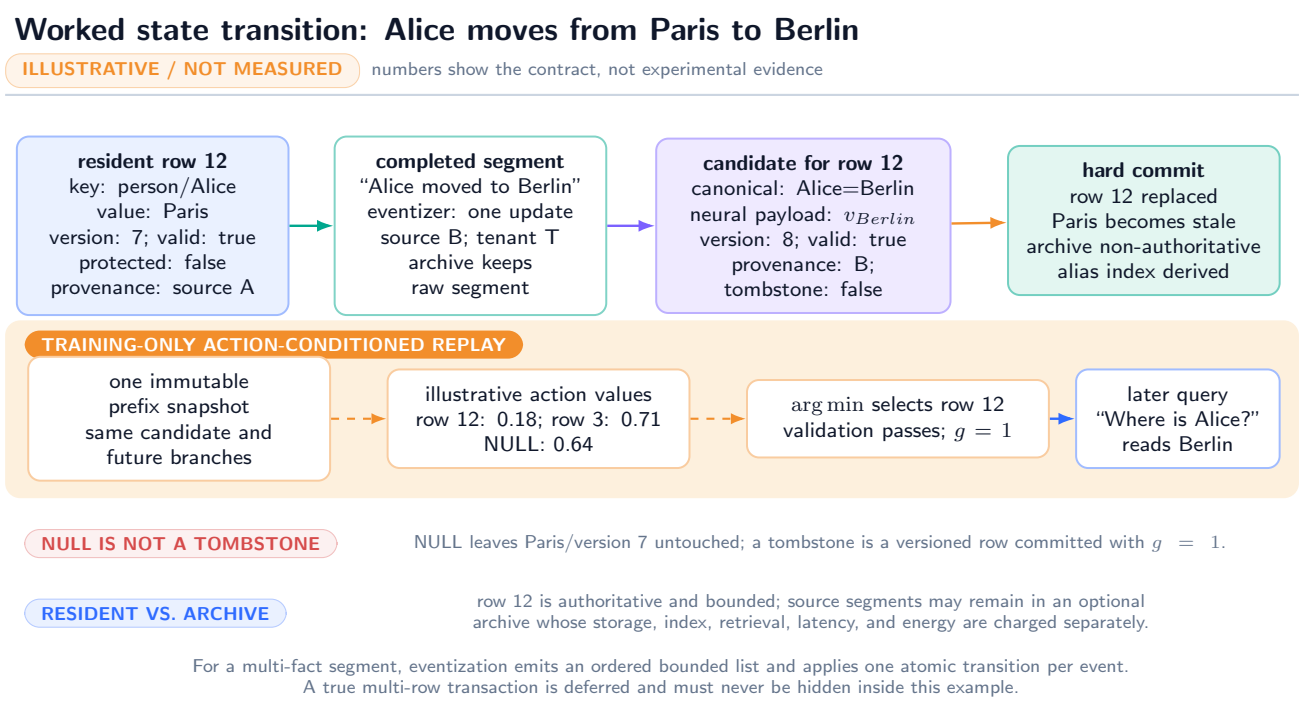

Figure 6: **Worked state transition.** Alice's authoritative row advances from Paris/version 7 to Berlin/version 8. Numerical action values are illustrative, not measured. NULL preserves version 7; a tombstone would be a committed version 8 row. The replay teacher replays each target-conditioned action from the same snapshot; the archive remains non-authoritative and separately charged.

System and recovery contract. The action mask enforces protection and tenant scope before values are compared. A failed schema, signature, provenance, monotone-version, or security check returns NULL. A committed row and its audit record publish atomically. Recovery restores a prior versioned snapshot and invalidates deterministic derived indexes. Verified deletion writes a tombstone and applies the declared archive-retention policy; it is never conflated with NULL. These are formal requirements, not end-to-end validated mechanisms.

2.4 Multi-event, multi-future risk teacher and causal action values

Let the bounded deployment summary for event j be $s_{n,j} = S_\theta(h_{b_n-w+1:b_n}, z_n, M_{n,j-1}, e_{n,j})$. The notation $x_{\le b_n}$ describes only the causal information set: deployment cannot reread the full prefix unless archive retrieval and its cost are explicit. A segment is one finite-horizon decision episode with events $j = 1, \dots, K_n$. For any event-action sequence $\mathbf{a}_{1:j} = (a_1, \dots, a_j)$, define the prefix state recursively by

$$M_{n,0}^{()} = M_{n-1}, \qquad M_{n,j}^{(\mathbf{a}_{1:j})} = \mathrm{Commit}\Big(M_{n,j-1}^{(\mathbf{a}_{1:j-1})}, a_j, \widehat{r}_{n,j,a_j}^{(\mathbf{a}_{1:j-1})}\Big). \tag{15}$$

The event summary, candidate, feasible mask, and later event state are recomputed from that branch's validated prefix; mixing a candidate from one prefix with another prefix is invalid. The future continuation F remains random under $P(F \mid s_{n,1})$.

The exact sequence teacher prices a policy or complete action sequence $\mathbf{a} = (a_1, \dots, a_{K_n})$ from one immutable pre-segment snapshot. A deterministic sequence is a special future-blind policy and is useful for exhaustive enumeration:

$$\begin{aligned} J_H^{\mathrm{seq}}(\mathbf{a}; s_{n,1}, F) = &\sum_{j=1}^{K_n} \lambda_{\mathrm{write}} \mathbf{1}[a_j \neq \varnothing] + \lambda_{\mathrm{int}} D_{\mathrm{collateral}}(M_{n,K_n}^{(\mathbf{a})}) \\ &+ \sum_{k=1}^{H} \gamma^{k-1} \ell\Big(F_k; M_{n,K_n}^{(\mathbf{a})}\Big). \end{aligned} \tag{16}$$

For a feasible action a at event j after branch prefix $\mathbf{a}_{<j}$, let $\Pi_{j+1:K_n}^{\mathrm{fb}}$ be the preregistered class of deterministic or stochastic continuation policies whose action at event q is measurable only with respect to the completed-segment event history and branch state $s_{n,q}$, never the identity or tokens of sampled F. Its teacher value is the future-blind cost-to-go

$$Q_{n,j}^{\mathrm{T}}(a \mid s_{n,j}^{(\mathbf{a}_{<j})}) = \min_{\pi \in \Pi_{j+1:K_n}^{\mathrm{fb}}} \mathbb{E}_{F, \mathbf{a}_{j+1:K_n} \sim \pi}\left[J_H^{\mathrm{seq}}((\mathbf{a}_{<j}, a, \mathbf{a}_{j+1:K_n}); s_{n,1}, F)\right]. \tag{17}$$

The order of minimization and expectation is essential: moving the minimum inside $\mathbb{E}_F$ would allow a clairvoyant continuation action chosen after seeing the sampled future. Exact dynamic programming minimizes the grouped expected return at every branch prefix; an approximation uses one frozen future-blind rollout policy declared before held-out futures are opened. Thus later events are either explicitly optimized or explicitly governed; they are never silently ignored. When $K_n = 1$, this reduces to the single-event risk below. For the first matched causal intervention, the registered protocol deliberately sets $K_n = 1$ and forbids later writes inside the scoring horizon, so that every branch isolates exactly one action:

$$J_H(a; s_{n,j}, F) = \sum_{k=1}^{H} \gamma^{k-1} \ell\Big(F_k; M_{n,j}^{(a)}\Big) + \lambda_{\text{int}} D_{\text{collateral}}(a) + \lambda_{\text{write}} \mathbf{1}[a \neq \varnothing], \tag{18}$$

where $M_{n,j}^{(a)}$ is the event-level state after action a. The frozen reader must replay the continuation under that post-action state; adding memory only after a future hidden trace has already been cached is a fixed-trace diagnostic, not an action-conditioned counterfactual. The first matched experiment forbids later writes inside this horizon, so that J_H identifies the consequence of exactly one intervention. The collateral term measures damage to facts that were not modified, leakage of obsolete values, or destruction of a still-useful slot.

The deployable single-event objective is the conditional action value

$$\overline{J}_H(a \mid s_{n,j}) = \mathbb{E}_{F \sim P(\cdot \mid s_{n,j})} \left[J_H(a; s_{n,j}, F) \right]. \tag{19}$$

For either the sequence return in Eq. 16 or its registered $K_n = 1$ restriction, the formal benchmark approximates the future expectation with K_F physically grouped continuations sharing the identical prefix and pre-decision memory snapshot,

$$\widetilde{J}_H(a \mid s_{n,j}) = \sum_{k=1}^{K_F} w_k J_H(a; s_{n,j}, F_k), \qquad \sum_k w_k = 1, \tag{20}$$

with fixed preregistered weights w_k that are normalized within a group and never inferred from the preferred action. Group IDs, not continuation branches, are split across fit, calibration, and held-out data. For $K_n > 1$, the same grouped futures price every branch prefix in the exact dynamic program or the declared frozen rollout policy. A future-blind value model $\widehat{Q}_\psi(a \mid s_{n,j})$ regresses the detached cost-to-go target for *every* feasible action, for example with a Huber loss,

$$\mathcal{L}_Q = \sum_{a \in \mathcal{A}_{n,j}} \text{Huber}\Big(\widehat{Q}_\psi(a \mid s_{n,j}) - \text{stopgrad}\, \widetilde{Q}^{\text{T}}_{n,j}(a \mid s_{n,j}) \Big). \tag{21}$$

At deployment it processes events in order, choosing $\arg\min_a \widehat{Q}_\psi(a \mid s_{n,j})$, committing the chosen row, recomputing $s_{n,j+1}$ from the resulting state, and optionally requiring a calibrated margin over NULL. It receives no continuation, future hidden state, answer, or gold action. In the first causal intervention, $K_n = 1$ and one payload with $g_{n,1} = 1$ may deliberately be shared across slot actions to isolate *where or whether to write*. That controlled simplification is not the complete target-conditioned architecture of Equations 8–9 and cannot establish multi-event credit assignment.

This distinction repairs a failure of the earlier formulation. In general,

$$\mathbb{E}_F[\text{softmax}(-J_F/\tau)] \neq \text{softmax}(-\mathbb{E}_F[J_F]/\tau). \tag{22}$$

Consequently, KL imitation of a Gibbs policy built from one realized future does not by itself optimize conditional expected risk. The earlier code remains useful for interface qualification, but it is not the decisive predictive-consolidation estimator. A learned latent rollout may later approximate $P(F \mid s_{n,j})$, as in world-model and latent-imagination systems [37, 38]; it is not the first source of supervision, because a misspecified predictor can reward memories that merely make its own imagined future self-consistent.

2.5 Motivation only: predictive information under finite capacity

The persistent state is a lossy code of the observed past. We want it to approach a sufficient statistic for the future,

$$P(X_{>b_n} \mid X_{\le b_n}) \approx P(X_{>b_n} \mid M_n), \tag{23}$$

rather than reconstruct every previous token. A predictive information-bottleneck objective is

$$\max_{q(M \mid X_{\le b_n})} I(M; X_{>b_n}) - \beta I(M; X_{\le b_n}) - \lambda\, \mathbb{E}[N_{\text{overwrite}}]. \tag{24}$$

This is the information-bottleneck principle with future observations as the relevant variable [99, 8]. If the deployed slots use a finite representation of b bits each, then

$$\begin{aligned} I(M; X_{\le b_n}) &\le H(M) \le Sb, \\ I(M; X_{>b_n}) &\le I(X_{\le b_n}; X_{>b_n}), \end{aligned} \tag{25}$$

where the second inequality follows from data processing. The architecture cannot create predictive information or evade finite storage; it can only learn which predictive information to preserve. A variational implementation may replace the mutual-information penalty with a KL term to a bounded prior [2].

For the finite action set, let $E(a) = \overline{J}_H(a \mid s_{n,j})$ and $Z = \sum_a \exp[-E(a)/\tau]$. Any entropy-regularized action distribution q obeys the exact identity

$$\begin{aligned} \mathcal{F}[q] &= \mathbb{E}_q[E(a)] - \tau H(q) \\ &= \tau D_{\mathrm{KL}}(q \| q^*) - \tau \log Z, \\ q^*(a) &= \frac{e^{-E(a)/\tau}}{Z}. \end{aligned} \tag{26}$$

Equation 26 is a regularized decision identity for one *already averaged* conditional-risk vector. It does not justify averaging Gibbs policies formed separately for sampled futures; Eq. 22 shows why. The first intervention trains action values directly. Here τ is an optional optimization temperature, not the physical temperature of a GPU or a brain.

For completeness, an overwrite is logically irreversible when it discards the previous slot. Landauer's ideal lower bound is

$$Q_{\min} \ge b_{\mathrm{erase}} k_B T \ln 2 \tag{27}$$

for erasing b_{erase} independent bits [55]. This observation has no evidential role in the architecture or experiments. It establishes compatibility with thermodynamic accounting, not measured energy efficiency; present hardware operates far above this limit, and fewer logical state changes need not reduce wall-plug power.

2.6 Bounded resident state and the full system cost

Let T be total stream length, W the local window, S the resident slots, $K_r \ll S$ the active slots read per token, K_e the maximum events per segment, and C the minimum tokens per consolidation. Let $R_q(S, d)$ be the routing cost for one token, $C_{\mathrm{cons}}(W, d)$ the completed-segment encoding/eventization cost, $C_G(d, L_c)$ the cost of constructing one target-conditioned semantic proposal, $C_A(B_m)$ the trusted assembly/dual-view validation cost per candidate, $C_Q(S, d)$ the action-scoring cost per event after candidates exist, and $C_{\mathrm{commit}}(B_{\mathrm{row}})$ the bounded row-commit cost. If the complete architecture materializes all S target candidates before choosing an action, the honest deployment context-mixing budget is

$$\begin{aligned} C_{\mathrm{deploy}}(T) &= O(TWd) + O\big(TR_q(S, d)\big) + O(TK_r d) \\ &\quad + O\left(\frac{T}{C}\left[C_{\mathrm{cons}}(W, d) + K_e\left(S(C_G + C_A) + C_Q + C_{\mathrm{commit}}\right)\right]\right), \\ \mathrm{bits}(M_n) &\le B_{\mathrm{resident}} = S(B_i + B_c + B_v + B_k + B_m). \end{aligned} \tag{28}$$

Here C_G and C_A are arguments of their displayed shapes; the shorthand suppresses them only inside the bracket. A shared precomputation may reduce SC_G to $C_G^{\mathrm{shared}} + SC_G^{\mathrm{target}}$, and a preregistered target shortlist of size $L \ll S$ may replace S only if shortlist construction, misses, and excluded actions are charged and reported. Neither optimization may omit complete-candidate construction from accounting. An exact read scan has $R_q(S, d) = \Theta(Sd)$. An approximate index can reduce query work only by adding explicit build, update, storage, and miss costs; those costs do not disappear from the system boundary.

Training cost is larger because the teacher branches actions and futures. For exact multi-event dynamic programming, let $N_{\mathrm{node}} \le \sum_{j=0}^{K_e-1}(S+1)^j$ be the number of reachable branch prefixes before the next action after feasibility pruning. With K_F grouped futures and replay cost $C_{\mathrm{replay}}(H)$ per complete sequence leaf, let $N_{\mathrm{leaf}} \le (S+1)^{K_e}$ be the corresponding number of complete feasible action sequences. The additional teacher budget is

$$C_{\mathrm{teacher}} = O\left(\frac{T}{C}\left[N_{\mathrm{node}}\left(S(C_G + C_A) + C_Q + N_{\mathrm{child}} C_{\mathrm{commit}}\right) + N_{\mathrm{leaf}} K_F C_{\mathrm{replay}}(H)\right]\right). \tag{29}$$

where $N_{\mathrm{child}} \le S$ is the maximum number of feasible non-NULL children materialized from a prefix (NULL reuses the parent state). The registered first intervention has $K_e = 1$, so $N_{\mathrm{node}} = 1$, $N_{\mathrm{leaf}} = S + 1$, and $N_{\mathrm{child}} = S$ before feasibility pruning: all S complete candidates are built, all S non-NULL successor rows are materialized, and all $S + 1$ actions, including NULL, are replayed across every grouped future. If an implementation uses immutable structural sharing, it may replace SC_{commit} by a separately measured branch-state materialization cost, but it may not charge only the ultimately selected deployment commit. Any multi-event successor must report pruning or rollout-policy approximation separately rather than hiding the exponential branch factor.

Sparse fallback over an archive A_T makes the *resident* state bounded but not the total retained information: archive and audit-ledger size, index size, retrieval latency, and retrieval energy must be reported separately. With fixed W, S, K_e, C and a specified router/candidate policy, online deployment cost remains linear in the stream, but this is not a claim of constant total memory or small constant factors. The bound also remains aspirational until consolidation, candidate construction, validation, routing, and overwrite are fused into a few large kernels; an implementation dominated by small launches can lose despite favorable asymptotics. BF16 is the reference training path, FP8 is appropriate for qualified matrix multiplications, and normalization, accumulated risk, and discrete action values retain the FP32 islands required for stability. The design introduces no FP64 requirement. FP4 is a later inference or post-training option whose overwrite stability must be tested separately.

2.7 The registered minimal intervention

The next comparison freezes one completed small-model checkpoint and its reader, then trains only parameter-matched consolidation/controller modules on new, physically separated data. Four deployable arms and one oracle share the state size, candidate payload, gate, action candidates, overwrite budget, future-blind deployment interface, optimization exposure, and final checkpoint:

Arm	Segment representation	Write supervision
No-op memory	causal	always NULL
Immediate writer	causal final state	current-only risk
Local consolidation	bidirectional completed segment	current-only risk
Expected-risk consolidation	same local consolidator	grouped multi-future action-value regression
Expected-risk oracle	same local consolidator	empirical conditional risk at evaluation; non-deployable ceiling

The immediate-to-local difference isolates post-segment reinterpretation. The local-to-expected-risk difference isolates the use of a continuation distribution rather than current-only risk. Every benchmark group contains an identical observed prefix with four to eight continuations, including ambiguous prefixes for which NULL is optimal in expectation and cases whose value appears only later. No later write is allowed during the scoring horizon. The oracle first tests whether the data and action set contain useful conditional future value; if it does not beat current-risk writing, scale cannot rescue this intervention. Student evaluation reports future loss, action regret, strict overwrite success, stale-value leakage, preservation of untouched facts, overwrite rate, latency, resident and archive memory, routing/index cost, kernel count, and energy.

2.8 Evidence-gated training allocation

The proposed memory capability is primarily a *mid-training or post-training* intervention, not a reason to begin with an expensive from-scratch model. Let θ_0 be one frozen backbone, ω a current-state interface, and ψ the future-blind action-value model. The evidence order is

$$\begin{aligned} \hat{\omega} &= \arg\min_{\omega} \mathcal{L}_{\text{interface}}(\omega; \theta_0), \quad \theta = \theta_0, \\ \hat{\psi} &= \arg\min_{\psi} \mathcal{L}_{Q}(\psi; \theta_0, \hat{\omega}), \qquad \theta = \theta_0. \end{aligned} \tag{30}$$

The first line asks whether the resident state can be read and reconstructed at all; the second asks whether a deployable controller can exploit that interface to reduce future risk. Only if both fixed-checkpoint stages beat matched controls without collateral damage does joint training become scientifically justified:

$$\min_{\theta,\omega,\psi} \mathcal{L}_{\text{LM}}(\theta) + \lambda_{\text{mem}} \mathcal{L}_{\text{interface}} + \lambda_Q \mathcal{L}_Q. \tag{31}$$

Equation 31 is a conditional scale study, not a completed result. This sequence keeps the comparatively cheap representation and policy hypotheses falsifiable before allocating a full pretraining budget.

Current evidence boundary. An earlier one-realization CPU loop remains a useful interface demonstration, but its cached future and realization-specific Gibbs target do not estimate Eq. 19. Three subsequent three-seed current-state qualifications failed their preregistered semantic conditions. The first used an external semantic adapter; the second asked whether a raw hidden representation was directly usable. The third compared a dense row, a structured 4×192 span, and the checkpoint-native reader under matched data, training, and final-only evaluation. A versioned repair corrected the mismatch between conceptual audit names and physical safetensors keys before any successful training result existed. All nine repaired jobs then completed 2,048 updates, froze predictions before evaluator truth, and were evaluated unconditionally. Candidate exactness remained 0–45/23,040, query exactness 10–1,536/9,216, and record-macro Brier 0.998676–0.999093. Old-value suppression and physical-map invariance passed, but every semantic qualification failed.

PM-I3 reuses those immutable final checkpoints without optimizer updates and evaluates the complete registered fit population before opening the historical fresh summaries. Across nine jobs, direct candidate reconstruction is 40/73,728,

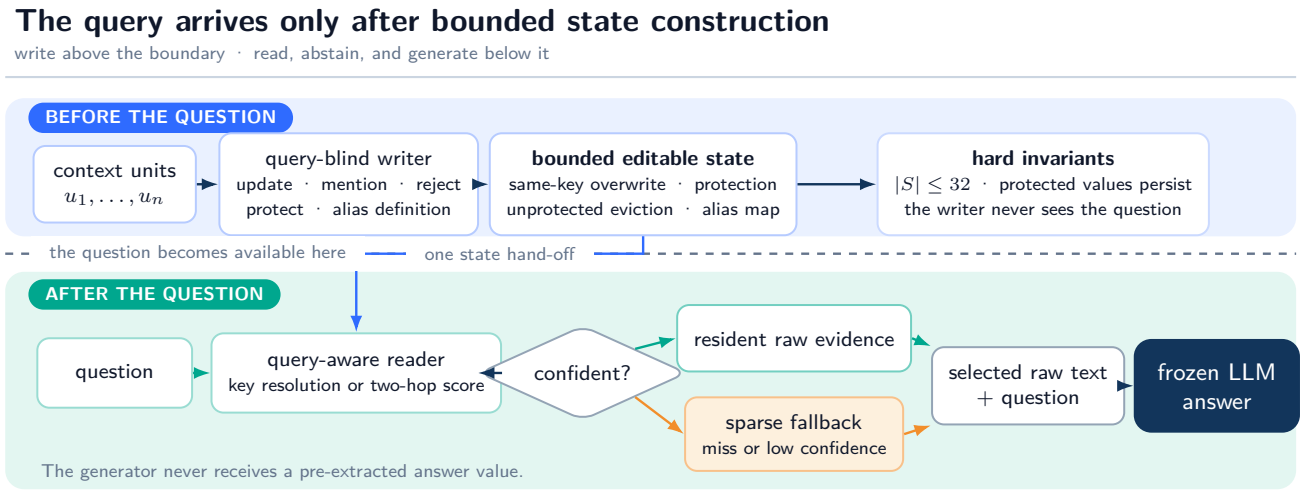

Figure 7: The implemented path. State construction finishes before the question crosses the dashed boundary. At query time, the reader either uses resident evidence or abstains to budgeted sparse retrieval. The frozen language model receives the selected raw text and the question, never a pre-extracted answer value.

direct present readout and learned-content lattice exactness are both 0/73,728, and a routing oracle still leaves learned content at 0/73,728. Oracle content with learned routing reaches 18,544/73,728, exactly the learned routing count, while the fully oracle-specified path is the required mechanical ceiling at 73,728/73,728. Historical fresh results remain near zero on content-bearing probes. The preregistered tree therefore takes its fit/support-failure branch: the evidence cannot distinguish representation geometry, decoder, and optimization within that branch, but it does rule out fresh composition as the primary failure. No arm comparison is admissible, the same pure latent-row interface is not extended, and expected-risk overwrite remains unopened.

The analogy to complementary learning systems is useful but limited: fast editable state and slow parameter learning play different computational roles [66]. We do not claim fixed slots, atomic overwrite, or conditional-risk supervision as a biological model. More generally, a system that can maintain revisable world state under a fixed budget could be an important substrate for persistent agents. That conditional possibility is narrower than a claim of artificial general intelligence: planning, goals, exploration, causal modeling, social reasoning, safety, and slow continual learning remain separate problems.

3 Implemented Evidence Path

3.1 Components

Let a request context be segmented into candidate units $u_1, \ldots, u_n$ (event lines in the lifecycle study; whole passages in the multi-hop QA study). The path executes in the following order.

Writer (query-independent). A classifier scores each unit from unit-local features only; its interface does not accept the final question. A unit test enforces this restriction, and identifiers, values, and digits are masked in the controlled lifecycle features. In that study the writer labels each line as `update`, `mention`, `reject`, `lock`, or `alias-def`. In the multi-hop QA study it predicts passage salience.

Controlled-language lifecycle executor. Units stream in document order into a memory of at most 32 slots, keyed by entity in the lifecycle study and ranked by salience in the QA study. `update` writes; `lock` writes and protects, making later writes to that key void; `reject` and `mention` never write; alias definitions populate a separate alias map. When a new key arrives at capacity, the least-recently-updated unprotected slot is evicted. This executor tests lifecycle semantics under controlled cues. Because its alias map is independently mutable, it is not itself an implementation of the single-row atomic state contract in Eq. 13.

Reader (query-aware). At question time the reader either resolves the requested key, directly or through the alias map, or scores resident passages with a learned two-hop reranker. The reranker combines dense-retrieval-style features [49], BM25 [83], lexical overlap, and bridge features. Its dense feature is a frozen-language-model hidden state, not a separately trained DPR encoder.

Calibrated sparse fallback. A confidence signal — the writer probability of a slot's source line in the lifecycle study, or the mean of the two highest resident-passage scores in the QA study — is compared with a threshold chosen from calibration data only. A miss or a score below that threshold sends the request to budget-matched sparse retrieval over the full context. The threshold must be calibrated at the intended context length. When it was calibrated on short contexts, where 32 slots contain every passage, fallback never fired and extended-length evidence coverage fell from 0.88 to 0.31 (§10).

Generation. A frozen instruction-tuned model receives only the *raw selected evidence* (verbatim context lines/passages —

asserted at prompt-build time, never a pre-extracted answer value) plus the question, and generates the answer greedily.

The state transition and the fallback decision can be written compactly. Let $S_t = \{(k_i, v_i, p_i, \rho_i)\}_{i=1}^{m_t}$ contain each resident key, value, protection bit, and retention score. For a writer action a_t, the lifecycle operator obeys

$$\begin{aligned} S_{t+1} &= \mathcal{L}(S_t, u_t, a_t), \\ |S_{t+1}| &\le M, \\ p_i = 1 &\Rightarrow (k_i, v_i) \text{ is protected.} \end{aligned} \tag{32}$$

After the question q becomes available, the evidence passed to generation is

$$E(q, S_n, x) = \begin{cases} R(q, S_n), & \text{hit, } c(q, S_n) \ge \tau, \\ \text{SparseRetrieve}(q, x; B), & \text{otherwise,} \end{cases} \tag{33}$$

where B is the fixed evidence budget and τ is chosen on calibration data. Equations 32–33 expose the two guarantees that are easy to conflate: the lifecycle constrains how resident state may change, while fallback handles evidence that was absent from that state.

Two instantiations, disclosed. The controlled versioning study exercises typed events, same-key overwrite, protection with void writes, LRU eviction, and alias resolution. The multi-hop QA study contains static encyclopedia passages, so its memory reduces to a *bounded salience cache*: the 32 passages with the highest query-independent writer scores. No overwrite or protection event fires in that setting. Its writer is query-free at inference, although its training labels come from question-linked supporting facts in the training split. These differences limit what the real-text result can say about lifecycle semantics.

3.2 How the completed evidence maps to the proposal

The implemented path predates the conditional-risk teacher and does not instantiate the entire model in Figure 3. Its role is to test semantics that a learned overwrite must eventually preserve. The controlled versioning study exercises same-key update, protection, stale rejection, eviction, and NULL-like non-writes under a controlled grammar. The multi-hop QA study measures the writer's information boundary and the value of abstention. The relational study tests a candidate-construction side path before lifecycle execution. None compares immediate writing with local or expected-risk consolidation.

A separate native scaffold uses pre-norm RMSNorm, rotary embeddings, grouped-query local attention, SwiGLU, and intermittent memory-managed mixers [116, 93, 1, 45, 91]. It has completed a 50,003,968-token real-data pilot. Predictive consolidation and conditional-risk supervision were not part of that training, so the run is evidence of trainability and checkpoint availability rather than evidence for the new mechanism. The staged studies in Eq. 30 reuse its final checkpoint precisely to avoid attributing backbone or data-order differences to memory-interface learning.

The first CPU implementation enforces useful interface invariants: reads depend only on prior persistent state; local consolidation sees only a completed segment; deployment methods reject future-bearing arguments; and a hard update changes no more than one payload row. It does not replay future tokens under each action, estimate risk from multiple continuations with the same prefix, or include all lifecycle metadata in the resident row. It is therefore a mechanical ancestor, not positive evidence for the learned architecture.

Interface	Seed	Candidate exact	Query exact	Mean Brier	Preservation	Physical map
External semantic adapter	2026082101	358/6656	244/2048	0.6490	13/1536	pass
	2026082102	314/6656	441/2048	0.6263	3/1536	pass
	2026082103	406/6656	258/2048	0.6342	20/1536	pass
Raw representation probe	2026083101	12/2048	343/131072	1.4201	—	pass
	2026083102	43/2048	263/131072	1.4230	—	pass
	2026083103	36/2048	513/131072	1.4023	—	pass

Table 3: Registered frozen-checkpoint interface negatives. "Physical map" denotes the preregistered invariance checks, not semantic success. The external adapter also passed old-value suppression but failed reconstruction, query, calibration, and unrelated-fact preservation. The raw probe failed its first five semantic conditions and passed only two mapping conditions on every seed. These experiments localize an interface problem; they do not reject the native memory mechanism or predictive consolidation.

Seed	Reader	Candidate exact	Query exact	Missing → absent	Mean Brier	Map	Qualified
2026091101	D1 dense row	0/23040	1536/9216	1536/1536	0.998981	pass	no
	S1 structured span	15/23040	1536/9216	1536/1536	0.998676	pass	no
	N1 native reader	30/23040	645/9216	640/1536	0.998992	pass	no
2026091102	S1 structured span	21/23040	10/9216	0/1536	0.999035	pass	no
	N1 native reader	45/23040	10/9216	0/1536	0.999046	pass	no
	D1 dense row	0/23040	1536/9216	1536/1536	0.998815	pass	no
2026091103	N1 native reader	30/23040	10/9216	0/1536	0.999093	pass	no
	D1 dense row	0/23040	1536/9216	1536/1536	0.998818	pass	no
	S1 structured span	0/23040	1536/9216	1536/1536	0.998977	pass	no

Table 4: PM-I2 current-interface qualification from one frozen V28 release. Predictions were materialized before evaluator truth; all 55,296 record rows were separately reduced. Mean Brier is near the 1,025-class uniform reference $1024/1025 = 0.999024$. Every job passes old-value suppression and the registered 64-row mapping checks, but none passes the semantic conditions. Because no interface qualifies at all three seeds, no arm comparison or preferred-reader claim is defined. A separately implemented within-project adjudicator publishes the registered no-interface terminal and marks bootstrap comparison not applicable.

PM-I2 moves one level closer to the model than Table 3: D1 reads a dense row, S1 preserves a structured 4×192 span, and N1 calls the checkpoint-native memory path. Table 4 shows that the result is not a narrow failure of one external codec. The nine jobs preserve the registered structural invariants yet remain close to chance on semantic output. The comparison therefore rules out these three frozen interfaces under their matched budgets; it does not show that the V28 backbone lacks potentially useful information, nor does it distinguish an optimization bottleneck from an unsuitable compositional output geometry. A first launch-only package stopped before production because a post-freeze test used the receipt builder at the wrong lifecycle stage. A versioned repair changed only that lifecycle test and launch identity; the separately implemented within-project adjudicator then reopened the admitted factors and truth, reproduced all nine false decisions, published the registered no-interface-qualified terminal, and performed no bootstrap draws. The exact machine identifier is retained in Appendix H and the evidence-admission audit.

PM-I3 probe	Exact	Population	Diagnostic reading
Direct candidate reconstruction	40/73,728	registered fit records	fit support is not reconstructed
Learned route / learned content (L/L)	0/73,728	grouped fit probes	ordinary interface fails on fit
Oracle route / learned content (O/L)	0/73,728	grouped fit probes	target-row isolation does not repair content
Learned route / oracle content (L/O)	18,544/73,728	grouped fit probes	equals learned row-routing count
Oracle route / oracle content (O/O)	73,728/73,728	grouped fit probes	mechanical ceiling, not learned success
Historical candidate exactness	141/207,360	PM-I2 fresh records	fresh semantics also fail
Historical known/latest query exactness	7/13,824 each	PM-I2 fresh queries	content-bearing queries remain near zero
Historical map / old suppression	82,944/82,944; 13,824/13,824	PM-I2 checks	structural invariants still pass

Table 5: Accepted PM-I3 fit-versus-generalization fault localization. The nine fixed PM-I2 final checkpoints are reused without retraining, checkpoint selection, threshold changes, or arm preference. O/L supplies the registered target-row oracle while retaining learned content; L/O supplies oracle content while retaining learned routing. The complete O/O path is a mechanical ceiling. Because learned content fails even under target-row isolation on the fit population, the preregistered terminal is fit/support failure rather than fresh-composition-only failure.

PM-I3 is diagnostic, not a second qualification and not a retrospective reader contest. It narrows the next direction to a new matched interface study centered on canonical token spans, typed canonical tuples, and the proposed dual-view row. It does not prove which of representation geometry, decoder design, or optimization caused the fit failure, and it does not establish that the V28 backbone lacks useful memory information.

3.3 Preregistration and test discipline

Before generating confirmatory data, we wrote down the comparisons, the numerical conditions that would count as support, and the circumstances that would stop the run. Frozen holdouts were then read once; the multi-hop QA holdouts were read once for each of the two reader models specified in advance. Completion markers prevent accidental re-execution, and every artifact records the code revision that produced it. An earlier version of the lifecycle task was discarded before its test split was read because a lexical rule already solved every example. Section 8 preserves this history and the later evaluator corrections.

4 Controlled Lifecycle Study

Data. 1,200/300/300 train/dev/test records (split-disjoint sentence frames, shared cue vocabulary — a disclosed controlled-language boundary), six balanced scenarios: `base-overwrite`, `alias-query` (the query names the key only through an alias defined once in context), `stale-remention` (post-final-update audit lines repeat old values), `invalid-write` (rejected change requests propose wrong values), `protected-slot` (a lock freezes the value; later updates are void), and `slot-pressure` (48 distinct written keys exceed the 32-slot bound). Generator tests prove that "take the last line mentioning the queried key" is wrong by construction on the four middle scenarios — and returns exactly the designed wrong value — while remaining correct on the other two.

Method	Held-out exact match (3 seeds)	Purpose
Lexical first-key	0.087	floor control
Lexical last-key	0.333	**strongest non-learned baseline**
Naive all-writes lifecycle	0.333	no event typing
Writer allowed to see the question	0.333	leakage control
Learned writer; lifecycle disabled	0.490/0.490/0.493	component removal
Lifecycle active; fallback disabled	0.990 (all seeds)	component removal
Complete controlled lifecycle path	**1.000** (all seeds)	proposed path
Lifecycle with oracle event labels	1.000	diagnostic ceiling

Table 6: Controlled-language lifecycle results on 300 held-out records, read once. Before the split was opened, we required improvement over the strongest hand-written rule on every seed with a positive paired-bootstrap 95% confidence interval, substantially fewer lifecycle errors than the learned writer without lifecycle control, at least 0.90 accuracy on protected values, recovery of most capacity misses, and no more than 32 resident slots. The complete path met each condition: a +66.7-point margin with 95% CI [+61.3, +72.0], zero lifecycle errors versus 0.51, protected-value accuracy 1.0, recovery of all 3/3 target evictions, and peak occupancy 32. These numbers validate controlled execution, not learned natural-language memory management.

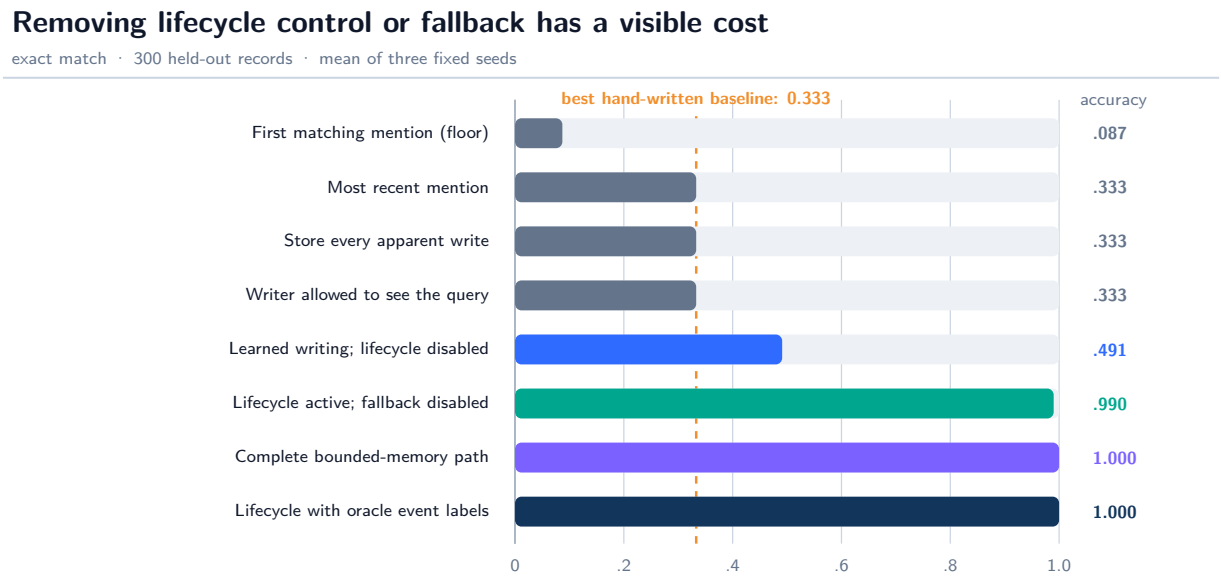

Figure 8: Held-out exact match in the controlled lifecycle study, averaged over three fixed seeds. The dashed line is the strongest hand-written rule. Event typing and query-independent writing are necessary to rise above that rule; disabling lifecycle execution roughly halves accuracy, while disabling fallback loses only the capacity-pressure cases.

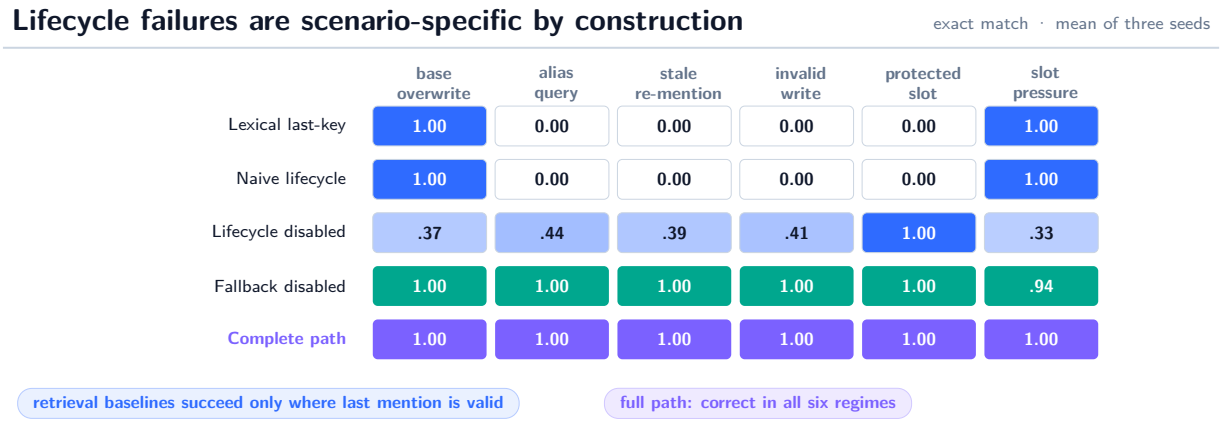

Figure 9: Held-out exact match by lifecycle scenario, averaged over three fixed seeds. Lexical and untyped rules solve only the cases in which the most recent mention is valid. Without lifecycle execution, the learned writer cannot arbitrate versions; without fallback, only capacity-pressure evictions are lost. The complete path is correct in all six scenarios.

Table 6 and Figures 8–9 make the component costs visible. Removing event typing or allowing the writer to depend on the question returns accuracy to the 0.333 lexical level. Keeping the learned writer but skipping lifecycle execution yields about 0.49. Keeping lifecycle execution but removing fallback loses three capacity-pressure cases, reducing exact match from 1.000

Reader model	Method	Natural (25% tier) F1	Natural (25% tier) tokens	Extended 8.2k words (10% tier) F1	Extended 8.2k words (10% tier) tokens
Llama-3.1-8B	Full context	0.714	1477	0.666	12024
	Dense (budget-matched)	0.516	432	0.622	1300
	BM25 (budget-matched)	0.601	437	0.636	1293
	Reader only (no state bound)	0.659–0.663	434	0.695–0.706	1310
	Resident cache + archive fallback	**0.659–0.663**	434	**0.677–0.684**	1323
	Oracle evidence (ceiling)	0.717	438	0.786	1301
Qwen2.5-14B	Full context	0.805	1521	0.715	12646
	Dense (budget-matched)	0.594	425	0.753	1338
	BM25 (budget-matched)	0.701	430	0.771	1330
	Reader only (no state bound)	0.758–0.760	428	0.827–0.828	1351
	Resident cache + archive fallback	**0.755–0.760**	428	**0.813–0.830**	1367
	Oracle evidence (ceiling)	0.804	432	0.854	1340

Table 7: Held-out multi-hop QA results, with 300 questions in each length condition. Ranges show three fixed selector seeds; token counts are whole-prompt averages. The selector — frozen-Llama hidden-state features, a learned two-hop reranker, a query-independent writer, and calibrated thresholds — was developed with Llama and then applied to Qwen without re-tuning.

to 0.990. The writer also transfers across disjoint sentence frames (development macro-F1 1.0; shuffled-label control 0.069).

Fallback has a narrow role in this controlled grammar. All three calibrated thresholds reached the minimum of the search grid because the writer was both confident and correct, and keys and values were recoverable by the controlled parser. The low-confidence branch therefore never fired; fallback was used only for the three target slots evicted under capacity pressure, and recovered all three. The multi-hop QA study supplies the complementary setting, with fallback rates of 0.39–0.72.

Generation. Given selected raw lines rather than pre-extracted values, the complete path answers 300/300 with an average prompt length of 146 tokens. The same frozen Llama-3.1-8B [27], when given the entire log, answers 172/300 with 729 tokens. Its errors concentrate on protection and alias resolution, the two operations executed explicitly by the lifecycle. The improvement therefore cannot be explained by shorter prompts alone.

5 Held-Out Multi-Hop QA at Two Lengths

Data hygiene. An early selector had already consumed labels for 20 examples from the local LongBench HotpotQA slice, so that pool is retained only for diagnostics. The confirmatory questions instead come from the official HotpotQA training split [111]. After removing exact and near-duplicate questions against all 500 local LongBench examples, we froze 1,200 training, 300 calibration, and 300 held-out records. Their median context length is 933 words, far below the roughly 7.9k words used by LongBench [3]. We therefore created a second held-out condition with a median of 8,221 words by adding answer-excluded distractor passages drawn from the training-record pool. The questions and gold passages remain held out, but the distractor text is train-derived; this is a test of held-out questions among familiar distractor text, not of wholly unseen documents. Before developing the selector, we fixed evidence budgets at 25% of context words for natural records and 10% for extended records. Frozen Llama-3.1-8B and Qwen2.5-14B models serve as readers [27, 77].

For selected passages E and full context x, the intended evidence budget is

$$b(E;x) = \frac{\sum_{p\in E} |p|_{\text{word}}}{|x|_{\text{word}}} \le \beta, \qquad \beta = 0.25 \text{ (natural)}, \quad 0.10 \text{ (extended)}. \tag{34}$$

The first Llama execution used whole-prompt tokens rather than Eq. 34 and is retained as failed under that frozen definition; the corrected Qwen follow-up enforces this equation record by record. Section 8 gives the full accounting.

The comparison with budget-matched dense retrieval was fixed in advance. Across all selector seeds, the learned path improves F1 by 14.3–16.6 points at natural length and 5.5–7.6 points at extended length. The lower end of every paired-bootstrap 95% confidence interval remains positive: at least +8.9 and +1.45 for Llama, and +10.6 and +2.0 for Qwen. That dense feature is a weak frozen-hidden baseline (train-only recall 0.14), so the more informative margin is against BM25: 4.0–6.2 F1. A trained modern retriever, answer-masked and no-evidence controls, and end-to-end archive cost remain necessary before making a general retrieval claim.

The long condition changes the usual cost–quality trade-off. Reading all 8.2k words lowers F1 for both readers (0.714 → 0.666 for Llama and 0.805 → 0.715 for Qwen), consistent with previously reported lost-in-the-middle behavior [61]. Selecting at most 10% of the evidence words instead reaches 102–103% of full-context F1 for Llama and 114–116% for Qwen. No part of the selector was re-tuned for Qwen, yet its margins are larger.

The reader-only rows clarify where the gain comes from. Removing the 32-slot bound does not reduce quality; on the extended set it is slightly better than the cache-plus-fallback path for Llama and roughly tied for Qwen. The learned selector creates the quality margin, while the resident cache preserves most of it. Calibrated fallback fires on 39% of natural and 72%

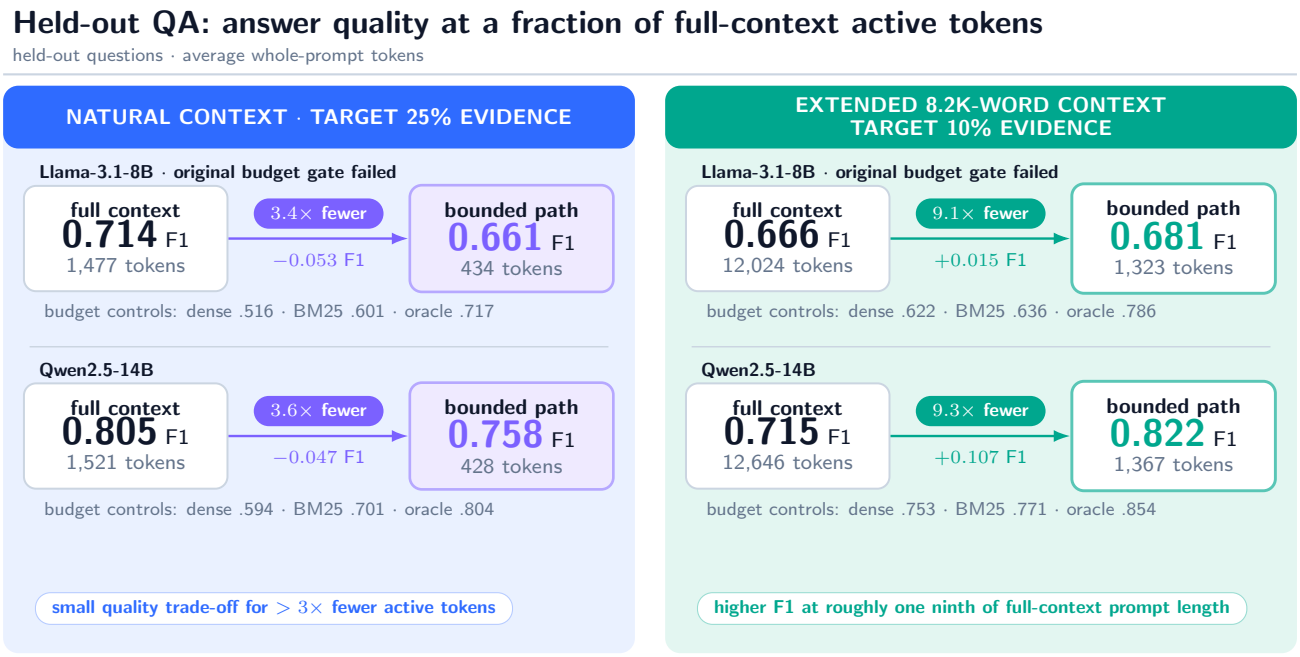

Figure 10: Answer quality and active-prompt cost on the two held-out question sets. Each row compares the resident-cache plus archive-fallback path with the same reader given the full context; budget-matched dense, BM25, and oracle selectors are reported alongside. At natural length, selection trades a small amount of F1 for 3.4–3.6× fewer prompt tokens. At 8.2k words, it improves F1 for both readers while using roughly one ninth as many prompt tokens. These counts exclude archive storage, indexing, retrieval latency, and retrieval energy.

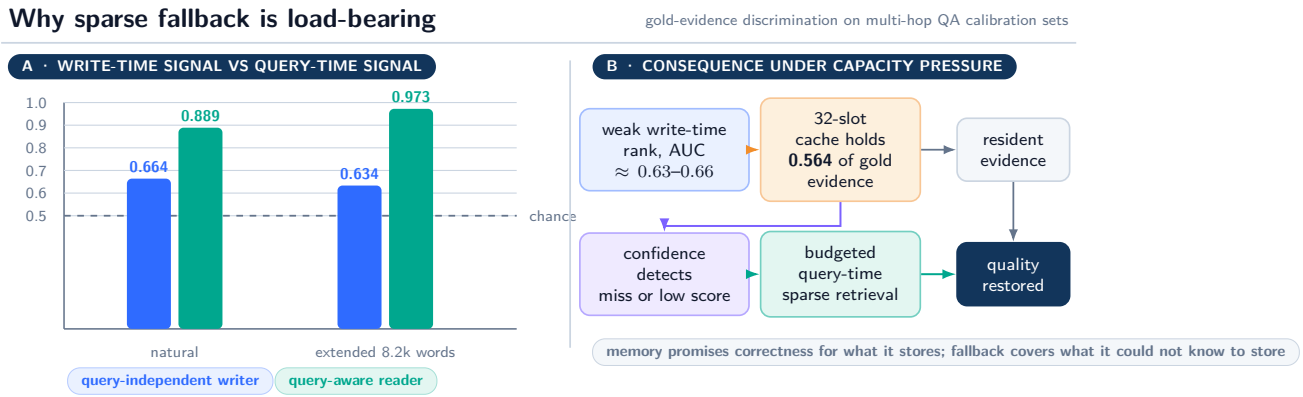

Figure 11: Gold-evidence discrimination on calibration questions. Passage-only features provide little information about which evidence a future question will need. Once the question is available, the same selection problem becomes much easier. A bounded writer therefore needs an explicit way to abstain and retrieve at query time.

of extended records and searches the retained full context, so total memory is not bounded by 32 slots. Because these passages never overwrite one another, this study supports selection and abstention with bounded hot state, not bounded total memory or the overwrite and protection claims established by the lifecycle study.

6 Why Fallback Is Necessary When Relevance Is Unknown

Figure 11 measures the asymmetry directly. A query-independent salience probe over passage-only features reaches AUC 0.664 on natural records and 0.634 on extended records, only modestly above chance. The query-aware reranker reaches 0.889 and 0.973, although part of the extended-set signal comes from the construction procedure. Under capacity pressure, the 32-slot cache consequently contains only 0.564 of the gold evidence. Without fallback, every missing gold passage is unrecoverable; with a threshold calibrated at the deployment length, most of the selector's quality is retained. A bounded memory can enforce correct lifecycle behavior for resident information. It cannot promise to store an ordinary fact whose future relevance was unavailable when the write decision was made.

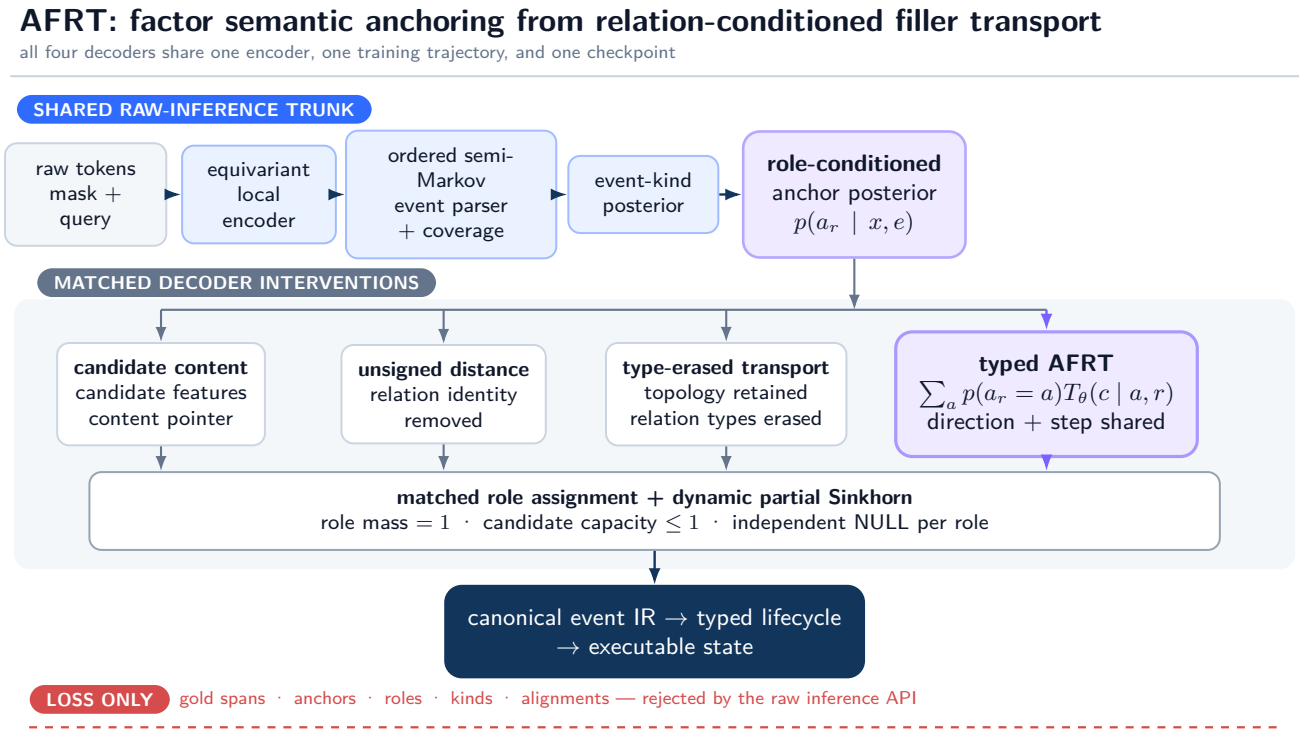

Figure 12: **Relational-binding architecture and matched controls.** The raw encoder, semi-Markov event parser, event-kind head, role-anchor posterior, canonical intermediate representation, and typed lifecycle are shared. The four decoders use candidate content, unsigned distance, type-erased topology, or typed AFRT transport. Direction and path-step parameters are shared in the typed decoder. Gold spans, anchors, roles, kinds, and alignments enter the losses only and are absent from raw inference.

7 Relational Binding with Factorized Anchor–Filler Transport

The lifecycle study relies on a deliberately controlled event parser. We therefore ask a narrower trainability question: once event boundaries and semantic role anchors must be predicted, what inductive bias lets a differentiable compiler bind each role to the correct filler under unseen compositions? Pointer networks motivate content-conditioned selection [101], while relational models motivate explicit structural bias [88, 4]. Our first group-disjoint compiler fit every training example but obtained 0/70 held-out successes for role binding and whole-record execution. Optimization was sufficient; the content-matching readout did not compose. This failure motivated AFRT as a structured side path, not as a replacement for all self-attention.

For event e, role r, and candidate filler c, AFRT factorizes the pointer as

$$p(f_r = c \mid x, e) = \sum_a p(a_r = a \mid x, e)\, T_\theta(c \mid a, r, x, e), \tag{35}$$

where $p(a_r = a \mid x, e)$ is a predicted semantic-anchor posterior and T_θ transports its mass along registered relation paths. Transport depends on relation type, while the two directions and repeated path steps share parameters. Candidate logits are masked by predicted event coverage. They then enter a dynamic partial Sinkhorn assignment P. For the real candidate set C and the private NULL candidate $\varnothing_r$ of each role,

$$\begin{aligned} \sum_{c \in C} P_{rc} + P_{r\varnothing_r} &= 1, \\ \sum_r P_{rc} &\le 1 \quad (c \in C), \\ P_{r\varnothing_{r'}} &= 0 \quad (r \neq r'). \end{aligned} \tag{36}$$

Thus every role places one unit of mass, real fillers cannot be reused, and each role may independently remain unfilled. The typed decoder receives no filler-content feature, absolute position, signed-direction parameter, template or wrapper identifier, or gold span, anchor, role, kind, or alignment.

Matched controls. Each seed trains one model and publishes one final checkpoint. Four jointly trained decoders are read from it: a candidate-content decoder, an unsigned-distance decoder, a type-erased transport decoder, and the typed AFRT decoder. They share the raw encoder, event partition, coverage, kind and anchor heads, initialization, data order, optimizer, token exposure, and checkpoint. Their paired differences do not mix decoder structure with independently selected checkpoints. They do, however, permit co-adaptation through the jointly trained shared encoder, and none is a type-aware non-factorized

Predefined check	Seed 0	Seed 1	Seed 2	Required result
Each of 11 typed-decoder output checks	480/480	480/480	480/480	$\geq 432/480$
Cross-view consistency	240/240	240/240	240/240	$\geq 216/240$
Whole-record exactness	240/240	240/240	240/240	$\geq 216/240$
Each of six stress subsets	40/40	40/40	40/40	$\geq 34/40$
Gain over candidate-content decoder	+240	+240	+240	$\geq +24$
Gain over unsigned-distance decoder	+240	+240	+240	$\geq +18$
Gain over type-erased transport	+240	+240	+240	$\geq +12$
All conditions met for this seed?	yes	yes	yes	yes

Table 8: Fresh three-seed confirmation on physically disjoint fitting and held-out records. The eleven output checks cover boundary, active anchor, present anchor, NULL anchor, role, kind, canonical intermediate representation, execution trace, state, resident selection, and payload exactness. Every matched control has zero whole-record successes on every seed. These are controlled synthetic results, not natural-language, long-context, or full-language-model results.

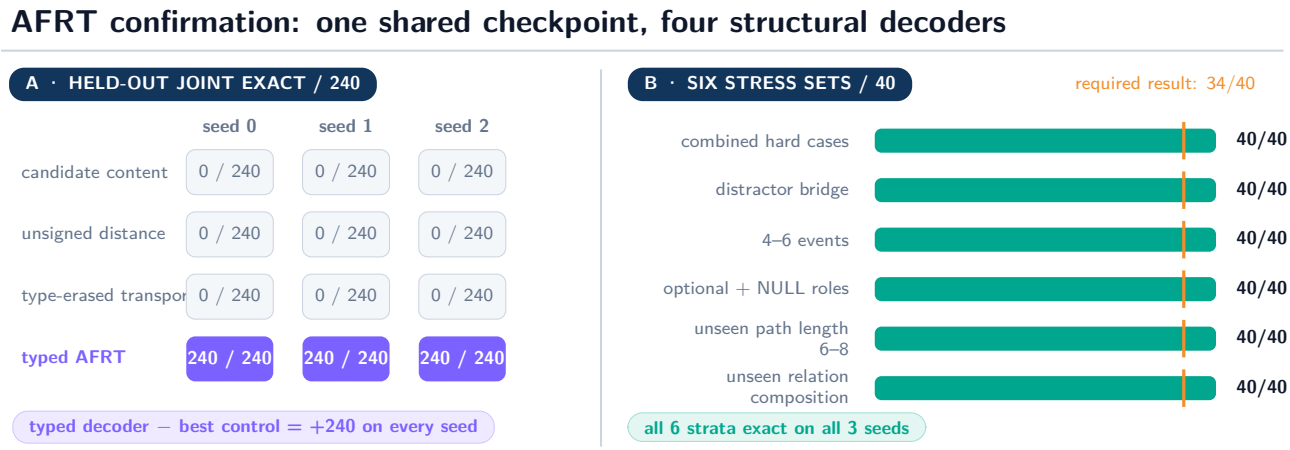

Figure 13: Matched-control result for relational binding. Every decoder is read from the same checkpoint for a given seed. Typed AFRT is exact on all 240 held-out records for each of three fresh seeds; candidate-content, unsigned-distance, and type-erased transport decoders have no whole-record successes. The right panel separates the six predefined stress subsets.

control. The present comparison therefore identifies a need for typed topology relative to the registered ablations; it does not isolate factorization as the unique cause.

Richer topology and fixed repetitions. Each of three fresh data seeds contains 512 fitting records and 240 physically disjoint held-out records. The generator varies path length from 2–8, relation composition, distractor bridges, event count from 2–6, and optional or NULL roles. Six stress subsets contain 40 records each. Every model is trained for 32 complete epochs and 512 optimizer steps at effective batch 32. We fixed the three data/model seed pairs before training and did not use early stopping, choose among checkpoints, replace a failed seed, or tune a threshold after seeing a held-out result.

For decoder d and seed s, the paired whole-record gain over a matched control b is

$$\Delta^{(s)}_{d-b} = \sum_{i=1}^{240} \left(\mathbf{1}[\hat{y}_{i,d} = y_i] - \mathbf{1}[\hat{y}_{i,b} = y_i] \right). \tag{37}$$

This quantity is positive only when the proposed decoder solves more of the same records than its control.

Result. We counted the hypothesis as supported only if every one of the three fixed seed pairs met every condition in Table 8; no failed seed could be replaced. All three did. Typed AFRT obtains 240/240 whole-record exactness and cross-view consistency on each seed, and all six stress subsets are 40/40. Each registered control has no whole-record successes, yielding $\Delta = +240$ against every control on every seed. The result supports a narrow causal statement: in this controlled topology and joint-training setup, typed relation-conditioned transport supplies a reproducible binding signal absent from content, unsigned-distance, and type-erased alternatives. A decisive factorization claim still requires a parameter-matched, type-aware non-factorized decoder and independently trained same-initialization/equal-compute repetitions. The result does not establish natural-language relation discovery, arbitrary graph reasoning, long-context scaling, or superiority to external sequence architectures.

Failure and evaluator audit. The evidence did not improve monotonically. The initial group-disjoint compiler failed as described above. A later single-seed qualification isolated an 80/80 typed-transport result against 0/80 for candidate content, but was not enough for a confirmation. The first three-seed richer-topology study remains failed under its original evaluation contract. Two post-hoc audits then found that payload exactness omitted a separately stored lifecycle version and that anchor

equality counted inactive padded events. The fresh confirmation used new data and model seeds, with both definitions fixed before training and before any metric was observed.

One evaluation-only recovery is also disclosed. Canonical JSON sorted the metric keys, while the original adjudicator incorrectly required dictionary insertion order. A versioned adjudicator replaced that requirement with exact key-set equality; missing or extra metrics still fail closed. No model, data, prediction row, score, seed, numerical condition, or threshold changed, and the superseded aggregate path remains unused.

Originality boundary. Ordered semi-Markov segmentation [89], Sinkhorn assignment [18], canonical intermediate representations, typed lifecycle execution, pointer-style selection [101], and generic relational inductive bias [4] are inherited tools. The proposed mechanism is the anchor-posterior/filler-pointer factorization in Eq. 35, filler-content-invariant typed transport with direction and step sharing, the partial assignment constraints in Eq. 36, and the same-checkpoint structural intervention. We do not attribute novelty to the standard components that make the mechanism executable.

8 Preregistration History and Forward-Looking Corrections

We report the full preregistration history, including the experiments that did not support the hypothesis.

Lifecycle-task re-registration. The original versioning benchmark accidentally made "take the last line mentioning the queried key" correct on every example. No learned method could exceed that 1.0 baseline by the three points required in the original protocol, and the data never exercised fallback, capacity, or protection. Cross-review found the defect before the test split was opened. We then registered the harder generator in §4; the original test split remains unread.

Evidence-budget definition. The multi-hop QA protocol intended to limit selected evidence to a fraction of the context words. Its frozen implementation instead divided whole-prompt tokens by context tokens, thereby counting roughly 60 fixed question and instruction tokens against every method. Under that implementation, the Llama run exceeded its allowances: 29.4% rather than 26.25% at natural length and 11.0% rather than 10.5% at extended length. We retain that run as failed under its frozen definition even though its quality comparisons were positive on every seed.

The saved prediction rows permit a separate evidence-word reading. The extended set respects the intended 10% limit on every record. Natural contexts use 22.6% on average, but a greedy first-passage exception reaches 61% on one short record. For later work only, we adopted the evidence-word definition and replaced that exception with a hard per-record cap, truncating the top passage only when no whole passage fits. The Qwen follow-up uses this corrected packer and respects both limits exactly. We therefore describe it as a forward-looking follow-up, not as a second successful execution of the original Llama protocol.

Relational-binding evaluator contracts. The first three-seed richer-topology result remains "not demonstrated" under its frozen evaluator. We did not recompute that verdict after finding the payload and padded-anchor defects. Instead, the defects were documented in two post-hoc audits, and a new confirmation used fresh data and model seeds with corrected definitions fixed beforehand. We required all three fixed seed pairs to meet every predefined condition, and they did. The JSON-key-order recovery described in §7 happened before aggregate scoring, wrote to a new path, and changed no scientific quantity. Keeping the failed run, the audits, and the fresh confirmation separate prevents evaluator repair from becoming retrospective model selection.

9 Supporting Controlled Evidence (Corrected)

The integrated path builds on a longer sequence of controlled studies. A logic audit found nine issues, including a fixed target position, two evaluation shortcuts, an entity-identity split overlap, and invalid pooled confidence intervals over repeated use of the same examples. We corrected and reran the affected studies and report repetitions separately.

- **2M-token synthetic stress (collision-free rerun):** explicit memory + sparse fallback scores $49/50$ at 2,097,152 tokens with ≤ 132 active chunks; fixed-state and sparse-only proxies each fail complementary scenario classes.
- **Trainability:** a 2.74M-parameter causal event-token backbone learns routing signals (199, 196, and 200 of 200 across three seeds with lite write supervision).
- **Frozen-hidden bridge:** all 18 predefined model–seed comparisons across six model families reach at least $57/60$ on entity-disjoint splits, showing that published-model hidden states contain lifecycle signal when key identity is supplied.
- **Controlled grounding and generation:** oracle-free five-digit lifecycle at $90/90$ per model/seed (Llama/Qwen); causal writer + hard lifecycle + generation at $90/90$; trained soft-token value compression is a documented negative (unstable 85–$89/90$ across regularizers), with lossless span replay strictly better ($90/90/88$).
- **Open-domain selector precursor (IDF multi-hop):** a non-learned IDF multi-hop selector recovers structured variable-tracking chains (+58.0 points, CI $[49.8, 65.8]$) but fails open-domain HotpotQA (-7.1 F1, CI $[-14.6, 0.25]$), motivating the learned two-hop selector.
- **Differentiable compiler sequence:** the first compiler fits but does not generalize role binding; a single-seed same-checkpoint study isolates an $80/80$ typed-transport result against $0/80$ for content matching; the first three-seed richer-topology study

remains failed; two audits identify evaluator defects; and a fresh three-seed confirmation tests the corrected question (Table 8).

10 Negative Results

All artifacts are retained. Raw frozen-language-model hidden states are weak retrievers (layer-probe recall 0.14; dense-only F1 0.47 at natural length) and become useful only as features in a learned reranker. An IDF-weighted bridge feature and two anchors did not improve that reranker. Presenting evidence in document order rather than score order cost six F1 points at extended length. Calibrating fallback at the wrong context length reduced evidence coverage from 0.88 to 0.31. Trained soft-token value compression never reached the predefined accuracy requirement and was replaced by lossless span replay.

Two earlier results are explicitly superseded or retained as failures. A first scaffold reported 0.02 accuracy without lifecycle control, but its tie-break deliberately preferred stale answers; Table 6 uses a neutral tie-break and obtains 0.49. The first group-disjoint compiler fits its compact training distribution yet has no held-out role-binding successes. The first three-seed richer-topology study also remains failed even though later audits show that part of its apparent payload and anchor failure came from the evaluator. We answer the corrected question with fresh data and seeds rather than rewriting either verdict.

11 Related Work

Efficient sequence backbones. Linear attention, structured state spaces, recurrent hybrids, and long convolutions compress history into a recurrent or convolutional state [50, 16, 36, 98, 75, 34, 21, 76, 110, 39]. Sparse attention limits the set of positions read by each query [115, 114, 14, 22]. Both directions primarily redesign token mixing. We study the semantics imposed on a bounded state when it must support overwrite, protection, eviction, and abstention.

Kernel substrate. FlashKDA is a high-performance CUTLASS implementation of Kimi Delta Attention with token-parallel and head-parallel recurrent updates [15]. Its recurrent state has shape $[B, H, V, K]$; current source documentation reports BF16 or FP32 state, FP32 update accumulation in the optimized path, automatic dispatch through Flash Linear Attention, and H20 kernel speedups over earlier KDA/GDN implementations. Those are upstream kernel measurements, not MMLA measurements. FlashKDA does not define semantic row authority, hindsight supervision, overwrite versus NULL, or lifecycle evaluation. It is nevertheless a plausible complementary substrate for a recurrent/linear MMLA branch if the state geometry and precision contract match. We have not integrated or benchmarked that combination.

Addressable and external memory. Memory Networks and Neural Turing Machines introduced differentiable addressable stores [107, 28]. Recurrent-memory and virtual-context systems carry state across segments or calls [11, 73]. Other Transformer memory systems use recurrence, compression, nearest-neighbor retrieval, retrieved text, or learned memory layers [19, 78, 109, 104, 7, 56, 9, 51, 5, 70]. H2O, Scissorhands, SnapKV, and Landmark Attention retain or index selected KV-cache entries [117, 62, 59, 69]. The closest system-level comparison is sparse memory over static documents (MSA [14]). Our experiments add same-key versioning, protected state, explicit eviction rules, and calibrated abstention when future relevance is unavailable at write time.

Work	Problem	Persistent state	Operation	Training	Inference update	Evaluation / unresolved limit
MMLA	Future-use-aware bounded persistence	S authoritative rows plus completed-segment workspace	One complete-row overwrite or NULL per event	Frozen-backbone interface qualification, then grouped-future action values; joint training is conditional	Causal value model selects one row or NULL per event	Controlled component evidence; three interface negatives; no learned predictive-overwrite result
FlashKDA [15]	Fast delta-attention recurrence	Recurrent matrix $[B, H, V, K]$	Numerical recurrent update, not semantic overwrite	Kernel implementation; no memory-policy objective	Fused KDA state transition	Upstream H20 kernel results only; no MMLA integration or lifecycle semantics

Table 9: Primary-source comparison. "Operation" distinguishes numerical state evolution from semantically authoritative memory mutation. The four systems are complementary rather than points on one leaderboard.

Predictive state and consolidation. Predictive state representations describe latent state through predictions of future observations rather than a reconstruction of history [60]. Predictive information formalizes the mutual information shared by past and future [8], and the information bottleneck asks for a compact code that preserves a designated relevant variable [99, 2]. Complementary learning systems separate fast episodic storage from slower integration into distributed parameters [66]. World models and latent-imagination agents learn compressed dynamics for prediction and control [37, 38]. Our construction combines these motivations in a narrower language-model operation: grouped actual futures estimate the conditional action value of one bounded overwrite, then a causal value model predicts those values for deployment. We do not claim novelty for predictive state, value estimation, or consolidation in isolation.

Retrieval and relational binding. The QA study combines BM25 [83] with dense-retrieval-style features [49], evaluates on HotpotQA [111] at LongBench-scale lengths [3], and interprets the full-context comparison in light of lost-in-the-middle behavior [61]. The relational study draws on pointer selection [101], relational modules [88, 4], ordered semi-Markov inference [89], and Sinkhorn normalization [18]. We claim neither the inherited components nor a general graph reasoner; the tested mechanism is their anchor-posterior/typed-transport factorization under same-checkpoint content, distance, and type-erasure

controls. RULER supplies complementary long-context stress tests [42]; Llama and Qwen are frozen readers [27, 77], served with vLLM [54].

12 Reproducibility

Every experiment in §4–7 records the runner commit, SHA-256 of every frozen split, generation and optimization seeds, calibration and bootstrap seeds, per-record predictions, and a completion marker. Protocols and later corrections are versioned and were committed before the data read they govern. The controlled versioning seeds are 202607091–93; the ledger-controlled HotpotQA split excludes near duplicates, and the extended contexts use seeds 202607098/99. Selector seeds are 202607102–104 and bootstrap seeds are 20260709/20260710. The fresh relational confirmation uses data seeds 2026080411–13 and model seeds 2026080421–23. Its protocol, freeze receipt, and final aggregate have SHA-256 prefixes `c759ac80...462be`, `0ca2b847...4688`, and `e74d92ce...fb9ea`. Superseded artifacts remain preserved rather than overwritten.

These hashes and paths provide an internal audit trail, not external reproducibility by themselves. A public release must map every cited digest to an accessible repository revision, data manifest, checkpoint or deterministic constructor, executable environment, and evaluator command. Until that package is published, readers outside the project cannot independently resolve the internal artifact paths in this report.

13 Discussion: Memory Beyond Context Length

Ultra-long context remains a stringent stress test, but it is not the final objective. A capable persistent system should improve its state as evidence arrives rather than keep replaying an ever-growing transcript. The distinction matters for long-running assistants, agents that revisit a changing world, and any model expected to learn during use without immediately changing all of its weights. In this view, memory quality is measured by revision, preservation, deletion, reuse, and future consequence, not by the largest accepted token count.

The architecture also suggests a testable scaling hypothesis. Let $G^*(c)$ be the future-risk reduction available to an oracle memory policy at model capability c, and let $\epsilon_Q(c) = \sup_{s,a} |\widehat{Q}_c(a \mid s) - \overline{J}_c(a \mid s)|$ be the action-value error. The greedy decision bound in Appendix J gives

$$G_{\text{value}}(c) \geq G^*(c) - 2\epsilon_Q(c). \tag{38}$$

If stronger models both extract more value from correct memories and estimate conditional action values more accurately, $G^*(c)$ may rise while $\epsilon_Q(c)$ falls. Memory could then become more useful as the model becomes more capable, rather than merely compensating for a weak backbone. This is an empirical scaling hypothesis. It is not an AGI theorem.

Persistent, revisable state may nevertheless be an important ingredient of more general intelligence. It offers a middle time scale between transient KV state and slow weight updates: experience can change behavior immediately, remain bounded, be corrected, and later supervise slower learning. That substrate is insufficient on its own. General intelligence also requires goal formation, active exploration, planning, causal world models, social reasoning, calibration, safety, and stable parameter-level continual learning. The appropriate claim is therefore conditional: predictive memory consolidation is a plausible architectural line for persistent adaptive intelligence, and its value can be falsified by the matched and scaling experiments above.

14 Limitations and Next Steps

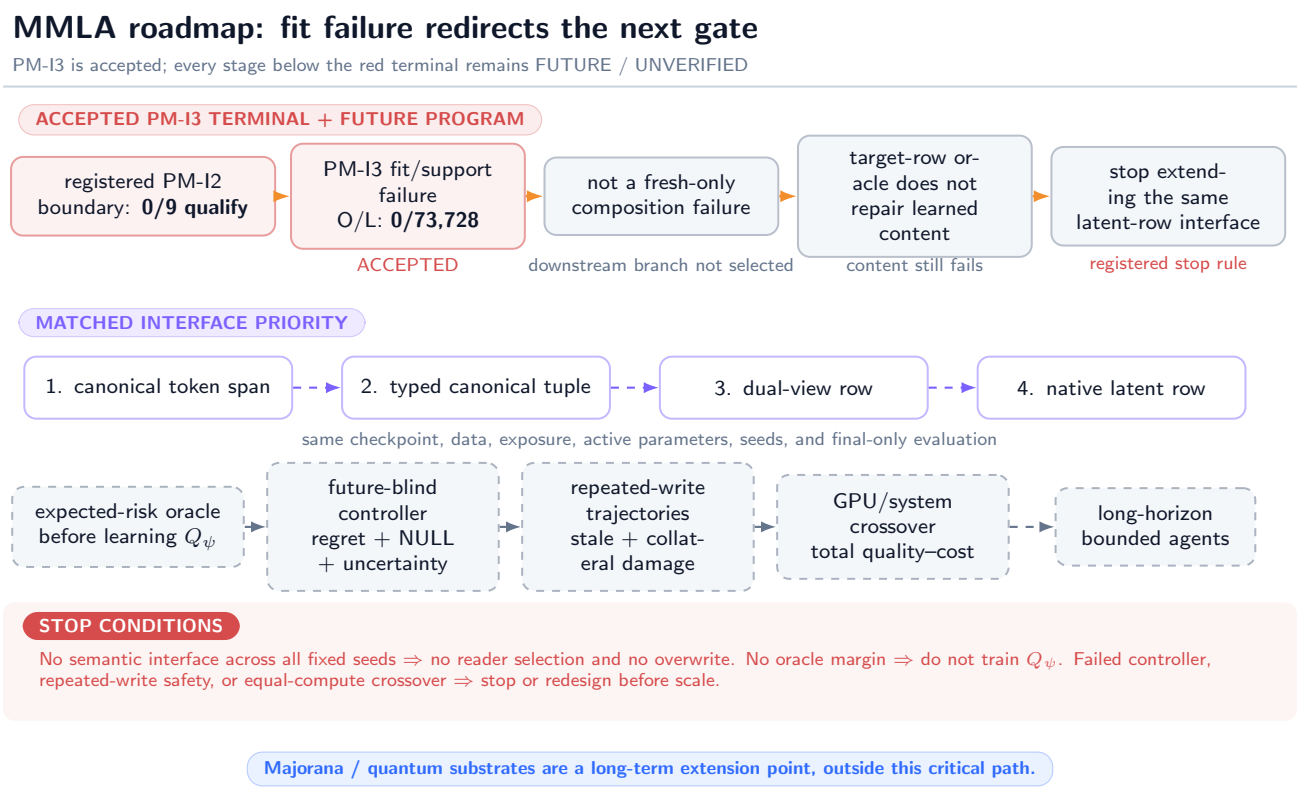

Figure 14: **Conditional roadmap, not an achieved system.** Every dashed successor is proposed or future work. Progress stops when the preceding frozen matched gate fails; Majorana and quantum backends remain outside the critical path.

The lifecycle and multi-hop QA experiments use fixed language models; only the writer, reranker, and thresholds are trained. The versioning language is controlled and shares cue words across otherwise disjoint sentence frames, so it does not establish open-language event parsing. The QA passages are static and exercise only the bounded cache and fallback, not same-key overwrite. Its natural-length budget may also require truncating a passage in very short contexts, and passage-level selection leaves sentence-level budgeting unexplored.

The relational experiment closes one binding failure in a compact synthetic compiler. Its perfect result is tied to the registered generator, seen relation primitives, short sequences, and a jointly trained shared encoder. It is not evidence that AFRT improves a full language model, that factorization is uniquely responsible, or that the model discovers arbitrary relations in natural text. The three negative interface studies in Tables 3 and 4 add a more immediate limitation: physical memory rows can be stable while their semantic content remains unusable to registered lightweight, structured, and native readers. PM-I3 narrows this boundary further. Table 5 shows that the failure is already present on fitted support and is not repaired by target-row isolation. It still does not distinguish representation geometry, decoder design, and optimization, and it does not reject the V28 memory mechanism or MMLA as a whole. The predictive architecture adds further open questions: whether conditional future utility is predictable from the past, whether a local bidirectional encoder helps beyond equal-compute causal encoding, whether repeated atomic overwrites remain stable, whether learned payload construction can be separated from action selection, and whether branch-generation cost is amortized by deployment savings.

Persistent writable memory also creates a security surface that the present experiments do not test. Relevant failures include poisoning a candidate so that it survives consolidation, exhausting slots through malicious writes or locks, cross-user leakage through shared resident state or archives, alias or routing collisions, unauthorized deletion, and rollback that resurrects stale values. A deployable system needs provenance, tenant isolation, write-rate limits, protected-capacity policy, auditable tombstones, deletion verification, and recovery semantics. These are system requirements, not consequences of the information-bottleneck objective.

The immediate next step is not a larger model and not predictive overwrite training. PM-I3 is complete and takes the preregistered fit/support-failure branch: learned content is wrong on fitted examples even when the target row is supplied, so the same pure latent-row interface is not extended. This is a stop rule for that interface family, not permission to retrain the nine jobs, change thresholds, or retrospectively prefer one reader.

The next fresh matched interface study therefore prioritizes canonical token span, typed canonical tuple, the proposed dual-view row, and pure native latent row. Checkpoint, data, exposure, active parameters, seeds, and final-only evaluation

must match. Reconstruction, fresh composition, query, calibration, preservation, stale suppression, and mapping must all pass across fixed seeds. The pure native latent arm remains a preregistered control, not a favored successor. Failure of canonical and dual-view rows even under route/content oracles triggers backbone redesign rather than another decoder. Failure of any semantic gate keeps overwrite closed.

Only then may an expected-risk oracle replay all registered slot actions and NULL from the same snapshot. No preregistered oracle margin means Q_ψ is not trained. A future-blind controller must report action regret, top-1 agreement, NULL precision/recall, calibrated uncertainty, stale leakage, collateral damage, and write rate. Repeated-write trajectories test churn, rollback, protected rows, adversarial writes, and long-horizon preservation before any system-scale claim. GPU/system comparison comes last and must include archive storage, indexing, retrieval, branch replay, kernel count, memory traffic, latency, and energy. Majorana and quantum backends are long-term extension points outside this critical path.

15 Conclusion

Memory management asks a different question from context extension: not how much history can be presented, but which parts should be allowed to change persistent state. We formalized one answer. A causal model predicts normally; a completed local segment may be reinterpreted and eventized; grouped true future outcomes estimate the conditional risk of each feasible event-level action; a causal value model chooses one bounded overwrite or NULL per event. The construction is compatible with finite information capacity. That compatibility does not guarantee learnability, efficiency, safety, or intelligence.

The completed evidence establishes useful components. Explicit lifecycle execution resolves overwrite, protection, and eviction in controlled language; calibrated archive fallback handles evidence whose relevance was unknowable at write time; typed transport repairs one controlled relational-binding failure relative to its registered ablations. It also establishes a reproducible bottleneck: three current-state interface studies fail their semantic conditions despite stable physical mapping, and PM-I3 shows that the latest failure already occurs on fitted support and survives a routing oracle. The predictive closed loop therefore remains two evidence gates away: first a new canonical or dual-view same-checkpoint interface must qualify, then a grouped-future overwrite policy must improve future behavior without increasing interference, compute, or hidden leakage. A positive answer to both would justify joint pretraining and scale studies. The present negative answer retires further extension of the same latent-row reader and redirects the next study toward a more explicit semantic contract; it does not by itself reject bounded predictive memory.

Part I
Reasoning-Time Training Foundations

Branch status. This part imports the complete substantive formal branch from the frozen R01 focus paper. It defines a typed within-problem policy-update regime and its qualification obligations; it contributes no new experiment and does not establish strict-RTT utility, a qualified semantic-memory interface, or a learned admission policy.

16 Introduction

Suppose a system receives one difficult problem, produces an unsuccessful attempt, observes a compiler error or verifier score, and later succeeds. The success alone does not tell us what changed. The system may have sampled a luckier trajectory from the same policy, expanded a larger search tree, appended the error to its prompt, stored a reflection in external memory, or changed a state that parameterizes the policy. All five mechanisms can be useful. Only the last is a candidate for the phenomenon studied here, and even then the update must occur and be reused before that same problem is resolved.

We use *Reasoning-Time Training* (RTT) for this within-problem temporal regime. The name is descriptive rather than a claim of lexical priority. "Training" refers to a change in a declared policy-generating state, not necessarily to backpropagation. "Reasoning time" is the interval between admission of one problem instance and its resolution or abandonment. The boundary is therefore operational: state type, event order, read trace, reset scope, and intervention response must be inspectable.

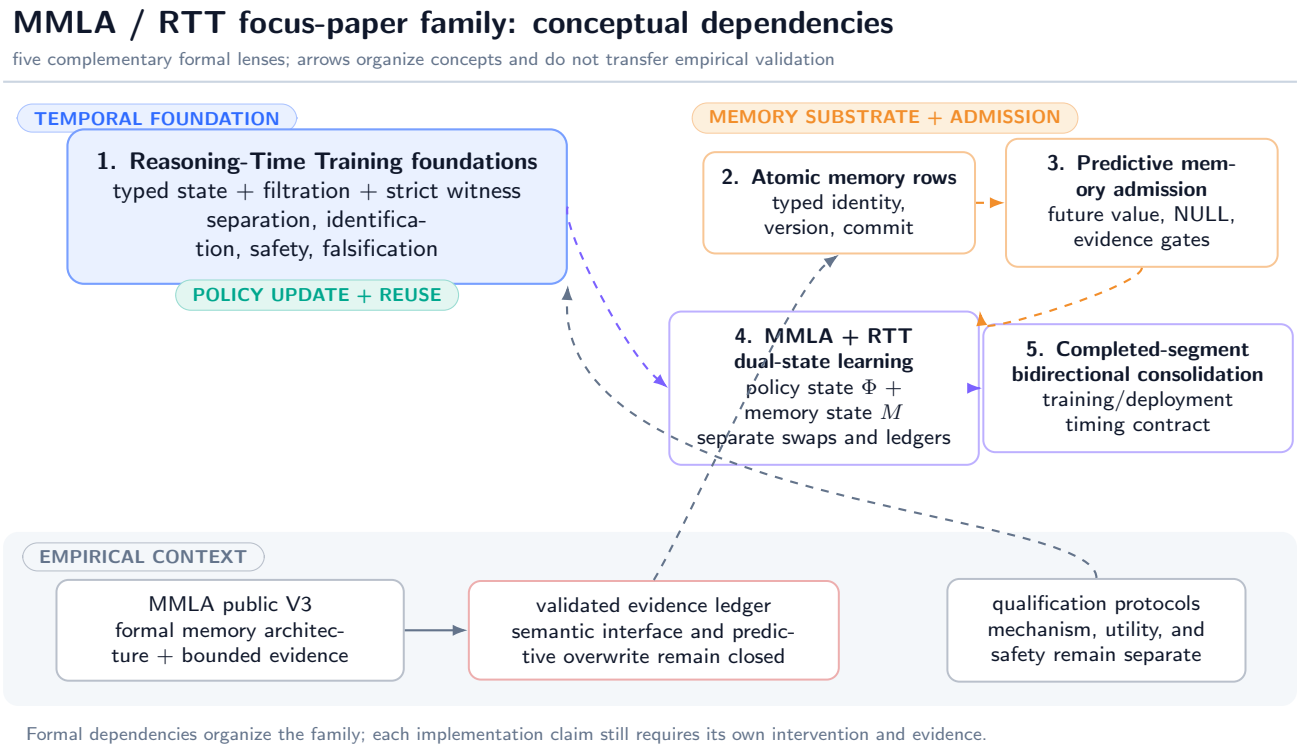

Figure 15: **Five-paper conceptual map.** This paper supplies the temporal and causal foundations used by the four companion focus papers. Arrows encode mathematical dependencies, not empirical validation.

The relationship to MMLA is deliberately narrow. MMLA formalizes a bounded authoritative memory between context and slow weight updates [124]. The public evidence record does not qualify same-checkpoint semantic memory or learned predictive overwrite. We do not import those component results as evidence for RTT. Instead, we separate policy mutation from external-memory mutation so that a combined system can ablate their causal roles independently.

16.1 Contributions

The contributions are:

1. a typed stochastic process for a single unresolved problem, with attempt, feedback, policy, context, memory, workspace, verifier, lifetime, and budget objects;

2. a strict witness definition that requires an in-lifetime policy-state update and an actually post-update-policy-conditioned later attempt;
3. separation and non-identifiability results clarifying what transcripts alone cannot establish;
4. causal qualification designs and estimands that isolate update use, feedback semantics, reset leakage, verifier reliability, and local harm;
5. an update-operator taxonomy, outer objective, budget ledger, rollback contract, implementation obligations, and a staged falsification ladder.

Formal statements carry their assumptions locally. Status tags distinguish definitions, derived results, design prescriptions, and future tests; none converts a theorem into implementation evidence. No experiment in this document demonstrates RTT, no theorem guarantees that an optimizer finds a useful update, and no result establishes superiority over search, context refinement, memory, or offline training. Policy mutation may be unnecessary, unstable, or inefficient. We neither relabel arbitrary cache mutation as training nor infer strict RTT from a method name.

16.2 Episode boundary and paper map

The unit is a *problem episode*: one admitted instance, its ordered attempts, its within-episode feedback, and a terminal resolution or abandonment decision. A benchmark dataset is a collection of such episodes, not one giant problem. An update learned from earlier dataset items and used on later items is test-time or continual adaptation, but is not strict RTT for a later item unless that later item itself contains an update–reuse witness before its own endpoint. Conversely, a harmful within-problem update can satisfy the mechanism definition even though it fails every utility gate. Mechanism qualification and benefit qualification are separate questions throughout.

Section 17 specifies the typed process; Section 18 gives the strict witness; Sections 19–20 establish separations and identification limits; and the remaining sections develop update objectives, safety contracts, implementation obligations, related work, and falsification tests. Full proofs, symbols, and audit-oriented derivations appear in the appendices.

17 A Typed Within-Problem Stochastic Process

17.1 Probability space and typed interfaces

Fix a probability space $(\Omega, \mathcal{A}, \mathbb{P})$. A problem episode draws $X : \Omega \to \mathcal{X}$ and is assigned an immutable episode identifier $I \in \mathcal{I}$. The identifier distinguishes a retry of the same problem from a later, merely similar item. Within the episode, attempts are indexed by $j \in \mathbb{N}$ and token/tool microsteps by $t \in \mathbb{N}$. The basic spaces are collected in Table 10; they are measurable spaces even when implemented as structured byte records.

Object	Space	Intended content	Required boundary
Problem X	$\mathcal{X}$	Immutable task statement, tools, and success contract.	Same episode identifier across witness attempts.
Observation $O_{j,t}$	$\mathcal{O}$	Tokens, tool returns, environment state visible by time (j,t).	Timestamped; no future returns.
Action $A_{j,t}$	$\mathcal{A}$	Token, tool call, branch, halt, or answer action.	Sampled from the declared policy kernel.
Attempt Z_j	$\mathcal{Z}$	Finite marked trajectory of observations and actions.	Start/end and policy-state receipt recorded.
Feedback Y_j	$\mathcal{Y}$	Verifier output, execution result, critique, or reward record.	Provenance and availability time recorded.
Policy state Φ_j	$\mathcal{P}$	Mutable state directly parameterizing the action kernel.	Versioned, bounded, intervenable, and read-traced.
Context C_j	$\mathcal{C}$	Prompt and transient in-context transcript.	Not silently counted as policy state.
Memory M_j	$\mathcal{M}$	External or resident episodic/semantic records.	Typed separately from policy state.
Workspace W_j	$\mathcal{W}$	Files, search tree, scratchpad, caches, tool state.	Charged and reset-scoped explicitly.
Auxiliary state Ξ_j	$\mathfrak{X}^{\mathrm{read}} \times \mathfrak{X}^{\mathrm{aux}}$	Policy-read fields; RNG, optimizer, router, scheduler, callback, and epoch records.	Typed, bounded, versioned; every read path declared.
Budget B_j	$\mathcal{B} \subseteq \mathbb{R}_+^d$	Remaining tokens, calls, time, energy proxy, update steps.	Monotone ledger with hard stop.

Table 10: **Typed state decomposition (DEFINITION).** Physical storage does not determine the type. The causal interface and declared read path do.

Every admitted event e receives the immutable key

$$\kappa(e) = \big(g(e), p(e), \mathrm{id}(e)\big), \tag{39}$$

where g is an atomically assigned monotone ingress sequence, p is a frozen event-class precedence, and id is a stable receipt identifier. Events are replayed in lexicographic key order; wall-clock timestamps are descriptive and never break a causal tie. Let S_n be the full state immediately before event n in that order. If σ_j is the start event of attempt j, then the pre-attempt state is

$$S_j := \mathsf{S}_{\sigma_j} = (X, I, C_j, M_j, W_j, \Phi_j, \Xi_j, B_j), \tag{40}$$

where $\Xi_j = (\Xi_j^{\mathrm{read}}, \Xi_j^{\mathrm{aux}})$. The first component contains every persistent non-Φ field directly read on an action-kernel path. The second contains non-policy auxiliary variables, including random-generator state, optimizer moments, routing and batching metadata, the reset epoch, and pending-callback receipts. Each component is assigned a measurable, implementation-bounded space with declared caps on serialized bytes, queue length, optimizer slots, and callback count. Excluding Ξ from an audit does not make it causally irrelevant.

Define the *policy-read closure* at an action call as the versioned manifest of every persistent field reachable by that call. A field in this closure is either serialized into Φ, placed in Ξ^{read}, or the policy-use claim stops. Write $\Psi := (\Phi, \Xi^{\mathrm{read}})$ for the effective policy state. A contrast may be called a Φ effect only when Ξ^{read} is byte-identical across arms; otherwise it is a bundled Ψ effect. Fields in Ξ^{aux} remain causally relevant and must be restored, randomized, or included in the estimand.

Definition 17.1 (Policy kernel and functional policy state)**.** DEFINITION For each microstep, the system exposes a stochastic kernel

$$\pi_\phi(da \mid x, h, c, m, w, \xi^{\mathrm{read}}, b), \tag{41}$$

where h is the causally available within-attempt history. A declared component Φ is a *functional policy state* on a history set $\mathcal{H}_0$ if there exist $h \in \mathcal{H}_0$, admissible states ϕ, ϕ', and fixed $(x, c, m, w, \xi^{\mathrm{read}}, b)$ such that

$$\mathrm{TV}\Big(\pi_\phi(\cdot \mid x, h, c, m, w, \xi^{\mathrm{read}}, b), \pi_{\phi'}(\cdot \mid x, h, c, m, w, \xi^{\mathrm{read}}, b)\Big) > 0. \tag{42}$$

The declaration must identify the serialization, version, update rule, and action-kernel call site. A semantic relabeling of context or memory without such an interface is insufficient.

This definition is intentionally substrate-neutral. Full weights, a low-rank adapter, a learned-optimizer state, a recurrent fast-weight state, or a hypernetwork code can all be functional policy state. Conversely, a vector called an "adapter" that is never read is not functional on the observed episode. External memory can influence actions after retrieval, but its mutation remains a memory operation unless it is also the declared direct policy-generating state under an auditable interface.

17.2 Filtration, attempt times, and resolution

Let $\{\mathcal{F}_n\}_{n\geq 0}$ be the filtration generated by the deterministically ordered event log. It contains the problem and every admitted observation, action, random-seed draw, verifier return, state version, reset/drop receipt, and external side effect available before event $n{+}1$. For attempt j, let σ_j and ϵ_j be start and end stopping times with $\sigma_j \leq \epsilon_j < \sigma_{j+1}$ on episodes that retry. Let u_j be the committed-update event triggered after attempt j. The resolution time τ is a stopping time taking values in $\mathbb{N} \cup \{\infty\}$; resolution includes verified success, explicit abandonment, budget exhaustion, or a safety stop. A remote or asynchronous return becomes causally available only when its receipt is admitted as an event. Returns carrying a stale reset epoch are rejected and the rejection itself is logged; they cannot re-enter the post-reset state.

Within an attempt, $n(j,t)$ is the unique event-log embedding of microstep (j,t) and $\mathcal{F}_{j,t} := \mathcal{F}_{n(j,t)}$ denotes the information set before action $A_{j,t}$. Causality requires

$$A_{j,t} \text{ is sampled from an } \mathcal{F}_{j,t}\text{-measurable kernel}, \qquad O_{j,t+1} \in \mathcal{F}_{j,t+1}. \tag{43}$$

Feedback Y_j becomes available at a stopping time $v_j \geq \epsilon_j$ and is $\mathcal{F}_{v_j}$-measurable. An admissible update has the form

$$(\Phi_{j+1}, \Xi_{j+1}, \rho_j) = U_j(\Phi_j, \Xi_j, X, Z_{\leq j}, Y_{\leq j}, B_j; R_j), \tag{44}$$

where R_j is fresh update randomness revealed no later than u_j, and ρ_j is a receipt containing input/output hashes, times, costs, and status. The entire right-hand side must be $\mathcal{F}_{u_j}$-measurable. In particular, no observation after u_j may select or tune the update used before that observation.

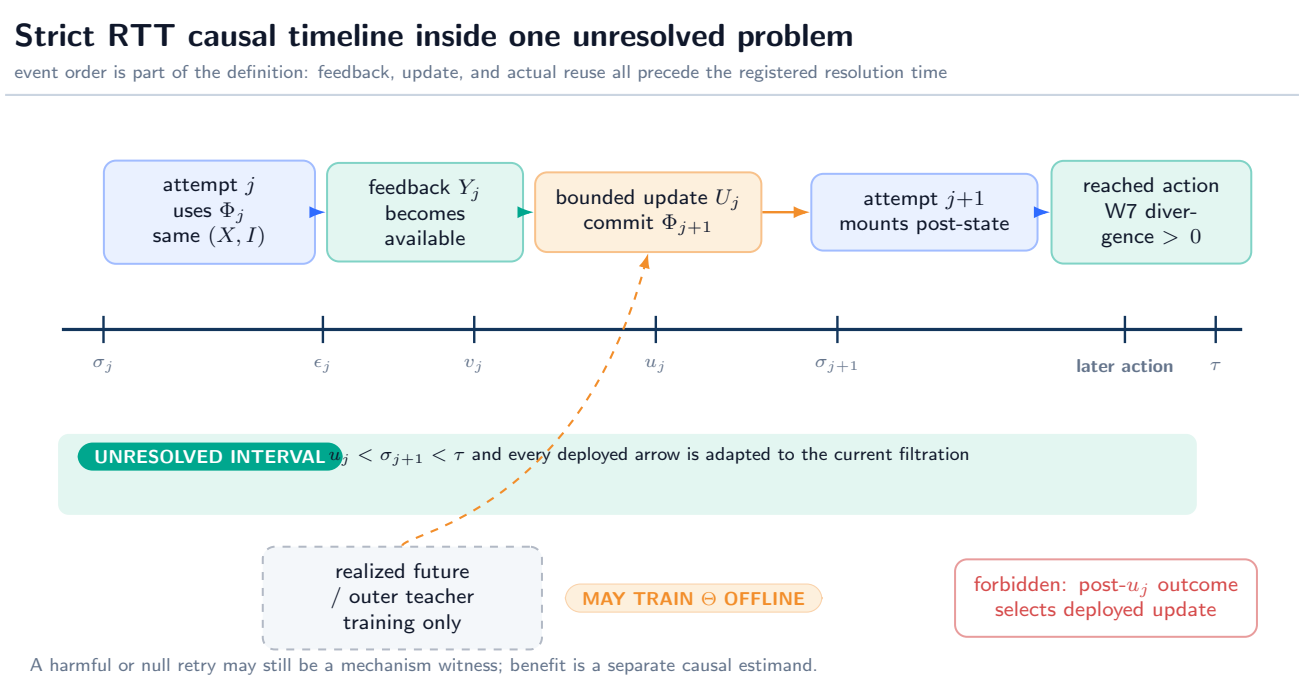

Figure 16: **Strict within-problem causal timing (DEFINITION).** Solid arrows are deployed, causally available paths. Attempt j ends, feedback becomes available, and update U_j commits while the same episode is unresolved; only then may attempt $j{+}1$ invoke Φ_{j+1}. The dashed amber future branch is allowed only for an outer training teacher and may not enter the deployed update.

Definition 17.2 (Unresolved interval and lifetime)**.** DEFINITION The episode is unresolved at event n on $\{n < \tau\}$. A policy-state version ϕ has a declared lifetime interval $L(\phi) = [\ell^-(\phi), \ell^+(\phi))$ in event time. A within-problem version must satisfy

$$\sigma_1 \le \ell^-(\phi) < \ell^+(\phi) \le \tau + 1, \tag{45}$$

unless an explicitly separate cross-problem continual-learning protocol owns the carryover. Retaining a version after τ silently changes the regime.

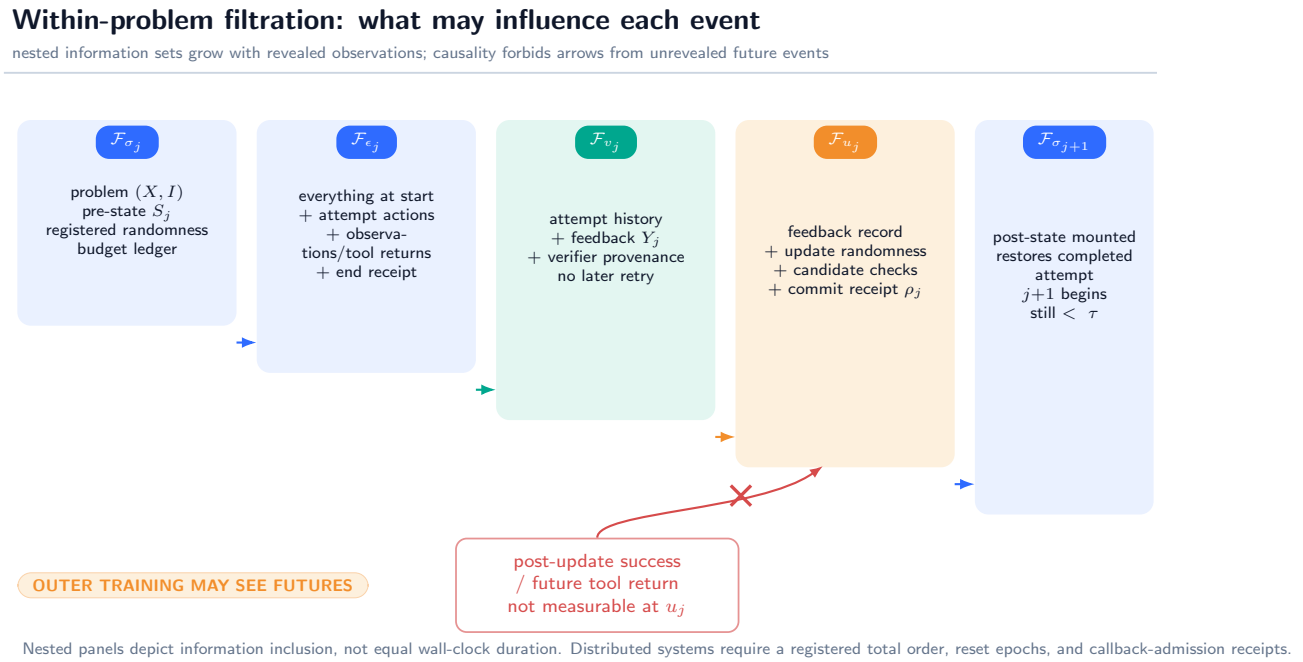

Figure 17: **Information sets and admissible arrows (DEFINITION).** Each blue filtration panel contains only events already revealed. Policy actions, feedback processing, and the update receipt are adapted to the current filtration. The crossed coral arrow is future leakage: later success or a post-update tool result cannot be used to choose an earlier deployed update.

17.3 Standing assumptions for formal results

Results invoke only the assumptions they cite; no assumption is globally true by typography.

Assumption 17.3 (Typed observability)**.** PROVED UNDER STATED ASSUMPTIONS The declared state components in Equation (40), their bounds, versions, policy-read closure, and read/write events are logged without omission for the analyzed episode.

Assumption 17.4 (Causal availability)**.** PROVED UNDER STATED ASSUMPTIONS Actions and updates are adapted to $\{\mathcal{F}_n\}$; event keys obey Equation (39); stale-epoch returns cannot mutate current state; and τ is a stopping time for that filtration.

Assumption 17.5 (Stable episode identity)**.** PROVED UNDER STATED ASSUMPTIONS Witness attempts share the same immutable pair (X, I); a generated paraphrase or neighboring dataset item is not silently substituted.

Assumption 17.6 (Well-defined interventions)**.** PROVED UNDER STATED ASSUMPTIONS When a result compares state versions or feedback arms, all non-target state fields and environment kernels are either held fixed, randomized as specified, or included in the estimand.

Assumption 17.7 (Positivity and consistency)**.** PROVED UNDER STATED ASSUMPTIONS Every randomized experimental arm has positive assignment probability, and the observed outcome equals the potential outcome under the assigned arm without cross-episode interference.

Assumption 17.8 (Bounded evaluation)**.** PROVED UNDER STATED ASSUMPTIONS Utility and harm variables used in stability bounds lie in $[0, 1]$, or are rescaled with an explicitly reported range.

18 Strict Reasoning-Time Training

18.1 A witness is a trace plus a functional state test

The definition below is intentionally stronger than "an optimizer ran." A no-op optimizer, an update committed after resolution, or an updated adapter that is never invoked does not establish within-problem learning.

Definition 18.1 (Strict RTT witness)**.** DEFINITION A strict RTT witness for attempt index j is a record

$$\mathfrak{W}_j = (I, j, \epsilon_j, v_j, u_j, \sigma_{j+1}, \tau, h_j^-, h_j^+, \chi_j^-, \chi_j^+, \rho_j, \mathcal{R}_{j+1}, d_j) \tag{46}$$

that satisfies all of the following:

- **W1. Same problem.** Attempts j and $j+1$ carry the same immutable (X, I).
- **W2. Available feedback.** Attempt j ends at ϵ_j, feedback Y_j is available by v_j, and $Y_j \in \mathcal{F}_{v_j}$.
- **W3. Nontrivial policy-state transition.** Update U_j commits at $u_j \geq v_j$, producing a version change $h_j^- := H(\Phi_j) \neq H(\Phi_{j+1}) =: h_j^+$ under a collision-resistant serialization hash and a valid receipt ρ_j. The receipt also records the policy-read-closure manifests $\chi_j^- := H(\Xi_j^{\text{read}})$ and $\chi_j^+ := H(\Xi_{j+1}^{\text{read}})$.
- **W4. Causal update.** $(\Phi_{j+1}, \Xi_{j+1}, \rho_j)$ is $\mathcal{F}_{u_j}$-measurable; its commit carries the admitted event key and current reset epoch, and no event after u_j selects the committed update.
- **W5. Problem still unresolved.** $u_j < \sigma_{j+1} < \tau$.
- **W6. Actual post-update read.** The later attempt contains a signed read trace $\mathcal{R}_{j+1}$ showing that the action-kernel call loaded version h_j^+ and the registered closure χ_j^+, not merely that either version existed.
- **W7. Functional reuse on a reached history.** For at least one reached pre-action history $H_{j+1,t}$, hold every registered non-policy input fixed at its recorded value. Then

$$d_j := \mathrm{TV}\Big(\pi_{\Phi_{j+1}}(\cdot \mid H_{j+1,t}, \Xi^{\text{read}}), \pi_{\Phi_j}(\cdot \mid H_{j+1,t}, \Xi^{\text{read}})\Big) > 0. \tag{47}$$

  This is a Φ-only contrast only when $\chi_j^- = \chi_j^+$ and all other fields are fixed. If a policy-read auxiliary field changes, the target is the bundled effective state $\Psi = (\Phi, \Xi^{\text{read}})$ and the witness is labeled accordingly. The distance may be computed exactly, lower-bounded with a registered probe, or established by a randomized state-swap test whose uncertainty is reported.
- **W8. Charged lifetime.** Update, retained state, retry, and verification costs are charged to B_j, and the state version expires or transfers under a declared rule at τ. Expiry, rollback, and late-callback rejection are scoped by the recorded reset epoch.

No positive reward, correctness, or improvement condition is part of the mechanism definition.

The hash inequality in W3 is necessary but not sufficient: different bytes can implement the same kernel. W6 catches dead updates; W7 catches semantically inert updates. The closure hashes prevent an unlogged router, gate, recurrent carrier, or normalization field from being attributed to Φ. Conversely, W7 does not show that the change helped. It only shows that a later attempt was genuinely conditioned on a declared effective policy state produced from earlier within-problem feedback.

Definition 18.2 (Syntactic, functional, and beneficial qualification)**.** DEFINITION A trace satisfying W1–W6 and W8 is a *syntactic update–reuse trace*. It becomes a *strict functional RTT witness* only with W7. A *beneficial causal RTT effect* additionally requires a prespecified outcome estimand and an intervention or randomized design supporting a positive effect under its assumptions. These three levels must not be conflated.

Item	Required evidence	Typical falsifier	Status if absent
W1	Immutable problem/episode receipt	New item or regenerated task ID	Different regime
W2	Timestamped feedback provenance	Feedback created after update	Causal failure
W3	Φ hashes, closure hashes, and commit	No-op, failed commit, or undeclared read carrier	Not strict RTT or bundled Ψ
W4	Event key, epoch, filtration/future-read audit	Later success or stale callback tunes update	Leakage
W5	$u_j < \sigma_{j+1} < \tau$	Update after answer acceptance	Too late
W6	Kernel call reads post-state and closure	Adapter never mounted or route differs	Dead or bundled update
W7	Exact/probed swap with all non-target fields fixed	Zero divergence or closure mismatch	Functionally inert or misattributed
W8	Cost/lifetime/epoch ledger	Uncharged state or late return re-enters	Protocol invalid

Table 11: **Strict witness audit (DEFINITION).** Each row has an independently inspectable artifact and an explicit failure interpretation.

18.2 Fresh-proposal criterion

Search workspaces can strongly influence which node is expanded next. To avoid declaring every search statistic a learned policy, the state partition uses a fresh-proposal intervention.

Definition 18.3 (Fresh-proposal policy-state criterion)**.** DEFINITION Let r restore $(X, C, M, W, \Xi^{\text{read}}, \Xi^{\text{aux}}, B)$ to a registered pre-attempt snapshot while independently choosing policy version ϕ. A mutable state component qualifies as policy state for the episode only if, under some admissible restored snapshot and common randomization scheme, swapping that component changes the distribution of a *fresh proposal* before any updated search node, appended reflection, or new memory record is read. A Φ-only attribution additionally requires byte-identical Ξ^{read}; otherwise the intervened component is Ψ. Candidate-specific tree values, visited-node sets, and cached tool outputs are workspace state by default.

This criterion does not assert that workspace search is inferior. It enforces a stable meaning: RTT changes how new reasoning actions are proposed; search changes which already represented possibilities are explored or selected. A hybrid can do both and should report both.

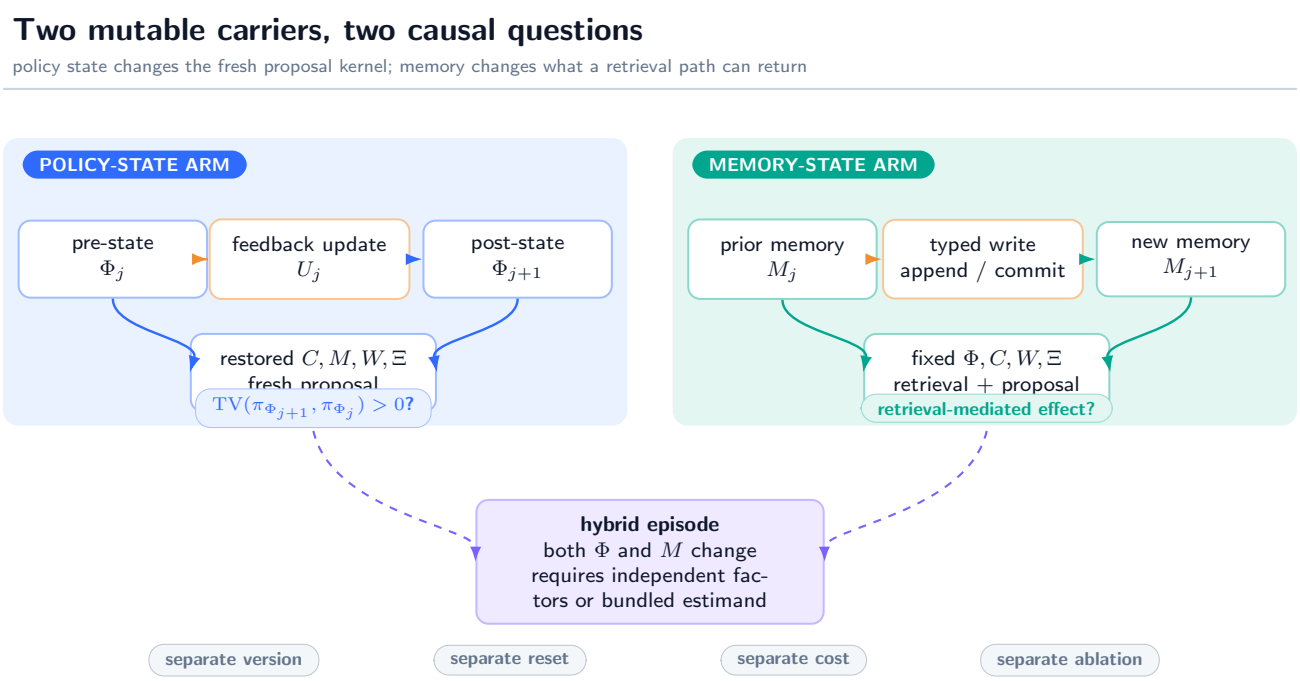

Figure 18: **Mutable policy state versus external memory and workspace (DEFINITION).** The navy state-swap holds context, memory, workspace, the policy-read closure, auxiliary state, and randomness fixed and asks whether a fresh proposal distribution changes. Teal memory swaps test retrieval-mediated effects separately. If the closure cannot be held fixed, the navy contrast is a bundled Ψ effect. Each state type needs its own version, reset, cost, and ablation receipt.

18.3 A worked strict witness

Consider a coding instance with a deterministic compiler verifier. Attempt 1 produces program z_1 and error y_1. A rank-r adapter $\Phi_1 = (A_1, B_1)$ is updated to Φ_2 by K bounded optimizer steps using only (X, z_1, y_1), and the commit receipt is written before retry. The prompt, external memory, compiler cache, workspace, RNG streams, optimizer/router state, pending-callback set, reset epoch, and the entire policy-read closure are then restored to their registered pre-attempt values. Attempt 2 mounts Φ_2. A common-random-number logit probe at a reached prefix reports positive total variation between the distributions under Φ_2 and restored Φ_1. This is a strict functional Φ witness because the closure is byte-identical, regardless of whether program 2 passes.

The example also shows why "the second answer differed" is inadequate. Different sampling randomness can change an answer with no update, and hidden compiler caches can change it despite a reset. The receipt must cover the state version, actual call path, controlled fields, and measurement uncertainty.

18.4 Hybrid systems and dual accounting

A system may update Φ, change a policy-read field in Ξ^{read}, append a reflection to C, write M, and expand W after the same attempt. Such a trace is not rejected, but the total causal effect is then a mixture. The minimal factorial decomposition uses independent restore/swap factors for Φ, Ξ^{read}, and non-policy information carriers. If cost permits, context, memory, workspace, auxiliary state, verifier feedback, and policy version receive separate randomized factors. If cost does not permit, the reported estimand must be the bundled Ψ or wider system effect; it may not be called a Φ-only policy-update effect.

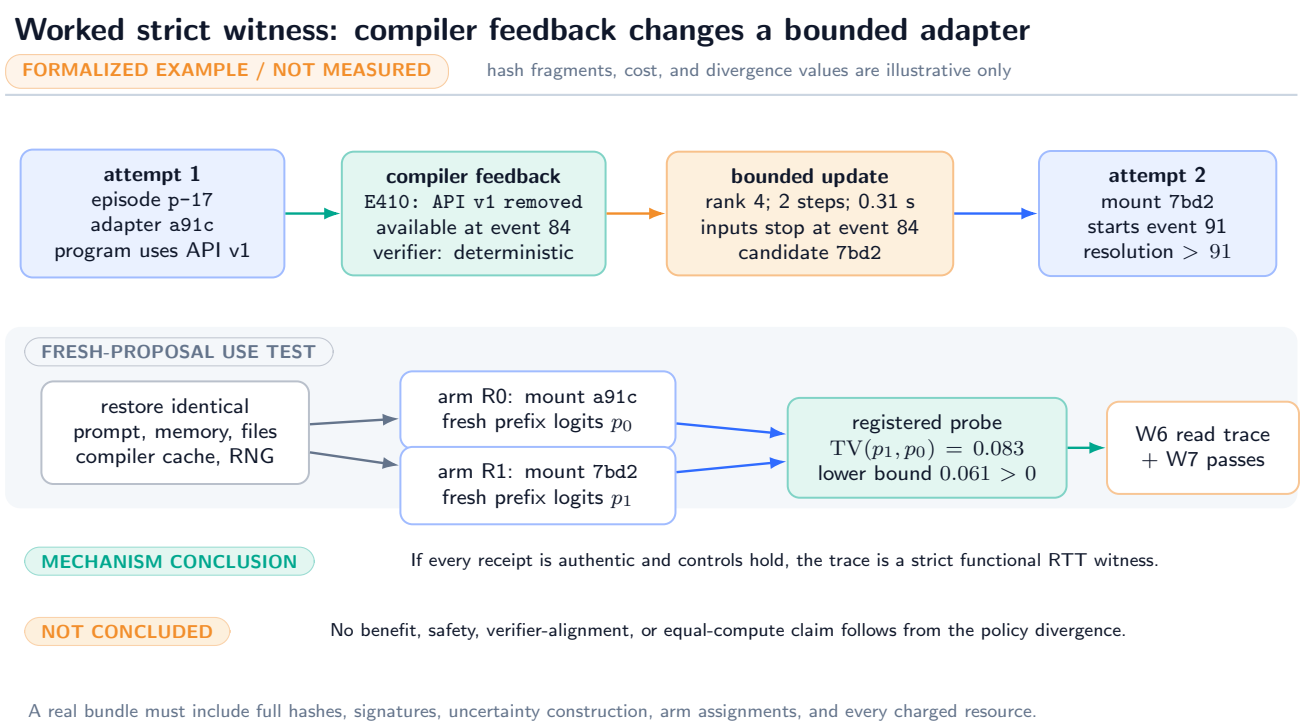

Figure 19: **Worked multi-attempt trace (formalized example, not measured).** Attempt 1 yields compiler feedback; a bounded adapter update commits while the episode is unresolved; attempt 2 actually mounts the new version. The lower restored-state branch is the required policy-use probe. All numerical hashes, costs, and distances are illustrative and carry no empirical claim.

19 Separation from Search, Context, and Memory

19.1 Mechanism taxonomy

The adjacent regimes differ along three axes: what state changes, when it changes, and which future unit uses it. Table 12 and Figure 20 use the strict witness rather than community labels.

Regime	Primary mutable carrier	Reuse horizon	Strict RTT condition
Fixed-policy sampling/search	W (samples, nodes, scores)	Same problem	No, unless Φ also changes and passes fresh-proposal reuse.
Prompt/context refinement	C	Same problem/session	No when Φ is fixed.
External memory / RTMA	M	Later attempts or problems	No when only M changes.
Classical TTT/adaptation	Φ	Current sample, batch, or stream	Only episodes with update and reuse before that same problem ends.
Continual/online learning	Φ	Later problems	Not strict RTT for the earlier completed problem; may contain nested RTT episodes.
Strict RTT	Φ	Later attempt on same unresolved (X, I)	Yes, if all W1–W8 hold.

Table 12: **Operational regime taxonomy (DEFINITION).** Names used by individual papers do not override the trace-level test.

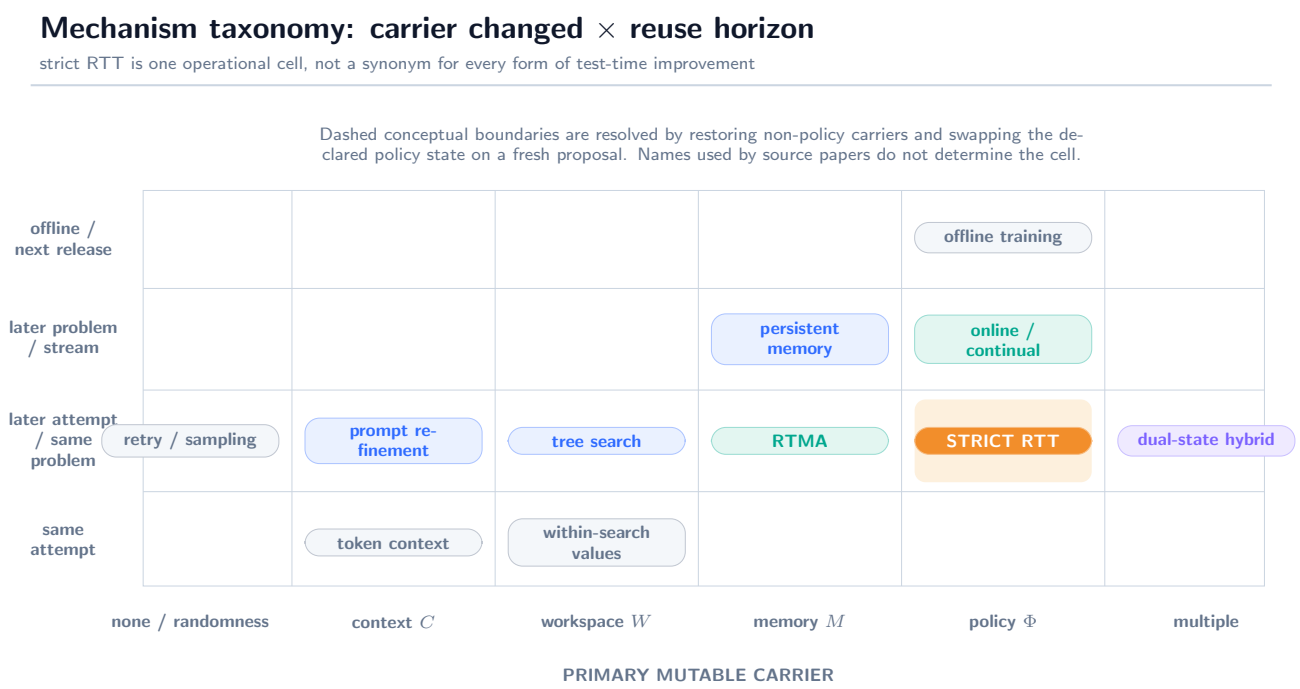

Figure 20: **RTT versus neighboring mechanisms (DEFINITION).** Horizontal position records the carrier that changes; vertical position records reuse time. Strict RTT occupies the policy-state/same-unresolved-problem cell. Dashed borders indicate hybrids whose classification requires state-specific swaps; no cell is presented as universally superior.

19.2 No-read and fixed-policy results

Lemma 19.1 (Unused policy update cannot change the later law)**.** ***PROVED UNDER STATED ASSUMPTIONS*** *Fix an episode and suppose that, conditional on* $(X, C, M, W, \Xi^{\text{read}}, \Xi^{\text{aux}}, B)$*, every transition kernel from* u_j *through attempt* $j{+}1$ *is independent of* Φ_{j+1}*. Then replacing* Φ_{j+1} *by any other admissible state while holding the policy-readable closure byte-identical leaves the conditional law of the later attempt and its outcome unchanged.*

Proof. Condition on the recorded non-policy state at u_j. The first post-update transition has the same kernel under every value of Φ_{j+1} by assumption. If the joint law through event n is the same, integrating the common event-$(n{+}1)$ kernel against that common law gives the same joint law through $n{+}1$. Induction continues through the finite attempt and its outcome. Thus the conditional laws coincide. Full details and the kernel factorization are restated in Appendix 27. □

The lemma distinguishes a state commit from actual reuse. A framework can spend substantial update compute and still have exactly zero causal path to later actions because the adapter was not mounted, the route bypassed it, or the state affected only a dead branch.

Proposition 19.2 (Fixed-policy search separation)**.** ***PROVED UNDER STATED ASSUMPTIONS*** *Consider any procedure that, during one episode, samples, scores, branches, backtracks, or votes using an action kernel with constant effective policy carrier* $\Psi_j = (\Phi_j, \Xi_j^{\text{read}}) \equiv \psi_0$*. If its only mutations are to* $(C, M, W, \Xi^{\text{aux}}, B)$*, the procedure has no strict RTT witness, regardless of its compute budget or success rate. Constancy of* Φ *alone is insufficient when the kernel-readable closure changes.*

Proof. W3 in Definition 18.1 requires a nontrivial transition of the declared functional carrier, and W7 requires a later fresh-proposal law that differs under the post-update version. Byte-identical Ψ violates W3 directly and leaves no carrier contrast for W7. Changes to the other typed components do not repair that violation. The conclusion is classificatory, not a claim that search is ineffective. □

Proposition 19.3 (Context- and memory-only separation)**.** ***PROVED UNDER STATED ASSUMPTIONS*** *If* $C_{j+1} \neq C_j$ *or* $M_{j+1} \neq M_j$ *while* $\Psi_{j+1} = \Psi_j$*, later behavior may change, but the trace is not strict RTT under Definition 18.1. Equality of* Φ *alone does not establish this premise when* Ξ^{read} *changes.*

Proof. The policy-state transition condition W3 fails. Because Definition 18.3 holds C and M fixed when testing policy state, a retrieval- or prompt-mediated behavior change cannot substitute for W7. Such a trace should be reported as context refinement or memory adaptation, possibly RTMA, rather than erased from the analysis. □

Corollary 19.4 (More inference compute is not sufficient)**.** ***PROVED UNDER STATED ASSUMPTIONS*** *Increasing the number of samples, tree nodes, tokens, tool calls, or retries is not sufficient for strict RTT.*

Proof. Each increase can be implemented with constant Φ and mutations confined to W, C, or B. Proposition 19.2 applies. □

19.3 Counterexamples to common classifications

Counterexample 19.5 (Lucky retry)**.** DEFINITION A fixed stochastic policy fails once and succeeds on a second independent sample. The outcome changes, but there is no policy-state update. This is repeated inference, not strict RTT.

Counterexample 19.6 (Reflection in the prompt)**.** DEFINITION A critique is appended to C_{j+1} and a frozen model retries. The later attempt is feedback-conditioned through context, not post-update-policy-conditioned. This is prompt/context refinement.

Counterexample 19.7 (Episodic memory write)**.** DEFINITION The system writes "avoid API v1" to M and retrieves it on retry while weights and fast policy state remain unchanged. The mechanism is memory adaptation, not strict RTT.

Counterexample 19.8 (Update after termination)**.** DEFINITION The correct answer is accepted at τ, after which the model trains on the completed trace for future problems. That may be online or continual learning, but $u_j \geq \tau$ violates W5 for the completed problem.

Counterexample 19.9 (Mounted but inert adapter)**.** DEFINITION A new adapter hash is mounted, but a zero gate makes every reached action distribution identical to the pre-update distribution. W6 holds while W7 fails; this is only a syntactic update–reuse trace.

Counterexample 19.10 (Hidden cross-arm cache)**.** DEFINITION An apparent update benefit disappears when a compiler cache and random-number stream are restored. The original comparison bundled Φ with W, Ξ^{read}, and Ξ^{aux}; it identified neither a Φ-only effect nor, without a declared readable closure, a well-typed Ψ effect.

Figure 21: **False-classification gallery (formal counterexamples).** Each panel can produce a different or better later answer without satisfying all strict-witness conditions. Coral marks the single missing or confounded link; it does not mark the underlying method as scientifically failed.

19.4 Ambiguous recurrent and hypernetwork state

Some architectures blur conventional storage labels. A recurrent fast state that persists across attempt resets and directly parameterizes the fresh proposal kernel may be Φ even if it is not called a weight. A hypernetwork code that regenerates adapter weights is also policy state if the code swap changes the fresh proposal law. By contrast, a recurrent token cache rebuilt solely from the appended transcript is context state. The decisive audit is not the tensor's name or device; it is the typed intervention with non-target carriers restored.

20 Identification, Intervention, and Impossibility

20.1 Transcripts do not identify the hidden mechanism

Theorem 20.1 (Finite-history emulation). **PROVED UNDER STATED ASSUMPTIONS** *Let an adaptive controller induce a joint distribution over a finite set of observable within-episode histories and actions, possibly through an unobserved mutable policy state. There exists a single fixed history-dependent policy $q(a \mid h)$, with no mutable policy state, that induces the same joint distribution over those observable histories and actions under the same environment kernels.*

Proof. For every observable history h with positive probability under the adaptive controller, define

$$q(a \mid h) := \mathbb{P}_{\mathrm{ad}}(A = a \mid H = h), \tag{48}$$

where the right side marginalizes any hidden policy state and update randomness. Define q arbitrarily on zero-probability histories. At the first decision, q matches the adaptive conditional action law. If the joint observable law matches through a history h, the action conditional matches by construction and the next-observation conditional matches because the environment kernel is shared. Induction over the finite event horizon gives equality of the full observable joint law. The map q is fixed before the emulated episode; it conditions on history but does not update policy state. □

Corollary 20.2 (Observational non-identifiability of strict RTT). **PROVED UNDER STATED ASSUMPTIONS** *A finite transcript distribution, even one showing systematic improvement after feedback, cannot by itself identify a strict RTT mechanism.*

Proof. Theorem 20.1 provides an observationally equivalent fixed-policy mechanism. Distinguishing the mechanisms therefore requires additional observables (state/read receipts) or interventions (state restoration/swaps). □

The theorem is an existence result, not a claim that the emulator is compact or learnable. Its purpose is epistemic: behavioral logs cannot certify the hidden update type merely because a fixed lookup policy might be impractically large.

20.2 A randomized state–feedback factorial design

Let $R \in \{0, 1\}$ denote the policy-state deployment arm and $F \in \{0, 1\}$ the feedback-semantic arm. In all arms, attempt 1 and update compute are executed. When $F = 1$, the update receives the true registered feedback; when $F = 0$, it receives a prespecified within-block permutation or matched sham feedback. When $R = 1$, the resulting post-update state is mounted for the fresh retry; when $R = 0$, the registered pre-update state is restored. Context, memory, workspace, tool state, budgets, Ξ^{aux}, and randomization are restored or randomized identically. The policy-read closure Ξ^{read} is byte-identical for a Φ-only estimand; if it changes with R, the registered target is instead the effective state $\Psi = (\Phi, \Xi^{\mathrm{read}})$. Let $Q(r, f) \in [0, 1]$ be the potential qualification outcome for that declared target.

Define the true-feedback state effect and semantic interaction

$$\theta_{\mathrm{state}} = \mathbb{E}[Q(1, 1) - Q(0, 1)], \tag{49}$$

$$\theta_{\mathrm{int}} = \mathbb{E}[Q(1, 1) - Q(0, 1) - Q(1, 0) + Q(0, 0)]. \tag{50}$$

The first measures mounting the true-feedback update. The second asks whether that effect exceeds the effect of a compute-matched sham update. Neither estimand is automatically positive.

Theorem 20.3 (Unbiased factorial estimator). **PROVED UNDER STATED ASSUMPTIONS** *Under Assumptions 17.6 and 17.7, independent randomized assignment of (R, F) makes each observed cell mean $\widehat{\mu}_{rf}$ unbiased for $\mathbb{E}[Q(r, f)]$. Consequently,*

$$\widehat{\theta}_{\mathrm{int}} = \widehat{\mu}_{11} - \widehat{\mu}_{01} - \widehat{\mu}_{10} + \widehat{\mu}_{00} \tag{51}$$

is unbiased for θ_{int}, and $\widehat{\mu}_{11} - \widehat{\mu}_{01}$ is unbiased for θ_{state}.

Proof. By randomized assignment, (R, F) is independent of the vector of potential outcomes and has positive cell probabilities. Consistency gives $Q_{\mathrm{obs}} = Q(R, F)$. Therefore $\mathbb{E}[Q_{\mathrm{obs}} \mid R = r, F = f] = \mathbb{E}[Q(r, f)]$ for each cell. Taking the stated linear combinations and using linearity of expectation proves both claims. □

Randomization does not repair a wrong outcome definition, a broken reset, or interference across episodes. It only identifies the displayed potential-outcome contrasts under the declared protocol.

Protocol 20.4 (Minimum strict-RTT qualification). **PLANNED TEST** Before measuring task success, a future implementation should preregister: (i) immutable episode IDs; (ii) a fresh-proposal snapshot; (iii) policy, policy-read-closure, and non-policy state hashes; (iv) the deterministic event-key order and reset epoch; (v) a balanced (R, F) assignment; (vi) common-random-number or sufficiently replicated distribution probes; (vii) the total-variation lower bound and uncertainty rule for W7; and (viii) a reset-negative control including deliberately late callbacks. Failure to pass W1–W8 stops the benefit claim.

20.3 Reset isolation

Definition 20.5 (Registered reset snapshot and completion)**.** DEFINITION A registered reset snapshot is

$$r^\star = (s^\star, n^\star, \kappa^\star, e^\star, H(s^\star)), \tag{52}$$

where $s^\star$ is the complete event state, $n^\star$ and $\kappa^\star$ identify its event boundary, $e^\star$ is its reset epoch, and $H(s^\star)$ is a content hash over every carrier in Equation (40), external-service disposition, and pending-callback set. A reset operator $\mathcal{D}_{r^\star}$ is *complete* only if it atomically restores $s^\star$, cancels or drains registered pending work, advances to a fresh epoch, and makes every return from an older epoch ineligible to mutate current state. Reset completion q is the admitted event at which these conditions first hold. Reapplying $\mathcal{D}_{r^\star}$ before any new admitted event is idempotent up to the fresh-epoch receipt.

Theorem 20.6 (Complete reset implies post-reset equality)**.** PROVED UNDER STATED ASSUMPTIONS *Let two arms complete registered resets at stopping events q_a and q_b. Suppose their post-completion current states are exactly the same complete state $s^\star$ in Equation (40), including policy-read and auxiliary state, external-service disposition, random-generator state, budget, and an empty or identically restored pending-work set. Suppose stale-epoch returns cannot re-enter either arm, there is no cross-arm interference, and thereafter both arms use the same transition kernels under the deterministic event-order rule. Then every observable after reset completion has the same conditional distribution in the two arms.*

Proof. Re-index each arm locally from its reset-completion event. Conditional on the common state $s^\star$, the first admitted post-reset event has the same kernel. Stale-epoch exclusion prevents pre-reset asynchronous work from adding an unmodeled transition. Applying the same induction over successive transition kernels as in Lemma 19.1 gives equality of all finite-dimensional post-reset distributions. With common random numbers and a shared measurable transition realization, the same premises strengthen the conclusion to pathwise equality; without that coupling, the theorem asserts distributional equality only. □

Corollary 20.7 (Reset-negative control)**.** PROVED UNDER STATED ASSUMPTIONS *Under Theorem 20.6's assumptions, a systematic post-reset arm difference falsifies at least one reset, kernel-equality, or no-interference assumption; it cannot be attributed to the erased policy update.*

The word "complete" is demanding. Files, retrieval indices, tool sessions, compiler caches, policy-read closure, scheduler state, numerical nondeterminism, optimizer moments, random streams, reset epoch, and queued or in-flight callbacks all require an explicit disposition. A byte-equal model file is not a complete reset certificate, and wall-clock simultaneity is not a reset boundary.

20.4 Verifier uncertainty and reward circularity

Let $S \in \{0, 1\}$ denote true task success under a prespecified contract and $V \in \{0, 1\}$ a verifier decision.

Proposition 20.8 (Verifier alignment bound)**.** PROVED UNDER STATED ASSUMPTIONS *For any policy arm k,*

$$|\mathbb{E}[V_k] - \mathbb{E}[S_k]| \le \mathbb{P}(V_k \neq S_k). \tag{53}$$

For two arms a, b with error rates at most $\varepsilon_a, \varepsilon_b$,

$$|\{\mathbb{E}[V_a] - \mathbb{E}[V_b]\} - \{\mathbb{E}[S_a] - \mathbb{E}[S_b]\}| \le \varepsilon_a + \varepsilon_b. \tag{54}$$

Proof. Because S, V are binary, $|V - S| = \mathbf{1}\{V \neq S\}$. Thus $|\mathbb{E}[V - S]| \le \mathbb{E}|V - S| = \mathbb{P}(V \neq S)$. Apply this once per arm and use the triangle inequality for the second claim. □

Theorem 20.9 (Sign non-identifiability without a verifier link)**.** PROVED UNDER STATED ASSUMPTIONS *Suppose only transcripts, arm assignments, and a proxy verifier V are observed, while no assumption or audit links V to latent success S. Then the sign of a measured arm effect on S is not identified in general.*

Proof. Fix any observed joint law of transcripts, arms, and V. One compatible latent model sets $S = V$; another sets $S = 1 - V$. Both preserve every observed distribution. Their arm effects satisfy $\mathbb{E}[S_a - S_b] = \mathbb{E}[V_a - V_b]$ and $\mathbb{E}[S_a - S_b] = -\mathbb{E}[V_a - V_b]$, respectively. Unless the proxy effect is zero, the signs oppose. Therefore observed data alone do not identify the true-success direction. □

This impossibility is especially relevant when the same model family generates feedback, trains the update, and judges the final answer. Agreement inside that loop can increase while external correctness falls. A held-out executable contract, independent judge, or bounded verifier-error audit is not optional evidence decoration; it is part of the estimand.

21 Update Operators, Budgets, Lifetimes, and Objectives

21.1 RTT is not defined as gradient descent

Let $G_j = (X, Z_{\le j}, Y_{\le j}, C_j, M_j, W_j, \Xi_j^{\text{read}}, \Xi_j^{\text{aux}}, B_j)$ be the causally available update record. A general bounded update of the declared carrier is

$$\Phi_{j+1} = \Pi_{\mathcal{K}(B_j)}\big(\mathcal{U}_\Theta(\Phi_j, G_j; R_j)\big), \tag{55}$$

where $\mathcal{U}_\Theta$ is an update operator, $\Pi_{\mathcal{K}(B_j)}$ enforces a budget- and safety-dependent admissible set, and Θ is an outer-trained rule or fixed algorithm. If the operator also changes a value read by the later action kernel, the registered target is $\Psi = (\Phi, \Xi^{\text{read}})$ and the receipt must expose both components and their closure hashes; it is not a Φ-only intervention. Table 13 lists nonexclusive realizations.

Operator family	Mutable policy state	Within-problem update	Distinct audit risk
Full/partial gradient	Selected weights and optimizer moments	One or more bounded gradient steps	Optimizer state, nondeterminism, support drift
Low-rank/adapters	Factor matrices or gates	Gradient, closed-form, or learned step	Mounted path may be gated off; rank/cost accounting
Learned optimizer	Optimizer recurrent state plus target parameters	Meta-learned map from gradients/feedback	Outer-training leakage and hidden lifetime
Recurrent/fast weights	Persistent policy-generating hidden state	State-transition rule from feedback	Confusion with ordinary token cache
Hypernetwork	Code or generator parameters	Feedback changes code/conditioning	Generated weights must be versioned and swapped
Bounded controller	Rules, logits, routing policy, or finite controller	Verified edit or constrained solver	Relabeling search workspace as policy

Table 13: **Update-operator taxonomy (DEFINITION).** An operator enters strict RTT only through the same W1–W8 witness; its mathematical form does not confer status. LoRA is one possible low-rank substrate [43], not part of the definition.

The projection in Equation (55) can enforce norm, rank, sparsity, layer, latency, or memory limits. It does not itself guarantee useful learning. A learned optimizer may use an outer parameter Θ trained across tasks, while Φ is reset and adapted inside one episode. Their lifetimes and evidence must be separated: performance of Θ does not prove that a particular $\Phi_j \to \Phi_{j+1}$ transition—or a bundled $\Psi_j \to \Psi_{j+1}$ transition—was mounted and read.

21.2 Within-problem and outer objectives

For a fixed causal record at update time, a local update may target

$$J_j(\phi) = \mathbb{E}\big[R_{j+1} - \lambda_c C_{j+1} - \lambda_h H_{j+1} \,\big|\, \mathcal{F}_{u_j}, \mathrm{do}(\Phi_{j+1} = \phi)\big], \tag{56}$$

where R_{j+1} is a prespecified retry utility, C_{j+1} is charged cost, and H_{j+1} is a harm variable. This conditional objective is not observable without assumptions or interventions; a proxy verifier defines a different objective unless its link to R is audited.

An outer design parameter Θ should be evaluated over complete episodes:

$$\begin{aligned}\mathcal{J}(\Theta) = \mathbb{E}_{X\sim\mathcal{D}}\Big[&U(X, Z_{1:J_\tau}) - \lambda_{\text{tok}} C_{\text{tok}} - \lambda_{\text{tool}} C_{\text{tool}} - \lambda_{\text{upd}} C_{\text{upd}} \\ &- \lambda_{\text{state}} C_{\text{state}} - \lambda_{\text{harm}} H - \lambda_{\text{drift}} D(\Phi_{\tau\wedge J}, \Phi_1)\Big],\end{aligned} \tag{57}$$

subject to hard per-episode budget, lifetime, verifier, and safety constraints. J_τ is the number of attempts begun before resolution. Reporting only final accuracy removes the defining cost of within-problem learning and can make an arbitrarily expensive method look dominant.

Definition 21.1 (Training-only realized-future teacher)**.** DEFINITION An outer teacher may use completed future trajectories to estimate the counterfactual value of update actions during offline training. Any value, label, hyperparameter choice, or state version selected with post-u_j information is *training-only*. Deployment remains valid only if its committed update is $\mathcal{F}_{u_j}$-measurable. The future teacher is not a deployed RTT witness.

21.3 Policy-gradient identity and its assumptions

Let a retry trajectory Z have density $p_\phi(z \mid \mathcal{G})$ conditional on a pre-retry record $\mathcal{G} \subseteq \mathcal{F}_{u_j}$, and let $R(Z)$ be integrable.

Proposition 21.2 (Conditional score-function identity). **PROVED UNDER STATED ASSUMPTIONS** *Suppose p_ϕ is differentiable, its support does not depend on ϕ, differentiation and integration may be interchanged, and $b(\mathcal{G})$ is any integrable baseline independent of the sampled retry actions conditional on $\mathcal{G}$. Then*

$$\nabla_\phi \mathbb{E}[R(Z) \mid \mathcal{G}] = \mathbb{E}\big[(R(Z) - b(\mathcal{G}))\nabla_\phi \log p_\phi(Z \mid \mathcal{G}) \mid \mathcal{G}\big]. \tag{58}$$

Proof. Using the stated interchange, $\nabla_\phi \int R(z)p_\phi(z \mid \mathcal{G})dz = \int R(z)p_\phi(z \mid \mathcal{G})\nabla_\phi \log p_\phi(z \mid \mathcal{G})dz$. The baseline term has conditional expectation $b(\mathcal{G}) \int \nabla_\phi p_\phi(z \mid \mathcal{G})dz = b(\mathcal{G})\nabla_\phi 1 = 0$. Subtracting it yields Equation (58). □

The proposition proves an identity, not low variance, verifier validity, safe optimization, or improvement after a finite step. If support changes under decoding truncation or tool constraints, the stated form may fail.

Lemma 21.3 (Group-relative degeneracy). **PROVED UNDER STATED ASSUMPTIONS** *Consider a group update whose per-sample weight is*

$$A_i = \frac{R_i - \overline{R}}{s_R + \varepsilon}, \qquad \varepsilon > 0. \tag{59}$$

If $R_1 = \cdots = R_K$, then every $A_i = 0$ and any update linear in the A_i is exactly zero.

Proof. Equal rewards imply $R_i = \overline{R}$ for every i, so every numerator vanishes. A linear combination of score terms with zero coefficients is zero. □

This is a within-group credit result, not a statement that all group-relative algorithms always fail. It yields a concrete stop condition: if the verifier collapses to one value within update groups, more optimizer steps cannot recover a signal from Equation (59).

21.4 Budget and lifetime ledgers

Let the remaining budget be a vector

$$B_j = (b_j^{\text{tok}}, b_j^{\text{tool}}, b_j^{\text{sec}}, b_j^{\text{upd}}, b_j^{\text{byte}}, b_j^{\text{energy}}), \tag{60}$$

where energy may be an explicitly labeled proxy if direct measurement is unavailable. Every attempt/update pair incurs a nonnegative cost vector c_j.

Proposition 21.4 (Hard-ledger feasibility). **PROVED UNDER STATED ASSUMPTIONS** *Suppose an operation is admitted only when $c_j \le B_j$ componentwise and then sets $B_{j+1} = B_j - c_j$. If $B_1 \ge 0$, every admitted prefix has $B_j \ge 0$ and cumulative cost at most B_1.*

Proof. Componentwise subtraction preserves nonnegativity by the admission rule. Telescoping gives $\sum_{i<j} c_i = B_1 - B_j \le B_1$. □

Lifetime class	Required reset/transfer statement	Scientific interpretation
Attempt-local	State expires before another attempt	Not a cross-attempt RTT carrier
Problem-local	State survives retries and expires at τ	Canonical strict RTT lifetime
Session-local	State may cross problem IDs	Mixture with online/continual adaptation
Persistent	State enters later sessions/releases	Continual/offline learning artifact; separate governance

Table 14: **Policy-state lifetime classes (DEFINITION).** Strict RTT concerns the in-problem update–reuse event; cross-problem retention is an additional mechanism that must not be hidden.

An episode-local adapter may still leak through optimizer caches, compilation artifacts, retrieval indices, or scheduler state after its nominal deletion. Lifetime certification therefore concerns the complete causal state, not only the primary tensor file.

22 Stability, Rollback, and Stop Conditions

22.1 Local drift bounds are conditional, not global safety proofs

For a fixed admissible history h, write $\pi_i(\cdot \mid h)$ for the fresh-proposal law after update i.

Theorem 22.1 (Cumulative local drift). **PROVED UNDER STATED ASSUMPTIONS** *Suppose for $i = 1, \dots, K$ and a fixed history h,*

$$\mathrm{KL}(\pi_i(\cdot \mid h) \,\|\, \pi_{i-1}(\cdot \mid h)) \le \kappa_i(h) < \infty. \tag{61}$$

Then

$$\mathrm{TV}(\pi_K(\cdot \mid h), \pi_0(\cdot \mid h)) \le \sum_{i=1}^{K} \sqrt{\frac{\kappa_i(h)}{2}}. \tag{62}$$

For any measurable $g : \mathcal{A} \to [0, 1]$,

$$\left|\mathbb{E}_{\pi_K}[g(A) \mid h] - \mathbb{E}_{\pi_0}[g(A) \mid h]\right| \le \min\left\{1, \sum_{i=1}^{K} \sqrt{\frac{\kappa_i(h)}{2}}\right\}. \tag{63}$$

Proof. Pinsker's inequality bounds each adjacent total variation distance by $\sqrt{\kappa_i(h)/2}$. The triangle inequality for total variation gives Equation (62). The difference in expectations of a $[0, 1]$ function is at most total variation; total variation is at most one, yielding Equation (63). □

The theorem is pointwise in h. A KL constraint measured on update trajectories does not automatically control unseen histories, tool calls, or external side effects. Nor does small local drift imply positive utility: a small harmful change is still harmful.

Theorem 22.2 (No nontrivial update is uniformly improving). **PROVED UNDER STATED ASSUMPTIONS** *Let π and π' be distinct probability measures over actions at some fixed history. There exists a bounded utility $g : \mathcal{A} \to [0, 1]$ such that $\mathbb{E}_{\pi'} g < \mathbb{E}_{\pi} g$.*

Proof. Because $\pi' \neq \pi$, there is a measurable set A on which their probabilities differ. If $\pi'(A) < \pi(A)$, choose $g = \mathbf{1}_A$. Otherwise the complement satisfies $\pi'(A^c) < \pi(A^c)$, so choose $g = \mathbf{1}_{A^c}$. In either case the expected utility decreases. □

This simple impossibility explains why a universal "safe learning rate" cannot follow from nontriviality alone. Safety requires a specified utility/harm family, coverage assumptions, conservative gates, and rollback; it cannot mean improvement for every possible downstream objective.

22.2 Transactional update contract

Definition 22.3 (Rollback-capable policy-state transaction). **DEFINITION** A policy update transaction contains: (i) an immutable pre-state snapshot; (ii) a candidate state and complete auxiliary state; (iii) typed validation receipts; (iv) canary probes on registered histories; (v) an atomic mount or NULL decision; (vi) post-mount monitoring; (vii) a bounded-time rollback path restoring the complete pre-state; and (viii) quarantine of rejected candidate artifacts. Partial mounts do not count as commits.

Gate	Prespecified check	Mandatory action on failure
Causal	Update inputs are $\mathcal{F}_{u_j}$-measurable	Quarantine; label leakage
Integrity	Hashes, shapes, optimizer state, route, and version agree	Atomic NULL; no retry mount
Budget	Worst-case update/retry cost fits remaining ledger	Skip update or stop episode
Functional	Fresh-proposal W7 lower bound clears threshold	Label inert; no RTT claim
Local stability	KL/norm/logit and tool-policy deltas under caps	Roll back candidate
Canary safety	No registered severe-harm event; harm CI below bound	Roll back and stop branch
Verifier	Error/abstention and group diversity within gate	Do not optimize proxy
Reset control	Negative-control arms exchangeable within tolerance	Stop causal attribution

Table 15: **Transactional qualification and stop rules (DESIGN PRESCRIPTION).** Thresholds must be registered before seeing target-arm outcomes.

22.3 What rollback can and cannot undo

Rollback can restore a declared digital state. It cannot automatically undo an external side effect already caused by a tool call, disclosure, transaction, or human decision. A safe episode therefore separates *proposal actions* from *irreversible actions*. Candidate policy versions may generate proposals in a sandbox; an independently governed executor decides whether to commit side effects. If the task necessarily includes irreversible actions, the harm estimand and authorization boundary must include them before the update is mounted.

Definition 22.4 (Stop condition)**.** DEFINITION A stop condition is a precommitted predicate on causally available audit variables whose truth forces one of: skip update, return NULL, roll back, abandon the episode, or escalate to external review. A stop condition is invalid if it is relaxed after observing an unfavorable target-arm result without recording a new protocol.

Minimum stops include: future leakage; missing state/read receipts; zero functional divergence; collapsed within-group feedback; failed complete-reset control; verifier error exceeding its bound; safety-canary breach; budget overrun; nonfinite update; version mismatch; and inability to restore the full pre-state. An episode may stop even when the current candidate would have solved the problem; evidence discipline takes precedence over retrospective success.

22.4 Stability across repeated within-problem updates

Repeated updates create a feedback loop: the current policy determines which attempts are seen, those attempts determine feedback, and feedback changes the next policy. Three ledgers should therefore be distinct:

1. **Mechanism ledger:** hashes, update inputs, W1–W8, and state-swap divergence for each transition;
2. **Utility ledger:** true/proxy outcomes, verifier uncertainty, and prespecified episode utility;
3. **Risk ledger:** local drift, canary harms, side effects, rollback latency, and residual state after expiry.

A positive utility ledger cannot repair a missing mechanism witness, and a valid mechanism witness cannot waive a risk failure.

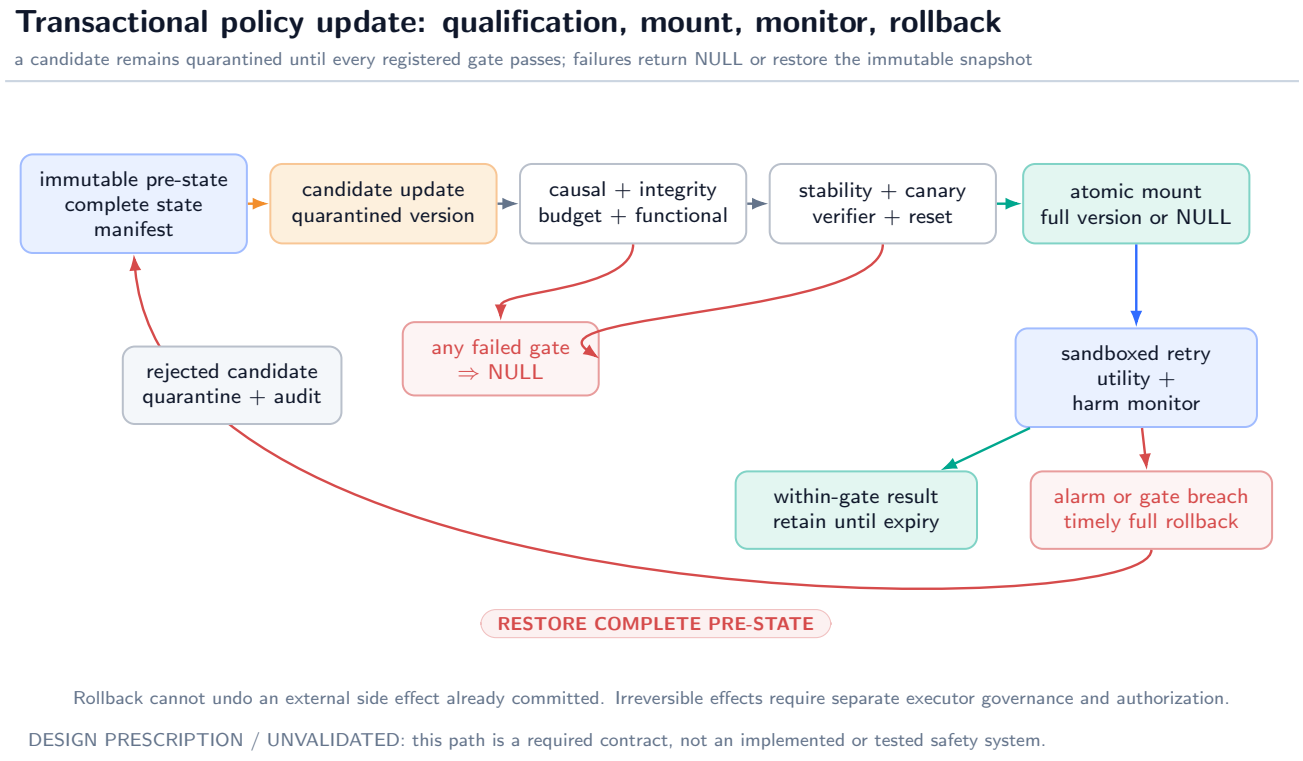

Figure 22: **Safety, rollback, and stop-condition path (DESIGN PRESCRIPTION).** A candidate update remains quarantined until causal, integrity, budget, functional, stability, canary, verifier, and reset gates pass. Any bounded coral failure returns NULL or restores the immutable snapshot. Deployment of a candidate is atomic; the figure reports a contract, not an implemented system.

23 From Theorems to Implementation Obligations

23.1 Proof dependency map

The formal claims are shallow by design: each conclusion depends on a small, inspectable assumption set. Figure 23 and Table 16 prevent a common category error—implementing the conclusion's name while omitting the assumptions that make it true.

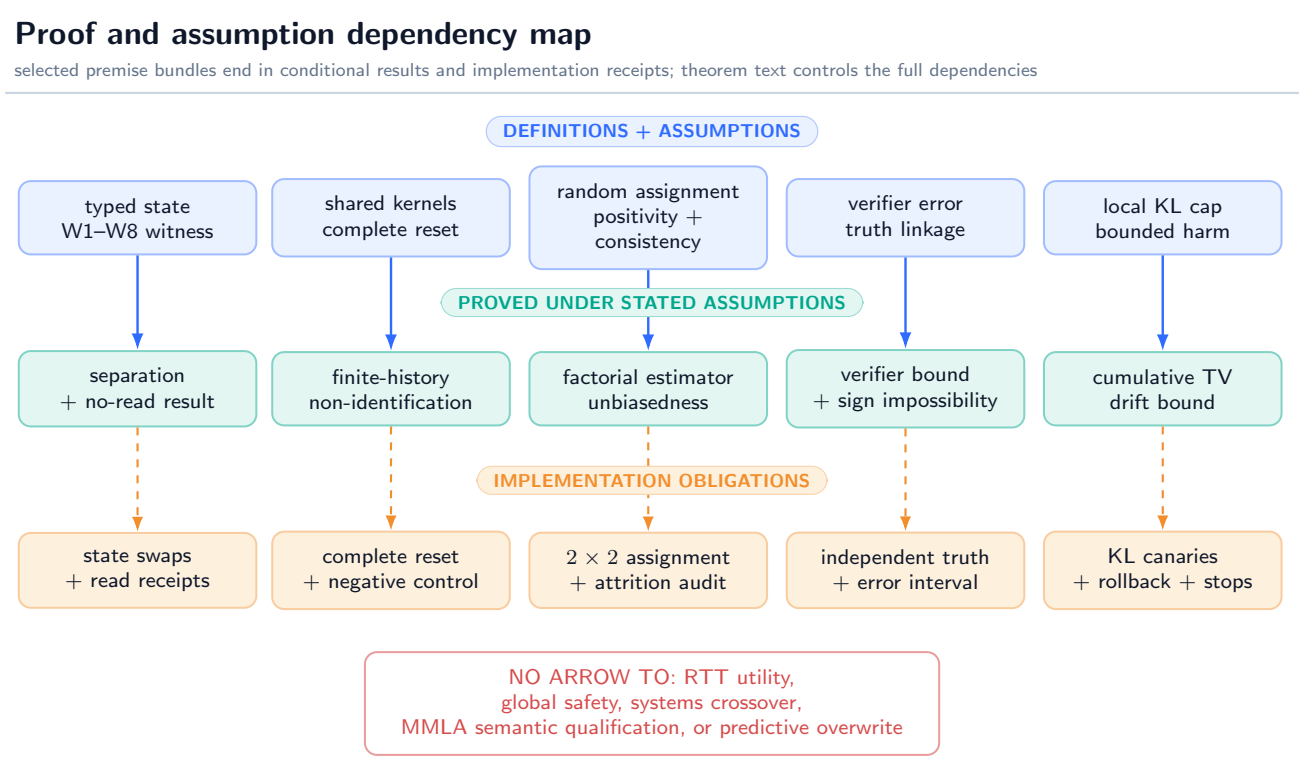

Figure 23: **Assumption and proof dependency map (PROVED UNDER STATED ASSUMPTIONS for the drawn logical arrows).** Blue nodes are definitions/assumptions, teal nodes are proved statements, and amber nodes are design consequences. No arrow enters an empirical-success node because this paper contains none. Appendix 27 provides the expanded derivations.

Table 16: **Theorem-to-implementation obligations.** Formal results become usable only when their premise receipts exist.

Formal statement	Essential assumptions	Required implementation evidence	Does not establish
Lemma 19.1	Conditional independence of every later kernel from Φ at fixed Ξ^{read}	Typed route/read traces; readable-closure hashes; state-swap or route-disable probe	Update usefulness
Proposition 19.2	Stable typed state partition and byte-identical Ψ	Fresh-proposal snapshot; readable-closure hash; workspace inventory	Search inferiority
Theorem 20.1	Finite observable horizon; same environment kernels	Transcript schema and hidden-state inventory	Efficient fixed emulator
Theorem 20.3	Randomization, positivity, consistency, no bundled carrier changes	Assignment receipt; arm counts; Φ and readable-closure hashes; restore hashes; attrition audit	Correct outcome/verifier
Theorem 20.6	Complete state equality, fresh epoch, deterministic event order, stale-return exclusion, and common kernels after completion	Registered snapshot/restore map/completion event; full-state manifest; pending-callback ledger; stale-return receipts; external-service reset; seed receipt	That any practical reset is complete
Proposition 20.8	Binary success and measured verifier error	Independent adjudication sample and error CI	Small error off the audited support
Theorem 20.9	No constraint linking verifier to truth	Explicit absence/presence of external truth contract	That the proxy is actually adversarial

Formal statement	Essential assumptions	Required implementation evidence	Does not establish
Proposition 21.2	Differentiability, fixed support, integrability, legal interchange	Decoder support audit; finite-gradient checks	Low variance or finite-step gain
Lemma 21.3	Centered group-relative weights; linearity in advantages	Per-group rewards and computed advantages	Failure under noncollapsed rewards
Proposition 21.4	Nonnegative charged costs; hard admission rule	Monotone signed resource ledger	Accuracy or efficiency dominance
Theorem 22.1	Per-history finite adjacent KL	Common-history log-probability probes	Global behavioral safety
Theorem 22.2	Nonidentical action laws; arbitrary bounded utilities	Utility/harm family stated before update	Harm on the chosen real task

23.2 Minimum trace schema

Definition 23.1 (RTT trace bundle)**.** DEFINITION A qualification bundle contains one immutable manifest linking:

1. episode/problem/version identifiers and complete problem contract;
2. attempt start/end events, observations, actions, tool calls, deterministic keys $\kappa(e)$, wall-clock attributes, and randomization receipts;
3. feedback bytes, producer identity, availability time, and truth/proxy status;
4. pre/candidate/post Φ bytes, typed Ξ^{read} and Ξ^{aux} manifests, content-addressed references, and readable-closure hashes χ^-, χ^+;
5. context, memory, workspace, external-service, and budget manifests at each restore boundary;
6. update inputs, code/config hash, numerical mode, cost vector, candidate validation, commit/NULL decision;
7. later policy-kernel read trace and fresh-proposal state-swap outputs, stating whether the intervention is on Φ or on Ψ;
8. reset snapshot, complete restore-map receipt, completion event, old/new epoch, pending-callback disposition, and every rejected stale-epoch return;
9. outcome, verifier, safety, rollback, expiry, and residual-state receipts.

Every omission is a declared unknown, not an implicitly controlled variable.

23.3 Hidden-state and future-leakage audit

The following channels require explicit disposition before causal attribution:

- model weights, adapters, gates, normalization statistics, optimizer moments, activation/attention/KV caches, recurrent state, quantization scales, compiler graphs, and numerical kernels, classified explicitly as Φ, Ξ^{read}, or Ξ^{aux};
- prompts, system messages, hidden chain-of-thought carriers, retrieved documents, episodic/semantic memory, search trees, scratch files, databases, environment variables, tool sessions, and service-side caches;
- random-number generators at every library/device, scheduler order, batching neighbors, deterministic event keys, reset epochs, asynchronous callbacks and stale returns, and retry policies;
- verifier models, reference answers, unit tests, search or tool results, and any post-update outcome used to choose checkpoints or thresholds.

Protocol 23.2 (Leakage challenge)**.** PLANNED TEST Construct a canary field revealed strictly after update time and guaranteed irrelevant to the problem. Search every update input, serialized state, log, and selection decision for dependence on that field; additionally train a held-out decoder to predict the canary from the committed policy-state delta. Any above-chance result beyond the registered tolerance stops the causal claim until the channel is explained. A negative canary test is evidence about tested channels only.

23.4 Circularity audit

Four loops are especially dangerous:

1. **Self-verification:** the policy family produces answers and labels them;
2. **Selection on target outcomes:** learning rates, checkpoints, or stop thresholds are chosen using the same episodes used for the claim;
3. **Adaptive exclusion:** difficult failures are discarded after arm assignment;
4. **Mechanism by outcome:** only successful retries are inspected for update receipts.

The remedy is not to ban internal feedback. It is to define whether it is a proxy, bound or externally audit its error, freeze selection rules on a disjoint split, report intention-to-treat and attrition, and inspect mechanism receipts for every assigned episode. Appendix 29 gives a printable audit matrix.

23.5 Reproducible state swaps

A state swap must be semantically complete. For an adapter, this includes factor matrices, scaling, gates, routing tables, normalization state, optimizer-coupled forward state, dtype, and mount location. The pre- and post-state should be evaluated on the same registered histories under common random numbers where possible. For sampled text, a lower bound on total variation may be estimated from repeated draws with simultaneous confidence intervals, but a failure to reject equality is not proof of equality. Exact logit comparison is preferable when the action space and inference stack permit it.

24 Relationship to Adjacent Work

This section classifies *described protocols*, not communities or papers as wholes. A paper may contain several mechanisms, and a future implementation may differ from its prose. Strict RTT status belongs to a concrete trace satisfying Definition 18.1.

24.1 Test-time training and adaptation

Classical test-time training updates model parameters on an auxiliary self-supervised objective derived from an unlabeled test sample before the main prediction [97]. Fully test-time adaptation such as Tent updates normalization parameters online by entropy minimization on test batches [102]. These works establish the broader idea that parameters can change during evaluation. Their unit and timing need not match strict RTT: an auxiliary update before a first task attempt, or an update accumulated across a stream and used on later samples, is test-time adaptation without necessarily containing a feedback–update–retry witness inside one still-unresolved reasoning problem.

TTRL performs reinforcement learning on unlabeled test data using consensus-derived rewards and online parameter updates [135]. At the dataset level it is test-time reinforcement learning. Whether a particular item is strict RTT depends on whether feedback from that item updates policy state before the item ends and a later attempt on that same item uses the update; dataset-level improvement cannot substitute for that trace.

24.2 Inference-time scaling, search, and refinement

Self-consistency samples multiple reasoning paths and aggregates answers [105]. Tree of Thoughts explicitly searches over intermediate reasoning states with evaluation and backtracking [112]. ReAct interleaves reasoning traces with actions and tool observations [113]. These mechanisms enlarge computation, workspace, or context and can substantially change outcomes while keeping the underlying proposal policy fixed. Under our state partition they are not strict RTT unless an additional policy-state update passes the fresh-proposal test.

Self-Refine iteratively produces feedback and revised outputs without supervised training, additional training, or reinforcement learning [64]. Reflexion explicitly reinforces agents through linguistic feedback and an episodic memory buffer rather than weight updates [92]. They are canonical reasons to preserve context/memory categories: trial-to-trial learning-like behavior need not be policy training. The label RTMA is useful for ordinary reasoning-time memory adaptation, but this paper makes no claim that it is the original name for such mechanisms.

24.3 Within-instance policy evolution

Policy of Thoughts describes within-instance online optimization: execution feedback drives GRPO updates to a transient LoRA adapter, which is reused during subsequent search and discarded after the problem [47]. StarOR describes node-level GRPO updates to a transient LoRA adapter maintained for the current optimization-modeling instance and reset for each new problem [57]. Their primary sources therefore describe candidate strict-RTT architectures more closely than fixed-policy search. This paper does not revalidate their experiments or infer W1–W8 from descriptions alone. In particular, an independent strict audit would still need exact update times, unresolved status, actual mounted-state reads, fresh-proposal divergence, reset completeness, budgets, and verifier provenance.

24.4 Continual learning, low-rank state, and memory

Continual learning studies policy retention across sequences of tasks and the prevention of catastrophic forgetting; elastic weight consolidation is a representative example [53]. Its primary reuse horizon is later tasks, whereas strict RTT requires reuse before the current problem ends. The regimes can be nested: a strict within-problem adapter may be reset at each endpoint while an outer continual learner changes the initialization across episodes.

LoRA provides a parameter-efficient low-rank adaptation substrate [43]. A LoRA update may be offline, continual, test-time, or strict RTT depending on timing and reuse; the matrix factorization alone determines none of those regimes.

MMLA formalizes bounded authoritative memory, causal event-conditioned writes, and training-only future valuation while reporting a current semantic-interface blocker [124]. Its public evidence does not validate RTT, and this theory paper does not validate MMLA predictive overwrite. A combined system would need independent policy-state and memory-state interventions as in Figure 18.

Protocol family	Described changing carrier	Typical reuse unit	Trace-level RTT interpretation
TTT / Tent [97, 102]	Model/statistical parameters	Current sample, batch, stream	Adjacent; inspect whether same-problem feedback precedes an actual retry.
Self-consistency / ToT [105, 112]	Samples/search workspace	Same problem	Fixed-policy form is not strict RTT.
ReAct / Self-Refine [113, 64]	Context, tool state, revision text	Same problem	Context/workspace adaptation unless policy state also changes.
Reflexion [92]	Episodic linguistic memory	Later trials	Memory adaptation; source explicitly contrasts with weight updates.
TTRL [135]	Policy parameters	Test data stream	Per-item strict status requires an item-level update–reuse trace.
PoT [47]	Per-instance transient LoRA	Subsequent same-instance search	Candidate strict RTT; W1–W8 not independently audited here.
StarOR [57]	Per-instance transient LoRA	Later nodes/iterations on same instance	Candidate strict RTT; W1–W8 not independently audited here.
Continual learning [53]	Persistent parameters	Later tasks	Different primary horizon; may contain nested RTT.
MMLA [124]	Bounded external/resident memory	Later tokens/segments	Memory program, not RTT evidence.

Table 17: **Related-work protocol map (source-verified, not a replication).** The final column applies this paper's operational boundary cautiously and makes no priority or empirical-ranking claim.

The literature suggests a continuum rather than a winner: compute can be spent on broader exploration, richer nonparametric state, faster parametric adaptation, or combinations. The scientific task is to identify which carrier caused which effect at what cost, not to collapse all adaptation into one label.

25 A Future Qualification and Falsification Program

25.1 Stage gates

The program begins with mechanism identity and stops on failure. It does not begin with benchmark accuracy. Figure 24 shows the dependency order; Table 18 makes each rung falsifiable.

The gates are sequential because later quantities are uninterpretable when earlier premises fail. A utility difference cannot establish policy-state use if the declared carrier was never read, and a carrier-swap contrast cannot isolate Φ if the policy-read closure, a tool cache, context suffix, random-number stream, or workspace also changed. When the policy-read closure cannot be held byte-identical, the registered contrast concerns the effective carrier $\Psi = (\Phi, \Xi^{\text{read}})$ rather than Φ alone. Likewise, an apparently favorable verifier score does not identify true utility until the verifier–truth error relation is bounded on the relevant arms. Passing a gate therefore means only that its registered receipts survived their own audit. It neither repairs a failed lower gate nor supplies evidence for a higher one.

Each gate must specify its analysis population before execution. The intention-to-treat population contains every assigned episode, including update crashes, refusals, timeouts, rollback events, and missing receipts. A separately reported qualification population may restrict attention to episodes whose W1–W8 records are complete, but that restriction is descriptive unless the selection mechanism is itself identified. Attrition, verifier abstention, and failed mounts are outcomes, not preprocessing inconveniences. Negative controls and deliberately corrupted traces belong in the same ledger so that an auditor can estimate whether the qualification machinery rejects mechanisms it was designed to reject.

Thresholds, budgets, and stop decisions must be frozen before outcome inspection: the minimum W7 divergence, verifier-error tolerance, local drift cap, severe-harm definition, equal-compute ledger, and minimum worthwhile utility gain. A negative terminal is a valid result at its gate. It should preserve the failed receipt, assigned arm, and available downstream measurements while withholding the claim that depended on the failure. This convention separates scientific falsification from system promotion: an implementation can be informative even when it never becomes eligible for scaling.

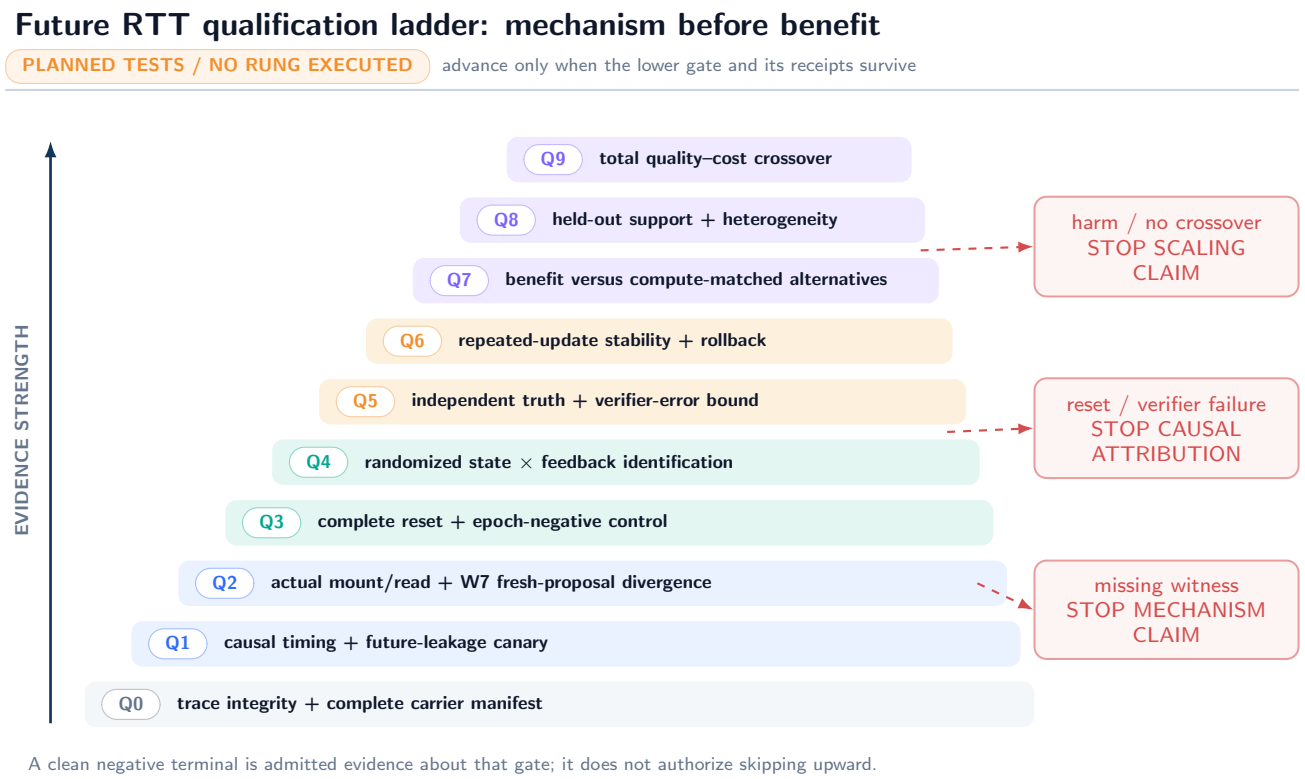

Figure 24: **Future empirical qualification ladder (PLANNED TEST).** Evidence may advance only upward: integrity and timing precede functional state use; identification precedes benefit; safety precedes scale; equal-compute crossover is last. Coral side arrows are registered stops. No rung has been executed or passed in this paper.

Table 18: **Planned qualification tests and stop rules (PLANNED TEST).**

Gate	Question	Minimal protocol	Stop condition
Q0	Is the trace complete?	Hash code/config/state; immutable episode IDs; typed carrier, policy-read-closure, callback, and cost inventories.	Missing, inconsistent, or unbounded receipt.
Q1	Did timing satisfy W1–W5?	Audit deterministic event keys, reset epochs, callback admissions, filtration, and stopping times; run a future canary.	Update after resolution, ambiguous order, stale return admitted, or detectable future dependence.
Q2	Was post-state actually used?	Mount/read trace with closure hashes plus an exact or replicated fresh-proposal swap; label the target Φ or Ψ.	W6 absent, closure target ambiguous, or W7 lower bound ≤ 0.
Q3	Are other carriers isolated?	Restore a registered complete snapshot, complete the restore event, enter a fresh epoch, reject stale callbacks, and run separate carrier swaps.	Reset receipt incomplete or reset-negative control differs.
Q4	Is feedback semantic?	Randomized 2×2 state/feedback factorial; intention-to-treat analysis.	Assignment, positivity, attrition, or sham-compute failure.
Q5	Is the verifier linked to truth?	Independent adjudication, abstention and error bounds, adversarial cases.	Error bound too wide for claimed effect or group signal collapse.
Q6	Is repeated updating stable?	Registered KL/norm/canary caps; atomic rollback; residual-state audit.	Severe harm, nonfinite update, or rollback failure.
Q7	Does it help under matched compute?	Compare fixed-policy search, context, memory, sham update, and RTT under equal ledgers.	Confidence interval excludes the prespecified minimum gain in the wrong direction.
Q8	Does it transfer within declared support?	Held-out problem families, verifiers, seeds, and update counts; report heterogeneity.	Benefit confined to tuned items or hidden carryover.

Gate	Question	Minimal protocol	Stop condition
Q9	Is total quality–cost favorable?	End-to-end wall time, tokens, tool calls, update compute, retained bytes, harm and failure cost.	No registered crossover or budget breach.

25.2 White-box mechanism benchmark

Protocol 25.1 (RTT-M0 mechanism falsifier)**.** PLANNED TEST Construct a synthetic family with a hidden per-instance rule that cannot be identified from the initial prompt alone. Attempt 1 reveals a deterministic verifier bit about one proposed rule; an update may change only a bounded registered carrier; attempt 2 begins only after a registered complete restore has completed, a fresh reset epoch is active, and stale callbacks from the prior epoch are ineligible to mutate current state. Include (a) true update, (b) pre-state restore, (c) sham-feedback update, and (d) search-only workspace arms under one deterministic event-order rule. The primary endpoint is W7 fresh-proposal divergence in the registered direction, not final accuracy. Call the endpoint a Φ effect only when Ξ^{read} and all non-policy carriers are byte-identical across the swap; otherwise preregister the effective target $\Psi = (\Phi, \Xi^{\text{read}})$ and hold every carrier outside Ψ fixed. Closure hashes, reset-completion receipts, epoch transitions, and rejected stale returns are mandatory.

The benchmark should contain negative controls where the feedback is uninformative, the update route is gated off, the adapter is byte-different but functionally null, and the apparent rule is recoverable from a hidden cache. These cases test the auditor rather than the learner.

25.3 Executable reasoning benchmark

Protocol 25.2 (RTT-M1 executable retry)**.** PLANNED TEST Use generated programs with isolated deterministic unit tests unavailable to the policy except through a bounded first-attempt feedback record. Freeze the task generator and hidden tests before method development. Preregister one retry, one bounded update, a hashed complete-state snapshot, the restore map and completion event, a fresh reset epoch with stale-return rejection, the 2×2 factorial, verifier-error spot checks, update/cost caps, and a severe-side-effect sandbox. Report every assigned episode and distinguish syntax, hidden-test truth, Φ-only or bundled-Ψ divergence, reset qualification, and final utility.

Compiler feedback is not automatically ground truth: passing public tests can overfit, and test-generation bugs can reward incorrect behavior. Hidden-test construction, contamination, and false-pass/false-fail audits remain part of Q5.

25.4 Repeated-update stress test

Protocol 25.3 (RTT-M2 repeated-update stability)**.** PLANNED TEST On episodes requiring up to K retries, register a per-update KL cap κ_i, cumulative bound from Theorem 22.1, independent anchor histories, tool-action canaries, rollback latency, and a hard maximum update count. Randomize rollback challenges by injecting known-bad candidate states. Stop the full arm if any irreversible side effect, unbounded ledger, or failed complete-state restoration occurs.

The purpose is not to prove global safety. It is to falsify specific local contracts under adversarial but declared conditions and to measure how often the theoretical premises hold.

25.5 Equal-compute comparisons

The final comparison should allocate the same multidimensional budget, not merely the same number of sampled answers. Search may use its budget for more rollouts; context refinement for longer critique/revision; memory adaptation for writes and retrieval; RTT for update, state storage, probes, and retry. Wall-clock parallelism, accelerator occupancy, tool latency, failure recovery, and verifier calls must be included. A method that wins only after omitting update or safety costs has not established a quality–cost crossover.

Primary reporting should include:

- intention-to-treat and per-protocol estimates with arm counts and attrition;
- θ_{state}, θ_{int}, W7 divergence, verifier-error bounds, and true utility as distinct endpoints;
- distributions over problem families and seeds, not only pooled means;
- compute, time, retained-state, and side-effect ledgers with uncertainty;
- all preregistered stops, including negative and null terminals.

No successful result is predicted by this ladder. A clean failure at Q2 or Q4 would be scientifically useful: it would show that an implementation marketed as adaptive did not create or use the claimed policy-state path.

26 Discussion, Limitations, and Conclusion

26.1 Why adopt a strict definition?

A broad definition in which any feedback-conditioned retry counts as training would be easy to satisfy but difficult to falsify. It would merge sampling, search, prompt revision, memory, tool state, and policy adaptation even though they have different costs, safety properties, and scientific controls. The strict definition instead reserves one term for a specific causal pattern while leaving neighboring mechanisms first-class. A hybrid system can still be studied; it simply cannot attribute a bundled effect to policy learning without state-specific interventions.

The boundary is partly conventional. One could define "policy" to include the entire search tree or prompt, making every stateful computation a policy update. Definition 18.3 rejects that collapse for this paper by holding non-policy carriers fixed and testing fresh proposal laws. This choice is useful because it yields operational separation, but it is not a theorem that every community must adopt the same vocabulary.

26.2 Limitations of the framework

First, exact state interventions may be infeasible in opaque hosted systems. There, strict qualification may remain unidentifiable; behavioral improvement should be reported without a mechanism claim. Second, total variation at token-level histories can be expensive to estimate in large action spaces. Registered lower bounds or exact logit probes cover only tested histories. Third, the deterministic event-key rule presumes an auditable ingress sequencer and frozen precedence classes. Distributed systems need an implementation-specific total-order extension, epoch discipline, and receipts proving that concurrent or stale returns did not bypass it; wall-clock timestamps alone are insufficient. Fourth, potential-outcome consistency may fail when shared services create interference across arms. Fifth, a binary resolution time simplifies tasks with graded or revisable success. Sixth, the framework treats policy and memory as typed interfaces, but architectures with fully entangled fast state may resist clean partitioning. Such systems should report an inseparable bundled carrier rather than force a false decomposition.

The mathematical results are deliberately modest. The finite-history emulator may be exponentially large. The factorial theorem does not solve external validity. The verifier bound is only as useful as the audited error rate. The drift bound is local and can become vacuous after many updates. The no-uniform-improvement theorem does not imply that safe gains are impossible on a restricted task distribution. None of these results predicts that a practical optimizer will learn within an episode.

26.3 Boundary with MMLA evidence

This paper inherits the public author identity, style, and claim grammar of the MMLA V3 line, not an empirical success claim. The complete technical-report evidence record localizes a semantic-interface failure and keeps learned predictive overwrite closed. No MMLA result is used to claim that policy state can be updated and reused during reasoning. Conversely, this paper's definitions do not repair or validate the MMLA memory interface. A future combined MMLA+RTT system would require four independent questions: did policy state change, did memory state change, did each state affect the appropriate fresh/retrieval path, and did either cause verified benefit under matched cost?

26.4 Conclusion

RTT is most useful as a falsifiable trace property: feedback from an attempt on one unresolved problem changes a declared functional policy state, the change commits before resolution, and a later attempt on that same problem actually uses it. This temporal and typed definition separates policy learning from spending more inference compute or carrying more nonparametric state. The accompanying theorems show why logs and proxy success are insufficient, while the qualification ladder states what future evidence would have to survive.

The paper's strongest conclusion is therefore negative in form: neither a method name, an optimizer call, a different retry, nor a proof can establish strict RTT benefit. Establishing it requires state identity, causal timing, functional reuse, interventions, verifier alignment, budget accounting, and rollback evidence together. Those obligations are demanding by design. If future systems pass them, "learning before a single problem ends" will denote an identified mechanism rather than a metaphor.

27 Expanded Proofs

This appendix expands every numbered formal result. All statements retain the status **PROVED UNDER STATED ASSUMPTIONS**. The proofs concern the abstract model only and create no implementation or empirical evidence.

27.1 Kernel notation

Let E_n denote the nth complete event in the deterministic event log, ordered lexicographically by the registered key $\kappa(E_n) = (g_n, p_n, \mathrm{id}_n)$, and let $H_n = (E_0, \ldots, E_n)$ be its observable history. Wall-clock timestamps remain recorded attributes

but do not determine causal order. Conditional on a complete state and intervention arm, write $K_{n+1}(de \mid h_n)$ for the next-event transition kernel. For a finite horizon N, the joint law factors as

$$P(de_{0:N}) = P_0(de_0) \prod_{n=0}^{N-1} K_{n+1}(de_{n+1} \mid e_{0:n}). \tag{64}$$

All induction arguments below are applications of this factorization.

27.2 No-read and separation

Lemma 19.1. Condition on the non-policy state $N_{u_j} := (X, C, M, W, \Xi^{\text{read}}, \Xi^{\text{aux}}, B)$ at update completion. The declared policy-only intervention is meaningful only when the bytes of the policy-readable closure Ξ^{read} are identical; otherwise the intervened functional carrier is the bundle $\Psi = (\Phi, \Xi^{\text{read}})$. Let P^{ϕ} and $P^{\phi'}$ be the post-update laws under two mounted policy states with the same readable closure. By hypothesis, for every reachable history and each later event,

$$K^{\phi}_{n+1}(\cdot \mid h_n, N_{u_j}) = K^{\phi'}_{n+1}(\cdot \mid h_n, N_{u_j}). \tag{65}$$

The initial conditional distributions at u_j are identical because only Φ is intervened on and the later kernels do not read it. Suppose the laws of H_n agree. For any measurable set D of histories through $n{+}1$,

$$P^{\phi}(H_{n+1} \in D \mid N_{u_j}) = \int \mathbf{1}\{(h_n, e) \in D\} K^{\phi}_{n+1}(de \mid h_n) P^{\phi}(dh_n) \tag{66}$$

$$= \int \mathbf{1}\{(h_n, e) \in D\} K^{\phi'}_{n+1}(de \mid h_n) P^{\phi'}(dh_n) \tag{67}$$

$$= P^{\phi'}(H_{n+1} \in D \mid N_{u_j}). \tag{68}$$

Induction reaches the attempt endpoint and any outcome measurable there. Integrating over N_{u_j} gives the unconditional version if its distribution is shared.

Propositions 19.2 and 19.3; Corollary 19.4. Each is a direct application of the conjunction in Definition 18.1. Strict RTT requires W3, a nontrivial declared functional-carrier transition, and W7, a fresh-proposal difference after non-policy carriers are fixed. A process with constant Φ and byte-identical Ξ^{read} cannot satisfy W3; a change only to C, M, W, or Ξ^{aux} remains fixed away in W7. If the policy-readable closure changes, the admissible conclusion concerns Ψ, not Φ alone. Increasing compute changes neither logical fact. These are necessary-condition proofs; they make no claim about utility.

27.3 Finite-history emulation and non-identifiability

Theorem 20.1. Let $\mathcal{H}_n$ be the finite set of observable histories through decision n with positive probability under the adaptive mechanism P_{ad}. Hidden state, including Φ_n, may be stochastic conditional on H_n. Define the fixed behavioral kernel

$$q_n(da \mid h_n) = \int \pi_{\phi}(da \mid h_n) P_{\text{ad}}(d\phi \mid H_n = h_n). \tag{69}$$

This equals $P_{\text{ad}}(A_n \in da \mid H_n = h_n)$ by the law of total probability. The collection $q = (q_1, \ldots, q_N)$ is fixed before emulation; it does not modify an internal parameter in response to the realized episode. On zero-probability histories choose any probability kernel.

At decision 1, q_1 matches the adaptive action conditional. The shared environment then matches the next observation conditional. Suppose the observable joint law through decision n matches. Conditioning on any positive-probability h_n, Equation (69) matches the next action, and the environment matches the next observation. Multiplying these conditionals by the matched history probability establishes the induction step. Thus all observable finite-dimensional distributions match. Corollary 20.2 follows because one compatible mechanism contains mutable policy state and another does not.

For a countable but almost surely finite stopping horizon, the construction applies to every finite prefix consistently. We state the finite version in the main text to avoid imposing additional extension conditions that are irrelevant to finite logged episodes.

27.4 Randomized factorial identification

Theorem 20.3. Let $Z_{rf} = \mathbf{1}\{R = r, F = f\}$ and $p_{rf} = \mathbb{P}(Z_{rf} = 1) > 0$. A Horvitz–Thompson version of the cell mean is

$$\widetilde{\mu}_{rf} = \frac{1}{N} \sum_{i=1}^{N} \frac{Z_{i,rf} Q_i^{\text{obs}}}{p_{rf}}. \tag{70}$$

By consistency, $Z_{i,rf}Q_i^{\text{obs}} = Z_{i,rf}Q_i(r,f)$. Randomization makes $Z_{i,rf}$ independent of $Q_i(r,f)$, hence

$$\mathbb{E}\left[\frac{Z_{i,rf}Q_i^{\text{obs}}}{p_{rf}}\right] = \frac{\mathbb{E}Z_{i,rf}\,\mathbb{E}Q_i(r,f)}{p_{rf}} = \mathbb{E}Q_i(r,f). \tag{71}$$

Averaging proves unbiasedness. Under fixed balanced cell counts, the ordinary within-cell mean has the same conditional expectation. Linearity proves unbiasedness of both contrasts in Equations (49)–(51). If outcomes are missing after assignment, the result applies to intention-to-treat only when missingness is included in the outcome contract or handled under additional stated assumptions.

27.5 Reset isolation

Theorem 20.6 and Corollary 20.7. Let the registered snapshot be $r^\star = (s^\star, n^\star, \kappa^\star, e^\star, H(s^\star))$, and let the two reset operations complete at events q_a and q_b. Completeness of the registered restore map D gives byte-identical post-completion states after reindexing: model and runtime state, Φ, both components of Ξ, context, memory, workspace, RNG, budget, external-service handles, and the pending-callback ledger all equal $s^\star$. Each arm enters the same fresh reset epoch, and any callback carrying an earlier epoch is rejected and recorded rather than applied. By the shared-kernel premise and the deterministic event-order rule, the first events after q_a and q_b therefore have the same law $K_1(\cdot \mid s^\star)$. If their reindexed history distributions agree through event n, integrating the same K_{n+1} against the same history law in Equation (64) matches event $n{+}1$. Induction establishes distributional equality for every post-completion observable. With common random numbers and deterministic kernels, the same argument strengthens to pathwise equality. A detected systematic difference contradicts the conjunction of complete restoration, epoch isolation, shared kernels, deterministic ordering, and no interference; it does not by itself identify which reset premise failed, nor does a null difference certify reset completeness.

27.6 Verifier results

Proposition 20.8. For binary variables, $|V - S|$ is exactly the disagreement indicator. Jensen's inequality for absolute value gives

$$|\mathbb{E}V - \mathbb{E}S| = |\mathbb{E}(V - S)| \le \mathbb{E}|V - S| = \mathbb{P}(V \neq S). \tag{72}$$

For arms a, b, expand the difference between proxy and truth effects and apply the triangle inequality:

$$|\mathbb{E}(V_a - S_a) - \mathbb{E}(V_b - S_b)| \tag{73}$$

$$\le |\mathbb{E}(V_a - S_a)| + |\mathbb{E}(V_b - S_b)| \le \varepsilon_a + \varepsilon_b. \tag{74}$$

Theorem 20.9. Let P_{obs} be the complete observed law. Extend it to model P^+ by defining latent $S := V$ deterministically. Extend the same P_{obs} to P^- by defining $S := 1 - V$. Both extensions integrate back to the identical observed law. For any two arms,

$$\Delta_S^+ = \mathbb{E}[V \mid a] - \mathbb{E}[V \mid b], \tag{75}$$

$$\Delta_S^- = \mathbb{E}[1 - V \mid a] - \mathbb{E}[1 - V \mid b] = -\Delta_S^+. \tag{76}$$

Thus the observed law admits opposing truth-effect signs whenever the proxy effect is nonzero. A calibration, monotonicity, bounded-error, or external-truth assumption would rule out some extensions, but none is assumed in the theorem.

27.7 Optimization identities

Proposition 21.2. Write $J(\phi) = \int R(z)p_\phi(z \mid \mathcal{G})\,\nu(dz)$. The domination/interchange assumption gives

$$\nabla J(\phi) = \int R(z)\nabla p_\phi(z \mid \mathcal{G})\,\nu(dz) \tag{77}$$

$$= \int R(z)p_\phi(z \mid \mathcal{G})\nabla \log p_\phi(z \mid \mathcal{G})\,\nu(dz). \tag{78}$$

Moreover,

$$\int b(\mathcal{G})p_\phi(z \mid \mathcal{G})\nabla \log p_\phi(z \mid \mathcal{G})\,\nu(dz) = b(\mathcal{G})\nabla \int p_\phi\,d\nu = 0. \tag{79}$$

Subtracting this zero term proves the identity. If the action support depends on ϕ, boundary terms may appear; that case is outside the proposition.

Lemma 21.3. If $R_i = r$ for all i, then $\overline{R} = r$, $s_R = 0$, and $A_i = (r - r)/\varepsilon = 0$. Any update $\sum_i A_i g_i$ or scalar multiple thereof is zero. Nonlinear algorithms that add terms not weighted by A_i are not covered and must audit those terms separately.

27.8 Budget, drift, and non-uniform safety

Proposition 21.4. Induct on j. The base $B_1 \geq 0$ is assumed. If $B_j \geq 0$ and admission requires $0 \leq c_j \leq B_j$, then $B_{j+1} = B_j - c_j \geq 0$. Telescoping componentwise yields

$$\sum_{i=1}^{j} c_i = B_1 - B_{j+1} \leq B_1. \tag{80}$$

Theorem 22.1. For each i, Pinsker's inequality gives

$$\mathrm{TV}(\pi_i, \pi_{i-1}) \leq \sqrt{\tfrac{1}{2}\,\mathrm{KL}(\pi_i \| \pi_{i-1})} \leq \sqrt{\kappa_i/2}. \tag{81}$$

The triangle inequality for the total variation metric gives

$$\mathrm{TV}(\pi_K, \pi_0) \leq \sum_{i=1}^{K} \mathrm{TV}(\pi_i, \pi_{i-1}) \leq \sum_{i=1}^{K} \sqrt{\kappa_i/2}. \tag{82}$$

By the variational characterization of total variation, for any $g \in [0,1]$, $|\mathbb{E}_{\pi_K} g - \mathbb{E}_{\pi_0} g| \leq \mathrm{TV}(\pi_K, \pi_0)$. Total variation is at most one.

Theorem 22.2. Distinct probability measures differ on at least one measurable set A. If $\pi'(A) < \pi(A)$, $g = \mathbf{1}_A$ has lower expectation under π'. If $\pi'(A) > \pi(A)$, then $\pi'(A^c) = 1 - \pi'(A) < 1 - \pi(A) = \pi(A^c)$ and $g = \mathbf{1}_{A^c}$ works. The constructed utility may not be relevant to a given task; the theorem rules out uniform improvement over the unrestricted bounded-utility class only.

28 Symbol and State Dictionary

Table 19: **Canonical symbols (DEFINITION).** Symbols are local to this paper unless explicitly shared across the family.

Symbol	Type	Meaning
X, I	$\mathcal{X} \times \mathcal{I}$	Immutable problem and episode identifier.
j, t, n	$\mathbb{N}$	Attempt, within-attempt microstep, and deterministic global event indices; wall-clock time is a recorded attribute, not the causal order.
$\kappa(e) = (g, p, \mathrm{id})$	ordered key	Registered generation, phase priority, and unique identifier used to order complete events lexicographically.
Z_j	$\mathcal{Z}$	Complete marked trajectory for attempt j.
$O_{j,t}, A_{j,t}$	$\mathcal{O}, \mathcal{A}$	Observation available before the next action, and action.
Y_j	$\mathcal{Y}$	Feedback generated from attempt j, with provenance/time.
C_j	$\mathcal{C}$	Prompt and transient context state.
M_j	$\mathcal{M}$	External/resident memory state.
W_j	$\mathcal{W}$	Search tree, files, scratch state, tool/cache workspace.
Φ_j	$\mathcal{P}$	Declared functional policy state mounted for attempt j.
Ξ_j^{read}	bounded typed product	Policy-readable auxiliary closure: every registered route, gate, normalization statistic, adapter selector, or other non-Φ value read by the action kernel.
Ξ_j^{aux}	bounded typed product	Non-readable auxiliary state: optimizer moments, bookkeeping, RNG state, schedules, and other registered carriers not read by the current action kernel.
$\Psi_j = (\Phi_j, \Xi_j^{\mathrm{read}})$	product space	Effective policy carrier. A claim about Φ alone is licensed only when the readable closure is byte-identical across the compared arms.
B_j	$\mathcal{B} \subseteq \mathbb{R}_+^d$	Remaining multidimensional resource budget.
S_n	product space	Complete state after deterministic event n, including reset epoch and pending-callback ledger.
$S_j = \mathsf{S}_{\sigma_j}$	product space	Complete pre-attempt state in Equation (40).
π_ϕ	stochastic kernel	Action/proposal law parameterized by policy state ϕ.

Symbol	Type	Meaning
$U_j, \mathcal{U}_\Theta$	measurable maps	Concrete update and general outer-parameterized update rule.
ρ_j	receipt space	Update inputs/outputs, hashes, deterministic event keys, times, costs, validation and commit.
χ_j^-, χ_j^+	hash strings	Pre-update and post-update hashes of the declared policy-readable closure.
$\mathcal{F}_n, \mathcal{F}_{j,t}$	σ-fields	Causally available information at event and microstep.
σ_j, ϵ_j	stopping times	Attempt j start and end.
v_j, u_j	stopping times	Feedback availability and update completion.
τ	stopping time	Episode resolution, abandonment, budget, or safety endpoint.
e	$\mathbb{N}$	Reset epoch attached to callbacks and events; stale-epoch returns are rejected and logged.
$r^\star$	registered record	Complete reset snapshot $(s^\star, n^\star, \kappa^\star, e^\star, H(s^\star))$: bytes, event index/key, epoch, and state hash.
D, q	map, event index	Registered complete restore map and the event at which restoration, draining/cancellation, and fresh-epoch activation finish.
$\mathfrak{W}_j$	product record	Strict-witness bundle in Equation (46).
R, F	$\{0, 1\}$	Mounted-state and feedback-semantic randomized factors.
$Q(r, f)$	$[0, 1]$	Potential qualification outcome under factorial arm (r, f).
θ_{state}	$[-1, 1]$	True-feedback post-state versus pre-state effect.
θ_{int}	$[-2, 2]$	State-by-feedback semantic interaction.
S, V	$\{0, 1\}$	True success and proxy verifier decision.
TV, KL	divergences	Total variation and Kullback–Leibler divergence.
Θ	outer parameter	Meta-learned or engineered update-rule parameters.
$\mathcal{J}(\Theta)$	$\mathbb{R}$	Complete episode-level quality–cost–risk objective.

28.1 Reserved terminology

Attempt. A bounded, version-receipted policy invocation intended to advance or solve the current problem; internal token steps are not automatically separate attempts.

Feedback. A causally available record derived from an attempt or environment interaction. Its presence does not imply truth.

Policy state. The declared carrier Φ that directly parameterizes the fresh action/proposal kernel. Policy-only attribution additionally requires a byte-identical Ξ^{read} closure; otherwise the intervened functional carrier is $\Psi = (\Phi, \Xi^{\text{read}})$.

Memory state. A typed store read through a memory/retrieval path; mutation alone is RTMA, not strict RTT.

Workspace. Candidate-specific search and tool state, including trees, caches, files, and scratch computations.

Resolution. The registered terminal event for success, abandonment, budget exhaustion, or safety stop.

Strict RTT. A complete W1–W8 functional update–reuse witness, without an implied benefit sign.

RTT benefit. A prespecified causal outcome contrast; separate from mechanism identity.

29 Printable Leakage, Circularity, and Claim Audits

29.1 Carrier/reset matrix

Table 20: **Hidden-state and reset matrix (DESIGN PRESCRIPTION).** Each future experiment must replace "declare" with a concrete receipt.

Carrier	Hash/version	Restore or randomize	Failure interpretation
Base weights and adapters	Content hash, mount route, dtype	Swap only registered policy factor	Wrong or bundled intervention
Declared policy carrier Φ	Content hash, mount route, dtype	Swap only the registered policy factor	Wrong, dead, or bundled intervention
Policy-readable closure Ξ^{read}	Typed manifest and pre/post closure hashes χ^-, χ^+	Hold byte-identical for a Φ-only contrast, or intervene on Ψ	Unregistered functional carrier or invalid Φ attribution

Carrier	Hash/version	Restore or randomize	Failure interpretation
Non-readable auxiliary state Ξ^{aux}	Moments, step, scaler, recurrent code, scheduler	Restore with candidate/pre-state	Hidden state or reset failure
Normalization/routing/gates	Running statistics and realized decisions	Register in Ξ^{read} when kernel-readable; otherwise fix	Dead/bypassed or misclassified update
Prompt/context/KV state	Serialized prompt and cache receipt	Restore before fresh proposal	Context confounding
External/resident memory	Rows, versions, index, retrieval seed	Restore or independent memory factor	Memory confounding
Search/workspace/files	Tree, node scores, files, cache keys	Restore before proposal swap	Search/workspace confounding
Tools, services, and callbacks	Session IDs, remote cache/version, pending ledger, epoch tags	Drain/cancel before completion; reject and log stale-epoch returns	Environment interference or incomplete reset
Randomness and batching	All RNG states, batch peers, scheduler	Common random numbers or assignment	Stochastic imbalance
Event order and reset epoch	Lexicographic keys $\kappa(e)$, event log, epoch transitions	Replay the registered order; activate one fresh epoch at completion	Ambiguous causality, stale mutation, or non-replayable reset
Verifier/reference	Model/data/config hash, provenance	Freeze before target outcomes	Reward circularity
Budget and stop logic	Ledger hash, thresholds, timers	Equal rules across arms	Differential exposure

29.2 Future-leakage checklist

L1. Is every update input admitted under a deterministic key no later than u_j?

L2. Was any later success, hidden test, future tool return, or final answer used to choose an update, checkpoint, learning rate, or gate?

L3. Were failed candidates omitted after their later outcomes were known?

L4. Does a post-u_j random canary predict the committed delta above the registered tolerance?

L5. Were outer-training realized futures physically and logically separated from deployment artifacts?

L6. Does every complete event carry a unique registered key $\kappa(e)$, and does the log follow its deterministic lexicographic order independently of wall-clock arrival time?

L7. Are reset snapshot, restore completion, fresh epoch, pending-callback disposition, and every rejected stale-epoch return receipted?

L8. Are all target-outcome-dependent revisions recorded as a new protocol rather than silently folded into the old one?

Any "yes" to L2–L4 or L8, or an unresolved answer to L1/L5–L7, blocks a deployed causal-update claim. A clean callback ledger is necessary but does not by itself certify that the registered reset map was complete.

29.3 Circularity checklist

C1. Is the feedback producer independent of the policy family? If not, is it explicitly labeled a proxy?

C2. Is true success externally defined and audited on a disjoint sample?

C3. Were reward, state divergence, and final utility reported separately?

C4. Were all arms, including stopped and failed updates, included in intention-to-treat reporting?

C5. Were hyperparameters and stop thresholds frozen before target-arm inspection?

C6. Was mechanism qualification performed irrespective of outcome success?

C7. Can the verifier be optimized without task progress (reward hacking), and are adversarial cases included?

29.4 Manuscript claim audit

Claim class	Strongest admitted basis	Explicit exclusion
Strict RTT vocabulary	Authorial definitions and typed interfaces	No priority claim
Separation results	Logical consequences of W1–W8	No performance ranking
Identification results	Randomization/intervention assumptions	No guarantee assumptions hold in a system
Optimization results	Differentiability/support/group assumptions	No finite-step improvement
Safety results	Local KL and bounded-utility assumptions	No global or deployment safety
Related-work map	Verified primary-source protocol descriptions	No replication or independent result validation
MMLA boundary	Public V3 and complete technical-report evidence records	No RTT evidence; no repair of closed overwrite gate
Empirical ladder	Planned protocols and stop rules	No gate executed or passed

Table 21: **Claim–evidence audit.** The strongest paper-level claim is a proved conditional statement or a source-verified protocol description; no empirical RTT success is admitted.

Part II
Atomic Memory Rows

Branch status. This part imports the complete substantive formal branch from the frozen R01 focus paper. It defines a bounded complete-row authority and transaction contract under explicit assumptions; it contributes no new experiment and does not establish a qualified semantic-memory interface, useful neural readout, learned admission policy, safety result, or hardware advantage.

30 Introduction

30.1 The state problem hidden by the word memory

A transcript can retain an observation without deciding whether it is current. A vector store can retrieve neighbors without identifying which record is authoritative. A differentiable memory matrix can change continuously without specifying deletion, provenance, tenant scope, or a version boundary. An append-only audit stream can preserve history while growing without limit. These are useful objects, but none alone is a bounded editable fact state.

The missing object is a complete unit of authority. Suppose a resident record says that Alice is in Paris and a later observation proposes Berlin. Before "write Berlin" is a defined operation, a system must answer at least the following questions. Which logical object and physical slot are involved? Which tenant and principals may read or modify it? Does the proposal replace an entire version or blend with the old value? Are the canonical and neural representations synchronized? Which source and software builds produced the candidate? Does the version advance without wraparound? What happens to stale index entries? Is declining the write distinguishable from deletion? Can rollback restore Paris without restoring obsolete permissions? Which resident, index, archive, audit, and durable-write bytes are charged?

AMR answers those questions at the substrate level. It does not decide whether Berlin is true or useful, and it does not decide whether the proposal deserves admission. The distinction between state correctness and decision quality is the paper's central evidence boundary.

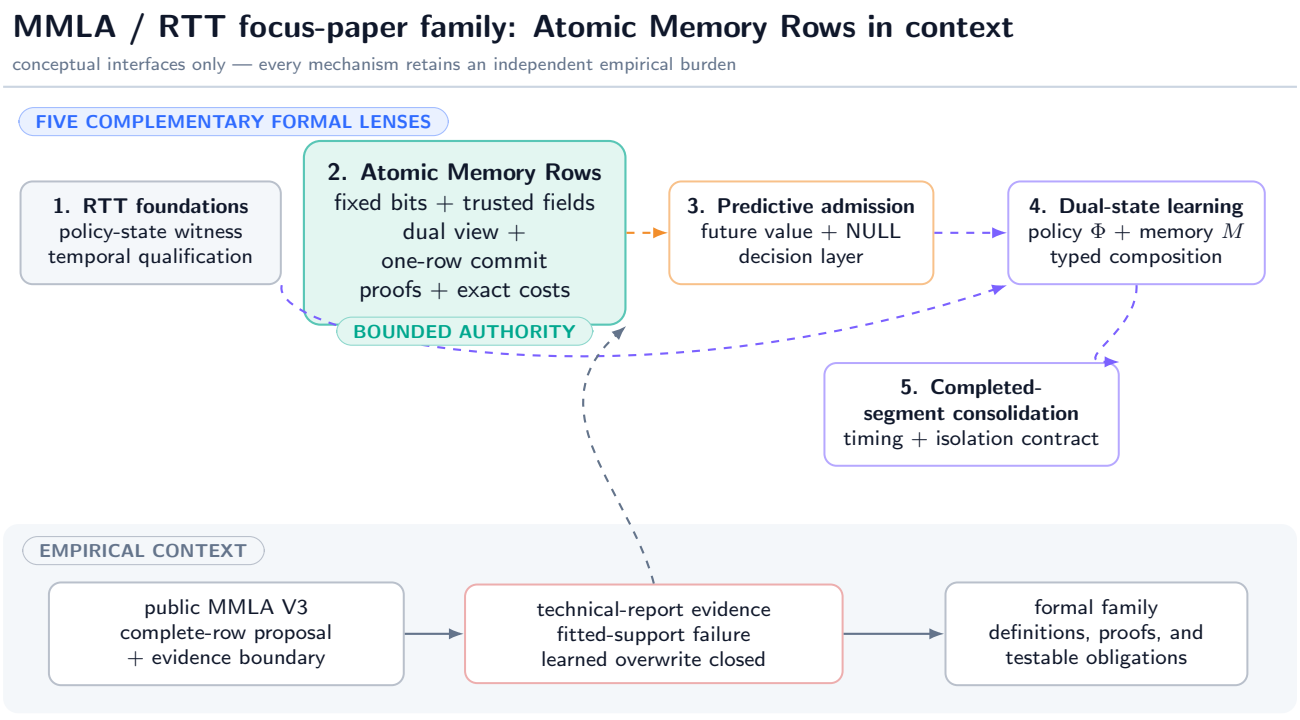

Figure 25: **Atomic rows occupy the bounded substrate layer of a five-paper conceptual family.** Dependency arrows organize formal interfaces; they do not transfer empirical validation. The semantic-interface and learned-overwrite gates remain closed.

30.2 Authority is narrower than storage

DEFINITION

Definition 30.1 (Authoritative state)**.** A mutable component is *authoritative* if changing it can change the system's declared current resident object, value, deletion status, security decision, or permitted transition without first validating against a current row. The authoritative resident memory is an ordered table

$$M = (r_1, \ldots, r_S) \in \mathcal{R}_\sigma^S, \tag{83}$$

where $S < \infty$ and schema σ fixes the exact row serialization length B_R bytes.

An alias map that can redirect a query without revalidating the current row is authoritative. So is a security map that can grant access independently of the row. Both are outside the contract unless included in the same fixed budget and commit boundary. By contrast, an index entry is a non-authoritative hint when every hit is rechecked against the current row's tenant, identity, slot sequence, status, and receipt.

We distinguish five state classes.

1. **Transient context** contains tokens, activations, and scratch reasoning for the current computation.
2. **Authoritative resident memory** is the fixed table M and is the only resident state permitted to declare a live, deleted, protected, or quarantined object.
3. **Deterministic derived views** include exact key/alias and neural-routing indexes. They are rebuildable functions of validated rows and never grant authority.
4. **External archives and audit ledgers** may retain source documents, superseded values, or receipts. Their storage and retention policies are separately charged; they are not hidden inside the resident bound.
5. **Model or policy state** includes fixed parameters and any separately typed fast or trainable state. Its mutation is not an AMR row operation.

DESIGN PRESCRIPTION Every implementation must publish a state-carrier inventory assigning each mutable byte to exactly one class, with its capacity, lifetime, authority, and update rule. "Cache," "metadata," or "log" is not an exemption from accounting.

30.3 Contributions and scope

This manuscript owns the following formal and systems-contract questions:

- a typed, fixed-width authoritative row and a reconciled concrete profile;
- neural-proposed versus trusted system-owned fields;
- canonical/neural conformance, receipts, re-encoding, and fail-closed mismatch behavior;
- one complete-row commit or bit-identical NULL per ordered event;
- insert, update, consolidating update, eviction, alias rename, deletion, rollback, quarantine, repair, and purge semantics;
- per-event and segment-level edit-locality bounds;
- deterministic derived indexes, stale-entry suppression, tenant partitioning, ACLs, protection, provenance, tombstones, and bounded rollback;
- exact resident, index, candidate, validation, publication, archive, and audit accounting;
- invariant, concurrency, crash-recovery, impossibility, and counterexample results under explicit assumptions; and
- a theorem-to-implementation obligation graph and falsification ladder.

It does not own the event extractor, learned target ranker, predictive admission objective, future-valued controller, archive-retrieval policy, general multi-row transactions, distributed consensus, or end-to-end language-model quality. Those components can consume or produce AMR objects only after their own evidence gates are satisfied.

30.4 Relationship to frozen MMLA evidence

Public MMLA V3 proposed a bounded resident table, a canonical/neural successor row, a trusted assembly boundary, and one overwrite or NULL per event [125]. The subsequent technical-report evidence record sharpened the fixed-bit, event-recursive, dual-view, and complete-cost contract and admitted a stronger negative semantic-interface diagnostic. This paper isolates and extends the substrate mathematics. It does not reinterpret either source as validating AMR.

The empirical ceiling is exact. PM-I2 qualified zero of nine dense, structured-span, and checkpoint-native jobs. Its structural checks passed while semantic prediction failed. A later fitted-population diagnostic reported direct candidate reconstruction 40/73,728, learned route/learned content 0/73,728, oracle route/learned content 0/73,728, learned route/oracle content 18,544/73,728, and the mechanical full oracle 73,728/73,728. These numbers show that target-row isolation did not repair

learned content. They do not reject the whole MMLA architecture, and they do not evaluate the fixed-row profile introduced here.

UNVALIDATED No trained checkpoint has been shown here to read an AMR neural capsule semantically, construct a useful complete row, learn a safe admission policy, or improve reasoning. No crash-injection, security, latency, energy, or hardware experiment is reported.

Formal statements carry their assumptions locally. Definition, proof, prescription, and planned-test labels identify the kind of support offered; none is evidence that a checkpoint can use the row. The next sections specify the row schema and exact budget, separate proposed from trusted fields, define dual-view consistency and the transition algebra, then develop lifecycle, invariants, security, recovery, costs, counterexamples, and falsification tests. Whether these bytes improve a reasoner's behavior remains open.

31 Typed State and Complete-Row Schema

31.1 Finite universes and schema parameters

Fix a schema identifier σ and finite-width spaces

$$\begin{aligned} &\mathcal{S} = \{1, \dots, S\}, \quad \mathcal{T} = \{0,1\}^{b_T}, \quad \mathcal{U} = \{0,1\}^{b_U}, \\ &\mathcal{O} = \{0,1\}^{b_O}, \quad \mathcal{V}_s = \{0, \dots, 2^{b_s} - 1\}, \quad \mathcal{V}_o = \{0, \dots, 2^{b_o} - 1\}. \end{aligned} \tag{84}$$

They denote physical slots, tenants, principals, logical objects, physical-slot sequences, and same-object versions. A trusted static partition

$$P : \mathcal{S} \to \mathcal{T} \tag{85}$$

assigns each slot to exactly one tenant. Dynamic repartitioning is an administrative storage migration with its own transaction; it is not an ordinary memory event.

Schema σ fixes at least the following finite parameters: canonical alphabet and normalization, key and value lengths, alias count and alias length, neural dimensions and precision, ACL capacity, identifier widths, rollback depth, validation bitmap width, digest and authentication algorithms, time representation, status normal forms, and the exact byte order of every field. It also fixes a finite validation registry

$$\mathcal{V}_\sigma : b \longmapsto (\text{predicate-id}, \text{version}, \text{applicability}, \text{fail-mode}),$$

whose digest $d_\mathcal{V} = H(\mathrm{Ser}(\mathcal{V}_\sigma))$ is receipt-covered. An unknown bit, version, or applicability code fails closed. A schema or registry change is a migration, not an in-place reinterpretation.

DEFINITION

Definition 31.1 (Status)**.** Every parsed row has one status

$$z(r) \in \{\mathsf{EMPTY}, \mathsf{LIVE}, \mathsf{TOMBSTONE}, \mathsf{QUARANTINED}\}. \tag{86}$$

An EMPTY row has no logical object but retains physical slot, tenant, slot sequence, and a valid empty-row receipt. A LIVE row may supply content to an authorized reader. A TOMBSTONE row records a committed deletion and is unavailable to ordinary semantic reads. A QUARANTINED row records a bounded integrity failure and is unavailable until an authorized repair commits a new complete version.

EMPTY is not deletion, TOMBSTONE is not NULL, and QUARANTINED does not assert that the logical object was deleted.

31.2 The row product type

DEFINITION

Definition 31.2 (Atomic Memory Row)**.** An Atomic Memory Row is the product

$$r = (h, c, n, p, \lambda, \rho) \in \mathcal{H}_\sigma \times \mathcal{C}_\sigma \times \mathcal{N}_\sigma \times \mathcal{P}_\sigma \times \mathcal{L}_\sigma \times \mathcal{Q}_\sigma, \tag{87}$$

where h is the trusted control block, c the canonical capsule, n the neural capsule, p the provenance capsule, λ the bounded rollback lineage, and ρ the consistency receipt. Schema σ defines a deterministic serializer and partial parser

$$\mathrm{Ser}_\sigma(r) \in \{0,1\}^{8B_R}, \qquad \mathrm{Parse}_\sigma : \{0,1\}^{8B_R} \to \mathcal{R}_\sigma \cup \{\bot\}. \tag{88}$$

The row is the publication unit: no field in Eq. 87 is authoritative on a different commit boundary.

The serializer is injective over valid rows. Lengths, endianness, normalization, padding, ordering, floating or quantized numeric semantics, reserved-bit values, and prohibited encodings are part of σ. Deterministic encoding recommendations such as CBOR's deterministic profiles are useful background [10], but AMR requires one schema-specific bit string rather than merely equivalent decoded maps.

31.3 Trusted control block

The control block is

$$h = (\text{magic}, \sigma, z, \text{committed-op}, \text{flags}, a, t, o, u, \tau, \\ v_s, v_o, t_{\min}, t_{\max}, t_{\text{retain}}, \pi, \delta, \text{ACL}, q_{\text{rb}}), \tag{89}$$

where a is physical slot, t tenant, o object, u owner, τ object type, v_s slot sequence, v_o object version, the three times encode validity and retention, π is protection class, δ a deletion-reason code, and q_{rb} describes rollback-ring occupancy. The ACL is a fixed array of principal/right pairs. Its empty entries have a unique zero representation.

The slot sequence advances on every committed replacement of a physical slot. The object version advances when the same logical object remains resident. Insertion of a different object starts its object version at one while still advancing the slot sequence. This two-counter design makes stale physical pointers distinguishable from a new object that reuses the same slot.

31.4 Canonical capsule

The canonical capsule is

$$c = (\tau_c, k_c, v_c, A_c, f_c), \tag{90}$$

where τ_c is a type tag, k_c a typed key, v_c a typed value or canonical tuple, A_c an ordered bounded alias array, and f_c semantic flags. The reference profile places the authoritative type and flags in the trusted control block while the canonical byte region contains the normalized key, value, and aliases. Their joint interpretation is nevertheless one canonical capsule.

Canonical means inspectable and deterministically serialized, not factually true. Unicode normalization, case handling, locale, units, temporal interpretation, numeric canonicalization, and missing-value semantics are fixed by schema. A canonical `0` is data with a nonzero type/length encoding. Padding bytes are not content.

Aliases are alternate query handles for the same row identity. They do not create extra logical objects or independently mutable mappings. Alias rename is a complete same-object row update, and every derived alias-index hit is revalidated against the current row.

31.5 Neural capsule

The neural capsule is

$$n = (k_n, v_n, u_n, \alpha_n, \widetilde{\iota}), \tag{91}$$

where k_n is a fixed-dimensional routing vector, v_n a fixed-dimensional semantic payload, u_n bounded uncertainty metadata, α_n a bounded abstention score, and $\widetilde{\iota}$ proposed operation intent. The schema chooses a finite coordinate representation. AMR-1 uses signed 16-bit coordinates. A floating-point profile must fix byte order, rounding, negative zero, infinities, denormals, and NaNs; unrestricted NaN payloads are prohibited because bitwise equality and receipt hashing would otherwise be noncanonical.

The neural capsule is not authoritative by itself. A matching vector cannot override canonical content, tenant scope, a tombstone, or a failed receipt. A separately typed differentiable fast state may use soft reads and writes, but Section 35 prevents it from silently becoming the versioned fact table.

31.6 Provenance, rollback, and receipt

The provenance capsule records bounded identifiers or digests for the event, source material, proposer/model build, assembler, encoder, validator, observation time, source class, committed operation, and deletion authorization. Provenance concepts distinguish where a datum came from and how it was transformed [12]; the AMR capsule intentionally stores only a finite receipt, not an unbounded derivation graph.

PROVED UNDER STATED ASSUMPTIONS Under the cryptographic assumptions in Section 37.1, provenance can bind declared source and build identifiers to committed bytes. It cannot establish that the source was truthful, that the validator was competent, or that the neural representation is useful.

Rollback lineage λ is a ring of at most H_{rb} fixed-width same-object snapshots. A snapshot contains the canonical and neural semantic state plus bounded prior version, validity, protection, and digest metadata, but contains no nested rollback lineage. A different object evicted from a slot is not placed in the new object's rollback ring. Recovering it requires a separately charged archive or a new source event.

Let $\text{core}(r)$ be every serialized row byte except the receipt. A predecessor is not an untyped digest. Schema σ defines the disjoint normal forms

$$\text{pred}_{\text{gen}}(\eta, a), \qquad \text{pred}_{\text{live}}(\eta, a, v_s, d_{\text{core}}), \qquad \text{pred}_{\text{migrate}}(\eta_s, a_s, v_s, d_s; \eta_d, a_d),$$

where η is a non-reused namespace epoch. Domain tags make genesis, ordinary replacement, and migration byte-distinct; no all-zero string is interpreted as a predecessor. For a hash H and authentication scheme Tag_K, define

$$d_{\text{core}} = H(\texttt{AMR-CORE} \parallel \text{core}(r)), \tag{92}$$

and

$$\rho = (d_{\text{core}}, \text{pred}, d_{\mathcal{V}}, b_{\text{val}}, \text{alg-ids}, \text{auth-bind}, \text{event-bind}, \text{Tag}_K(m), \text{reserved}), \tag{93}$$

where

$$m = H(\texttt{AMR-RECEIPT} \parallel \text{schema-id} \parallel d_{\text{core}} \parallel \text{pred} \parallel d_{\mathcal{V}} \parallel b_{\text{val}} \parallel \text{alg-ids} \parallel \text{auth-bind} \parallel \text{event-bind} \parallel \text{assembler-id}). \tag{94}$$

The digest is the row's cryptographic checksum; the authentication tag may be a MAC such as HMAC or a signature according to the trust model [72, 71]. The receipt authenticates bytes and declared checks under stated assumptions. It is not a zero-knowledge proof, an attestation that the source is true, or evidence that the validator implements the intended semantics.

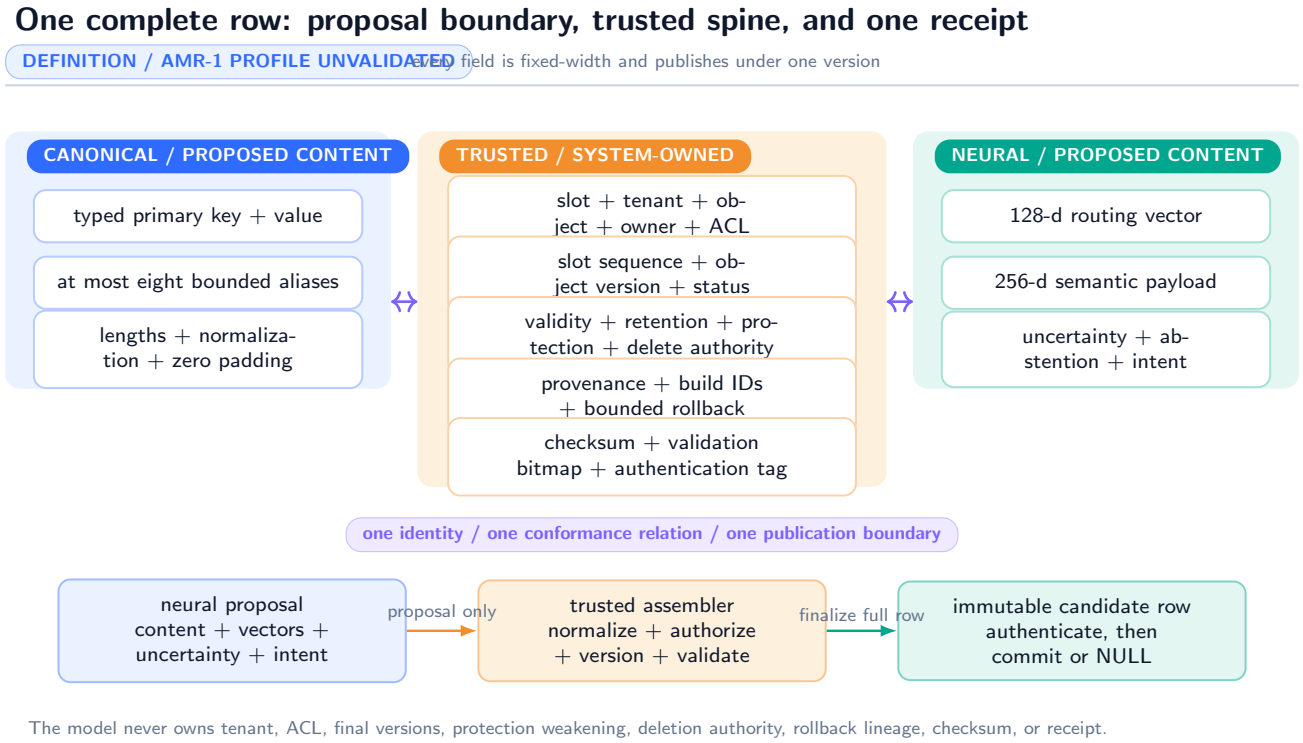

Figure 26: **Complete-row anatomy and ownership.** The figure defines one authoritative serialization. Neural components may propose bounded content, vectors, uncertainty, and intent; trusted code fixes identity, security, versions, policy, lineage, deletion authorization, validation state, and the authenticated receipt. Both views and every control field publish together or none becomes authoritative.

31.7 Normal forms and missing values

Status normal forms eliminate ambiguous interpretations.

- EMPTY has zero logical object, zero active canonical/neural regions, tenant and slot binding, current slot sequence, and a valid receipt.
- LIVE has a nonzero object, a well-formed canonical capsule, a conforming neural capsule, and readable status subject to current policy.
- TOMBSTONE retains object identity, versions, deletion authorization, retention, and receipt while active value and read payload regions take the schema's tombstone normal form.
- QUARANTINED retains only the trusted recoverable identity and bounded failure metadata permitted by policy; its active content is unavailable.

A schema-level optional value is encoded with an explicit presence bit or type tag. It is never inferred from an all-zero payload. NULL is an action, not a nullable value. This distinction becomes formal in Section 35.

32 Exact Bit and Resource Budgets

32.1 A symbolic finite contract

Let a schema allocate byte widths

$$B_R = B_h + B_c + B_n + B_p + B_\lambda + B_\rho. \tag{95}$$

Exactly S rows therefore consume

$$B_{\text{auth}} = SB_R \text{ bytes}, \qquad \text{bits}(M) = 8SB_R. \tag{96}$$

This is operational only if every summand is finite and includes the fields required to interpret and authorize a row. "S objects" is not a bound when an object can carry an unbounded alias list, provenance graph, source text, or rollback history.

Let B_I be fixed materialized-index capacity, B_Q a bounded diagnostic quarantine buffer, B_F separately typed non-authoritative fast state, $B_A(t)$ archive bytes retained at time t, and $B_L(t)$ audit-ledger bytes. Then

$$B_{\text{hot}} = SB_R + B_I + B_Q + B_F, \tag{97}$$

while

$$B_{\text{system}}(t) = B_{\text{hot}} + B_A(t) + B_L(t) + B_{\text{replica}}(t) + B_{\text{fs}}(t). \tag{98}$$

The resident contract bounds Eq. 97. A deployment may also cap every term in Eq. 98; otherwise it must say that total retained storage is unbounded. A finite row digest that points to an ever-growing archive does not make the archive finite.

32.2 AMR-1: a reconciled concrete profile

DESIGN PRESCRIPTION AMR-1 is an illustrative auditable profile, not an optimized standard and not an implemented format.

Block	Exact allocation	Bytes
Trusted control	identity, status, operation, versions, times, protection, fixed ACL, rollback pointers	256
Canonical capsule	key, value, eight aliases, lengths, deterministic padding	1,104
Neural capsule	128-dimensional route, 256-dimensional payload, uncertainty, abstention, intent	784
Provenance	event/source/model/build/time/deletion-authorization identifiers and digests	192
Rollback	two nonrecursive, same-object snapshots of 2,048 bytes each	4,096
Receipt	core/predecessor digests, validation bitmap, algorithms, event binding, auth tag	160
Complete row	one fixed serialization and publication unit	**6,592**

Table 22: AMR-1 exact top-level row allocation. Appendix 44 reconciles every subfield.

Thus

$$B_R = 6{,}592 \text{ bytes} = 52{,}736 \text{ bits}. \tag{99}$$

For $S = 256$,

$$B_{\text{auth}} = 256 \times 6{,}592 = 1{,}687{,}552 \text{ bytes} = 1.609375 \text{ MiB}. \tag{100}$$

The count excludes proposal buffers, model state, archives, filesystem metadata, allocation overhead, replicas, and diagnostic logs. Exclusion is not disappearance: Section 39 charges each separately.

32.3 Fixed deterministic base indexes

AMR-1 defines two optional, materialized, slot-major indexes. The canonical index stores one 112-byte entry for the primary key and each of eight aliases in every slot:

$$|I_C| = 256 \times 9 \times 112 = 258{,}048 \text{ bytes}. \tag{101}$$

The neural routing index stores one 336-byte entry per slot:

$$|I_N| = 256 \times 336 = 86{,}016 \text{ bytes}. \tag{102}$$

Hence

$$B_I = 344{,}064 \text{ bytes}, \qquad B_{\text{auth}} + B_I = 2{,}031{,}616 \text{ bytes} = 1.9375 \text{ MiB}. \tag{103}$$

Each index entry carries tenant, object, slot, slot sequence, and receipt digest. A canonical entry additionally carries a key digest and alias ordinal; a neural entry carries the quantized routing vector. Index capacity cannot overflow because every slot owns a fixed projection. Optional trees, graphs, approximate-nearest-neighbor structures, and learned routers require separate caps and miss accounting.

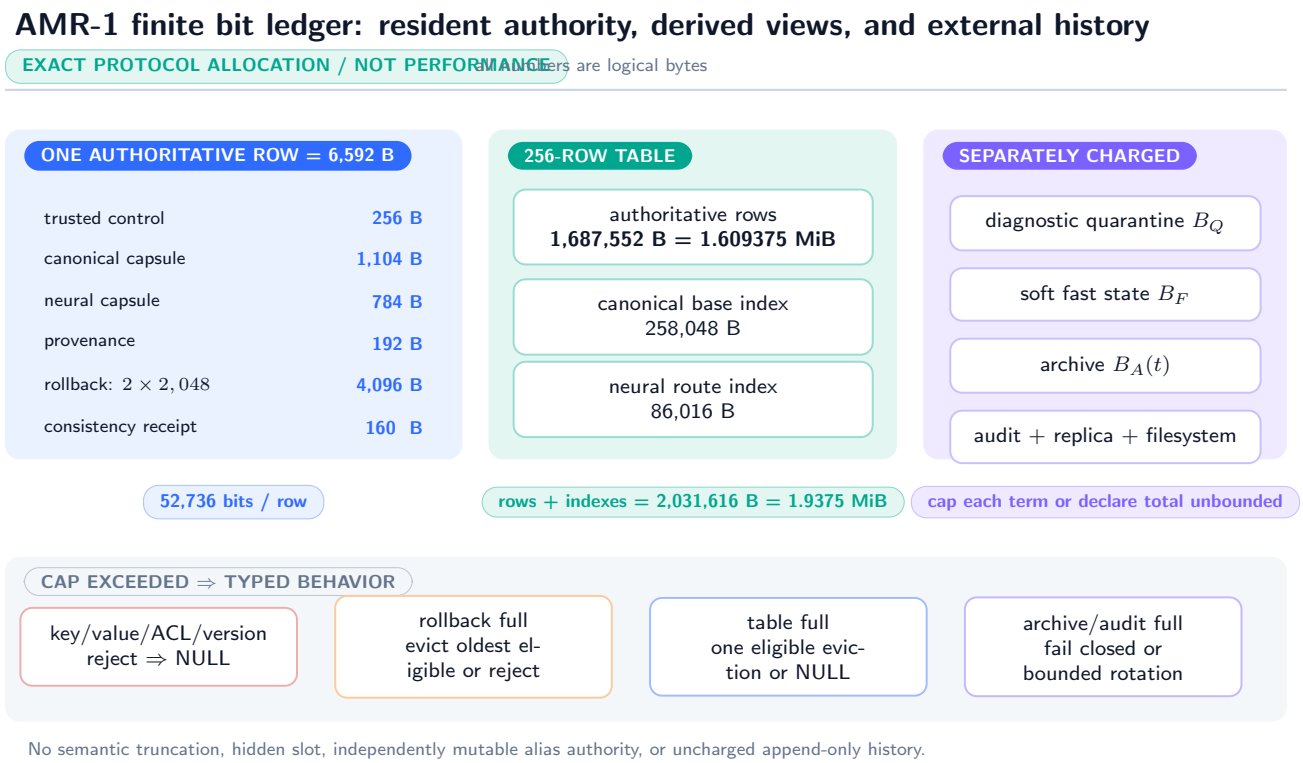

Figure 27: **Finite resident accounting versus separately charged history.** AMR-1 fixes every authoritative row and base-index byte. Quarantine and fast state require explicit caps. Archive, audit, replication, and filesystem terms are either independently bounded or declared unbounded; they never disappear behind the resident-table notation.

32.4 Overflow and cap semantics

DESIGN PRESCRIPTION Every cap has a typed action. Silent allocation, silent semantic truncation, and an implicit spill that changes authority are prohibited.

Exceeded resource	AMR-1 action	Reason
Canonical key/value length	reject candidate; event returns NULL	Truncation can change object identity or meaning.
Alias count or alias length	strict profile rejects; a different schema may drop only explicitly nonessential aliases and must receipt the loss	Alias loss must never be silent or alter the primary key.
Neural dimension/precision	reject candidate	Reallocation would change schema and digest interpretation.
ACL capacity	reject ACL mutation or use an explicitly bounded external capability system whose decision is re-bound into the row	Silent principal loss can grant or deny access.
Provenance document	store only fixed digest/identifier; optional source document spills to separately charged archive	The row never embeds an unbounded provenance graph.
Rollback ring	deterministically evict the oldest eligible snapshot; if policy forbids loss, reject the new commit	History loss is explicit and bounded.
Resident table capacity	choose one eligible eviction target and replace it completely; otherwise NULL	A hidden extra slot would violate the table bound.
Version counter	return NULL with F_VERSION_EXHAUSTED; external re-key/migration required	Counters never wrap, saturate, or reuse an identity/version pair.
Base index	impossible under the fixed slot-major layout; corrupt indexes are discarded and rebuilt	Derived views cannot force an authoritative mutation.
Optional index	evict/rebuild/disable within its own cap; never accept an unvalidated hit	Index failure may hurt recall or latency, not authority.
Quarantine diagnostics	drop non-authoritative diagnostic detail under a bounded policy; resident reads still fail closed	Diagnostic capacity cannot make corrupt content readable.
Required archive receipt	if the archive cap is full, reject commits whose policy requires archival retention	A required durability/provenance premise cannot be silently weakened.
Required audit ledger	fail closed before publication, or rotate through an authenticated bounded checkpoint protocol	An unaudited commit is not equivalent to an audited one.

Table 23: Cap behavior. NULL is the authoritative result of precommit rejection; diagnostic reasons live in a separately bounded channel.

32.5 Finite capacity theorem

PROVED UNDER STATED ASSUMPTIONS

Theorem 32.1 (Fixed information capacity)**.** *For a schema with S rows of exactly $8B_R$ bits,*

$$|\mathcal{M}| \le 2^{8SB_R}. \tag{104}$$

For every random history $X_{\le t}$ and resulting authoritative memory M_t,

$$H(M_t) \le 8SB_R, \qquad I(M_t; X_{\le t}) \le 8SB_R. \tag{105}$$

Proof. The complete table is a bit string of exactly $8SB_R$ bits, although only a subset may parse as valid. It therefore has at most 2^{8SB_R} possible values and entropy at most $8SB_R$. Mutual information is bounded by the entropy of either argument. The theorem concerns authoritative resident state only; adding external archives increases the corresponding system capacity. □

The theorem does not say that all bits are efficiently used, that the retained information is predictive, or that a language model can read it.

33 Neural Proposal and Trusted Assembly

33.1 The proposal is not a row

Let a neural constructor receive a completed event e, bounded context summary z, target slot a, prior row r_a, and a read-only memory snapshot M. Its output is

$$\widetilde{p} = G_\phi(z, e, a, r_a, M) = (\widetilde{c}, \widetilde{n}, \widetilde{\iota}, \widetilde{q}, \widetilde{s}), \tag{106}$$

where $\widetilde{c}$ is proposed canonical content, $\widetilde{n}$ a proposed neural capsule, $\widetilde{\iota}$ operation intent, $\widetilde{q}$ uncertainty/abstention, and $\widetilde{s}$ bounded source handles. This object is untrusted and may be malformed, adversarial, stale, or semantically wrong.

The trusted assembler is a partial function

$$\mathrm{Assemble}_\sigma(M, a, \widetilde{p}, \chi) \in \mathcal{R}_\sigma \cup \{\bot\}, \tag{107}$$

where authenticated context χ binds tenant, principal, event identifier, source-authentication result, and snapshot epoch. Authority is carried by a typed capability

$$\mathsf{cap} = (\text{principal}, \text{tenant}, \text{scope}, \text{rights}, \eta, t_{\min}, t_{\max}, \text{nonce}, \text{issuer}, \text{tag}),$$

and a trusted verifier $\text{Auth}_\sigma(\mathsf{cap}, \chi, e, M) \in \{\mathsf{PASS}, \mathsf{FAIL}\}$. The verifier checks issuer trust, tag, tenant/scope/right, epoch, time, nonce/revocation state, and the event target. The neural proposer neither emits nor extends cap; it may only request an operation. Returning $\perp$ means that no authoritative candidate exists; the event resolves to NULL.

33.2 Field ownership

DESIGN PRESCRIPTION

Field family	Neural role	Trusted/system role
Canonical key, value, aliases, type	propose bounded content	normalize, type-check, resolve identity, enforce caps and uniqueness
Routing vector and semantic payload	propose or request deterministic encoding	encode/re-encode or validate exact conformance; freeze encoder identity
Uncertainty, abstention, requested operation	propose signals and intent	interpret under fixed policy; choose committed operation/status
Target slot	score or propose candidate target	verify partition, occupancy, eviction eligibility, and snapshot freshness
Tenant, owner, ACL	no authority	authenticate and assemble; require explicit capability for ACL change
Object identity	suggest reference only	allocate or resolve stable identifier and same-object relation
Slot sequence and object version	no authority	increment exactly; reject stale snapshot or overflow
Validity and retention	suggest evidence times only	map to policy-approved intervals and legal holds
Protection	request change only	inherit/strengthen; require capability to weaken
Deletion/tombstone	request intent only	verify delete/resurrection authority and deletion receipt
Provenance	provide bounded source handles	authenticate sources; bind event and build identifiers/digests
Rollback lineage	no authority	push eligible same-object snapshot; apply bounded eviction policy
Checksum, validation bitmap, auth tag	no authority	compute after every check over the final serialized core

Table 24: Field ownership. A neural proposal cannot grant itself security scope, reset lineage, weaken protection, authorize deletion, or authenticate its own bytes.

System ownership is functional, not merely documentary. If a model can choose an arbitrary bit string that the assembler copies into the tenant or version field, then the model owns that field regardless of the API name.

33.3 Assembler sequence

For one target-conditioned proposal, the reference assembler executes:

1. authenticate χ, verify the typed capability with Auth_σ, and bind its decision digest to one snapshot epoch;
2. verify $P(a) = \chi$.tenant and target feasibility;
3. parse the requested operation and resolve whether the old and proposed records represent the same logical object;
4. normalize canonical bytes and enforce every key, value, alias, type, and padding bound;
5. derive or validate the neural capsule under the declared encoder/validator profile;
6. evaluate ACL, protection, validity, retention, deletion, resurrection, eviction, and legal-hold policies;
7. compute the next slot sequence and object version, rejecting stale inputs and overflow;
8. assemble fixed provenance and update the bounded same-object rollback ring;
9. serialize the complete core in canonical order;
10. evaluate row-local and memory-global invariants against the same snapshot;
11. compute the checksum, typed predecessor binding, registry digest, validation bitmap, authorization-decision binding, algorithm identifiers, and authentication tag; and
12. return one immutable candidate together with a compare-and-swap precondition.

Any failure returns a typed diagnostic outside authoritative state and no candidate. Candidate validation is not a menu from which the controller may select a partially checked row.

The legal-event predicate is therefore noncircular. For snapshot M, event e, context χ, target a, and proposed replacement r', define

$$\begin{aligned}\mathrm{Legal}_\sigma(M,e,\chi,a,r') \iff\ & \mathrm{Parse}_\sigma(e,\chi,r') \wedge \mathrm{Target}_\sigma(M,e,a) \wedge \mathrm{Auth}_\sigma(\mathsf{cap},\chi,e,M) = \mathsf{PASS} \\ & \wedge \mathrm{Pred}_\sigma(M,a,r') \wedge \mathrm{Assembly}_\sigma(M,e,\chi,a,r') \wedge \mathrm{Local}_\sigma(r',\chi) \wedge \mathrm{Global}_\sigma(M,a,r') \\ & \wedge \mathrm{Ready}_\sigma(M,e,a,r'). \end{aligned} \tag{108}$$

Here $\mathrm{Assembly}_\sigma$ means that every system-owned field equals the deterministic trusted result, while Ready_σ means that the complete receipt and any required audit/root dependencies are durably prepared for the same candidate. None of these predicates assumes that r' has already been published. A commit is permitted exactly when this conjunction holds at the publication snapshot; otherwise the transition is NULL.

33.4 Local and global validation

Row-local predicates check parsing, status normal form, field bounds, tenant/slot binding, versions, ACL encoding, protection, retention, dual-view conformance, rollback bounds, provenance completeness, reserved bytes, checksum, and authentication. Memory-global predicates check at least uniqueness of live/tombstoned object identifiers and primary typed keys within a tenant. A strict schema may also reject alias collisions; a permissive schema returns a set of candidates and still requires current-row revalidation.

Global validation must be snapshot consistent. Suppose two concurrent inserts both observe that key k is absent and then commit to different slots. Per-row compare-and-swap alone permits a duplicate. The contract therefore requires either one total commit order per tenant, a tenant-epoch compare-and-swap, or a serializable uniqueness index whose update is part of the same transaction. Section 38 makes this assumption explicit.

33.5 No self-authentication

PROVED UNDER STATED ASSUMPTIONS

Proposition 33.1 (Trusted-field nondelegation)**.** *Assume the assembler computes every trusted field as a function of authenticated context, the current validated snapshot, fixed policy, and validated proposal content, and assume the proposer cannot forge authentication. Then changing only the neural proposal cannot directly choose tenant, ACL, final versions, protection weakening, deletion authority, rollback lineage, or a valid receipt outside the image permitted by those trusted functions.*

Proof. By construction, none of the named output coordinates is copied from an unconstrained proposal coordinate. Each is either fixed by authenticated context and snapshot, derived by deterministic policy, or rejected. A proposal can alter a trusted field only through an explicitly authorized request whose policy preconditions hold. A receipt authenticating an otherwise forbidden row would require either assembler deviation or authentication forgery, both excluded by assumption. □

This proposition is a software-boundary theorem. It does not prove that a particular assembler is bug-free, that its authentication is uncompromised, or that policy is ethically or legally correct.

34 Canonical–Neural Consistency

34.1 Consistency is a relation, not co-location

Let c be canonical content, (k_n, v_n) the routing and semantic neural bytes, ω a frozen encoder artifact, and Q_σ a deterministic quantizer/serializer. The base AMR-1 conformance relation is

$$D_{\sigma,\omega}(c,n) = \mathsf{PASS} \iff \mathrm{Ser}_{k,v}(n) = \mathrm{Ser}_{k,v}(Q_\sigma(E_\omega(\mathrm{Can}_\sigma(c)))) \,. \tag{109}$$

The row receipt binds one identity and version to the canonical digest, declared encoder digest, derived neural digest, resident neural digest, validation bitmap, and authentication tag. Digest equality accelerates audit, but the contract's equality predicate is over the fixed serialized neural bytes. A hash collision cannot convert unequal bytes into a valid base-profile row because the validator recomputes Eq. 109.

DEFINITION

Definition 34.1 (Dual-view validity)**.** A LIVE candidate is dual-view valid when (i) its canonical bytes parse and normalize uniquely; (ii) its neural route and payload have the exact declared shape and finite representation; (iii) the registered conformance relation returns PASS; (iv) identity, version, encoder/validator IDs, and security scope agree across the receipt and control block; and (v) every required semantic qualification bit in the validation bitmap is present. A missing check is not a pass.

The last condition is an interface gate, not a theorem. A profile may distinguish mechanical byte equality from empirical semantic qualification. AMR-1 can mechanically derive a vector even when no checkpoint can use it. Such a profile remains UNVALIDATED for semantic readout until reconstruction, query, calibration, preservation, and stale-suppression tests pass.

34.2 Certified-proposal profiles

A non-derived profile may permit the proposer to supply n and use a fixed validator

$$D_{\sigma,\omega}(c, n) \in \{\mathsf{PASS}, \mathsf{FAIL}, \mathsf{UNKNOWN}\}. \tag{110}$$

The validator may combine deterministic decoding, typed relation checks, bounded distances, or a frozen semantic reader. Only PASS may produce a LIVE row. FAIL and UNKNOWN fail closed. The receipt records the exact validator and encoder builds; upgrading either creates a new schema or a versioned re-encoding event.

DESIGN PRESCRIPTION A learned validator must not be described as a proof. It is a component whose false-accept and false-reject behavior requires fresh, adversarial, and cross-implementation evidence. A receipt can make the responsible build attributable without making it correct.

34.3 Write, read, load, and migration obligations

The same relation is checked at more than one point.

- **Write.** The complete candidate is validated after final canonicalization and neural serialization, before receipt creation and action selection.
- **Commit.** Canonical bytes, neural bytes, encoder/validator IDs, validation bitmap, and receipt publish atomically.
- **Read.** The current row's status, checksum, authentication, tenant/ACL, validity, and conformance are checked, either directly or through a cached verification whose key includes the exact slot sequence and receipt digest.
- **Load/recovery/export.** Rows are parsed and revalidated before they enter a readable table or serve as teacher state.
- **Index rebuild.** Derived canonical and neural entries are emitted only from validated current rows.
- **Refresh/re-encoding.** A changed encoder, quantizer, or payload is a new complete-row event with new versions and receipt; no in-place neural drift is legal.

For request context χ, a read request carries the typed selector

$$s = (\tau, a, o, v_s, d_\rho, q), \qquad q \in \{\mathsf{canonical}, \mathsf{neural}\}, \tag{111}$$

where τ is the tenant, a the physical slot, o the logical object identifier, v_s the exact slot version, d_ρ the receipt digest, and q the requested view. Let $\mathcal{C}_\sigma$ and $\mathcal{N}_\sigma$ be disjoint canonical and neural output types. The authoritative selector is therefore

$$\begin{aligned}
&\mathrm{Read}_\sigma(M, s, \chi) \in \mathcal{C}_\sigma \uplus \mathcal{N}_\sigma \uplus \{\bot\},\\
&\mathrm{Read}_\sigma(M, s, \chi) = \begin{cases} c(M_a), & q = \mathsf{canonical} \text{ and all checks below pass},\\ n(M_a), & q = \mathsf{neural} \text{ and all checks below pass},\\ \bot, & \text{otherwise}, \end{cases}\\
&\quad z(M_a) = \mathsf{LIVE}, \quad (\tau, a, o, v_s, d_\rho) = \mathrm{SelectorKey}(M_a),\\
&\quad \mathrm{ReceiptOK}(M_a) = 1,\\
&\quad D_{\sigma,\omega}(c(M_a), n(M_a)) = \mathsf{PASS},\\
&\quad \mathrm{AuthorizedRead}(M_a, s, \chi) = 1, \quad \mathrm{TimeValid}(M_a, \chi) = 1.
\end{aligned} \tag{112}$$

The exact selector equality prevents an index cache from returning a different version or receipt. The view tag q fixes the output type before validation; no fallback chooses whichever view happens to decode.

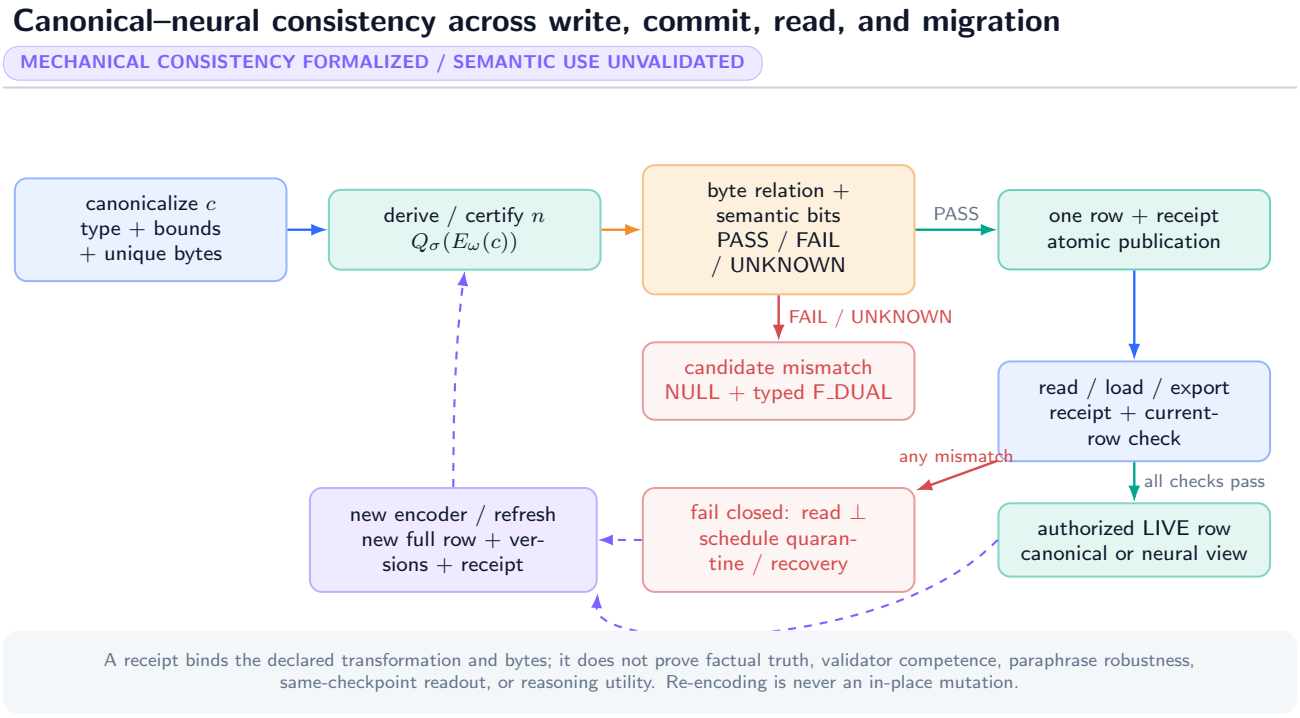

Figure 28: **Dual-view conformance is checked across the row lifecycle.** A candidate mismatch produces NULL; a resident mismatch suppresses the read and schedules a separately authorized quarantine or repair. Re-encoding is a new versioned commit. Mechanical equality is formalized; semantic usefulness remains an unpassed empirical gate.

34.4 Mismatch and quarantine

Candidate-time mismatch has the authoritative result

$$M' = M, \qquad \text{reason} = \mathsf{F\_DUAL}, \tag{113}$$

with an optional copy in a separately bounded non-authoritative diagnostic buffer. That copy may never be read as fact memory.

A mismatch discovered in a previously committed row can arise from storage corruption, lost encoder artifacts, a bug, or compromised infrastructure. The immediate read result is $\perp$; the read path itself does not silently mutate authoritative state. A maintenance event may replace the same slot with a valid QUARANTINED row if identity and version lineage can be recovered under trusted storage metadata. If the row receipt or version is corrupt beyond safe recovery, repair requires an authenticated replica, copy-on-write root, or external recovery record. A checksum detects inconsistency; it does not invent the lost state.

PROVED UNDER STATED ASSUMPTIONS

Theorem 34.2 (No torn dual-view authority)**.** *Assume one atomic publication exposes either the complete old row or the complete new row. A legal commit cannot expose the new canonical view with the old neural view, or the old canonical view with the new neural view.*

Proof. Both views are byte regions of one fixed row serialization. Atomic publication has exactly two visible row values: the precommit serialization and the postcommit serialization. Neither mixed serialization is in that set. □

Without atomic publication, a receipt can detect a tear after the fact but does not guarantee automatic recovery. Section 38 states the stronger storage assumption needed for crash-safe old-or-new recovery.

34.5 What conformance does and does not prove

PROVED UNDER STATED ASSUMPTIONS In the derived profile, equality proves that the resident neural bytes are the declared deterministic encoder/quantizer output on the resident canonical bytes.

PROVED UNDER STATED ASSUMPTIONS In a certified profile, an authentic receipt proves only that the declared validator returned PASS on the committed bytes, assuming the validator output and receipt path were faithfully implemented.

UNVALIDATED Neither statement proves factual truth, semantic sufficiency, robustness to paraphrase, downstream reasoning improvement, same-checkpoint readout, or learned admission quality. The negative frozen MMLA evidence is precisely why this boundary must remain visible.

35 Event-Recursive Transition Algebra

35.1 One row or none

For memory M, target a, and complete candidate $\widehat{r}_a$, define

$$\mathrm{Replace}(M, a, \widehat{r}_a)_i = \begin{cases} \widehat{r}_a, & i = a, \\ r_i, & i \neq a. \end{cases} \tag{114}$$

DEFINITION

Definition 35.1 (NULL)**.** The action NULL is exact state identity:

$$T_{\mathsf{NULL}}(M) = M \tag{115}$$

bit for bit. It increments no version, creates no row receipt, changes no index generation, writes no tombstone, and clears no payload. A separately charged decision log may record a rejected attempt, but that log is not authoritative resident memory.

For one validated event, the authoritative transition is

$$T(M, e, \chi, a, \widehat{r}_a) = \begin{cases} M, & a = \varnothing \text{ or validation fails}, \\ \mathrm{Replace}(M, a, \widehat{r}_a), & \text{authorized commit}. \end{cases} \tag{116}$$

An implementation can use copy-on-write pages, a write-ahead log, or an indirection root. Those are physical mechanisms for exposing the logical old-or-new row; they do not create a second authoritative update operator.

35.2 Completed segments and ordered events

Let completed segment n be eventized into

$$E_n = (e_{n,1}, \ldots, e_{n,K_n}), \qquad 0 \leq K_n \leq K_{\max}. \tag{117}$$

The within-segment state recursion is

$$M_{n,0} = M_{n-1}, \qquad M_{n,j} = T_{n,j}(M_{n,j-1}), \qquad M_n = M_{n,K_n}. \tag{118}$$

Every event is assembled and validated against its actual branch prefix $M_{n,j-1}$. A candidate computed for $M_{n,0}$ cannot be reused at event j after earlier commits unless its snapshot preconditions are revalidated.

Let

$$C_n = \sum_{j=1}^{K_n} \mathbf{1}[M_{n,j} \neq M_{n,j-1}] \tag{119}$$

be the number of accepted commits and let D_n be the number of distinct physical slots changed at least once. Repeated updates to one slot count once in D_n and multiple times in C_n.

Because rows are typed records rather than vectors, edit support is defined by serialized row inequality:

$$\Delta_{\mathrm{row}}(M', M) = \{i \in [S] : \mathrm{Ser}_\sigma(M'_i) \neq \mathrm{Ser}_\sigma(M_i)\}. \tag{120}$$

PROVED UNDER STATED ASSUMPTIONS

Theorem 35.2 (Event and segment edit locality)**.** *For every legal event j,*

$$|\Delta_{\mathrm{row}}(M_{n,j}, M_{n,j-1})| \leq 1, \qquad d_H(M_{n,j}, M_{n,j-1}) \leq 8B_R. \tag{121}$$

Across the segment,

$$C_n \leq K_n, \qquad D_n \leq \min(C_n, S), \qquad |\Delta_{\mathrm{row}}(M_n, M_{n-1})| \leq D_n \leq K_n. \tag{122}$$

If $V(M) \in \mathbb{R}^{S \times d_v}$ stacks decoded payloads, then

$$\mathrm{rank}(V(M_{n,j}) - V(M_{n,j-1})) \leq 1 \tag{123}$$

for one event, while only the correct segment bound

$$\mathrm{rank}(V(M_n) - V(M_{n-1})) \leq \min(D_n, d_v) \tag{124}$$

is generally valid.

Proof. NULL changes no row. A committed event replaces exactly target a, so every other row difference is zero and the Hamming distance is at most the complete row width. Each event contributes at most one commit and one target to the union of touched slots; therefore $C_n \le K_n$ and $D_n \le \min(C_n, S)$. The final difference is supported only on that union. A matrix with at most one nonzero row has rank at most one; across the segment it may have D_n nonzero rows, so rank is bounded by both D_n and d_v. □

The theorem does not say that a row edit is semantically small. Nor does it say that a whole segment has rank one. With two events updating linearly independent payload rows, the segment difference can have rank two.

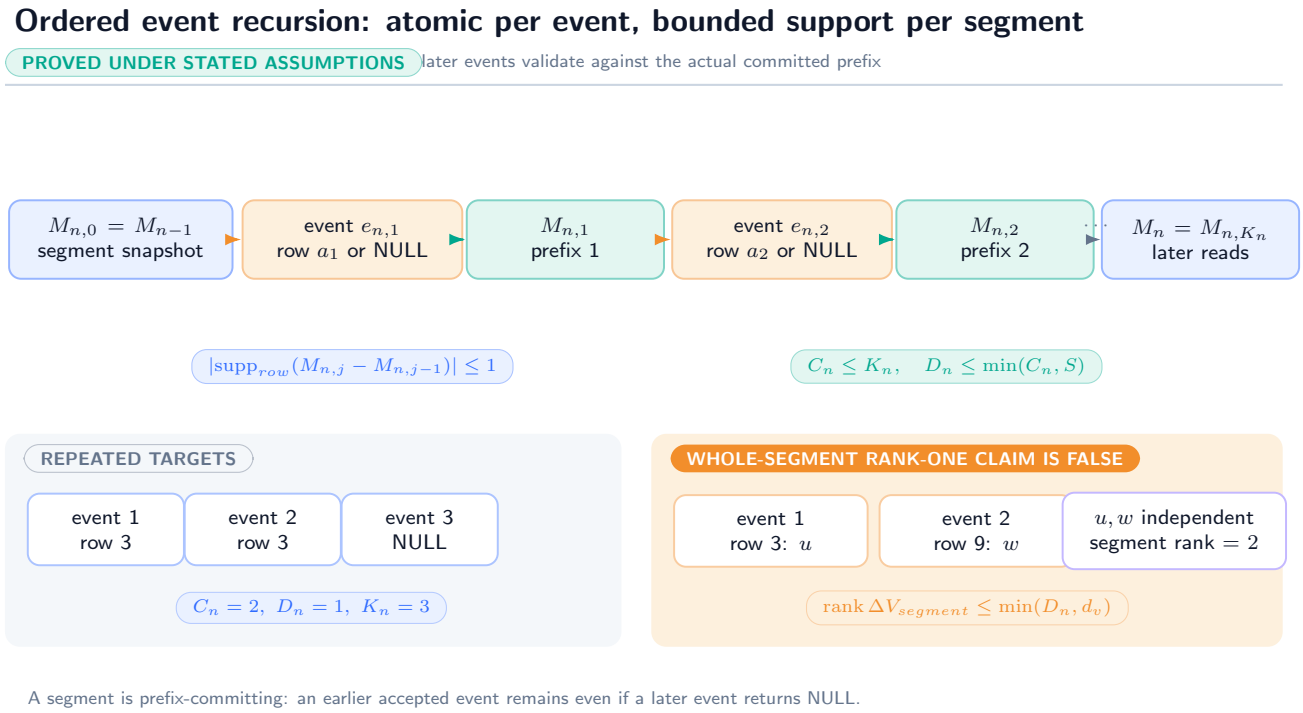

Figure 29: **Event-level atomicity composes into a segment support bound, not segment rank one.** Each event reads its current prefix, then commits one row or NULL. The union of changed targets has size at most K_n; repeated targets reduce the distinct-row count.

35.3 Hard authority and separately typed fast state

An authoritative update cannot be a soft interpolation

$$r'_a = (1-\gamma) r_a + \gamma \widehat{r}_a, qquad 0 < \gamma < 1, \tag{125}$$

because object IDs, ACLs, tombstone status, versions, type tags, and serialized canonical strings do not have a policy-independent convex meaning. Even restricting interpolation to the neural payload creates a third state unless its relation to the canonical version is revalidated and committed.

This does not prohibit differentiable computation. Let

$$F_t \in \mathbb{F}_q^{S_F \times d_F} \tag{126}$$

be a separately typed bounded fast state with its own lifetime and reset rule. Soft writes may update F_t; generation may consume it as a heuristic. Conformance requires that F_t cannot declare a row live, deleted, protected, current, or authorized, and cannot bypass a current AMR read check. Promoting information from F_t to M requires a normal complete candidate and hard commit.

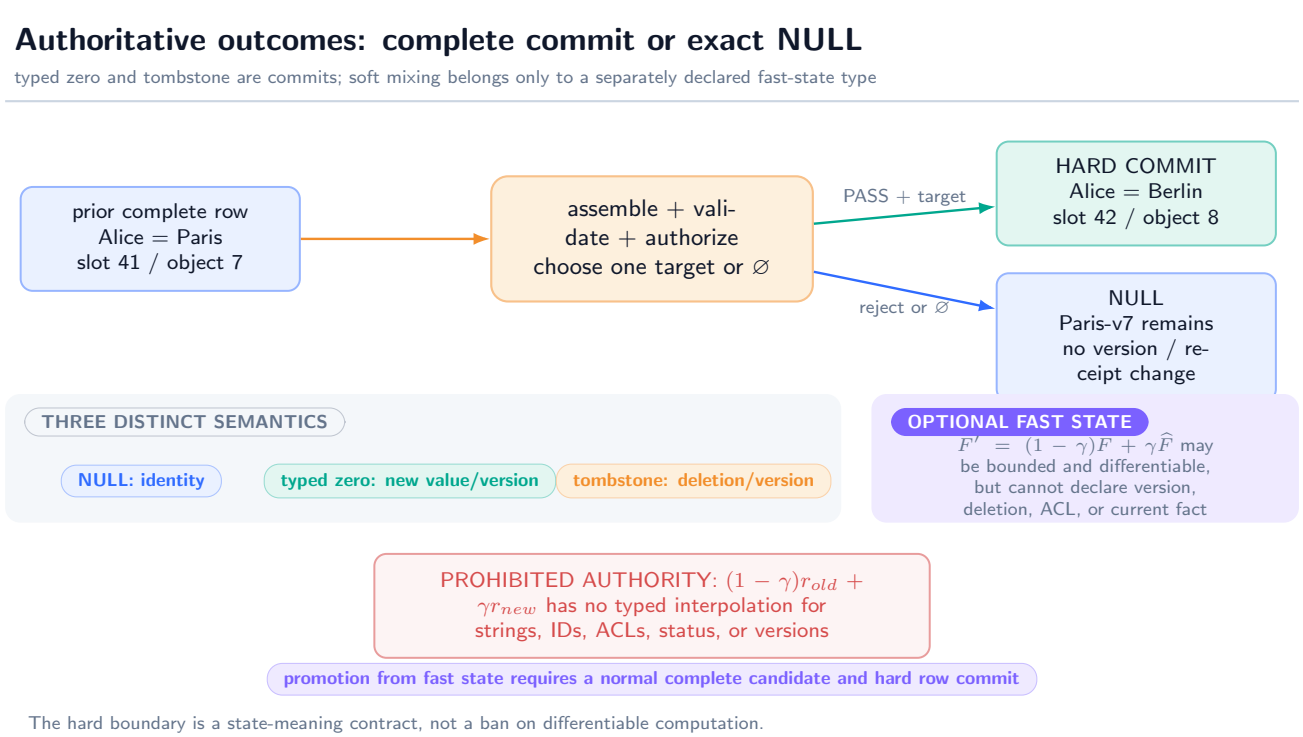

Figure 30: **Authority has two outcomes: a complete hard commit or exact NULL.** A typed zero is a committed value; a tombstone is a committed deletion state. Soft interpolation may exist only in a separately bounded, non-authoritative fast state and must pass ordinary assembly before promotion.

35.4 Zero, tombstone, and NULL

A typed canonical integer zero has a type and length; an all-zero byte string may be a valid value; and an all-zero quantized payload may be valid when its canonical relation passes. Therefore a legal complete-row replacement with a zero value is still a commit. It advances the slot sequence, normally advances object version, and creates a receipt:

$$\mathrm{Replace}(M, a, \widehat{r}_a^{\mathrm{zero}}) \neq T_{\mathsf{NULL}}(M). \tag{127}$$

A tombstone is likewise a complete new row with status, identity, versions, deletion authorization, and retention. Its active payload normal form may contain zeros, but the zeros do not define deletion.

35.5 Segment partial success

Equation 118 is prefix-committing. If event j commits and event $j+1$ returns NULL, the earlier commit remains. The whole segment is not automatically all-or-nothing. A bounded segment transaction would require a separate transaction object that lists every row, read/write set, precondition, rollback record, byte bound, and atomic publication mechanism. Such a protocol is future work and cannot be inferred from per-event atomicity.

36 Lifecycle Semantics

Lifecycle names describe admissible old/new row pairs. They do not introduce extra physical operators.

36.1 Version rules

Every accepted replacement satisfies

$$v_s' = v_s + 1. \tag{128}$$

If logical object identity is unchanged,

$$v_o' = v_o + 1; \tag{129}$$

if a different object is inserted, $v_o' = 1$. A required increment is defined only below the maximum representable value. At exhaustion the event returns NULL with a typed failure and requires an external re-key or migration protocol. It never wraps, saturates, or silently reuses a prior (η, a, v_s) or (η, o, v_o) tuple.

36.2 Namespace migration and ABA exclusion

A persistent reference is the typed tuple

$$\mathsf{ref} = (\eta, a, v_s, d_{\mathrm{core}}),$$

where the namespace-incarnation identifier η is part of the authenticated domain. Slot indices and slot versions therefore have meaning only inside one incarnation. Migration from η_s to η_d is the following fail-closed protocol:

1. allocate a destination incarnation η_d that has never previously been used;
2. seal the source ledger and authenticate its terminal root, schema identifier, validation-registry digest, and algorithm identifiers;
3. authenticate a one-to-one map from every migrated source reference (η_s, a, v_s, d) to a destination reference (η_d, a', v'_s, d');
4. publish the destination root only after every mapped row and receipt validates under the destination profile; and
5. reject every destination request carrying a source-incarnation reference rather than treating it as an alias.

If a finite incarnation space is exhausted, migration rejects. It never wraps or reuses an incarnation identifier.

Proposition 36.1 (Migration excludes namespace ABA)**.** *Assume fresh nonreused incarnation identifiers, authenticated migration roots and maps, and exact typed-selector checking. Then a reference valid before migration cannot become valid for a different row merely because its slot index and slot version are later repeated.*

Proof. Let $x = (\eta_s, a, v_s, d)$ be a pre-migration reference. A destination read requires equality of the full selector key, including η_d and the receipt digest. Freshness gives $\eta_d \neq \eta_s$, so x is rejected even if (a, v_s) is repeated. An authenticated migration map can issue a new reference $x' = (\eta_d, a', v'_s, d')$, but $x' \neq x$ and its receipt binds the destination schema, registry, algorithms, and predecessor token. Authentication prevents substitution of the map or either root. Exhaustion rejects rather than wrapping. Hence index or version reuse cannot make x denote a different destination row. □

Protection classes form a finite preorder $(\mathcal{P}, \preceq)$. An ordinary event may preserve or strengthen protection, $\pi_{\mathrm{old}} \preceq \pi_{\mathrm{new}}$. Weakening requires an authenticated capability bound into provenance. Retention is distinct: it determines eligibility for later deletion/purge, whereas protection governs authorization. The passage of time alone performs no memory mutation.

36.3 Operation table

DEFINITION

Operation	Required old state	Required new-row semantics
Insert	EMPTY	LIVE; new object; object version 1; slot sequence +1
Update	LIVE	LIVE; same object; complete canonical/neural replacement; both versions advance
Consolidating update	LIVE	LIVE; same object; one complete canonical capsule may summarize several observed facts; no second resident row is invalidated
Evict-and-insert	eligible LIVE/TOMBSTONE	LIVE; different object; old object is not copied into new rollback lineage
Delete	LIVE	TOMBSTONE; same object; content unavailable; deletion authority, reason, retention, and new receipt
Alias rename	LIVE/TOMBSTONE	same object/status; bounded alias array replaced; version advances; index projection rebuilt
Protect / ACL change	LIVE/TOMBSTONE	same object/content; policy-authorized control change; complete row reauthenticated
Rollback	LIVE/TOMBSTONE/ QUARANTINED	prior same-object semantic content; current security policy; new forward versions
Quarantine	recoverably identified corrupt row	QUARANTINED; active content unavailable; bounded reason and recovery provenance
Repair	QUARANTINED	newly validated LIVE or TOMBSTONE row from an authenticated source
Purge	retention-eligible TOMB- STONE	EMPTY; slot sequence advances; external deletion/archive policy applied
Reject	any	NULL; every authoritative bit unchanged

Table 25: Lifecycle interpretations of one complete-row replacement. Preconditions are part of validation.

36.4 Insert, update, and consolidation

Insertion allocates a new object identity or validates a preallocated one, starts object version one, and binds the row to the target tenant and slot. An update preserves object identity and replaces the complete semantic state. The old canonical or neural bytes do not survive as an independently readable shadow.

A consolidating update may combine several observations into one canonical tuple when they describe one logical object. For example, a row may replace separate observed location and timestamp fields with one typed current-location tuple. The assembler must declare which source receipts contribute and must fit them into bounded provenance. Consolidation does not authorize deleting a second resident object. If two resident rows represent duplicates, retiring one and enriching the other takes two ordered events or a future bounded transaction.

36.5 Eviction

Eviction is not deletion of an object from all storage. It is replacement of an eligible resident slot with a different object. The policy must respect tenant partition, protection, retention, legal hold, and any required archive precondition. The displaced object may be recoverable from a separately charged archive, but it is not hidden inside the new occupant's rollback ring. If no row is eligible, the event returns NULL.

The eviction target may be proposed by a learned ranker, but eligibility and final selection constraints are trusted. This paper does not validate a learned eviction policy.

36.6 Deletion and tombstones

A deletion request needs a capability distinct from ordinary write permission. A committed tombstone retains enough bounded identity, version, authorization, and retention information to suppress stale resurrection during its declared scope. It does not assert that source documents, backups, replicas, archives, or model parameters were erased.

An expired retention deadline does not mutate the row. A later authorized purge may replace the tombstone with EMPTY after archive, legal-hold, replica, and tenant policy checks. Once purged, perpetual non-resurrection over an unbounded object universe is impossible without an external authoritative deletion ledger; Theorem 37.15 makes this limit explicit.

36.7 Alias rename

An alias rename is a same-object complete replacement. The primary typed key remains authoritative; aliases are bounded query handles. Removing an alias means it disappears from the canonical capsule and therefore from the next deterministic index projection. An old alias-index entry contains the prior slot sequence and receipt digest and cannot be accepted after current-row revalidation.

Alias collision has two legal profile choices. A strict profile rejects the candidate. A permissive profile returns all current rows whose validated alias projections match and requires a later disambiguation step; it never makes "first index hit" authoritative.

36.8 Rollback moves content backward and lineage forward

Let snapshot s_j be a same-object entry in the bounded rollback ring. A legal rollback constructs

$$\begin{aligned} c' &= c(s_j), \\ n' &= \mathrm{Conform}_{\sigma,\omega}(c'), \\ v'_s &= v_s + 1, \\ v'_o &= v_o + 1. \end{aligned} \tag{130}$$

Current ACL, tenant binding, retention, and protection are not blindly copied from the snapshot. Restoring old content is a compensating transaction, not time reversal. Rollback from a tombstone is an explicit resurrection and requires the corresponding authority.

PROVED UNDER STATED ASSUMPTIONS

Proposition 36.2 (Monotone rollback)**.** *Under the rollback rule and authorization checks, a legal rollback may restore prior same-object semantic content but cannot decrease slot sequence, object version, or current protection without a separately authenticated weakening capability.*

Proof. The assembler reads semantic fields only from a same-object bounded snapshot. It computes both new versions from the current row by addition of one and rejects overflow. Current protection is inherited or strengthened unless χ carries a valid weakening capability. Therefore semantic content can revert while lineage moves forward. □

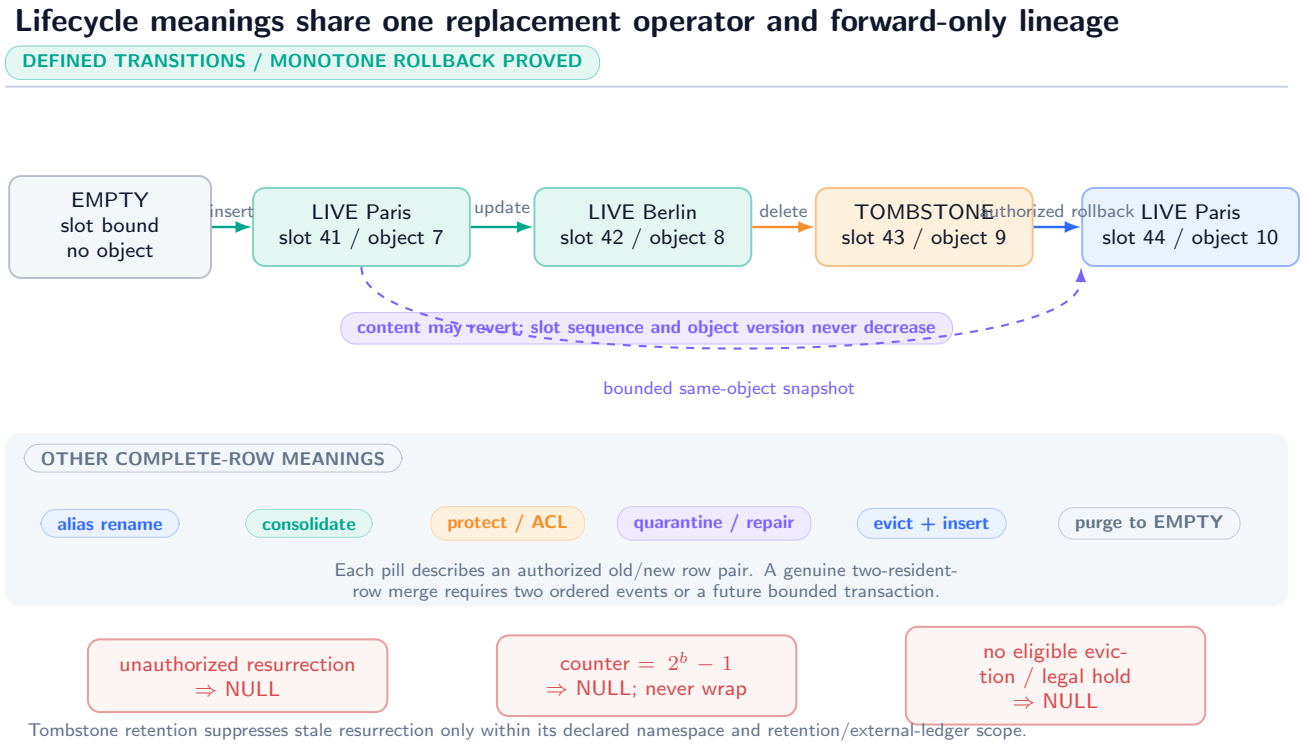

Figure 31: **Lifecycle states share one replacement operator and forward-only lineage.** Delete commits a tombstone; rollback restores bounded same-object content as a new version; purge is a later authorized event. The red paths are bounded rejection or overflow, not ordinary data flow.

36.9 Two-row merge limitation

PROVED UNDER STATED ASSUMPTIONS

Proposition 36.3 (One event cannot implement a genuine two-row merge)**.** *Suppose a merge specification requires both (i) row a to contain a combined object and (ii) distinct resident row $b \neq a$ to cease being live in the same atomic outcome. No transition of Eq. 116 implements that specification.*

Proof. A non-NULL transition changes exactly one target row. If it changes a, row b is untouched and remains live; if it changes b, row a cannot receive the combined object. NULL changes neither. Thus the required two-row postcondition is outside the event transition set. □

Two ordered events can express "write combined object, then tombstone duplicate," but a crash or rejection between them leaves a visible prefix. A future transaction can make them all-or-nothing only by adding an explicitly bounded multi-row protocol. Calling one-row consolidation a merge does not remove this limitation.

37 Invariants and Preservation Results

37.1 Assumption ledger

The results are conditional. They do not assert that a deployed system satisfies these premises.

Assumption 37.1 (Valid genesis)**.** The initial table M_0 contains exactly S parseable schema-σ rows, each in the appropriate empty normal form with valid tenant/slot bindings and receipts.

Assumption 37.2 (Trusted computing boundary)**.** The assembler, authentication context, policy engine, key material, canonicalizer, and commit coordinator are not controlled by the neural proposer. Their code may still contain bugs; this assumption excludes adversarial replacement of the trusted functions in the formal model.

Assumption 37.3 (Deterministic interpretation)**.** Serialization, parsing, canonicalization, quantization, the AMR-1 encoder, dual-view validation, policy evaluation, and base-index derivation are deterministic for their recorded schema/build identifiers.

Assumption 37.4 (Snapshot-consistent validation)**.** Every local and global check is evaluated against the same table snapshot and authenticated context from which versions and predecessor digests are computed.

Assumption 37.5 (Serializable publication)**.** Accepted events for one tenant are placed in a total order, or an equivalent tenant-epoch compare-and-swap rejects a stale snapshot. Global uniqueness checks and their publication share that order.

Assumption 37.6 (Atomic durable publication)**.** A storage transaction exposes and, after recovery, returns either the complete old table root or the complete new table root, with the corresponding required audit-root binding. Derived indexes may lag and are rebuilt from validated rows.

Assumption 37.7 (Cryptography)**.** H is collision resistant and the authentication scheme is unforgeable under chosen-message attack for parties without the relevant key. Key identifiers, rotations, and revocations are authenticated.

Assumption 37.8 (Static tenant partition)**.** P does not change during ordinary event execution, and the read observation of tenant t projects only slots in $P^{-1}(t)$ after current-row authorization.

Assumption 37.9 (Logical observation model)**.** Tenant noninterference excludes timing, cache occupancy, shared accelerator contention, global scheduling, shared-model memorization, allocator leakage, and malicious privileged administrators.

Assumption 37.10 (External-ledger contract)**.** When policy requires an archive or audit record, publication occurs only after that record is durably prepared and its fixed digest/root is bound into the commit. If capacity or durability fails, the authoritative result is NULL.

37.2 Invariant family

The contract is layered rather than treating every obligation as a row predicate:

$$\mathrm{Inv}_\sigma(M, h, \mathsf{obs}) = \mathrm{Inv}_\sigma^{\mathrm{row}}(M) \wedge \mathrm{Inv}_\sigma^{\mathrm{global}}(M) \wedge \mathrm{Inv}_\sigma^{\mathrm{history}}(h) \wedge \mathrm{Inv}_\sigma^{\mathrm{obs}}(M, h, \mathsf{obs}). \tag{131}$$

Row invariants are decidable from one parsed row; global invariants quantify over the table snapshot; history invariants constrain authenticated predecessor and event sequences; observational invariants constrain what typed reads, indexes, and tenant projections may disclose. The numbered clauses below state the concrete obligations. I1–I7 and I9–I12 are row-local; I2 and I13 are global; I8, I10–I12, and I14 constrain history; I6–I7 and I15–I16 constrain observations. Overlap is intentional because a receipt field can be locally well typed while its lineage or disclosure use is invalid.

Let $\mathrm{Inv}_\sigma(M)$ abbreviate the conjunction of all four layers for the current authenticated history and observation contract:

I1. Fixed shape. $|M| = S$, every row is B_R bytes, parses under σ, and has zero reserved bits.

I2. Slot and tenant binding. Row i declares slot i and tenant $P(i)$.

I3. Status normal form. EMPTY, LIVE, TOMBSTONE, and QUARANTINED obey the field constraints of Section 31.

I4. Canonical bounds. Type, length, alias, normalization, padding, and missing-value encodings are valid.

I5. Neural finiteness. Every coordinate is in the declared finite domain and forbidden encodings are absent.

I6. Dual-view conformance. Every readable LIVE row passes its declared mechanical relation and required validation bits.

I7. Security integrity. Tenant, owner, ACL, deletion authority, and protection fields were system assembled and parse uniquely.

I8. Version discipline. Every accepted predecessor relation follows Eqs. 128–129; no counter wraps.

I9. Protection discipline. Protection is not weakened without a bound authenticated capability.

I10. Provenance completeness. Every non-genesis row records the bounded event/source/build/operation fields required by its schema.

I11. Receipt validity. The core checksum, predecessor digest, validation bitmap, algorithm IDs, event binding, and authentication tag verify.

I12. Rollback bounds. Lineage has at most H_{rb} nonrecursive same-object snapshots and obeys current tenant scope.

I13. Identity uniqueness. Within a tenant, no two LIVE or retained TOMBSTONE rows claim the same nonzero object identifier or primary typed key.

I14. Tombstone discipline. A tombstone cannot become LIVE without explicit resurrection/rollback authority; purge obeys retention and legal hold.

I15. Read discipline. Only a current LIVE row passing receipt, conformance, tenant, ACL, validity, and protection checks may supply active content.

I16. Derived-view discipline. An index or cache can return only a hint that is revalidated against the current row; it cannot change authority.

37.3 Preservation

PROVED UNDER STATED ASSUMPTIONS

Theorem 37.11 (Invariant preservation)**.** *Under Assumptions 37.1–37.5, if* $\mathrm{Inv}_\sigma(M)$ *holds and* M' *is produced by a legal event transition, then* $\mathrm{Inv}_\sigma(M')$ *holds. Every finite sequence of legal transitions from genesis therefore preserves the invariant family.*

Proof. If the action is NULL, $M' = M$ and every layer is unchanged. Otherwise exactly target row a is replaced. For $i \neq a$, bytes are identical, so every row-local invariant persists. The trusted assembler fixes slot and tenant, normalizes and bounds canonical content, verifies finite neural encodings and dual-view conformance, applies status normal form, computes versions without overflow, enforces ACL/deletion/protection policy, constructs bounded same-object rollback, binds provenance, zeros reserved bytes, and authenticates the complete core. Thus the row layer, including I1–I7 and I9–I12, holds for the candidate.

Identity/key uniqueness and any strict alias rule are evaluated against the same snapshot used for candidate assembly. Serializable publication or tenant-epoch compare-and-swap prevents a conflicting interleaving between the global check and publication, preserving the global layer and I13. The authenticated predecessor normal form, forward version rule, and explicit resurrection predicates append a legal history edge, preserving the history layer and I8, I10–I12, and I14. Typed reads and derived views revalidate the exact current row and cannot change authority, preserving the observational layer and I15–I16. Therefore all four layers hold in M'. Induction on the finite event sequence proves the second claim. □

PROVED UNDER STATED ASSUMPTIONS

Corollary 37.12 (Version monotonicity)**.** *Along every accepted predecessor chain for one physical slot, slot sequence is strictly increasing until exhaustion. Along a resident same-object chain, object version is strictly increasing.* NULL *leaves both unchanged.*

Proof. The assembler rules are invariant I8 and are preserved by Theorem 37.11. A commit applies +1 and rejects overflow; NULL is identity. □

PROVED UNDER STATED ASSUMPTIONS

Corollary 37.13 (Untouched-row preservation)**.** *For event target a, every row* $i \neq a$ *is byte-identical before and after the event. Across a segment, every slot outside the union of committed targets is byte-identical between* M_{n-1} *and* M_n*.*

Proof. The first statement is Eq. 114. Composition can change only a row selected by at least one event, yielding the second statement. □

37.4 Finite versions and deletion history

PROVED UNDER STATED ASSUMPTIONS

Theorem 37.14 (Finite monotone-version exhaustion)**.** *A* b_s*-bit slot sequence initialized at zero and required to increase strictly on every commit permits at most* $2^{b_s} - 1$ *commits before the next commit must be rejected or the namespace must be migrated.*

Proof. There are 2^{b_s} representable values. A strictly increasing sequence starting at zero can visit each at most once. After $2^{b_s} - 1$, no larger value exists in the representation. □

Silent wraparound would make stale receipts and current versions indistinguishable. An epoch-renewal protocol may extend lifetime, but its epoch is additional authoritative state and must itself be bounded, authenticated, and migrated.

PROVED UNDER STATED ASSUMPTIONS

Theorem 37.15 (Bounded deletion-memory impossibility)**.** *A system with B total authoritative bits cannot, without external authoritative state, distinguish forever all deletion histories over an unbounded object universe.*

Proof. Choose distinct objects $o_1, o_2, \ldots$ and let M_j be the authoritative state after deleting the first j objects. Among $M_0, \ldots, M_{2^B}$, two states coincide by the pigeonhole principle: $M_p = M_q$ for some $p < q$. Object o_{p+1} is deleted in the history producing M_q but not in the history producing M_p. A deterministic query over identical final authoritative states cannot answer differently. □

The coherent guarantee is therefore scoped: a finite namespace, a bounded tombstone set, a retention window, or an external deletion ledger. "Permanent deletion memory for arbitrarily many identities at fixed total cost" is not a satisfiable specification.

37.5 Receipt tamper evidence

PROVED UNDER STATED ASSUMPTIONS

Theorem 37.16 (Receipt tamper evidence)**.** *Under Assumption 37.7, an adversary without the authentication key who changes any receipt-covered core bit of a valid row can make the changed row verify only by finding a hash collision or forging the authentication scheme.*

Proof. If the altered core has the old digest, the adversary found a collision. If its digest changes, the old authentication tag no longer authenticates the receipt message and acceptance requires a valid tag on a new message, which is a forgery. □

This theorem authenticates bytes and declared lineage. It does not authenticate real-world truth, protect a compromised key, or prove the correctness of the trusted code that created the receipt.

38 Security, Index, Concurrency, and Recovery Contract

38.1 Tenant and principal scope

For authenticated context χ with tenant t, the feasible targets satisfy

$$\mathcal{A}_t(M, \chi) \subseteq P^{-1}(t) \cup \{\varnothing\}. \tag{132}$$

The assembler additionally requires the candidate tenant to equal $P(a) = t$. A tenant cannot evict another tenant's row to obtain capacity. Static quotas can waste space, but they make the logical isolation statement auditable.

Within a tenant, each ACL entry contains a principal identifier and rights mask. A reference set is

$$\{\mathsf{READ}, \mathsf{WRITE}, \mathsf{DELETE}, \mathsf{PROTECT}, \mathsf{ROLLBACK}, \mathsf{ACL\_ADMIN}\}. \tag{133}$$

ACL changes require ACL_ADMIN; delete and rollback require their own rights. A general WRITE bit does not authorize security weakening or resurrection.

Formal noninterference traditionally asks whether one domain's behavior can affect another domain's observation [26]. Here the observation model is intentionally narrow.

PROVED UNDER STATED ASSUMPTIONS

Theorem 38.1 (Logical tenant noninterference)**.** *Under Assumptions 37.5, 37.8, and 37.9, for $t \neq t'$ and a legal event issued under tenant t,*

$$\mathrm{Obs}_{t'}(T_t(M)) = \mathrm{Obs}_{t'}(M). \tag{134}$$

Proof. Every feasible target of tenant t lies in $P^{-1}(t)$. Static partitions are disjoint, so no row in $P^{-1}(t')$ changes. The observation of t' projects only current authorized rows in its partition and rejects any index entry whose tenant/current-row binding does not match. Therefore its logical observation is unchanged. □

This theorem excludes timing, shared caches and accelerators, global backpressure, traffic analysis, shared-model memorization, and malicious administrators. A production confidentiality claim needs a stronger observation model and empirical side-channel analysis.

38.2 Derived indexes and stale-value suppression

Every base index is a deterministic function $I = D_\sigma(M)$. An entry is a hint containing

$$(\text{tenant}, \text{object}, \text{key-digest}, a, v_s, d_\rho, \text{projection}). \tag{135}$$

A lookup accepts the hit only if the current row at a has matching tenant, object, slot sequence, readable status, and receipt digest, then passes the ordinary receipt, dual-view, ACL, validity, and protection checks.

PROVED UNDER STATED ASSUMPTIONS

Theorem 38.2 (Stale-index safety)**.** *An arbitrarily stale derived index cannot make an obsolete, deleted, replaced, or cross-tenant row acceptable when every hit is revalidated by Eq. 135 and the current-row read predicate.*

Proof. If the row has not changed, validation reduces to the ordinary current read. If it has changed, then either the slot sequence, object identity, status, or receipt digest differs and the hit is rejected. A cross-tenant entry fails tenant/partition validation. A retained tombstone fails readable status. Thus a stale entry cannot authorize stale content. □

A stale or missing index can cause a false negative, fallback scan, extra work, or denial of service. It is safe only in the authority sense, not necessarily available or efficient. AMR-1 updates nine canonical entries and one neural entry after a row commit, for exactly

$$9 \times 112 + 336 = 1{,}344 \text{ logical index bytes}. \tag{136}$$

If the update is lost, rebuild from validated rows restores the deterministic projection.

38.3 Serial execution and linearization

Database transactions provide the classical all-or-nothing abstraction, while linearizability relates concurrent operations to a legal real-time-respecting sequential history [33, 40]. AMR adopts a narrower one-row event contract.

DEFINITION

Definition 38.3 (Commit linearization point)**.** The linearization point of an accepted event is the atomic publication of the new authenticated table root and required audit-root binding. A NULL event linearizes at the final validation decision and leaves the root unchanged.

PROVED UNDER STATED ASSUMPTIONS

Theorem 38.4 (Serializable event history)**.** *Under snapshot-consistent validation and serializable publication, every finite concurrent execution of accepted events is observationally equivalent to a sequential execution ordered by their successful publication points, with rejected stale attempts represented as NULL.*

Proof. The commit coordinator assigns each successful publication a unique position in a tenant-total order (or a stronger global order where uniqueness spans tenants). A candidate whose snapshot epoch is no longer current fails compare-and-swap and produces no authoritative mutation. Consider successful publications in their assigned order. Each transition was validated against the predecessor root it replaces and changes one complete row; therefore applying those transitions sequentially yields exactly the published roots. Reads validate against one published root and can be placed after the root publication they observe. The resulting sequential history respects successful publication order and has the same observations. □

The theorem would fail if two rows could pass a uniqueness check on different snapshots and publish independently. Per-row atomicity is therefore insufficient for memory-global uniqueness unless the uniqueness constraint has a serializable authority of its own.

38.4 Crash-safe publication protocol

ARIES and related recovery systems show that crash recovery needs an explicit log and ordering discipline rather than the word "atomic" alone [68]. The following is an abstract obligation, not an implementation claim.

Protocol 38.5 (Copy-on-write root publication)**.** For current root R and candidate row:

1. allocate a new immutable row/page and any required audit record; compute their digests;
2. construct a new immutable table root R' pointing to the new row and binding the required audit-ledger sequence/root;
3. durably write and verify the new row, audit record, and R' in an order that prevents a root from referencing unprepared data;
4. atomically publish one checksummed root selector from R to R';
5. update derived indexes after publication or rebuild them later;
6. on recovery, select the latest root whose selector, table, rows, and required audit binding validate; otherwise retain the prior valid root.

The root selector, durability primitive, sector/page atomicity, flush/fence ordering, replica protocol, and failure model are deployment parameters. A write-ahead implementation may satisfy the same abstract old-or-new condition with different steps.

PROVED UNDER STATED ASSUMPTIONS

Theorem 38.6 (Old-or-new crash recovery)**.** *Assume Protocol 38.5 is implemented correctly; durable writes do not acknowledge before reaching the stated failure domain; the root selector update is atomic; and recovery accepts only fully validated roots. After any single crash during one commit, recovery exposes either the complete precommit table and audit binding or the complete postcommit table and audit binding, never a readable mixture.*

Proof. Before the selector publication, the old selector still names R, whose referenced objects remain immutable and valid; unreferenced prepared objects are garbage after recovery. After selector publication, the protocol guarantees that every object reachable from R' was durably prepared. If the selector itself is torn or invalid, its checksum/redundancy causes recovery to choose the prior valid selector; if it is valid, recovery validates all reachable objects before accepting R'. Thus the recovered root is R or R'. Because a row is immutable and both views are inside it, neither root contains a torn canonical/neural or control/content combination. □

UNVALIDATED No storage engine, filesystem, device, replica set, flush sequence, or crash injector is evaluated in this paper. The theorem describes what an implementation must establish.

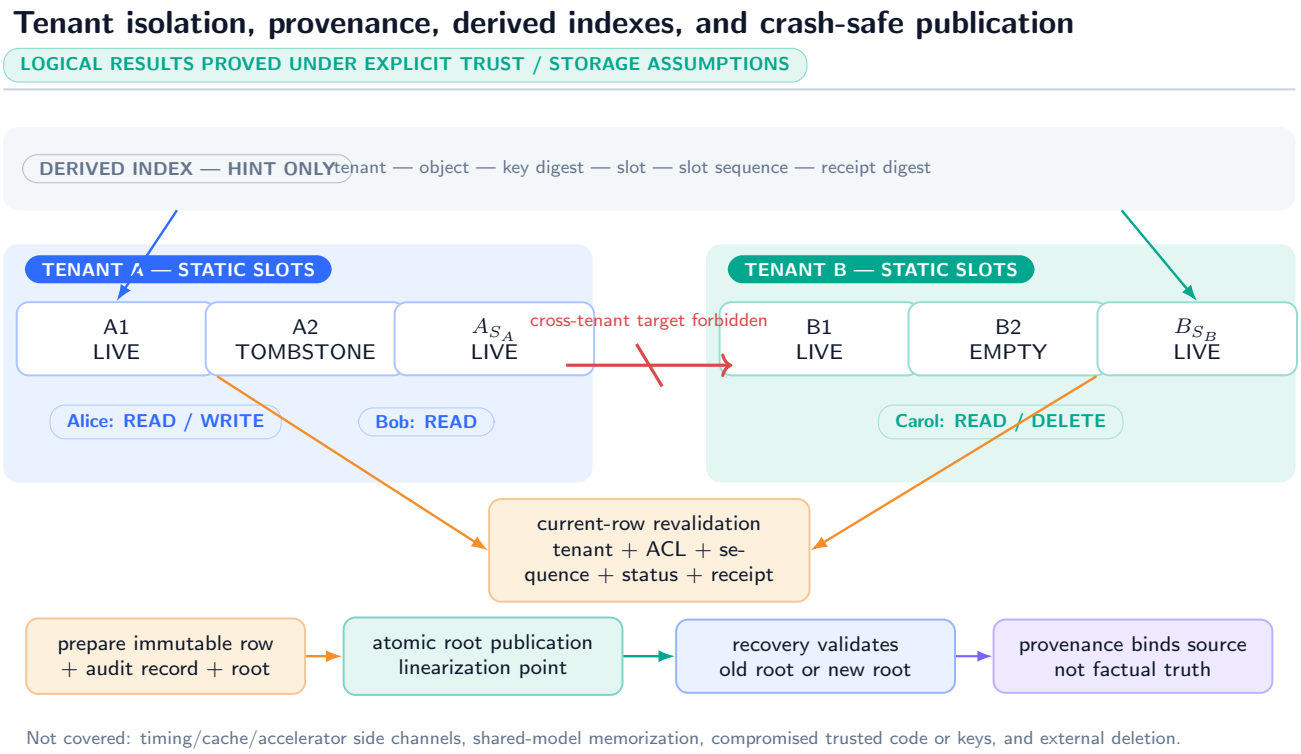

Figure 32: **Static tenant partitions, system-owned security, and current-row revalidation define the logical boundary.** Indexes are hints, provenance binds declared sources, and publication binds the row to an audit root. The proved path excludes side channels and depends on authenticated context, serializable checks, and crash-safe storage assumptions.

38.5 Threat model

The model includes malicious or erroneous neural proposals, replayed stale candidates, malformed serialized fields, alias collisions, cross-tenant target attempts, unauthorized deletion or rollback, storage bit corruption, stale indexes, interrupted commits, and parties without receipt keys attempting row tampering.

It does not defeat a compromised trusted assembler, policy engine, authentication service, signing/MAC key, storage administrator, firmware, or validator build. It does not cover covert channels, denial of service through shared compute, model-parametric memorization, source truth, or complete erasure from backups. The system can fail closed under many of those conditions, but availability and recovery may then require external administration.

39 Complete Cost Calculus

39.1 Why one scalar hides the contract

Candidate construction mixes neural inference, byte traffic, comparisons, policy checks, cryptography, storage barriers, and index work. We therefore report a work vector

$$\mathcal{W} = (B_{\mathrm{rd}}, B_{\mathrm{wr}}, N_H, N_{\mathrm{authV}}, N_{\mathrm{authC}}, N_E, N_D, N_{\mathrm{pol}}, N_{\mathrm{cmp}}, N_{\mathrm{barrier}}), \tag{137}$$

where the coordinates count logical bytes read/written, hash compression blocks, authentication verifications/creations, encoder and dual-view validator calls, policy evaluations, identity/key comparisons, and durable ordering barriers. Model FLOPs, accelerator traffic, kernel launches, wall time, replication, and energy are additional measured coordinates, not constants hidden in $O(1)$.

For a complete deployment, define:

- C_{event}: bounded segment eventization cost;
- C_G: neural proposal cost per target-conditioned candidate;
- C_A: trusted normalization, assembly, and local validation;
- C_D: dual-view derivation/certification;
- C_R: target routing or shortlist construction, including misses and excluded actions;

- C_U: memory-global uniqueness/protection checks;
- C_Q: action scoring after candidates exist;
- C_P: durable row/audit publication;
- C_I: index projection, update, rebuild, and lookup validation;
- C_X: archive write, lookup, retrieval, retention, and deletion work; and
- C_{audit}: audit append/checkpoint, authentication, and retention work.

No end-to-end bound may report only row-read cost while omitting C_G, C_A, or the candidate multiplicity used by routing.

39.2 General candidate accounting

Let B_P be proposal bytes, B_C canonical bytes scanned, B_D neural route/payload bytes compared, $B_{\text{core}} = B_R - B_\rho$, S_t tenant slots, B_{ctrl} trusted-control bytes scanned per row, and B_{key} primary-key bytes scanned per row. A conservative exact logical safe path that reads the old row, reads the proposal, normalizes canonical bytes, materializes and compares the neural view, hashes the complete core, and scans the tenant for object/key uniqueness has

$$\begin{aligned} B_{\text{rd}}^{\text{cand}} &= B_R + B_P + B_C + 2B_D + B_{\text{core}} \\ &\quad + S_t(B_{\text{ctrl}} + B_{\text{key}}), \\ B_{\text{wr}}^{\text{cand}} &= B_R + B_D. \end{aligned} \tag{138}$$

This is protocol traffic, not a prediction of cache misses: an implementation may fuse passes or reuse a cache line, but must report the measured physical traffic separately.

For a Merkle–Damgård-style hash whose compression block is r_H bytes, domain prefix is B_{dom} bytes, and padding consumes one delimiter plus an eight-byte length field, the receipt core needs

$$N_H = \left\lceil \frac{B_{\text{dom}} + B_{\text{core}} + 9}{r_H} \right\rceil \tag{139}$$

compression blocks. SHA-256's block structure supplies the illustrative AMR-1 numbers [72]; another profile must recalculate them.

One candidate has abstract cost

$$\begin{aligned} C_{\text{cand}} &= C_G + C_{\text{normalize}}(B_C) + C_D + C_{\text{pol}} + C_U \\ &\quad + N_H C_{H\text{-block}} + C_{\text{source-auth}} + C_{\text{receipt-auth}} \\ &\quad + C_{\text{traffic}}(B_{\text{rd}}^{\text{cand}}, B_{\text{wr}}^{\text{cand}}). \end{aligned} \tag{140}$$

Rejecting after step q costs the prefix actually executed. A schema-length rejection can occur before encoder and global scans; a late duplicate-key or authentication failure can incur almost the entire candidate cost while still producing zero authoritative row bytes.

39.3 AMR-1 candidate path

For AMR-1,

$$\begin{gathered} B_R = 6{,}592, \quad B_P = 1{,}888, \quad B_C = 1{,}104, \quad B_D = 768, \\ B_{\text{core}} = 6{,}432, \quad B_{\text{ctrl}} + B_{\text{key}} = 256 + 66 = 322. \end{gathered} \tag{141}$$

The proposal is exactly the 1,104-byte canonical block plus 784-byte neural block. The dual comparison covers only the 768 route/payload bytes; the 16 neural metadata/reserved bytes are separately validated.

With $S_t = 256$, the authoritative uniqueness scan reads

$$256 \times 322 = 82{,}432 \text{ bytes}. \tag{142}$$

The remaining local reads are

$$6{,}592 + 1{,}888 + 1{,}104 + 2(768) + 6{,}432 = 17{,}552 \text{ bytes}, \tag{143}$$

so

$$B_{\text{rd}}^{\text{cand}} = 99{,}984 \text{ bytes}. \tag{144}$$

Materializing the complete candidate and one 768-byte encoder scratch buffer writes

$$B_{\text{wr}}^{\text{cand}} = 6{,}592 + 768 = 7{,}360 \text{ bytes}. \tag{145}$$

With a 16-byte domain prefix and 64-byte SHA-256 compression blocks,

$$N_H = \left\lceil \frac{16 + 6{,}432 + 9}{64} \right\rceil = 101. \tag{146}$$

The reference operation ledger is therefore one neural proposal, one canonical normalization, one deterministic encoder call, one policy evaluation, 256 object/key comparisons, 101 hash compression blocks, one source-authentication verification, one receipt-authentication creation, 99,984 specified precommit read bytes, and 7,360 specified precommit write bytes. A strict alias-uniqueness scan adds

$$256 \times 524 = 134{,}144 \text{ read bytes} \tag{147}$$

and at most 8×256 alias comparisons.

On rejection, authoritative publication bytes are zero. On acceptance, logical publication adds one 6,592-byte row and one 1,344-byte fixed index projection. The row may be prepared and written more than once physically under journaling, copy-on-write, or replication; that amplification is measured rather than inferred.

39.4 Routing and candidate multiplicity

If event j constructs $N_{n,j}$ candidates, its construction/validation cost is

$$\sum_{a \in C_{n,j}} C_{\text{cand}}(a), \qquad |C_{n,j}| = N_{n,j}. \tag{148}$$

Exhaustive target conditioning gives $N_{n,j} = S_t$. A shortlist of size L may reduce this term only if C_R, index build/update/storage, shortlist misses, and omitted feasible targets are reported. Scoring S_t cheap target IDs while constructing only the winning row is not equivalent to the complete candidate set assumed by a controller whose action values depend on target-conditioned content.

For one segment,

$$C_{\text{segment}} = C_{\text{event}} + \sum_{j=1}^{K_n} \left(C_R + \sum_{a \in C_{n,j}} C_{\text{cand}}(a) + C_Q + C_{\text{selected publication}} \right). \tag{149}$$

The exact authoritative row traffic is $C_n B_R$ bytes. It is not $K_n B_R$ when events return NULL, and it does not count candidate buffers that never publish.

39.5 Re-encoding, archive, index, and audit costs

A refresh or re-encoding of N_{refresh} resident rows is N_{refresh} ordinary versioned candidates and commits. Its cost includes reading canonical content, running the new encoder, validating semantic gates, advancing versions, updating rollback, writing rows, and rebuilding index projections. There is no free in-place vector migration.

Let a commit emit B_A^+ archive bytes and B_L^+ audit bytes. Its full durable write volume includes at least

$$B^+_{\text{durable}} = B_R + B_A^+ + B_L^+ + B^+_{\text{root}} + B^+_{\text{replication}} + B^+_{\text{journal}}, \tag{150}$$

plus ordering barriers and verification reads. Terms may be zero only when the profile explicitly does not require the corresponding artifact. Archive retrieval later incurs index lookup, storage reads, authentication, decompression/decoding, and candidate reconstruction; it cannot be counted only at write time.

An audit ledger with maximum $L_{\max}$ entries of fixed receipt size B_ℓ has capacity $L_{\max} B_\ell$ plus its bounded index/checkpoint. When full, the system fails writes whose contract requires audit, or executes a separately specified authenticated rotation that commits a checkpoint root before releasing old detail. An append-only ledger with no $L_{\max}$ is explicitly unbounded total storage.

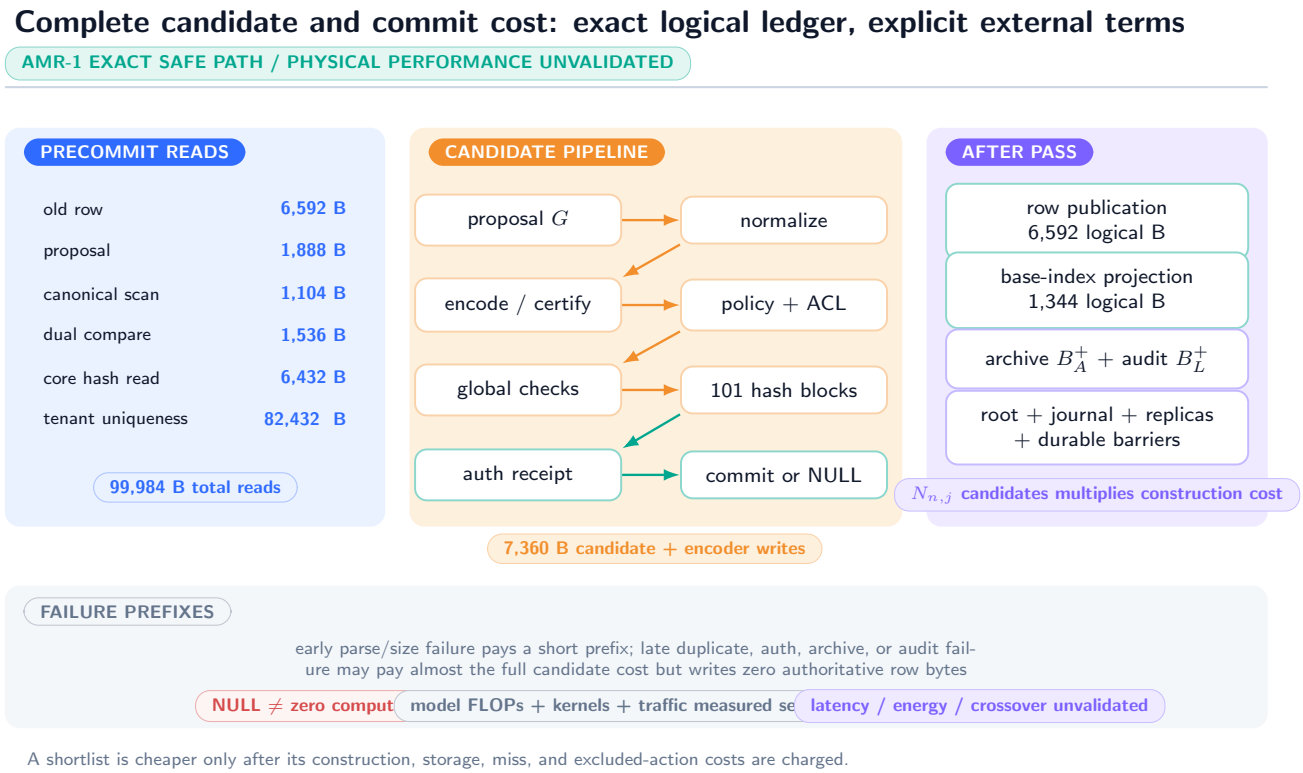

Figure 33: **A complete candidate and commit have a countable bill.** The AMR-1 waterfall gives exact logical safe-path traffic for one candidate. Routing multiplicity, global checks, publication, index projection, archive, audit, replication, barriers, model FLOPs, latency, and energy remain visible rather than disappearing into a row-count asymptotic.

39.6 Failure-path ledger

Failure	Latest required work	Authoritative writes	Remaining cost/risk
Parse/size/type failure	proposal read + bounded parse	0	diagnostic write if enabled
Tenant/ACL/protection denial	old control + policy/auth checks	0	possible authentication and rate-limit cost
Dual-view failure	normalization + encoder/validator	0	optional bounded quarantine copy
Duplicate identity/key	tenant scan or validated uniqueness index	0	nearly complete candidate cost
Version overflow/stale snapshot	version/snapshot check; may be early or late	0	retry/re-key/migration external
Archive/audit capacity failure	archive/audit reservation and policy checks	0	cleanup of unreferenced prepared data
Crash before root publication	durable preparation may have occurred	0 visible	recovery/garbage collection
Crash after root publication	full commit durable by assumption	one row visible	index rebuild may remain
Stale index	lookup + current-row mismatch	0	fallback scan or false negative
Receipt mismatch on read	row read + cryptographic validation	0 by read	fail-closed unavailability and recovery

Table 26: Failure costs. NULL means zero authoritative mutation, not zero computation or zero external logging.

UNVALIDATED AMR-1's byte ledger is exact only at the logical protocol layer under the declared AMR-1 profile. It does not predict cache behavior, latency, throughput, energy, device endurance, or quality–cost crossover. Those require an implementation and the stop-ruled measurements of Section 41.

40 Counterexamples and Failure Taxonomy

The contract's assumptions are not decorative. Each prevents a concrete violation.

40.1 Unbounded aliases and provenance defeat boundedness

Counterexample 40.1 (Hidden variable-width row)**.** Suppose a row stores a list $A(r)$ of every spelling ever used for its object, with no alias-count or length cap. After the mth rename, append a fresh one-byte-distinct alias. Then $|A(r)| \geq m$ and row storage grows with history even when $S = 1$. Likewise, storing the complete provenance document or all ancestors recursively makes B_p or B_λ history dependent. The statement "one fixed slot" no longer implies a bit bound.

Using a fixed digest pointer repairs resident boundedness only by moving the history to an external object. The total system remains unbounded unless that archive is independently capped.

40.2 Neural ownership of trusted fields breaks isolation and lineage

Counterexample 40.2 (Self-issued authority)**.** Let the assembler copy proposed tenant, ACL, and version bits without independent derivation. A malicious proposer targeting a tenant-A slot can emit tenant B, grant its principal READ/DELETE, or reset version from 42 to 1. The serialized row may be well formed and its neural/canonical views may agree, yet tenant isolation, authorization, monotonicity, stale suppression, and rollback lineage all fail. A checksum computed over these bytes merely authenticates the bad assignment.

The remedy is functional system ownership: trusted fields are computed from authenticated context, current state, and policy, never self-certified by the proposal.

40.3 In-place neural refresh creates dual-view drift

Counterexample 40.3 (Two independently mutable truths)**.** At version u, canonical bytes encode Paris and neural bytes equal E_ω(Paris). A background process upgrades to ω' and overwrites only the neural region with $E_{\omega'}$(Berlin) or even $E_{\omega'}$(Paris) while leaving the receipt and version unchanged. The canonical and neural views no longer share one declared transformation and version. A canonical read and neural read can disagree, or stale cached validation can accept bytes produced by an undeclared model.

Re-encoding must therefore be a complete new versioned event, and reads/cache entries must be keyed by the current receipt.

40.4 Soft authoritative overwrite has no unique discrete lineage

Counterexample 40.4 (Versioned mixture ambiguity)**.** Let a payload for Paris be v_P, a candidate for Berlin be v_B, and set $v' = (1-\gamma)v_P + \gamma v_B$ for $0 < \gamma < 1$. If the canonical row remains Paris, the neural view no longer conforms to it. If canonical content becomes Berlin, the payload is not the declared Berlin encoding. If both canonical strings or ACLs are "mixed," their interpolation is undefined. Assigning version 8 does not resolve which fact or security policy is authoritative.

Soft state is permitted only under the separate type and non-authority rule of Eq. 126.

40.5 Multi-event conflation breaks atomic claims

Counterexample 40.5 (Segment rank-one and merge error)**.** A segment contains two events: update Alice in row 3 and update Bob in row 9. Both commits succeed with linearly independent payload changes. The segment difference has two nonzero rows and can have rank two. Calling the entire segment "one atomic overwrite" hides a partial-success boundary. Similarly, merging row 3 into row 9 while making row 3 non-live requires two resident changes and cannot be one event.

The correct recursion is Eq. 118; per-event support is at most one and segment support is at most K_n.

40.6 Per-row CAS does not preserve global uniqueness

Counterexample 40.6 (Write skew)**.** Two assemblers read the same snapshot in which typed key k is absent. One prepares slot 7 and one prepares slot 8. Each uses a successful compare-and-swap only on its target row. Both commits can publish, leaving two live rows with key k. Every local row invariant holds; the memory-global uniqueness invariant fails.

A tenant epoch, serializable uniqueness index, or equivalent global commit check is required. This counterexample explains why the serializability assumption is stronger than atomic row bytes.

40.7 Failure taxonomy

Class	Example	Required authoritative behavior	Unresolved consequence
Syntax/bound	bad length, reserved bit, NaN, alias overflow	precommit NULL	proposal cost, possible denial of service
Authentication	bad tenant, ACL, capability, source tag	precommit NULL	compromised identity service out of model
Policy	protection/retention/legal-hold violation	precommit NULL	policy correctness not proven
Semantic consistency	canonical/neural mismatch or UNKNOWN	candidate NULL; resident read $\perp$	semantic validator errors unmeasured
Concurrency	stale epoch, uniqueness conflict	losing attempt NULL or retry	starvation/availability unproven
Storage integrity	checksum/tag mismatch, torn page	fail closed; recover old/new root	recovery depends on implementation
Index	stale/missing/corrupt hint	reject hit; scan or rebuild	latency and availability degradation
Version/history	counter exhaustion, rollback ring full	reject/migrate or evict declared snapshot	finite lifetime/history loss
Capacity	no eligible slot, audit/archive full	NULL unless explicit bounded rotation/eviction succeeds	memory stagnation or data loss trade-off
Adversarial truth	authenticated false source, poisoning	may still pass mechanical contract	truth/admission problem outside substrate
Key/build compromise	forged tag, malicious validator/assembler	outside theorem; revoke/recover administratively	rows may be untrustworthy
Side channel	timing/cache/traffic leakage	outside logical noninterference	separate security proof/test required

Table 27: Failure taxonomy. Fail-closed behavior preserves authority but can sacrifice availability.

40.8 Abuse cases and non-guarantees

Poisoning. An authenticated source can state a plausible falsehood whose canonical and neural views agree. Provenance enables attribution and later policy; it does not prove truth.

Capacity denial. An attacker can fill its tenant partition with low-value rows or retained tombstones. Static partitioning prevents cross-tenant eviction but not self-denial. Per-principal rate limits and protected reserve capacity are additional policies.

Rollback abuse. A valid old snapshot can still be obsolete or harmful. Rollback requires current authorization, tombstone-aware resurrection checks, and uniqueness validation; the substrate does not decide whether restoration is desirable.

Key or validator compromise. Receipt integrity ends at the trusted key and build boundary. Rotation and revocation are explicit migrations; receipts make a build attributable, not incorruptible.

Deletion scope. A row tombstone says nothing by itself about source documents, replicas, backups, archives, external logs, or learned model parameters. Each carrier needs its own deletion contract.

Protection deadlock. A policy can make all targets infeasible. NULL is safe for authority but can freeze useful learning. Administrative recovery needs its own bounded, authenticated, and audited path.

41 Falsification Protocols and Stop Rules

No protocol in this section has been executed for this manuscript. Passing a lower gate authorizes only the next prespecified test; it does not establish an end-to-end claim.

41.1 Contract-mechanics gates

PLANNED TEST

Protocol 41.1 (Finite-state model checking)**.** Instantiate a reduced schema with small slot, version, ACL, status, alias, and rollback spaces. Exhaustively enumerate legal transitions, concurrent schedules permitted by the model, and recovery points. **Pass:** no legal trace violates I1–I16 and every illegal trace is rejected. **Stop:** any legal trace reaches an invalid state; repair the specification before implementation claims.

PLANNED TEST

Protocol 41.2 (Cross-language serialization)**.** Implement the same schema independently in at least two languages. Generate valid and invalid boundary rows including Unicode normalization, maximum lengths, typed zeros, counter maxima, reserved-bit corruption, alias collisions, NaNs, and every status normal form. **Pass:** byte-identical serialization, checksum/tag input, parsing, and accept/reject decisions. **Stop:** any equivalent input yields different authoritative bytes or validator decision.

PLANNED TEST

Protocol 41.3 (Transition and segment property tests)**.** Generate segments with $K_n \in [0, K_{\max}]$, repeated targets, all-NULL segments, and mixed accept/reject outcomes. **Pass:** per-event row support ≤ 1, $C_n \leq K_n$, $D_n \leq \min(C_n, S)$, untouched rows

remain byte-identical, and published row traffic is exactly $C_n B_R$. **Stop:** any hidden second row or segment-rank-one assertion appears.

PLANNED TEST

Protocol 41.4 (Version, tombstone, and rollback traces)**.** Generate long randomized sequences of insert, update, alias rename, delete, rollback, protect, repair, purge, stale replay, and counter exhaustion. **Pass:** versions never decrease/wrap; rollback restores only bounded same-object content under current security; stale/unauthorized resurrection is zero during the declared scope. **Stop:** one unauthorized resurrection, version reuse, or obsolete ACL restoration.

41.2 Security and recovery gates

PLANNED TEST

Protocol 41.5 (Tenant and ACL adversary)**.** Run concurrent requests over disjoint partitions with forged slot IDs, tenant IDs, object IDs, stale index entries, ACL mutations, delete requests, and rollback requests. **Pass:** zero cross-tenant row changes and readable content; zero security weakening without capability. **Stop:** any logical cross-tenant effect. Timing and shared-resource leakage are measured in a later, stronger model.

PLANNED TEST

Protocol 41.6 (Receipt and dual-view corruption)**.** Flip every covered bit, substitute canonical/neural blocks and encoder IDs, replay predecessor receipts, alter validation bits, and corrupt cached verification keys. **Pass:** every unauthenticated alteration is rejected subject to the declared cryptographic model, and no mismatch yields a readable row. **Stop:** one changed covered core is accepted without acknowledged key compromise or cryptographic break.

PLANNED TEST

Protocol 41.7 (Crash injection)**.** Implement one storage profile, then inject process, kernel, and device failures at every durable-write and root-publication boundary, including torn-selector simulations and index lag. **Pass:** recovery exposes exactly the old or new complete table/audit binding and stale indexes never authorize old content. **Stop:** any mixed canonical/neural, control/content, tenant/object, or row/audit state becomes readable.

PLANNED TEST

Protocol 41.8 (Index determinism and stale suppression)**.** Rebuild indexes under different processing orders, drop updates, insert stale entries, and reuse slots for new objects. **Pass:** base rebuilds are byte identical; stale entries never authorize obsolete/deleted/cross-tenant rows; a full rebuild recovers the declared index. **Stop:** index state becomes independent authority.

41.3 Accounting gates

PLANNED TEST

Protocol 41.9 (Exact protocol-cost audit)**.** Instrument every AMR-1 pass and compare proposal bytes, row scans, neural comparison, hash blocks, authentication calls, candidate buffers, row/index writes, archive/audit bytes, barriers, and failure-prefix work with Section 39. **Pass:** exact equality at the logical protocol layer for the declared profile and an explicit reconciliation to physical traffic. **Stop:** any mandatory component is unmeasured or hidden in a constant.

PLANNED TEST

Protocol 41.10 (Capacity and retention exhaustion)**.** Fill rows, aliases, rollback rings, quarantine, optional indexes, archives, audit ledgers, tombstones, and counters to every boundary. **Pass:** the declared reject/evict/rotate/purge policy occurs with no silent allocation or authority spill. **Stop:** resident or total-system boundedness is claimed without reproducing the carrier inventory.

41.4 Semantic and reasoning gates

PLANNED TEST

Protocol 41.11 (Same-checkpoint semantic qualification)**.** Compare canonical span, typed tuple, derived dual-view, certified dual-view, and pure latent control under one frozen checkpoint, fixed exposure, matched active parameters, fresh seeds, final-only evaluation, and truth-free materialization before scoring. Require canonical reconstruction, neural readout, fresh composition, query accuracy, calibration, old-value suppression, unrelated-row preservation, routing consistency, and corruption rejection at every fixed seed. **Stop:** if no interface passes all preregistered gates, no learned admission or reasoning integration is opened.

This gate must retain the accepted MMLA boundary: PM-I2's 0/9 qualification and PM-I3's fitted learned-content failure motivate a new interface test; they do not count as a test of AMR-1.

PLANNED TEST

Protocol 41.12 (Repeated-write reasoning integration)**.** Only after substrate, crash, security, accounting, and semantic gates pass, compare an AMR-backed system with matched append-only context, canonical-only rows, and frozen retrieval. Measure factual update accuracy, stale leakage, untouched-fact preservation, unauthorized writes, tombstone/rollback correctness, answer quality, candidate multiplicity, total storage/index/archive/audit cost, latency, memory traffic, and energy. **Stop:** any reasoning gain accompanied by worse preregistered safety/integrity gates, or any gain that disappears under total-cost matching, blocks a system advantage claim.

41.5 Dependency and qualification graph

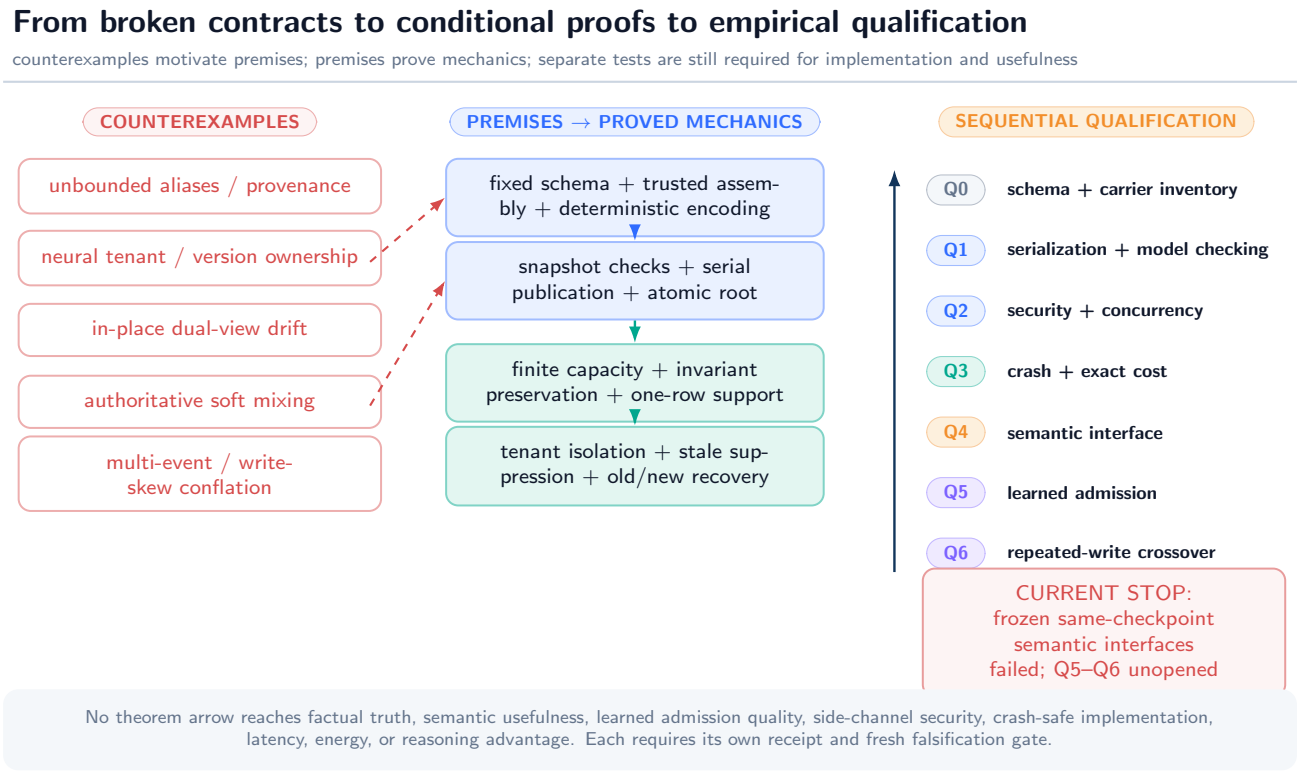

Figure 34: **Counterexamples motivate premise bundles; premises support conditional theorems; theorems create implementation obligations, not empirical success.** The qualification ladder is sequential. The frozen semantic interface remains failed, so learned admission and reasoning-integration rungs are unopened.

The complete theorem-to-obligation matrix is in Appendix 45. Its central rule is simple: an implementation may cite a theorem only after producing evidence for every premise used by that theorem. A successful checksum test cannot substitute for atomic storage; a successful crash test cannot substitute for semantic qualification; and a semantically useful reader cannot substitute for tenant or deletion authorization.

42 Related Work, Limitations, and Conclusion

42.1 Neural external memory

Memory Networks and Neural Turing Machines established influential differentiable read/write stores, while the Differentiable Neural Computer added structured access mechanisms and broad task demonstrations [108, 29, 31]. Their central contributions concern learnable addressing, differentiability, and task performance. AMR asks a complementary systems question: which exact finite bytes may become authoritative when canonical and neural representations, security, provenance, deletion, and rollback must agree?

The distinction allows coexistence. A model may maintain soft differentiable scratch state for computation and use hard AMR commits only for resident authority. This paper does not claim that hard rows should replace every neural memory mechanism or that the reference dimensions are optimal.

42.2 Rows, transactions, and recovery

Atomic state transformation, transaction boundaries, serialization, and recovery are classical database problems. Gray articulated the transaction concept and its limits; ARIES developed write-ahead recovery with fine-grained locking and partial rollback; linearizability gives a concurrent-object correctness condition; and Bigtable is a prominent row-oriented storage system [33, 68, 40, 13]. AMR does not claim invention of rows, atomicity, versions, tombstones, logs, or recovery.

Its narrower contribution is a reasoning-state contract in which canonical content, a neural payload/routing key, identity, security, provenance, deletion, rollback, and a consistency receipt share one fixed-width authoritative boundary. Its event transaction is intentionally weaker than a general database transaction: one event changes at most one resident row, while global uniqueness and crash recovery require explicit coordinator/storage assumptions.

42.3 Provenance, deterministic encoding, and isolation

Database provenance distinguishes source and transformation histories [12]. AMR-1 embeds only a bounded receipt and uses an external ledger for richer history. Deterministic encoding is essential for cross-implementation checksums and signatures; RFC 8949's deterministic CBOR discussion supplies useful precedent, while the profile here is stricter because each field has a fixed region [10]. SHA-256 and HMAC supply illustrative standardized digest and authentication profiles, not a mandate that every deployment use them [72, 71].

The tenant theorem follows the formal noninterference tradition but proves only equality of logical row observations under static disjoint partitions [26]. It does not claim constant-time execution, shared-hardware isolation, or protection from a privileged administrator.

42.4 Relationship to MMLA and the paper family

Public MMLA V3 is the historical source of the proposed complete-row, trusted-assembly, and one-overwrite-or-NULL boundary [125]. This manuscript turns that proposal into a standalone fixed-bit state machine, reconciled concrete profile, proof set, security/recovery assumptions, counterexamples, and complete cost ledger. It does not import V3's controlled lifecycle or relational-binding experiments as validation of AMR-1. Those components were tested under other representations and boundaries.

The five-paper family separates concerns. The first focus paper defines policy-state witnesses for Reasoning-Time Training; this paper defines memory-state authority; the remaining papers formalize predictive admission, dual policy/memory state, and completed-segment consolidation. These are conceptual dependencies only, and each mechanism retains an independent evidence burden.

42.5 Limitations

1. The trusted computing base is substantial: authentication, assembler, policy, canonicalizer, encoder/validator, key management, commit coordinator, and recovery logic.
2. Static tenant partitions can waste capacity and do not address timing or shared-resource leakage.
3. The AMR-1 uniqueness scan is linear in tenant slots; a faster index adds its own authority and concurrency obligations unless treated only as a hint.
4. A deterministic encoder can be reproducible and still semantically useless. A learned validator can share the reader's blind spots.
5. Fixed rollback forgets old history. A different evicted object requires an external archive or new observation.
6. Fixed-width versions eventually exhaust. Epoch renewal adds another bounded authenticated namespace.
7. Tombstones cannot remember an unbounded deletion set forever at fixed total cost.
8. Per-event atomicity is not segment-wide or multi-object transactionality.
9. Fail-closed validation protects authority at the cost of availability; corruption, key loss, or missing build artifacts can make rows unreadable.
10. A receipt binds bytes and declared checks, not truth, source reliability, validator competence, legal compliance, or ethical desirability.
11. Archive, audit, replication, filesystem, and backup costs are separate. Bounded hot state is not bounded total system storage.
12. Exact logical byte counts do not predict physical traffic, latency, energy, throughput, endurance, or hardware advantage.
13. No implementation, model checking, crash injection, security test, semantic qualification, or reasoning experiment is reported.

42.6 Explicit non-claims

This paper does not claim that:

- AMR-1 has been implemented or standardized;
- its 6,592-byte layout or dimensions are optimal;
- the public or technical-report evidence record validated a dual-view row;
- canonical/neural agreement establishes factual truth or semantic usefulness;
- checksums and receipts make a compromised trusted component safe;
- bounded rows improve language-model accuracy or reasoning;
- one-row atomicity implements general distributed transactions;
- tombstones guarantee global permanent deletion;
- rollback is always beneficial;
- static partitions eliminate all information leakage;
- deterministic indexes are fast or approximate indexes are safe without current-row revalidation;
- the profile lowers energy or wins a quality–cost comparison;
- memory-state mutation is itself strict policy training; or
- a theorem authorizes bypassing an empirical gate.

42.7 Conclusion

An Atomic Memory Row gives one resident object one typed identity, one canonical view, one bounded neural view, one tenant/security scope, one version lineage, one provenance receipt, and one publication boundary. The legal event is deliberately narrow: replace one complete row or perform bit-identical NULL.

Under explicit assumptions, that narrow rule yields useful formal consequences: fixed resident information capacity, one-row event locality and the correct K_n segment bound, invariant preservation, untouched-row preservation, monotone rollback, logical tenant isolation, stale-index rejection, serializable event histories, tamper evidence, and old-or-new crash recovery. It also exposes unavoidable limits: finite counters exhaust, bounded rollback forgets, finite tombstones cannot remember infinitely many deletions, and a genuine two-row merge is not one event.

The most important design choice is therefore not vector dimension but placement of authority. An alias map that bypasses row validation is another authority. An in-place neural refresh is another version. An uncharged deletion ledger defeats a total-system bound. A soft mixture is not a discrete fact version. Once those hidden states are made explicit, editable reasoning memory becomes a falsifiable systems object.

The result is not evidence that editable reasoning works. It is a bounded contract on which such evidence can later be demanded.

AI-Assisted Tooling Disclosure

The authors used AI-assisted programming and writing tools for drafting, editing, figure generation, and build verification. The authors reviewed the manuscript artifacts and retain responsibility for their content.

Author Contributions

Junyi Zou and Avrova Donz contributed equally to this work. Both authors share responsibility for the research direction and manuscript. No corresponding-author designation is made.

43 Additional Proofs and Formal Boundaries

43.1 Canonical serialization and bit equality

PROVED UNDER STATED ASSUMPTIONS

Lemma 43.1 (Unique valid serialization)**.** *Assume every variable-length region has an explicit length, deterministic normalization, fixed padding, fixed ordering, and zero reserved bytes. Then two valid parsed rows with the same typed field values have identical serialized bytes.*

Proof. Proceed through the schema regions in their fixed order. Fixed-width fields have a unique byte encoding by the schema's endianness and numeric rules. A variable-width field has one normalized content sequence, one encoded length, the exact content bytes, and uniquely determined zero padding to the fixed region boundary. Arrays have fixed capacity and canonical entry order; unused entries are the unique empty encoding. Reserved bytes are zero. Each region is therefore identical, hence their concatenation is identical. □

This lemma fails if map order, Unicode normalization, NaN payloads, or padding are unconstrained. A semantic equality relation over decoded objects is not enough for byte-addressed checksums.

43.2 Lifecycle reduction and exact NULL

PROVED UNDER STATED ASSUMPTIONS

Proposition 43.2 (Row-local lifecycle reduction)**.** *Every authorized lifecycle operation whose complete postcondition changes at most one row and yields a valid schema row is representable as Eq. 114. Every rejected operation is representable as Eq. 115.*

Proof. For a valid row-local postcondition, let a be its unique changed row and let $\widehat{r}_a$ be the complete postcondition serialization. Equation 114 produces exactly that table. A rejected operation has no legal postcondition and the contract assigns it NULL, which is state identity. □

The premise "changes at most one row" excludes a genuine two-row merge, as Proposition 36.3 shows.

PROVED UNDER STATED ASSUMPTIONS

Proposition 43.3 (Zero write is not NULL)**.** *If a legal candidate encodes a typed zero value and advances slot sequence or receipt, its commit is not NULL even when every byte in its active payload region is zero.*

Proof. NULL leaves every authoritative byte unchanged. The legal zero candidate contains a new slot sequence and receipt by premise, so its serialized row differs from its predecessor. Its replacement therefore changes authoritative state. □

43.3 Tombstone suppression

PROVED UNDER STATED ASSUMPTIONS

Theorem 43.4 (Stale-value suppression during retained tombstone scope)**.** *Assume the current row for object o is a valid retained TOMBSTONE row; reads require current LIVE status; every index hit is revalidated; and becoming LIVE requires an authorized version-advancing resurrection or rollback. Then no stale pre-deletion row or index entry can supply o's active content before such an authorized commit.*

Proof. The current row fails the LIVE predicate and therefore supplies no active content. A stale index entry for the pre-deletion row carries an earlier slot sequence or receipt digest, so Theorem 38.2 rejects it. Any attempt to replace the tombstone without resurrection/rollback authority fails validation and resolves to NULL, leaving the tombstone current. Thus only an authorized version-advancing commit can make o readable. □

The scope ends when the tombstone is purged or when external ledgers no longer remember the deleted identity. The theorem does not promise perpetual suppression over an unbounded universe.

43.4 Tenant-local capacity

PROVED UNDER STATED ASSUMPTIONS

Corollary 43.5 (Tenant resident capacity)**.** *For tenant t with $S_t = |P^{-1}(t)|$, its authoritative row projection has at most $2^{8S_tB_R}$ bit states and carries at most $8S_tB_R$ bits of mutual information about any history.*

Proof. Apply Theorem 32.1 to the concatenation of the tenant's S_t fixed-width rows. □

This is both a storage bound and a denial-of-capacity boundary: a tenant cannot borrow another tenant's rows under static partitioning.

43.5 Index rebuild determinism

PROVED UNDER STATED ASSUMPTIONS

Proposition 43.6 (Deterministic base-index rebuild)**.** *Under deterministic parsing, row validation, key hashing, quantization, and fixed slot-major output order, rebuilding I_C and I_N from the same authoritative table yields byte-identical indexes.*

Proof. Each output region is assigned to one fixed pair (a, q) for slot a and primary/alias ordinal q, or to one fixed slot a for the neural index. The validated row at that slot is fixed. Every projected field and digest is deterministic, and invalid/nonreadable projections have a unique empty encoding. Fixed output order eliminates scheduling dependence. Thus every output byte is a deterministic function of M. □

43.6 Segment rank counterexample in matrix form

Let $S \geq 2$ and $d_v \geq 2$. Choose two distinct rows $a \neq b$ and payload changes $u, w \in \mathbb{R}^{d_v}$ that are linearly independent. Event 1 changes only row a by u; event 2 changes only row b by w. Then

$$V(M_{n,2}) - V(M_{n,0}) = e_a u^\top + e_b w^\top, \tag{151}$$

where e_a, e_b are distinct standard basis vectors. The two nonzero rows are linearly independent, so the matrix rank is two. This construction satisfies every per-event rank-one bound and disproves an unconditional whole-segment rank-one claim.

43.7 Read-time fail-closed property

PROVED UNDER STATED ASSUMPTIONS

Proposition 43.7 (No view selection after mismatch)**.** *If Eq. 112 is the only active-content read rule, a row with checksum, authentication, status, ACL, time, or dual-view failure supplies neither canonical nor neural active content.*

Proof. The rule is a conjunction. Failure of any conjunct selects the otherwise branch $\perp$. No rule returns one view conditional on failure of the other. □

The proposition is only as strong as the carrier inventory. An undocumented vector cache that bypasses Eq. 112 is an additional authority and invalidates the premise.

43.8 Commit and audit binding

PROVED UNDER STATED ASSUMPTIONS

Proposition 43.8 (No accepted unaudited commit under a required-ledger profile)**.** *Assume policy marks an event as requiring audit, the candidate receipt binds the intended audit sequence/root, and publication verifies that the prepared audit record is durably reachable from that root before changing the table selector. Then an accepted recovered postcommit root has the corresponding audit binding; failure to prepare the audit record yields the old root.*

Proof. If audit preparation fails, the publication precondition is false and the selector remains on the old root. If the selector reaches the new root, the protocol's ordering premise guarantees the bound audit object was prepared and recovery validates its reachability. Therefore every accepted postcommit root satisfies the required binding. □

This result binds an audit record, not an unbounded human-readable log. A bounded checkpoint/rotation policy determines how long detail remains available.

43.9 Assumption sensitivity

Dropped premise	Formal result lost	Minimal witness
Fixed field widths	capacity theorem	append one alias per event
Trusted field derivation	isolation/version/protection	proposal self-assigns tenant or version
Deterministic encoding	byte equality/rebuild	map order or NaN payload differs
Snapshot-consistent global checks	identity uniqueness	concurrent duplicate inserts
Serializable publication	serial history and uniqueness	write skew across target rows
Atomic root publication	no-torn/crash theorem	visible partial page or mixed root
Current-row index validation	stale suppression	old alias hit returns old row
Cryptographic assumptions	tamper-evidence theorem	collision or tag forgery
Static partition/logical observation	tenant noninterference	row migration or timing channel
Required-ledger fail-closed rule	audit binding	row publishes after audit failure

Table 28: Premises are necessary to the indicated claims; later evidence must test them independently.

44 AMR-1 Profile Ledger

This appendix reconciles every byte used by the illustrative profile. It is a design artifact, not a deployed wire-format standard.

44.1 Trusted control: 256 bytes

Field	Bytes
Magic, schema, status, operation, flags	10
Physical slot identifier	4
Tenant identifier	16
Logical object identifier	16
Owner/principal identifier	16
Object type	2
Slot sequence	8
Object version	8
Valid-from, valid-until, retention-until	24
Protection class and deletion-reason code	4
Eight ACL entries: 16-byte principal + 2-byte rights	144
Rollback ring head and occupancy	2
Reserved zero bytes	2
Total	**256**

Table 29: AMR-1 trusted-control allocation.

The object type and header flags supply the canonical type/semantic flags referenced in Eq. 90; they remain system-owned.

44.2 Canonical capsule: 1,104 bytes

$$\underbrace{(2+64)}_{\text{key}} + \underbrace{(2+512)}_{\text{value}} + \underbrace{(1+3)+8(1+64)}_{\text{alias count/reserved + entries}} = 1{,}104. \tag{152}$$

Lengths count canonical bytes, not code points. Padding to each maximum is zero. The primary key and value use 16-bit lengths; each alias uses an 8-bit length because its maximum is 64. Alias count is at most eight.

44.3 Neural capsule: 784 bytes

$$\underbrace{128\times 2}_{\text{routing}} + \underbrace{256\times 2}_{\text{payload}} + \underbrace{4}_{\text{uncertainty + abstention}} + \underbrace{1}_{\text{proposed intent}} + \underbrace{11}_{\text{reserved}} = 784. \tag{153}$$

The 768 routing/payload bytes participate in exact derived-mode comparison. Uncertainty and abstention are two signed or unsigned 16-bit schema values; intent is one enumerated byte.

44.4 Provenance: 192 bytes

Field	Bytes
Event identifier	16
Source-material or source-receipt digest	32
Proposer/model-build digest	32
Assembler build identifier	16
Encoder build identifier	16
Validator build identifier	16
Observation time	8
Source class	2
Committed operation copy	1
Deletion-authorization digest	32
External audit handle	16
Reserved zero bytes	5
Total	**192**

Table 30: AMR-1 bounded provenance allocation. A source document or provenance graph is external.

44.5 Rollback: two 2,048-byte snapshots

Each snapshot contains no nested rollback data:

Field	Bytes
Prior canonical capsule	1,104
Prior neural capsule	784
Same-object identifier	16
Prior slot sequence	8
Prior object version	8
Prior validity and retention times	24
Prior protection/status/snapshot flags	8
Prior provenance digest	32
Prior receipt digest	32
Reserved zero bytes	32
One snapshot	**2,048**
Two-snapshot ring	**4,096**

Table 31: AMR-1 rollback allocation. The snapshot is sufficient to propose restoration of semantic state; current policy and new provenance are assembled at rollback time.

An evicted different object is excluded. A rollback snapshot does not carry an ACL or recursively embedded history; current security controls govern restoration.

44.6 Consistency receipt: 160 bytes

Field	Bytes
Core checksum/digest	32
Predecessor receipt digest	32
Validation bitmap	8
Schema/algorithm/key-epoch/format identifiers	16
Event binding	16
Authentication tag	32
Reserved zero bytes	24
Total	**160**

Table 32: AMR-1 receipt allocation.

The receipt-covered core is therefore $6{,}592 - 160 = 6{,}432$ bytes.

44.7 Base-index entries

One 112-byte canonical entry contains:

$$16_{\text{tenant}} + 16_{\text{object}} + 32_{\text{key digest}} + 4_{\text{slot}} + 8_{\text{sequence}} + 32_{\text{receipt digest}} + 4_{\text{ordinal/flags}} = 112. \tag{154}$$

One 336-byte neural entry contains:

$$256_{\text{route}} + 16_{\text{tenant}} + 16_{\text{object}} + 4_{\text{slot}} + 8_{\text{sequence}} + 32_{\text{receipt digest}} + 4_{\text{flags}} = 336. \tag{155}$$

The fixed slot-major arrays contain entries for empty/tombstoned/quarantined rows in the unique disabled normal form. Their size is therefore independent of occupancy.

44.8 Auxiliary and external carrier parameters

AMR-1's reported 1.9375 MiB total includes authoritative rows and both base indexes. It sets persistent non-authoritative fast state and diagnostic-copy quarantine to zero. A conforming extension publishes the following additional parameters:

$$\begin{aligned} B_Q &= Q_{\max} B_q && \text{diagnostic copies,} \\ B_F &= S_F d_F p_F + B_{F,\text{meta}} && \text{soft fast state,} \\ B_A(t) &\le A_{\max} \text{ or explicitly unbounded} && \text{archive,} \\ B_L(t) &\le L_{\max} B_\ell + B_{L,\text{index}} \text{ or explicitly unbounded} && \text{audit ledger.} \end{aligned} \tag{156}$$

If B_Q or B_F is nonzero, its authority and reset/lifetime rule must be audited. If $A_{\max}$ or $L_{\max}$ is finite, reaching it invokes Table 23; if absent, total-system boundedness is not claimed.

The safe-path ephemeral working set has at least the proposal, candidate, and encoder scratch regions,

$$B_{\text{work,min}} = 1{,}888 + 6{,}592 + 768 = 9{,}248 \text{ bytes}, \tag{157}$$

plus implementation-specific old-row, parser, hash/MAC, policy, index, and I/O buffers. A deployment must measure its peak physical working set rather than treating the minimum as exact RAM use.

44.9 Arithmetic reconciliation

$$256 + 1{,}104 + 784 + 192 + 4{,}096 + 160 = 6{,}592. \tag{158}$$

$$6{,}592 \times 256 = 1{,}687{,}552, \quad 256(9 \times 112 + 336) = 344{,}064. \tag{159}$$

$$1{,}687{,}552 + 344{,}064 = 2{,}031{,}616 = 1.9375 \times 2^{20}. \tag{160}$$

Every headline AMR-1 byte count follows from these fixed allocations.

45 Theorem-to-Implementation Obligations

Theorems are reusable only when the implementation supplies the corresponding premise evidence. Table 33 is a minimum receipt set.

Table 33: Formal result to implementation and falsification obligations.

Claim	Required premise bundle	Minimum implementation receipt/test
Fixed resident capacity (Theorem 32.1)	exact schema; complete carrier inventory; fixed S, B_R	schema-generated offset table; binary-size checks; allocator and external-carrier inventory; exhaustion tests
Unique serialization (Lemma 43.1)	deterministic normalization, lengths, padding, order, numeric rules	cross-language golden corpus and boundary/fuzz vectors with byte-identical results
Event/segment locality (Theorem 35.2)	all authority in M; one target or NULL per event	whole-process write tracing; state hash before/after every event; hidden-cache/alias-map inventory
Trusted-field nondelegation (Proposition 33.1)	proposer isolated from trusted functions and keys	data-flow/code audit; privilege separation; adversarial field-injection suite; key-access audit
No torn dual view (Theorem 34.2)	one atomic complete-row publication	crash/concurrency injection at every row-write boundary; no mixed serialization accepted
Invariant preservation (Theorem 37.11)	valid genesis; trusted deterministic assembly; snapshot-consistent serializable validation	model check on reduced schema; property-based transition tests; global-uniqueness concurrency tests
Version monotonicity (Corollary 37.12)	exact increment and overflow rejection	long trace; stale replay; maximum counter; restart and recovery tests
Untouched preservation (Corollary 37.13)	complete target isolation	byte hashes of all non-target rows across every event and crash point
Monotone rollback (Proposition 36.2)	same-object bounded snapshot; current security; version increment	rollback/resurrection adversary; obsolete ACL/protection fixtures; ring-overflow tests
Tombstone suppression (Theorem 43.4)	retained current tombstone; revalidated indexes; authorized resurrection only	stale row/index replay, aliases, backups in scope, privilege-negative tests
Stale-index safety (Theorem 38.2)	every hit bound to and checked against current row	fault injection dropping/reordering index updates; slot reuse; cross-tenant forged entries
Tenant noninterference (Theorem 38.1)	static partitions and logical observation model	cross-tenant mutation/read adversary; separate side-channel scope statement and tests
Serializable history (Theorem 38.4)	tenant-total order or equivalent CAS; global constraint serialization	concurrent history checker; write-skew fixtures; linearization receipt; stale snapshot rejection
Old-or-new recovery (Theorem 38.6)	correct storage protocol, atomic selector, durable ordering, validating recovery	power/process/device crash matrix; torn selectors/pages; flush/fence audit; replica failover if claimed
Receipt tamper evidence (Theorem 37.16)	collision resistance, unforgeability, protected keys	algorithm/key inventory; bit-flip/replay/substitution suite; rotation/revocation and compromise response

Claim	Required premise bundle	Minimum implementation receipt/test
Index rebuild determinism (Proposition 43.6)	deterministic projection and fixed order	repeated parallel/serial/cross-language rebuilds with identical hashes
Audit binding (Proposition 43.8)	required-ledger reservation and atomic root binding	audit-full, write-failure, rotation, recovery, and dangling-record tests
Semantic usefulness	*not implied by any theorem*	preregistered same-checkpoint reconstruction/query/calibration/preservation qualification
Reasoning or cost advantage	*not implied by any theorem*	repeated-write, security, and total-system compute/storage/latency/energy matched comparison

45.1 Implementation conformance statement

DESIGN PRESCRIPTION A system claiming AMR conformance should publish:

1. schema identifier and generated field-offset/size table;
2. complete mutable-carrier inventory and authority classification;
3. trusted computing boundary, privilege/key map, and build digests;
4. serializer/parser/validator golden vectors;
5. event trace with snapshot, candidate, decision, target, predecessor, new receipt, and NULL reason;
6. current-row read and index-revalidation trace;
7. tenant/ACL/protection/deletion/rollback policy fixtures;
8. storage publication ordering and recovery specification;
9. archive, audit, index, quarantine, replication, and backup capacity/retention policies;
10. exact logical and measured physical cost ledger; and
11. explicit list of theorems claimed, with every premise receipt linked.

Missing evidence weakens only the affected claim. For example, a correct serializer may support fixed-bit parsing while crash recovery remains unvalidated. A successful semantic reader may support interface usefulness while deletion authority remains unvalidated. Status must not be inherited by the whole system.

45.2 Stop-rule ordering

The intended order is:

$$\begin{aligned}&\text{schema/transition mechanics} \to \text{serialization and model checking} \to \text{security/concurrency}\\ &\to \text{crash recovery and exact cost} \to \text{semantic interface} \to \text{learned admission}\\ &\to \text{repeated writes} \to \text{total-system quality–cost comparison.}\end{aligned} \tag{161}$$

Failure at a rung stops claims that depend on it. It does not erase independent lower-rung evidence, and it does not authorize a post-hoc threshold, seed, checkpoint, or threat-model change.

The frozen MMLA result currently stops at the semantic-interface rung for the tested interfaces. This manuscript supplies a new formal substrate candidate, not evidence that the rung has been passed.

46 Symbol and Status Ledger

Table 34: Principal symbols.

Symbol	Meaning
σ	immutable row schema and deterministic serialization profile
$S, \mathcal{S}$	resident slot count and slot set $\{1, \dots, S\}$
M, r_a	complete authoritative table and row in slot a
B_R	complete serialized row bytes; AMR-1 uses 6,592
$h, c, n, p, \lambda, \rho$	trusted control, canonical, neural, provenance, rollback, and receipt row blocks
$P(a)$	trusted static tenant assigned to slot a
v_s, v_o	physical slot sequence and same-object version

Symbol	Meaning
$z(r)$	row status: EMPTY, LIVE, TOMBSTONE, or QUARANTINED
$\widetilde{p}$	untrusted neural proposal
χ	authenticated request/event context and capabilities
$D_{\sigma,\omega}$	versioned canonical–neural conformance relation
NULL	exact no-authoritative-change action
E_n, K_n	ordered event list for segment n and its length
$M_{n,j}$	authoritative state after event j of segment n
C_n, D_n	accepted commit count and distinct changed-slot count in segment n
B_I, B_Q, B_F	fixed index, diagnostic quarantine, and non-authoritative fast-state capacity
$B_A(t), B_L(t)$	separately charged archive and audit-ledger storage at time t
$\mathcal{W}$	heterogeneous logical work vector for a candidate/commit
B_P, B_C, B_D	proposal, canonical-scan, and compared neural route/payload bytes
N_H	hash compression-block count
Obs_t	logical authorized observation projected to tenant t
Inv_σ	conjunction of the sixteen state invariants

46.1 Status-to-evidence rule

Displayed status	Admissible support in this paper	Required before promotion
DEFINITION	internal typed construction	independent terminology and interface audit
PROVED UNDER STATED ASSUMPTIONS	deductive proof in source	independent proof audit; premise evidence for implementation use
DESIGN PRESCRIPTION	engineering necessity/counterexample	implemented mechanism and adversarial test
PLANNED TEST	protocol text only	immutable preregistration and completed evidence
UNVALIDATED	explicit absence of qualifying evidence	matched fresh evaluation under frozen gates
CONJECTURED	stated hypothesis only	theory proof or accepted experiment, depending on claim

Table 35: No status automatically promotes another.

46.2 Artifact boundary

The mathematical statements in this paper are candidate theory. Project-result statements are limited to the public V3 and technical-report evidence boundary. Source organization and arithmetic are not treated as scientific evidence without an explicit derivation or admitted measurement.

Part III
Predictive Memory Admission

Branch status. This part imports the complete substantive formal branch from the frozen R01 focus paper. It specifies typed completed-segment state, grouped alternative futures, action-conditioned replay, uncertainty-aware admission, proofs, counterexamples, audit obligations, and stop rules under explicit assumptions. It contributes no new experiment, measured oracle gap, learned value or admission policy, qualified semantic-memory interface, safety result, or efficiency result; predictive overwrite remains closed.

47 Introduction

47.1 Admission is a decision about authority

A transcript may retain a sentence, a retriever may return it, and a recurrent state may compress it. None of those operations alone decides whether the sentence should replace a current authoritative fact. Under capacity pressure, a write has at least three possible costs: it may admit false or stale content, displace a useful row, or spend resources without improving any later decision. Refusing a write has a different cost: the state may remain stale or omit information that would have helped a later task. Predictive memory admission is the decision problem created by those asymmetric consequences.

DEFINITION

Definition 47.1 (Predictive memory admission)**.** Predictive memory admission is the selection of one action from a finite feasible set containing exact NULL and zero or more complete target-conditioned row commits, using only information available at the decision boundary, with the objective and evidence gate fixed before evaluation futures are opened.

The word *predictive* refers to conditional future consequence, not access to a realized future at deployment. A training teacher may use later outcomes to estimate action risk. A deployed value model may use only the causal state. The difference is structural and will be expressed as a filtration constraint rather than a promise in prose.

47.2 Evidence boundary

This manuscript inherits architecture and identity from the public MMLA V3 manuscript and its technical-report evidence record [126]. Their evidence boundary is unchanged. The registered same-checkpoint semantic interfaces did not qualify. PM-I2 qualified zero of nine jobs. A subsequent fitted-population diagnostic reported learned content exact on 0/73,728 probes both with learned routing and when the target row was supplied mechanically; only the complete mechanical oracle was perfect. The result is a fit/support failure for the tested pure latent-row interface, not a rejection of every possible memory representation.

UNVALIDATED Consequently, the empirical expected-risk oracle in this paper has not been run, no positive oracle gap exists in the admitted record, and no future-blind value model or admission policy has been learned. Predictive overwrite remains closed by gate. Every numerical example below is identified as illustrative; every experimental protocol is future work.

The companion focus papers provide terminology and an external row contract only. The RTT foundation separates mutable policy state from memory state [132]. Atomic Memory Rows supplies complete-row, trusted-assembly, version, protection, and exact-NULL semantics [118]. Neither formal dependency supplies empirical evidence for admission.

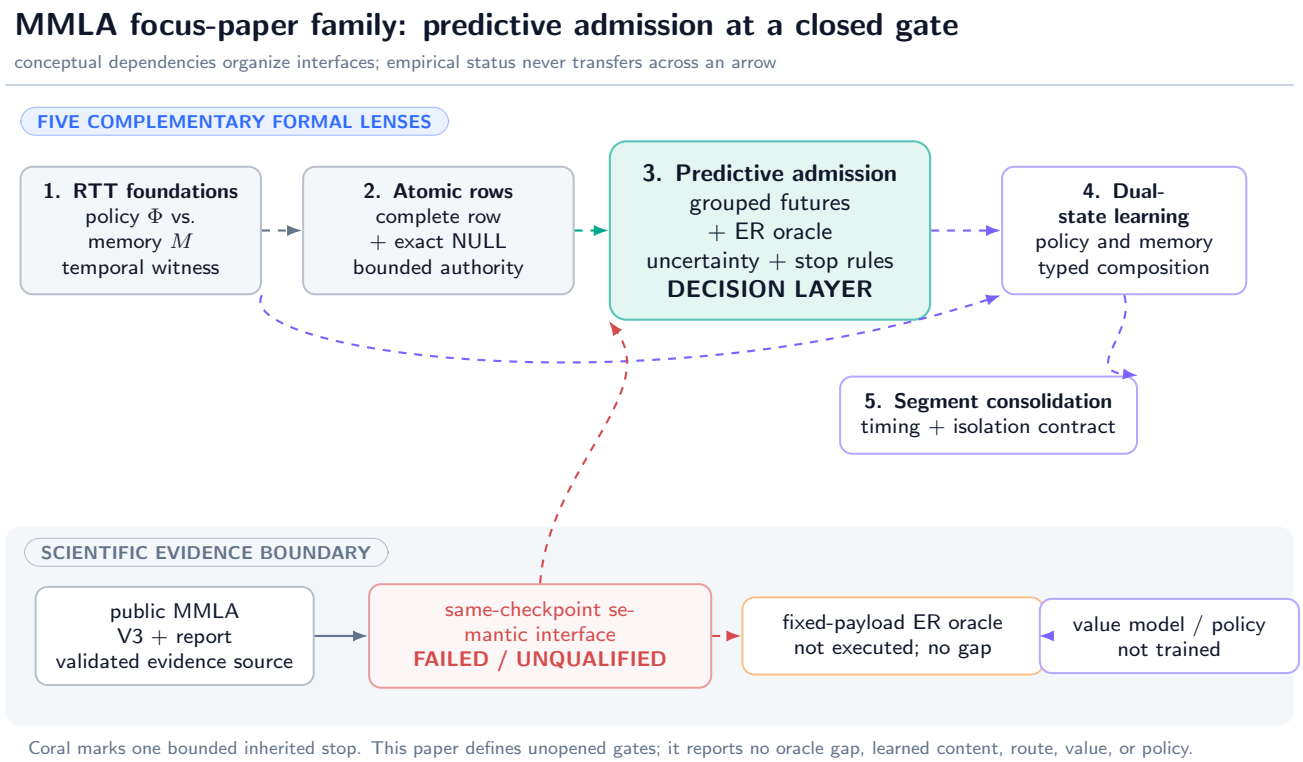

Figure 35: **Predictive admission in the five-paper conceptual map.** The coral stop at the inherited semantic-interface gate blocks any oracle, controller, or overwrite-success claim. Formal dependency arrows do not transfer empirical status.

47.3 Contributions

This paper owns the following questions.

1. It types the causal state, completed segment, event sequence, bounded resident memory, target-conditioned candidate, action set, grouped future, horizon loss, resource ledger, and deployable information set.
2. It gives event-indexed teacher semantics. Event j reads the branch state produced by earlier committed events; it never acts on a whole-segment candidate computed against an obsolete prefix.
3. It defines grouped-future sampling and action-conditioned replay, including branch weights, support, synthetic-future bias, stale leakage, collateral damage, write/NULL penalties, and resources.
4. It separates an empirical expected-risk oracle, a current-risk comparator, exact NULL, and a future-blind value model.
5. It supplies simultaneous concentration and conservative uncertainty rules, identification and impossibility results, metrics, leakage audits, full costs, stop rules, and theorem-to-implementation obligations.

This paper does not specify a row serializer, trusted assembler implementation, semantic reader, event extractor, archive policy, policy-state RTT update, or complete MMLA system. It does not run a training job, choose a checkpoint, or estimate an oracle gap. Definitions, conditional proofs, prescriptions, and planned tests are marked locally; none constitutes implementation evidence.

47.4 Central falsifiable question

For a causal prefix state $\widetilde{S}$, let $R(a \mid \widetilde{S})$ be the conditional horizon risk of feasible action a, let a^{cur} be a preregistered current-risk action, and let 0 denote NULL. The first scientific question is

$$G(\widetilde{S}) = R(a^{\mathrm{cur}} \mid \widetilde{S}) - \min_{a \in \mathcal{A}(\widetilde{S})} R(a \mid \widetilde{S}). \tag{162}$$

The relevant population claim is not that G can be positive in a hand-built example. It is that a frozen empirical procedure yields a stable, uncertainty-controlled improvement exceeding a prespecified materiality margin on fresh prefix groups and required seeds. If it does not, the value model remains closed.

CONJECTURED Some domains may contain predictable future-use structure for which a bounded authoritative memory benefits from selective admission. This conjecture can fail because the action set is poor, futures are not identified, the gap is too small, the value is not predictable from causal history, long-run pollution dominates, or total-system cost erases the benefit.

48 Typed Causal State and Deployable Information

48.1 Probability space and completed segments

Fix a probability space $(\Omega, \mathcal{B}, \mathbb{P})$ and a discrete event clock with filtration $(\mathcal{F}_t)_{t\geq 0}$. Stream instance n has identity $I_n \in \mathcal{I}$, observations $X_{n,t} \in \mathcal{X}$, and a completed segment

$$B_n = (X_{n,b_{n-1}+1}, \ldots, X_{n,b_n}) \tag{163}$$

that becomes available only at stopping time b_n. Prediction at or before b_n cannot use a memory mutation produced after B_n completes. A bounded eventizer maps the completed segment and prior state to an ordered list

$$E_n = \mathsf{Eventize}(B_n, M_{n-1}; \xi_n) = (e_{n,1}, \ldots, e_{n,J_n}), \qquad 0 \leq J_n \leq J_{\max}, \tag{164}$$

where ξ_n is recorded eventizer randomness. Eventization is itself a costed, unvalidated component; the theory conditions on its declared output.

Object	Space	Meaning	Boundary
I_n	$\mathcal{I}$	Prefix-group / stream identity	Split and receipt unit
B_n	bounded sequence	Completed observed segment	No unobserved continuation
$e_{n,j}$	$\mathcal{E}$	Event j in the ordered segment list	Processed only after events $< j$
$M_{n,j}$	$\mathcal{M} = \mathcal{R}^S$	S complete authoritative rows after event j	Fixed resident capacity
$c_{n,j,a}$	$\mathcal{R} \cup \{\perp\}$	Complete row candidate for target a	Built against current branch prefix
$A_{n,j}$	finite subset	Feasible commits plus NULL	Hard mask precedes value scoring
$F_{n,k}$	$\mathcal{F}$	Future branch k for the same prefix	Training/evaluation only
H	$\mathbb{N}$	Replay horizon	Fixed before outcomes open
L	$[0, L_{\max}]$	Registered aggregate action loss	Bounded for finite-sample results
$\mathfrak{R}$	$\mathbb{R}_+^d$	Resource ledger	Logical and measured coordinates
$S_{n,j}^-, S_{n,j}^+$	$\mathcal{S}^- \times \mathcal{S}^+$	Preconstruction / postvalidation state	Bounded; no future-bearing field

Table 36: Typed objects. Physical co-location does not merge their causal roles.

48.2 Bounded authoritative state

Let

$$M_{n,j} = (r_{n,j,1}, \ldots, r_{n,j,S}) \in \mathcal{R}^S \tag{165}$$

be the complete resident state. The row space $\mathcal{R}$ and slot count S are finite-width under the external contract. This paper assumes a trusted validator can distinguish a complete committable row from $\perp$; it does not assume that the row is factually true or semantically usable.

For event j, candidate construction for target $a \in \{1, \ldots, S\}$ is the partial map

$$c_{n,j,a} = \mathrm{Construct}(S_{n,j}^-, e_{n,j}, a, r_{n,j-1,a}, M_{n,j-1}; \zeta_{n,j,a}) \in \mathcal{R} \cup \{\perp\}, \tag{166}$$

with fixed or recorded constructor randomness $\zeta_{n,j,a}$. Target conditioning is essential because version, protection, provenance, collision, eviction, tenant, and rollback fields can depend on the old target row. A shared semantic payload may be a restricted causal probe, but the authoritative action still requires a complete target-specific assembly.

The feasible set is

$$\mathcal{A}_{n,j} = \{0\} \cup \left\{a \in \{1, \ldots, S\} : c_{n,j,a} \neq \perp,\ \mathrm{Valid}(c_{n,j,a}; M_{n,j-1}) = 1\right\}, \tag{167}$$

where action 0 denotes exact NULL. Authorization, protection, schema, version, tenant, and snapshot checks are hard predicates. A learned score cannot restore an excluded action.

48.3 Bounded, acyclic deployable information

Fix before the run a versioned observation map Ψ_{obs}. It emits

$$O_{n,j} = \Psi_{\mathrm{obs}}\Big(B_n, W_{n,j}, \{\rho_{n,j,r}^{\mathrm{read}} : r \in \mathcal{R}_{n,j}\}; v_{\mathrm{obs}}\Big) \in \mathcal{O}, \tag{168}$$

where $W_{n,j}$ is a fixed-width suffix of the completed segment and every nonlocal read has an authenticated receipt ρ^{read} whose bytes and latency are charged. The schema of $\mathcal{O}$, the suffix width, and the number and size of receipts are finite and frozen. The raw prefix $X_{n,\leq b_n}$ is therefore not a deployable input.

Let $t_{n,j}^-$ be the instant before candidate construction. The preconstruction sigma-field and state are

$$\mathcal{D}_{n,j}^- = \sigma\Big(I_n, O_{n,j}, e_{n,\leq j}, M_{n,j-1}, \mathfrak{R}_{n,j}^{\mathrm{remaining}}, \Xi_{n,j}^-, \{\zeta_{n,j,a}\}_{a=1}^S\Big) \subseteq \mathcal{F}_{t_{n,j}^-}, \tag{169}$$

$$S_{n,j}^- = \Psi_{\mathrm{dep}}^-(\mathcal{D}_{n,j}^-) \in \mathcal{S}^-. \tag{170}$$

Construction in Equation (166) uses only this state. After all candidates are constructed and hard-validated, their constructor and validator receipts enlarge the information set to

$$\mathcal{D}^{+}_{n,j} = \mathcal{D}^{-}_{n,j} \vee \sigma\Big(\{c_{n,j,a}, \rho^{\mathrm{con}}_{n,j,a}, \rho^{\mathrm{val}}_{n,j,a}\}_{a=1}^{S}, \mathcal{A}_{n,j}\Big), \tag{171}$$

$$S^{+}_{n,j} = \Psi^{+}_{\mathrm{dep}}(\mathcal{D}^{+}_{n,j}) \in \mathcal{S}^{+}, \qquad \widetilde{S}_{n,j} := S^{+}_{n,j}. \tag{172}$$

This order is acyclic: no candidate is used to define the state from which that same candidate is constructed. Scoring and action selection use S^{+}; the post-commit memory becomes readable only afterward.

DEFINITION

Definition 48.1 (Future-blind deployment)**.** A deployed scorer or decision rule is future blind at event (n, j) if its output is $\mathcal{D}^{+}_{n,j}$-measurable and its executable input graph has no continuation, future token, future answer, future evaluator output, branch identifier, or statistic computed from any of those values.

DESIGN PRESCRIPTION Declaring an input "history only" is insufficient. A conformance audit must enumerate every feature producer, cached tensor, RNG state, retrieved artifact, label join, and preprocessing path that can reach the deployment call.

48.4 Core assumptions

Assumption 48.2 (Causal timing)**.** Every event decision is $\mathcal{D}^{+}_{n,j}$-measurable, and the committed state becomes readable only after the decision time. Candidate construction is $\mathcal{D}^{-}_{n,j}$-measurable. No future branch affects eventization, construction, feasibility, or deployment features.

Assumption 48.3 (External row contract)**.** For every accepted action, the constructor and trusted assembler return one complete row bound to the current snapshot; NULL is exact identity; hard validation is deterministic for fixed inputs; and one event changes at most its selected target.

Assumption 48.4 (Finite action and loss)**.** $1 \le |\mathcal{A}_{n,j}| < \infty$ and every registered horizon loss lies in $[0, L_{\max}]$. If a natural loss is unbounded, the protocol fixes clipping or a different tail model before evaluation.

Assumption 48.5 (Frozen total order)**.** The event manifest fixes a deterministic total order $\prec_{n,j}$ over encoded action records, including NULL. Every set-valued optimization returns the $\prec_{n,j}$-least minimizer. The order is fixed before outcomes open and is included in the protocol hash.

These assumptions support later results but are not asserted to hold in the frozen system. In particular, Assumption 48.3's semantic precondition is presently unvalidated.

49 Event-Indexed Candidates, Actions, and Teacher Timing

49.1 Deployed recursion

Set $M_{n,0} = M_{n-1}$. After constructing $\mathcal{A}_{n,j}$ against $M_{n,j-1}$, a deployed decision rule selects the $\prec_{n,j}$-resolved action $A_{n,j} \in \mathcal{A}_{n,j}$ using only $\mathcal{D}^{+}_{n,j}$ and commits

$$M_{n,j} = T_{n,j}(M_{n,j-1}, A_{n,j}) = \begin{cases} M_{n,j-1}, & A_{n,j} = 0, \\ \mathrm{Commit}(M_{n,j-1}, A_{n,j}, c_{n,j,A_{n,j}}), & A_{n,j} \neq 0. \end{cases} \tag{173}$$

The segment output is $M_n = M_{n,J_n}$. Later events see earlier committed events through this state and no other within-segment authority.

PROVED UNDER STATED ASSUMPTIONS

Proposition 49.1 (Event-timing consistency)**.** *Under Assumptions 48.2 and 48.3, $M_{n,j}$ is measurable with respect to information available immediately after event j, and every candidate used at event $j+1$ is a function of $M_{n,j}$ rather than an obsolete pre-segment state.*

Proof. The base state $M_{n,0}$ is available before the first decision. Given an available $M_{n,j-1}$, Equation (169) fixes the preconstruction inputs; construction and validation then produce $\mathcal{D}^{+}_{n,j}$ without reading a post-commit value. Action selection and commit are causal by Assumption 48.2; hence $M_{n,j}$ is available after the event. Equation 166 then uses that exact state for event $j+1$. Induction over j proves the claim. □

The proposition prohibits a convenient but invalid shortcut: construct all segment candidates against $M_{n,0}$, choose several actions, and describe the result as event-recursive. Earlier commits can change target versions, feasibility, collision status, retained content, and later losses.

49.2 Branch-prefix recursion for a teacher

To price counterfactual action sequences, let $\mathbf{a}_{1:j} = (a_1, \ldots, a_j)$. Each branch has its own state

$$M_{n,0}^{()} = M_{n-1}, \qquad M_{n,j}^{(\mathbf{a}_{1:j})} = T_{n,j}^{(\mathbf{a}_{<j})}\left(M_{n,j-1}^{(\mathbf{a}_{<j})}, a_j\right), \tag{174}$$

where event summary, candidate set, and feasibility at depth j are recomputed against the branch prefix $M_{n,j-1}^{(\mathbf{a}_{<j})}$. The teacher may use one of three distinct protocols:

1. **Exact dynamic program:** enumerate every feasible branch prefix and use a future-blind continuation policy optimized outside the future expectation.
2. **Frozen rollout:** after the scored action, follow one preregistered future-blind continuation policy for later segment events.
3. **Single-event intervention:** set $J_n = 1$ or forbid later writes in the scoring horizon. This is the cleanest first causal test.

Mixing these protocols is not allowed. In particular, choosing later actions separately for each revealed future is a hindsight policy and is not a deployable cost-to-go.

For a sequence policy π whose event-q action is measurable with respect to the branch state before event q, define

$$Q_{n,j}^{\mathrm{T}}(a \mid \widetilde{S}_{n,j}^{(\mathbf{a}_{<j})}) = \inf_{\pi \in \Pi_{j+1:J_n}^{\mathrm{fb}}} \mathbb{E}\left[L_H\left((\mathbf{a}_{<j}, a, A_{j+1:J_n}^{\pi}), F\right) \,\middle|\, \widetilde{S}_{n,j}^{(\mathbf{a}_{<j})}\right]. \tag{175}$$

The infimum is outside the expectation over F. Moving it inside would let the teacher observe the realized future before selecting later events. If the infimum is attained by more than one policy, the manifest's frozen lexicographic policy order resolves the tie; no future outcome may do so.

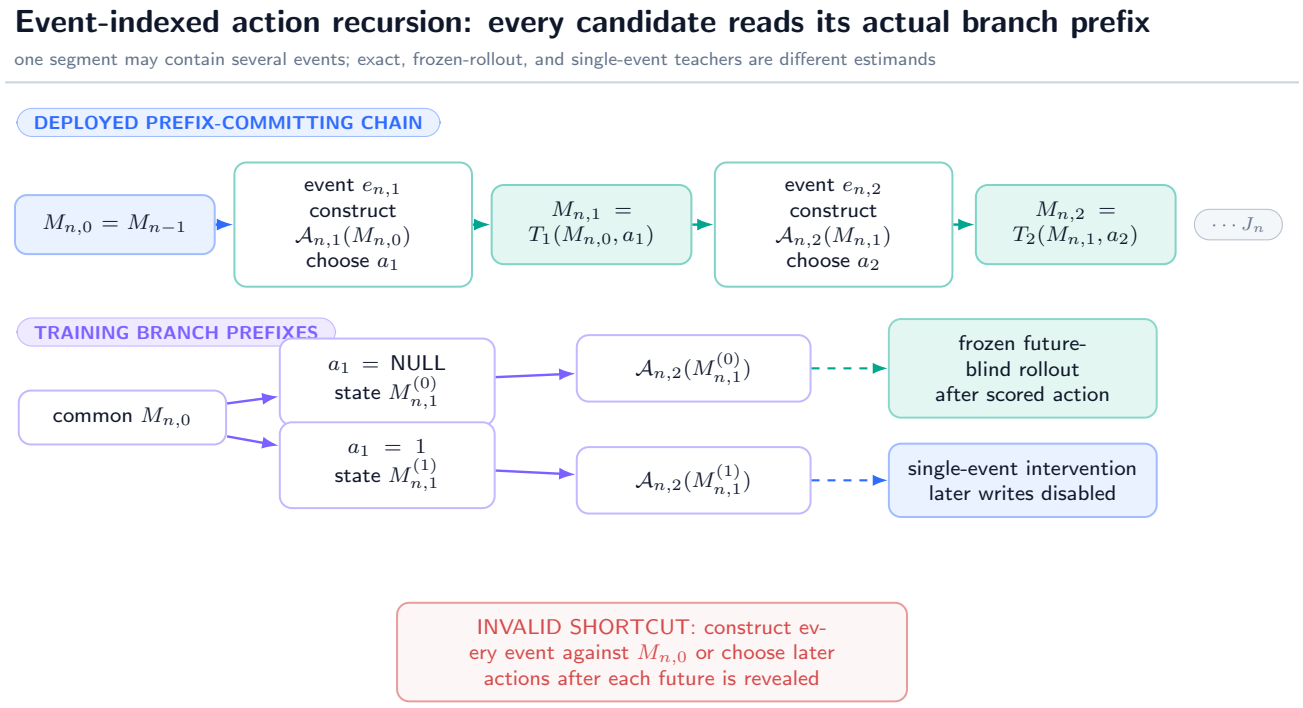

Figure 36: **Event-indexed recursion** (DEFINITION)**.** Candidate/action sets at event j are constructed from the branch state after earlier commits. Exact, frozen-rollout, and single-event teachers are distinct. No whole-segment action may reuse candidates from an obsolete prefix.

49.3 No conflated segment action

PROVED UNDER STATED ASSUMPTIONS

Theorem 49.2 (Sequential composition and support)**.** *Under Assumption 48.3, each legal event changes at most one resident row. Across a segment of J_n events, the final state can differ from M_{n-1} on at most J_n distinct slots. A later* NULL *leaves earlier committed prefixes unchanged.*

Proof. At one event, NULL changes zero rows and a non-NULL action replaces exactly its selected row. The union of target slots over J_n events therefore has size at most J_n, and every row outside that union remains unchanged. Equation 173 is prefix committing, so an identity transition at event j maps $M_{n,j-1}$ to itself rather than to M_{n-1}. □

This theorem establishes a mechanical support bound only. It does not show that a one-row change is semantically local, that a segment is all-or-nothing, or that a teacher can safely ignore later events.

49.4 Required timing receipt

DESIGN PRESCRIPTION Every future implementation must record, per event and per counterfactual branch: prefix-state digest, event digest, constructor/assembler version, candidate digests, feasibility mask, action, commit or NULL result, successor digest, future-group identifier, rollout protocol, and resource increments. A candidate whose recorded predecessor differs from the replay prefix is invalid even if its bytes parse.

50 Grouped Alternative Futures and Sampling Assumptions

50.1 Physical prefix groups

Fix one post-candidate/pre-action decision state $\widetilde{S}_g^+$ and its immutable pre-action memory snapshot M_g. A grouped-future record is

$$\mathcal{G}_g = \left(g, \widetilde{S}_g^+, M_g, \mathcal{A}_g, \{c_{g,a}\}_{a\in\mathcal{A}_g\setminus\{0\}}, \{(F_{g,k}, w_{g,k})\}_{k=1}^{K_g}\right), \tag{176}$$

where every branch shares exactly the same causal prefix, candidates, and pre-action snapshot; $w_{g,k} \geq 0$ and $\sum_k w_{g,k} = 1$. The group ID, not a future branch, is the split unit.

DEFINITION

Definition 50.1 (Grouped future)**.** A grouped future is a continuation sampled or constructed for one fixed causal prefix state. "Similar prompts," paraphrases with different hidden states, or branches joined only after answer labels are known are not grouped futures without an explicit causal equivalence argument.

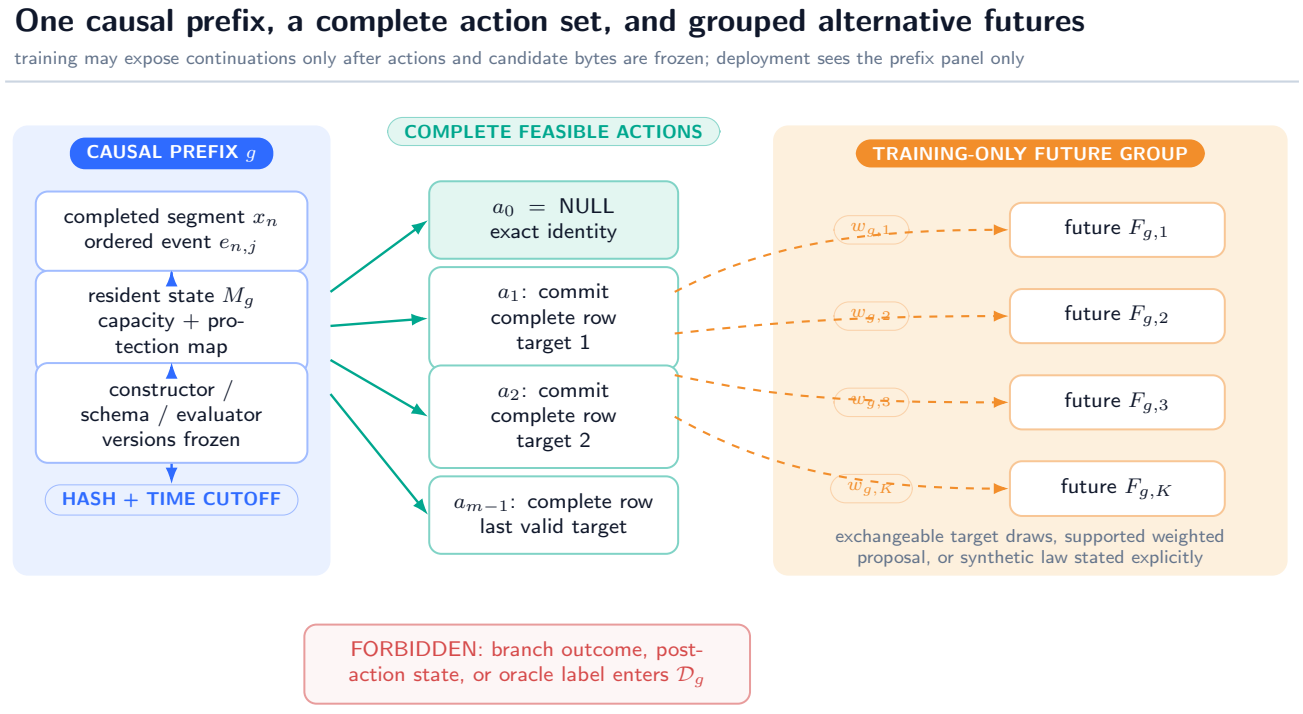

Figure 37: **Causal prefix and grouped futures.** Dashed amber branches are training/evaluation-only continuations from one frozen prefix. Solid navy and teal paths are deployable. Future tokens may price actions but no future-derived field crosses the deployment boundary.

50.2 Sampling models

Let P_g be the intended deployment continuation distribution conditional on $\widetilde{S}_g^+$. Three data regimes must be distinguished. Before any outcome is opened, the group record fixes one sampling flag

$$\mathsf{samp}_g \in \{\mathsf{iid}, \mathsf{stratified}, \mathsf{finite}\}. \tag{177}$$

The flag is part of the estimand: it selects the estimator, its center, and its finite-sample argument. It may not be changed after inspecting an action advantage.

Regime	Identifying requirement	Mandatory limitation
Repeated real continuations	Branches are conditionally exchangeable draws from P_g or a declared finite population	Repeated collection may alter users or environment
Controlled generator	Generator seed and branch law are fixed before action outcomes; support covers the declared target population	Identifies the generator distribution, not natural deployment
Model-generated futures	Generator is frozen and causally separated from action scoring; calibration compares its branch law with fresh real continuations	Self-consistency and mode collapse may bias values
Logged observational futures	Every evaluated action has positive logging support, propensities and confounders are recorded, and a valid off-policy estimator is used	Missing support cannot be repaired by modeling alone

Table 37: Grouped-future regimes. Synthetic and model-generated futures require an explicit target-distribution boundary.

Assumption 50.2 (Conditional exchangeability in the iid regime)**.** If $\mathsf{samp}_g = \mathsf{iid}$, then $(F_{g,1}, \ldots, F_{g,K_g})$ are conditionally independent draws from one declared branch law $Q_g(\cdot \mid \widetilde{S}_g^+)$, and the nonnegative weights are fixed independently of their realized losses. If this statement is false, the iid estimator and its concentration theorem are inapplicable.

If $\mathsf{samp}_g = \mathsf{stratified}$, the stratum map, sampling fractions, and target stratum masses are frozen, and risk is estimated by the corresponding stratum-weighted estimator with a stratum-aware variance or concentration calculation. If $\mathsf{samp}_g = \mathsf{finite}$, the declared estimand is a finite-population mean, branches are sampled without replacement by a frozen design, and the analysis uses a without-replacement bound (including its finite-population correction). Neither regime is silently relabeled iid.

Assumption 50.3 (Action support and consistency)**.** Every action whose risk is estimated is feasible at the common snapshot and has a well-defined potential replay outcome. The replay engine returns that outcome when the corresponding action is applied. If logged data rather than full replay are used, the logging propensity is positive wherever the target action has positive probability.

Assumption 50.4 (Exogenous or interventional future)**.** Either the future evaluation sequence is conditionally exogenous to the memory action, so the same branch can be replayed under all actions, or the environment supplies valid action-conditioned counterfactual futures. Reusing one observed future under all actions is not valid when the action changes the future-generating process.

These are causal-identification assumptions in the potential-outcome sense [86, 85]. They are stricter than ordinary predictive train/test separation.

50.3 Branch weights and effective sample size

In the iid regime, weights may encode fixed quadrature or replication weights and must not depend on realized losses. In the stratified and finite-population regimes, they are the design weights required by the frozen estimand. They must not be tuned to increase the preferred action's advantage. Define the descriptive concentration diagnostic

$$K_{\mathrm{eff},g} = \frac{1}{\sum_{k=1}^{K_g} w_{g,k}^2}, \tag{178}$$

which equals K_g for uniform weights and approaches one when a single branch dominates. Reporting only K_g can therefore overstate information. Equation 178 is not, by itself, a license to reuse the iid theorem under stratification, clustering, or sampling without replacement.

If branches are sampled under proposal law Q_g but target risk uses P_g, importance weights require $P_g \ll Q_g$ and bounded or otherwise controlled density ratios. Doubly robust off-policy estimators can combine a reward model with propensities [25]; they do not solve zero support or hidden confounding.

50.4 Synthetic-future bias

For any action loss bounded in $[0, L_{\max}]$, define total variation as

$$\mathrm{TV}(P, Q) = \sup_A |P(A) - Q(A)|. \tag{179}$$

Then the risk under a synthetic branch law can differ from deployment risk even with infinitely many synthetic samples. Section 54 proves the bound

$$|R_P(a) - R_Q(a)| \le L_{\max}\,\mathrm{TV}(P, Q). \tag{180}$$

The sampling error shrinks with more branches; the distributional bias does not. A model-generated teacher is therefore not an oracle for the real future unless the target relation is independently justified.

50.5 Split obligations

DESIGN PRESCRIPTION Fitting, calibration, and final evaluation must be disjoint at the prefix-family or generating-process level. All branches from one group remain together. Near-duplicate prefixes, shared answer templates, generator IDs, and deterministic seed descendants are audited before labels are opened. Final-only means one frozen model, threshold, horizon, loss vector, and confidence procedure are read once on the held-out groups; a failed seed is never replaced.

51 Action-Conditioned Replay and Horizon Loss

51.1 One immutable snapshot, every feasible action

For group g, action $a \in \mathcal{A}_g$, and future branch $F_{g,k}$, replay begins from the same immutable pre-action snapshot M_g. Define

$$M_g^{(a)} = \begin{cases} M_g, & a = 0, \\ \mathrm{Commit}(M_g, a, c_{g,a}), & a \neq 0. \end{cases} \tag{181}$$

The continuation is then executed under $M_g^{(a)}$. A cached hidden trace computed before the action and merely rescored afterward is not action-conditioned replay when memory can affect future hidden-state evolution.

The complete replay array is

$$\mathsf{Y}_g = \left(Y_{g,a,k,h} : a \in \mathcal{A}_g,\ 1 \le k \le K_g,\ 1 \le h \le H\right), \tag{182}$$

with action, future, and horizon axes. NULL occupies a full action slice. It is not a missing baseline.

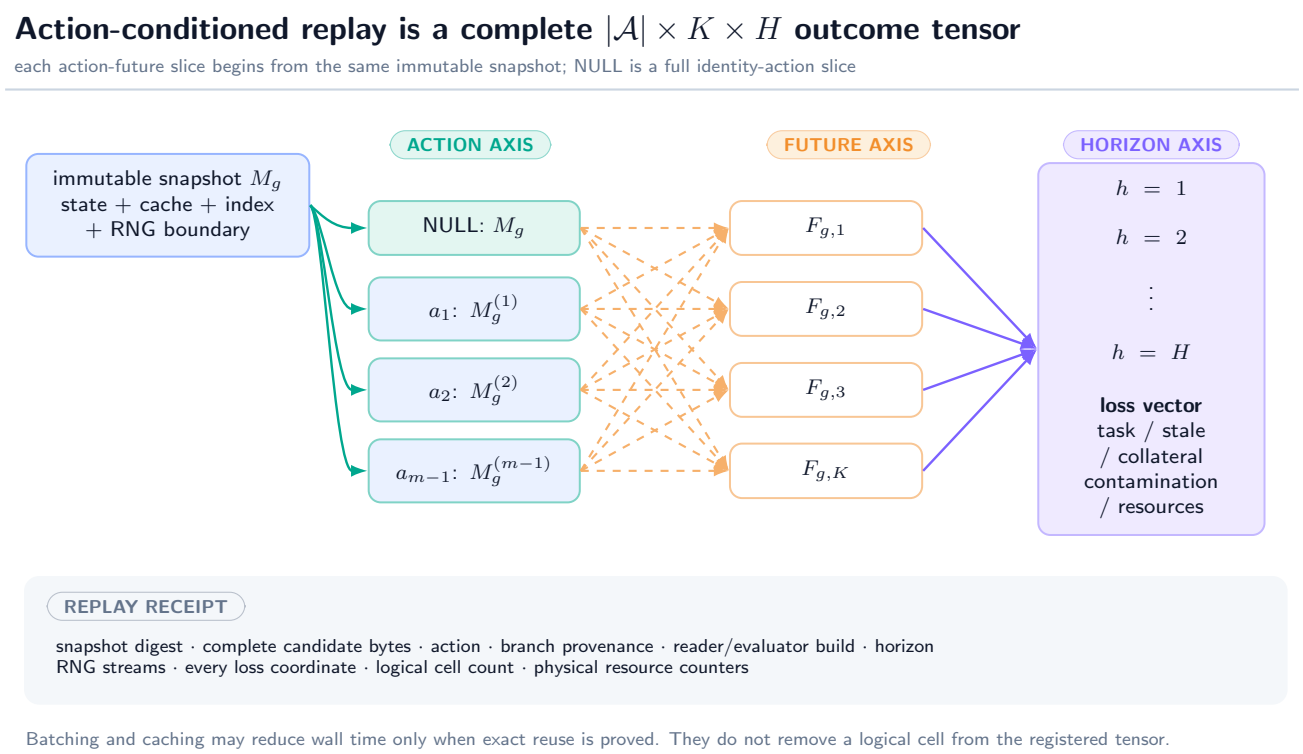

Figure 38: **Action-conditioned replay tensor (DEFINITION / training only).** Every feasible write and NULL starts from one immutable snapshot, is evaluated on every grouped future, and contributes all registered horizon terms. The visible $|\mathcal{A}| \times K \times H$ lattice is logical work even when kernels batch it.

51.2 Loss vector and scalarization

For horizon step h, let

$$\ell_{g,a,k,h} = \left(\ell^{\text{task}}, \ell^{\text{stale}}, \ell^{\text{coll}}, \ell^{\text{cont}}, \ell^{\text{sec}}, \ell^{\text{lat}}, \ell^{\text{traffic}}\right)_{g,a,k,h} \in \mathbb{R}^7_+. \tag{183}$$

The coordinates respectively measure task error, obsolete-value leakage, damage to unrelated information, admitted-content contamination, security/policy harm not already removed by hard infeasibility, latency, and memory/data traffic. The registered loss schema includes deterministic componentwise caps

$$0 \le \ell_{g,a,k,h} \le \overline{\ell} \in \mathbb{R}^7_+, \quad 0 \le D^{\text{miss}}_{g,k} \le \overline{D}_{mathrmmiss}, \quad 0 \le \mathfrak{R}_{g,a,k} \le \overline{\mathfrak{R}}, \tag{184}$$

where inequalities are componentwise and every cap is fixed from the measurement protocol before calibration. Values outside the registered domain are an audit failure; they are not clipped after outcome inspection. The event-level loss is

$$\begin{aligned} L_g(a, F_{g,k}) = &\sum_{h=1}^{H} \gamma^{h-1} \left\langle \lambda, \ell_{g,a,k,h} \right\rangle + \lambda_{\text{write}} \mathbf{1}[a \neq 0] \\ &+ \lambda_{\text{null}} \mathbf{1}[a = 0] D^{\text{miss}}_{g,k} + \left\langle \lambda_{\text{res}}, \mathfrak{R}_{g,a,k} \right\rangle, \end{aligned} \tag{185}$$

where $\gamma \in (0, 1]$, all weights are nonnegative and fixed before evaluation, D^{miss} prices a missed useful update without presuming every NULL is wrong, and $\mathfrak{R}$ is the replay resource vector. A hard safety violation should normally remove an action before scoring rather than be traded against task loss.

The missed-update coordinate is disjoint by construction: it may encode only opportunity loss not already represented in task, stale, collateral, contamination, security, latency, traffic, or resource coordinates. If the evaluator cannot establish that disjointness, λ_{null} is fixed to zero. With $\Gamma_H = \sum_{h=1}^{H} \gamma^{h-1}$, the declared scalarization therefore satisfies the auditable range

$$0 \le L_g(a, F) \le \Gamma_H \langle \lambda, \overline{\ell} \rangle + \lambda_{\text{write}} + \lambda_{\text{null}} \overline{D}_{\text{miss}} + \langle \lambda_{\text{res}}, \overline{\mathfrak{R}} \rangle =: L_{\max}. \tag{186}$$

The replay validator recomputes every coordinate and this bound from the frozen schema; a violation invalidates the group rather than expanding $L_{\max}$ retrospectively.

Term	Operational measurement	Boundary
Task loss	Frozen evaluator on future task/output	Evaluator error audited separately
Stale leakage	Old value emitted, selected, or remains authoritative when superseded	Must distinguish unknown from stale
Collateral damage	Positive loss change on unrelated facts/tasks versus the common snapshot	Neural cross-row effects measured, not assumed absent
Contamination	Severity of false, stale, malicious, or wrongly scoped authority	Requires truth/source contract
Write charge	Fixed cost for any committed row	Prevents free churn
NULL miss	Opportunity cost only when future evidence establishes a useful feasible write	Does not make abstention an error by default
Resources	Tokens, FLOPs, bytes, traffic, latency, energy, archive/index and validation work	Teacher and deployment ledgers stay separate

Table 38: Loss contract. Every coordinate and scalarization weight is preregistered; changing it changes the estimand.

DEFINITION

Definition 51.1 (Target, proposal, and empirical horizon risk)**.** For target continuation law $P_g(\cdot \mid \widetilde{S}^+_g)$,

$$R_{P_g}(a) = \mathbb{E}_{F \sim P_g}\left[L_g(a, F) \,\middle|\, \widetilde{S}^+_g \right]. \tag{187}$$

For the proposal law $Q_g(\cdot \mid \widetilde{S}^+_g)$,

$$R_{Q_g}(a) = \mathbb{E}_{F \sim Q_g}\left[L_g(a, F) \,\middle|\, \widetilde{S}^+_g \right]. \tag{188}$$

The raw grouped empirical proposal risk is

$$\widehat{R}_{Q_g}(a) = \sum_{k=1}^{K_g} w_{g,k} L_g(a, F_{g,k}). \tag{189}$$

The notation separates sampling and target distributions. When $Q_g = P_g$ and the iid assumptions hold, $\widehat{R}_{Q_g}$ estimates target risk. When $Q_g \neq P_g$, it estimates proposal-law risk unless a separately declared support-valid correction or shift certificate maps it to P_g. For compactness below, an unsubscripted R_g always means R_{P_g}; an unsubscripted empirical risk is never used.

51.3 Paired action differences

Because all actions share each future branch, the paired difference

$$D_{g,k}(a,b) = L_g(a, F_{g,k}) - L_g(b, F_{g,k}) \tag{190}$$

has the sign convention "positive means action a is worse than action b." It directly estimates the proposal-law contrast $R_{Q_g}(a) - R_{Q_g}(b)$ and cancels branch difficulty. Its variance for uniform independent branches is

$$\frac{1}{K_g}\left[\mathrm{Var}(L_a) + \mathrm{Var}(L_b) - 2\,\mathrm{Cov}(L_a, L_b)\right]. \tag{191}$$

Pairing reduces variance when action losses are nonnegatively correlated on the same future. It does not repair a biased branch law or invalid counterfactual replay.

51.4 Replay integrity

DESIGN PRESCRIPTION A valid replay record binds the source snapshot, complete candidate bytes, action, future branch, reader/evaluator build, horizon, RNG streams, loss coordinates, and resource increments. Action order is randomized or counterbalanced when shared hardware state could create interference. Each branch begins from an independently restored snapshot; caches and derived indexes are either restored or rebuilt under the selected action. A replay that omits NULL, prunes an action after seeing its outcome, or lets one branch inherit another branch's state is invalid.

52 Expected-Risk Oracle, Current-Risk Comparator, and Qualification Gate

52.1 Three distinct decision objects

The empirical proposal-risk oracle for group g is

$$\widehat{a}_g^{\mathrm{ER},Q} = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} \widehat{R}_{Q_g}(a). \tag{192}$$

It is an evaluation rule using training-only future branches, with the frozen protocol order $\prec_g$ resolving every tie. It is not deployed, it is not a learned policy, and it is not a target-risk oracle unless a valid Q_g–P_g bridge is supplied below.

The current-risk comparator is fixed independently:

$$a_g^{\mathrm{cur}} = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} C_g^{\mathrm{cur}}(a; \widetilde{S}_g^{+}), \tag{193}$$

where C^{cur} may include current reconstruction, verification, immediate stale suppression, and write cost but no realized continuation. Its exact definition, tie rule, and parameters are preregistered. NULL is also reported independently, even if $a^{\mathrm{cur}} = 0$.

The comparator registration record binds the exact formula, the named fields of the postvalidation state S_g^{+}, all feature normalizations and missing-value rules, constructor/validator/evaluator versions, scalar weights, $\prec_g$, a configuration hash, and the registration timestamp. A field absent from that record cannot be introduced after future outcomes are observed.

Finally, the per-future hindsight action

$$a_{g,k}^{\mathrm{hind}} = \min_{\prec_g} \arg\min_{a} L_g(a, F_{g,k}) \tag{194}$$

is an unattainable information ceiling. Averaging its loss answers a different question from choosing one action that minimizes expected loss before the branch is known.

52.2 Expected oracle gap

Define group-level advantages over current risk and NULL:

$$G_g^{\mathrm{cur}} = R_{P_g}(a_g^{\mathrm{cur}}) - \min_{a \in \mathcal{A}_g} R_{P_g}(a), \tag{195}$$

$$G_g^{0} = R_{P_g}(0) - \min_{a \in \mathcal{A}_g} R_{P_g}(a). \tag{196}$$

For a population of fresh prefix groups $g \sim \mathcal{P}$, the target gaps are $G^{\mathrm{cur}} = \mathbb{E}_g G_g^{\mathrm{cur}}$ and $G^{0} = \mathbb{E}_g G_g^{0}$. An oracle can have a positive advantage over current-risk writing but none over NULL, or vice versa. Both comparisons are required.

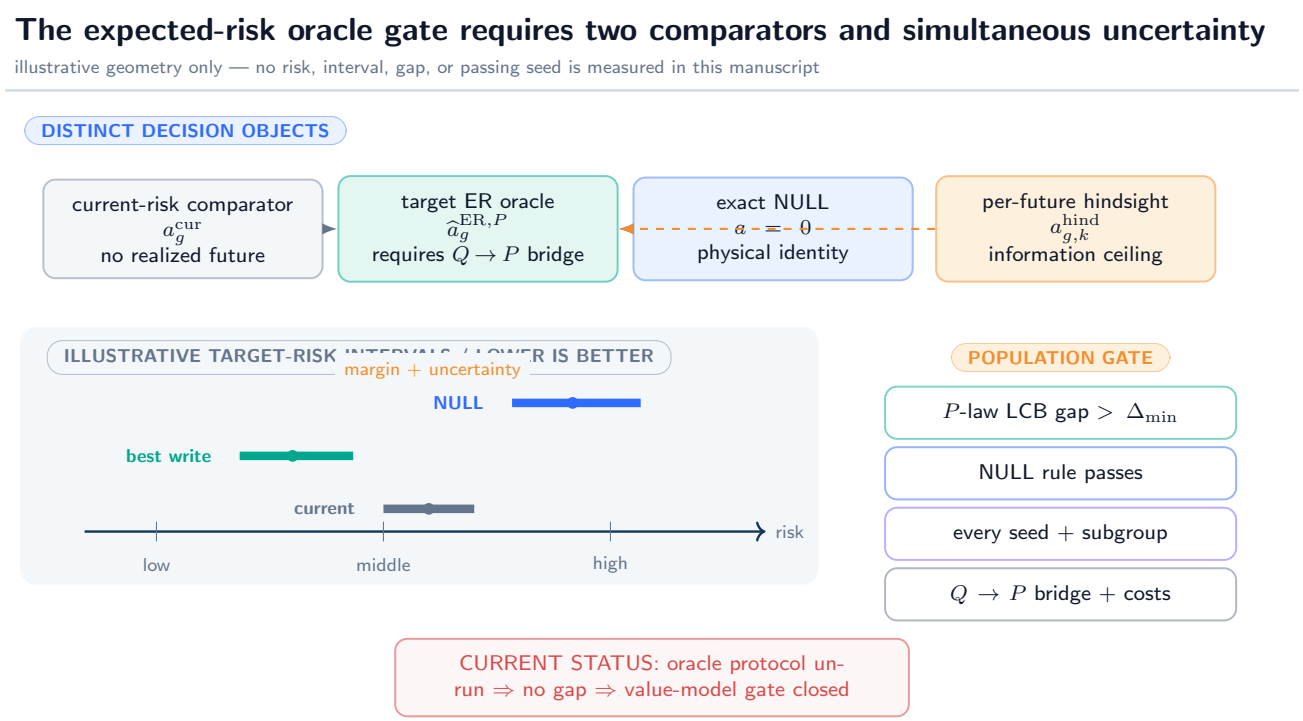

Figure 39: **Oracle qualification geometry (PLANNED TEST).** The empirical expected-risk oracle, current-risk comparator, and NULL are distinct. A stable simultaneous lower confidence bound must exceed the preregistered materiality margin on fresh groups and every required seed; otherwise the controller gate stays closed. No such gap is reported here.

52.3 Weighted concentration within one group

PROVED UNDER STATED ASSUMPTIONS

Theorem 52.1 (Simultaneous weighted Hoeffding bound)**.** *Assume 50.2, let $m_g = |\mathcal{A}_g|$, and suppose $0 \le L_g(a, F) \le L_{\max}$. Conditional on $\widetilde{S}_g^+$, with probability at least $1 - \delta$,*

$$\max_{a \in \mathcal{A}_g} \left| \widehat{R}_{Q_g}(a) - R_{Q_g}(a) \right| \le L_{\max} \sqrt{\frac{\sum_{k=1}^{K_g} w_{g,k}^2}{2} \log \frac{2m_g}{\delta}} =: \varepsilon_g(\delta). \tag{197}$$

Proof. For fixed a, weighted Hoeffding gives $\mathbb{P}(|\sum_k w_k(L_{a,k} - \mathbb{E}L_{a,k})| > \epsilon) \le 2\exp[-2\epsilon^2/(L_{\max}^2 \sum_k w_k^2)]$. A union bound over m_g actions makes the failure probability at most δ. Solving for ϵ gives Eq. 197. □

The bound controls action selection and makes the multiple-action penalty explicit. It centers on R_{Q_g}, not R_{P_g}, when the branch law differs from deployment.

PROVED UNDER STATED ASSUMPTIONS

Corollary 52.2 (Stable empirical action)**.** *Let a_g^* be the unique minimizer of R_{Q_g} and*

$$\Delta_g = \min_{a \ne a_g^*} \left(R_{Q_g}(a) - R_{Q_g}(a_g^*) \right). \tag{198}$$

If $\Delta_g > 2\varepsilon_g(\delta)$, then $\widehat{a}_g^{\mathrm{ER},Q} = a_g^*$ with probability at least $1 - \delta$.

Proof. On the simultaneous event, the optimal estimate can increase by at most ε_g and a competitor can decrease by at most ε_g. A true gap larger than $2\varepsilon_g$ cannot reverse. □

Variance-sensitive empirical Bernstein intervals may be used under their stated conditions [65], but the estimator, finite-sample constants, action multiplicity, and calibration split must be frozen. Selecting whichever interval makes the gap significant is not valid.

52.4 Mandatory proposal-to-target bridge

Proposal-law concentration alone cannot certify a deployment-law advantage. For every group, exactly one of the following bridge records must be frozen before qualification:

1. **Law equality:** a design argument and audit establish $Q_g = P_g$ on the declared continuation space.
2. **Support-valid reweighting:** $P_g \ll Q_g$, the density ratio $r_g = \mathrm{d}P_g/\mathrm{d}Q_g$ and an upper bound $0 \le r_g \le \bar{r}_g < \infty$ are fixed independently of outcomes, and a registered importance-weighted estimator with its own simultaneous finite-sample interval is used. The unweighted bound in Theorem 52.1 is not reused.
3. **Certified shift set:** an external, held-out procedure provides d_g such that $\mathrm{TV}(P_g, Q_g) \le d_g$. If $[L_g^Q(a), U_g^Q(a)]$ is a simultaneous proposal-risk interval, the target interval is conservatively inflated to

$$[L_g^P(a), U_g^P(a)] = [\max\{0, L_g^Q(a) - L_{\max} d_g\}, \min\{L_{\max}, U_g^Q(a) + L_{\max} d_g\}]. \tag{199}$$

The shift certificate declares whether generator, environment, and evaluator discrepancies are included. A discrepancy already absorbed into d_g may not be charged again as an independent uncertainty bonus. If no bridge applies, target intervals are undefined and the qualification gate is closed.

52.5 Population gate

Protocol 52.3 (Oracle-gap qualification)**.** PLANNED TEST For each fixed seed, estimate paired group-level target-risk advantages on a fresh namespace using one frozen bridge above. Construct simultaneous confidence intervals for current-risk and NULL advantages across actions, primary subgroups, horizons, and required seeds using a preregistered group-level procedure. The expected-risk oracle qualifies only if:

1. the proposal-to-target bridge is valid and its uncertainty is included exactly once;
2. the lower confidence bound for G^{cur} exceeds materiality margin $\Delta^{\mathrm{cur}}_{\min} > 0$;
3. the registered relation to NULL passes its separate noninferiority or superiority rule;
4. action rankings, sign, stale leakage, and collateral damage are stable on every required seed and primary subgroup;
5. no leakage, support, evaluator, or replay audit fails; and
6. full teacher and deployment costs are published.

If any condition fails, Q_ψ is not trained.

UNVALIDATED Protocol 52.3 has not been executed. This manuscript defines the gate and proves conditional properties; it does not claim an oracle advantage of any magnitude.

53 Uncertainty-Aware Conservative Admission

53.1 Simultaneous intervals

The registration declares one and only one simultaneous confidence family

$$\mathcal{J} = \{(s, g, z, h, a) : s \in \mathcal{S}_{\mathrm{req}},\ g \in \mathcal{G}_{\mathrm{final}},\ z \in \mathcal{Z}_{\mathrm{primary}},\ h \in \mathcal{H}_{\mathrm{reg}},\ a \in \mathcal{A}_g\}, \tag{200}$$

including every required seed, final prefix group, primary subgroup, registered horizon summary, and feasible action. The family, its alpha allocation, and its proposal-to-target bridge are frozen together; overlapping "primary" families or a second, more favorable correction are not permitted.

For each action, let $[L_g^P(a), U_g^P(a)]$ be the resulting interval for target risk $R_{P_g}(a)$, constructed without final-set tuning. The required actionwise statement below is a projection of the joint event over $\mathcal{J}$:

$$\mathbb{P}\left(R_{P_g}(a) \in [L_g^P(a), U_g^P(a)] \text{ for every } a \in \mathcal{A}_g\right) \ge 1 - \delta. \tag{201}$$

Marginal 95% intervals for many actions do not imply 95% coverage of the selected action. Bonferroni, finite-class concentration, or another frozen simultaneous procedure must charge action selection. If thresholds are also chosen on calibration data, final coverage is evaluated on untouched groups.

Let $\mathcal{W}_g = \mathcal{A}_g \setminus \{0\}$. The conservative write candidate is defined only when a feasible write exists:

$$i_g^+ = \begin{cases} \min_{\prec_g} \arg\min_{a \in \mathcal{W}_g} U_g^P(a), & \mathcal{W}_g \neq \varnothing, \\ \bot, & \mathcal{W}_g = \varnothing, \end{cases} \tag{202}$$

where $\perp$ is a non-action sentinel and $\prec_g$ is the fixed tie rule. For required benefit margin $\mu \geq 0$, define

$$\pi_{\text{cert}}(\widetilde{S}_g^+) = \begin{cases} i_g^+, & i_g^+ \neq \perp \text{ and } U_g^P(i_g^+) + \mu < L_g^P(0), \\ 0, & \text{otherwise.} \end{cases} \tag{203}$$

If there is no feasible write, the rule returns NULL.

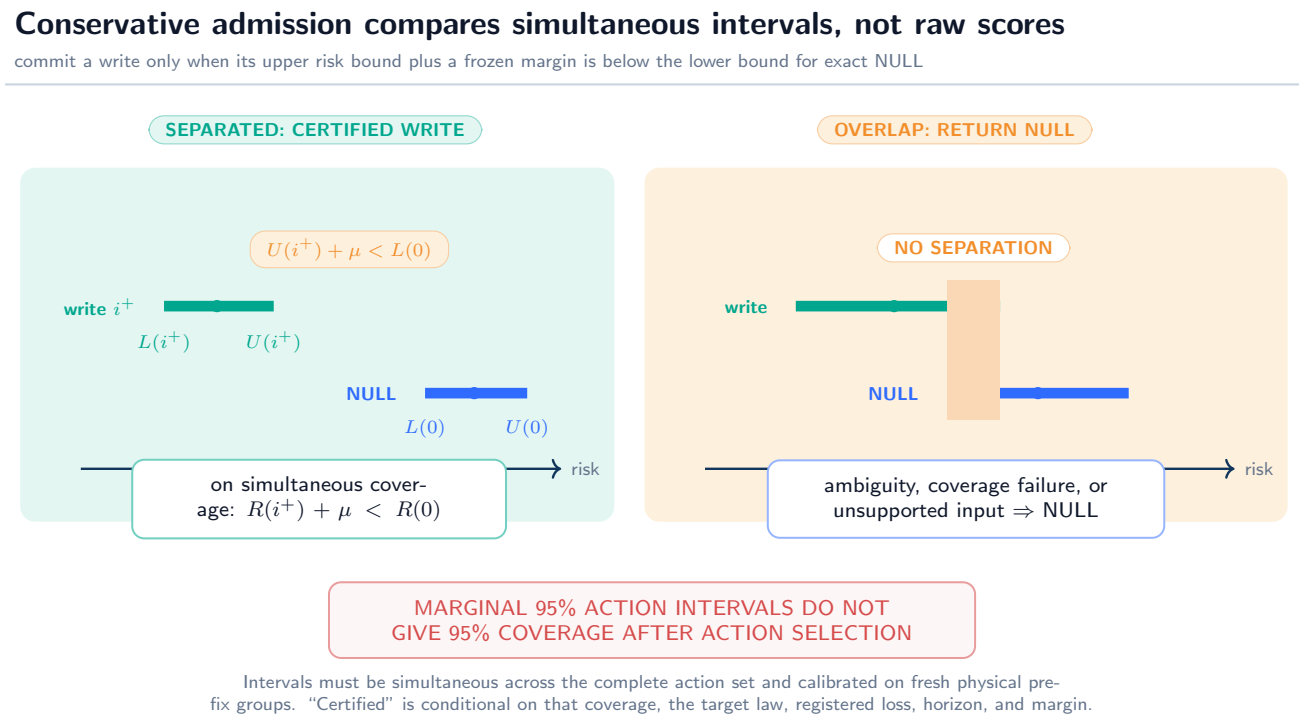

Figure 40: **Conservative UCB/LCB admission (PROVED UNDER STATED ASSUMPTIONS under simultaneous coverage).** The best feasible write commits only when its upper risk bound plus margin is below the lower bound for NULL. Overlap, multiplicity, or failed calibration returns NULL.

PROVED UNDER STATED ASSUMPTIONS

Theorem 53.1 (Certified improvement over NULL)**.** *Under Eq. 201, every non-NULL action selected by Eq. 203 satisfies*

$$R_{P_g}(i_g^+) + \mu < R_{P_g}(0) \tag{204}$$

with probability at least $1 - \delta$.

Proof. On the simultaneous-coverage event, $R_{P_g}(i_g^+) \leq U_g^P(i_g^+)$ and $L_g^P(0) \leq R_{P_g}(0)$. The commit rule therefore yields $R_{P_g}(i_g^+) + \mu \leq U_g^P(i_g^+) + \mu < L_g^P(0) \leq R_{P_g}(0)$. The event has probability at least $1 - \delta$. □

This result certifies improvement over NULL under the interval assumptions. It does not prove that i_g^+ is the globally best write. If target correctness matters, require the additional separation

$$U_g^P(i_g^+) + \eta < \min_{a \in \mathcal{W}_g \setminus \{i_g^+\}} L_g^P(a), \tag{205}$$

for a prespecified $\eta \geq 0$, or report the action as beneficial but target-ambiguous. Here and below $\min \varnothing := +\infty$; thus a sole feasible write has no unresolved write competitor, while an empty write set still returns NULL by Eq. 203.

53.2 NULL calibration

NULL is both a physical identity action and a statistical reject option, related to classical error-reject tradeoffs [17]. It needs its own calibration. A model can achieve low write error by returning NULL everywhere, or high recall by writing everywhere. Required outputs include:

- empirical interval coverage for NULL and non-NULL actions separately;
- precision and recall of NULL on preregistered write-worthy and null-worthy groups;

- risk-coverage curves as μ varies only on calibration data;
- subgroup calibration by horizon, capacity, action count, source reliability, and event type; and
- an ambiguity rate for groups where intervals do not separate.

DEFINITION

Definition 53.2 (Write-worthy, null-worthy, and ambiguous)**.** For materiality margin μ and true best-write target risk $R^W_{P_g} = \min_{a \in \mathcal{W}_g} R_{P_g}(a)$, a group is write-worthy if $R^W_{P_g} + \mu < R_{P_g}(0)$, null-worthy if $R_{P_g}(0) + \mu < R^W_{P_g}$, and ambiguous otherwise, using $\min \varnothing = +\infty$.

Ambiguous groups are not forced into a favorable binary label. A false write is a non-NULL decision on a null-worthy group; a false NULL is NULL on a write-worthy group; a wrong-target write has regret above a fixed target tolerance even when some write is beneficial.

53.3 Uncertainty sources

Source	Required treatment	Failure boundary
Finite future branches	Concentration or group bootstrap with action multiplicity	Small K_{eff} makes intervals wide
Finite prefix groups	Group-level resampling; no branch-level pseudoreplication	Shared prefixes are not independent units
Value-model uncertainty	Calibrated ensemble, quantile, or conformal-style method fixed before final use	Neural scores are not confidence bounds by naming
Distribution shift	Shift audit or declared robust set	Nominal coverage need not transfer
Evaluator uncertainty	Error/sensitivity analysis in the loss scale	Miscalibration can certify the wrong action
Constructor randomness	Freeze seeds or integrate over recorded draws	Selection can exploit lucky candidates

Table 39: Uncertainty sources. Aleatoric branch variation, estimation error, model error, and evaluator error are not interchangeable.

The registered resampling hierarchy mirrors the design: prefix families are the outer independent units; prefix groups remain intact within a family; all action slices for a branch remain paired; and constructor draws, evaluator draws, and branch draws are resampled only at their recorded nested levels. A branch-level bootstrap that treats $K_g|\mathcal{A}_g|H$ tensor cells as independent is forbidden. Analytical and resampling components may be composed only by the single frozen rule attached to $\mathcal{J}$.

DESIGN PRESCRIPTION Any monitoring failure, coverage shortfall, unsupported action, or out-of-scope group switches the deployed selector to NULL-only. This is a design rule, not evidence that a future implementation detects every shift.

54 Identification, Distribution Shift, and Impossibility

54.1 What full grouped replay identifies

Let $Y_g(a, F)$ be the complete potential replay outcome for action a and continuation F. Under Assumptions 50.2–50.4, full action replay from one common snapshot identifies $R_{Q_g}(a) = \mathbb{E}_{Q_g} L(Y_g(a, F))$ for every feasible action, up to sampling error. If $Q_g = P_g$, this is the deployment conditional risk. If the future is action dependent, identification instead requires valid samples from each interventional law $P_g(F \mid \text{do}(A = a), \widetilde{S}^+_g)$.

The conclusion is narrow. Grouped futures do not identify a different deployment branch law, an action absent from support, an evaluator's latent truth, or the value of features that would be unavailable before the action.

54.2 Missing support

PROVED UNDER STATED ASSUMPTIONS

Theorem 54.1 (Unsupported-action non-identification)**.** *Suppose action $a^\dagger$ is never replayed and has zero logging probability on a set of prefix states with positive target probability. Without an additional outcome model restriction, $R(a^\dagger)$ is not identified from the observed data.*

Proof. Construct two data-generating laws that agree on prefix states, logging actions, and every observed action outcome. Under the first law set the unobserved loss of $a^\dagger$ to 0 on the unsupported set; under the second set it to $L_{\max}$. Both induce exactly the same observed distribution because $a^\dagger$ is never taken there, yet their target risks differ by $L_{\max}$ times the set's positive probability. Therefore the risk is not identified. □

This result is why a learned shortlist cannot silently replace full-action replay in the first oracle test. Shortlist recall must itself be qualified before omitted actions can be ignored.

54.3 Branch-distribution shift

PROVED UNDER STATED ASSUMPTIONS

Theorem 54.2 (Total-variation risk and gap sensitivity)**.** *Let* $0 \le L(a, F) \le L_{\max}$ *and define* $d = \mathrm{TV}(P, Q)$*. Then for every action*

$$|R_P(a) - R_Q(a)| \le L_{\max} d. \tag{206}$$

For any two actions a, b*,*

$$\left|\{R_P(a) - R_P(b)\} - \{R_Q(a) - R_Q(b)\}\right| \le 2L_{\max} d. \tag{207}$$

Proof. For a measurable function in $[0, L_{\max}]$, the difference of expectations under P and Q is at most $L_{\max}\,\mathrm{TV}(P, Q)$. Apply this once for Eq. 206 and twice with the triangle inequality for Eq. 207. □

A synthetic oracle gap smaller than $2L_{\max} d$ can reverse under a deployment law within that distance. More synthetic branches cannot remove this bias. Distributionally robust objectives can optimize over a declared ambiguity set [24]; they do not guarantee coverage of an undeclared shift.

54.4 Evaluator misspecification

Let L^* be the target loss and $\widetilde{L}$ the evaluator loss.

PROVED UNDER STATED ASSUMPTIONS

Proposition 54.3 (Evaluator-error bound)**.** *If* $|\widetilde{L}(a, F) - L^*(a, F)| \le \epsilon_{\mathrm{eval}}$ *almost surely for every action, then*

$$|\widetilde{R}(a) - R^*(a)| \le \epsilon_{\mathrm{eval}}, \tag{208}$$

and every pairwise action gap can differ by at most $2\epsilon_{\mathrm{eval}}$*.*

Proof. Take expectations of the pointwise bound; apply the triangle inequality to two actions. □

Without any link between evaluator and target loss, even the sign of an action advantage is not identified: two latent target-loss laws can agree on every evaluator output and reverse which action is better.

54.5 Leakage and confounding

Counterexample 54.4 (Post-treatment feature)**.** The write changes a future hidden state. A feature extractor then reads that hidden state and supplies it to the value model as if it were a pre-action feature. The feature perfectly predicts which action produced the favorable replay, but it is unavailable before selection. Offline accuracy can be one while the deployed input is undefined. This violates $\mathcal{D}^{+}_{n,j}$-measurability.

Counterexample 54.5 (Hidden confounding in logged actions)**.** A production heuristic writes only when an unrecorded verifier is confident. Written cases later have lower loss than NULL cases. The association can reflect verifier confidence rather than the write. Without randomization, observed confounder adjustment, or valid instrumental structure, the action effect is not identified.

Counterexample 54.6 (Near-zero action gap)**.** Two actions have risks differing by $\Delta \le 2\varepsilon_g(\delta)$. Either action can be the empirical minimizer on the simultaneous concentration event. Treating the selected index as a reliable class label creates false precision; the group should be ambiguous or receive a soft value target with uncertainty.

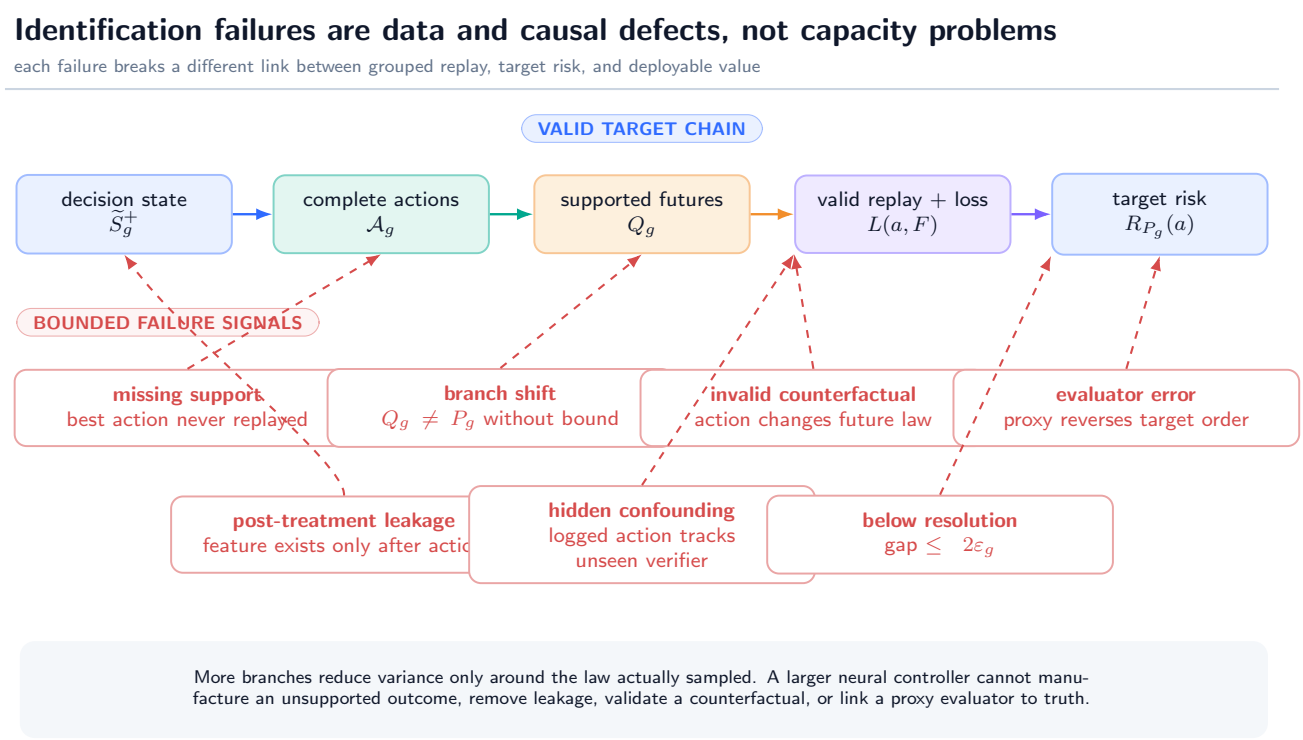

Figure 41: **Identification and failure counterexamples.** Coral marks bounded failure signals: missing action support, branch shift, post-treatment leakage, hidden confounding, evaluator error, and action gaps below sampling resolution. More controller capacity does not repair these defects.

54.6 Identification ledger

Target	Identified when	Not identified when
$R_{Q_g}(a)$	Full valid replay; exchangeable branches; fixed loss	Action absent, replay invalid, dependence ignored
$R_{P_g}(a)$	$Q_g = P_g$ or valid supported correction	Synthetic/model law differs without a bound
Action effect	Exogenous common future or interventional simulator	Action changes future and one observed branch is reused
Future-blind predictability	Fresh prefix groups; causal features only	Future-derived or post-treatment feature enters model
Truth-linked risk	Evaluator link audited	Proxy can reverse target ordering
Long-run value	Repeated-write state distribution evaluated	One-step groups only

Table 40: Identification summary. Each target has a separate premise bundle.

55 Future-Blind Value Estimation and Selection

55.1 A value model is not an oracle and not a policy

The expected-risk oracle in Eq. 192 sees training-only future branches. A deployable value model may see only the causal information available before action selection. Let

$$X_{g,a} = \phi(S_g^+, a, c_{g,a}) \tag{209}$$

be a typed, versioned feature record with $c_{g,0} = \varnothing$. Every coordinate of $X_{g,a}$ must be measurable with respect to the post-validation, preselection field $\mathcal{D}_g^+$ from Eq. 171; no coordinate may depend on a selected action, commit, or future branch. Define the target-law conditional value

$$q_P^*(x) = \mathbb{E}_{F\sim P_g}\left[L_g(a, F) \mid X_{g,a} = x\right]. \tag{210}$$

A model $Q_\psi(X_{g,a})$ may approximate Eq. 210 only from a teacher target whose proposal-to-target bridge is valid under Protocol 52.3. It does not receive F, a replay outcome, an oracle label computed from the same group, or any post-action state.

Three objects must remain separate:

1. the *value estimator* Q_ψ, which predicts action risk;
2. the *feasibility mask* $\mathrm{Valid}(S_g^+, a, c_{g,a})$, which enforces the row contract and hard constraints; and

3. the *selector*, which compares calibrated action intervals and may return NULL.

Calling the first object a policy erases two independently testable interfaces. This manuscript specifies all three but instantiates none.

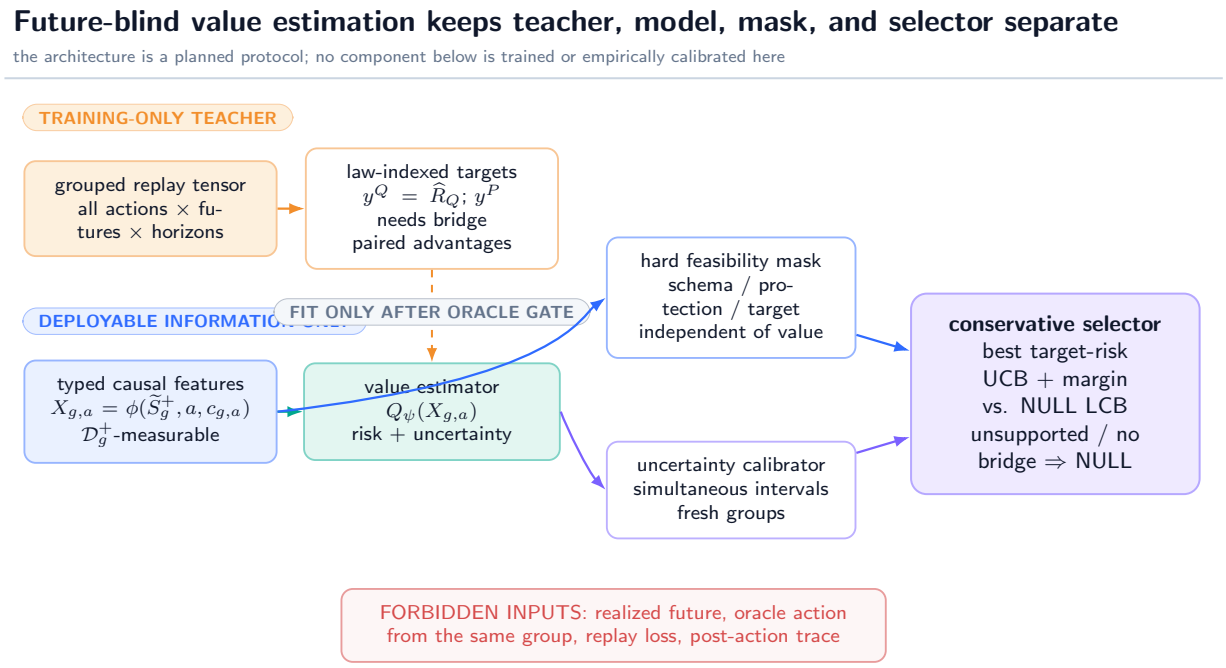

Figure 42: **Future-blind value architecture (PLANNED TEST).** Training-only grouped futures create action-risk targets. Deployment exposes only typed prefix state, action identity, and candidate metadata. The learned value, feasibility mask, uncertainty calibrator, and conservative selector remain separately auditable. No value model or policy is trained here.

55.2 Training targets and losses

The primary target is the continuous paired risk vector or its frozen scalarization, not merely the empirical argmin. The raw replay teacher is $y^Q_{g,a} = \widehat{R}_{Q_g}(a)$. It is a target-risk label only when the registered bridge establishes $Q_g = P_g$; under supported importance weighting, write the resulting target-risk estimate as $y^P_{g,a} = \widehat{R}^{\text{bridge}}_{P_g}(a)$. A total-variation bound alone yields a target-risk interval, not a point label; such a group is used by an interval-valued or robust loss and is not silently converted into $y^P_{g,a}$. Let $y_{g,a}$ denote the registered bridge-valid point teacher when it exists, and let $d_{g,a} = y_{g,a} - y_{g,0}$. A preregistered value objective may combine

$$\begin{aligned}\mathcal{L}_{\text{val}}(\psi) = &\sum_{g,a} \omega_{g,a}\, \rho_\kappa\big(Q_\psi(X_{g,a}) - y_{g,a}\big) \\ &+ \lambda_{\text{rank}} \sum_{g} \sum_{a<b} \mathbf{1}\{s_{g,a,b} \neq 0\} \log\big(1 + \exp\{-s_{g,a,b}[Q_\psi(X_{g,b}) - Q_\psi(X_{g,a})]\}\big)\,, \end{aligned} \tag{211}$$

where ρ_κ is the Huber loss and $s_{g,a,b} = \text{sign}(y_{g,b} - y_{g,a}) \in \{-1, 0, 1\}$ after applying the frozen ambiguity margin. Thus positive s means that b is riskier than a, while ambiguous pairs have $s = 0$ and receive exactly zero ranking charge. Group weights are fixed without using final outcomes. A heteroscedastic, quantile, ensemble, or conformal-style head may estimate uncertainty, but its coverage is an empirical property, not a consequence of architecture naming.

Regression error, action ranking, interval calibration, and selector regret are reported separately. Low mean squared error can coexist with wrong top actions near a decision boundary. Conversely, correct ranking can coexist with badly calibrated risk magnitudes and unsafe commit thresholds.

55.3 Value error and decision regret

For a fixed prefix group, let $a^*_g = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} R_{P_g}(a)$ and let

$$\widehat{a}_g = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} \widehat{Q}_g(a), \qquad \widehat{Q}_g(a) = Q_\psi(X_{g,a}), \tag{212}$$

with a fixed tie rule.

PROVED UNDER STATED ASSUMPTIONS

Theorem 55.1 (Uniform value error implies bounded action regret)**.** *If* $|\widehat{Q}_g(a) - R_{P_g}(a)| \le \epsilon_g$ *for every* $a \in \mathcal{A}_g$*, then*

$$0 \le R_{P_g}(\widehat{a}_g) - R_{P_g}(a_g^*) \le 2\epsilon_g. \tag{213}$$

If the true best action is unique and separated from every competitor by more than $2\epsilon_g$*, then* $\widehat{a}_g = a_g^*$*.*

Proof. Optimality of $\widehat{a}_g$ for the estimated values gives $\widehat{Q}_g(\widehat{a}_g) \le \widehat{Q}_g(a_g^*)$. Hence

$$R_{P_g}(\widehat{a}_g) \le \widehat{Q}_g(\widehat{a}_g) + \epsilon_g \le \widehat{Q}_g(a_g^*) + \epsilon_g \le R_{P_g}(a_g^*) + 2\epsilon_g.$$

The separation statement follows because choosing a competitor would contradict this bound. □

The premise is simultaneous and prefix-conditional. An average validation error does not establish it. Under branch-law shift and evaluator error, a useful sensitivity statement combines Theorem 54.2 and Proposition 54.3: a model error bound ϵ_g, distribution distance d_g, and evaluator error ϵ_{eval} can permit pairwise ordering error on the scale

$$2\epsilon_g + 2L_{\max} d_g + 2\epsilon_{\text{eval}}. \tag{214}$$

This is a diagnostic budget, not a claim that any term is currently bounded.

55.4 Training gate and abstaining deployment

PLANNED TEST Value-model training may begin only after Protocol 52.3 qualifies a reproducible, material expected-risk advantage on fresh prefix groups. The data split is by physical prefix group, source namespace, and registered temporal boundary. Hyperparameters, feature schemas, action ordering, calibration method, and commit margin are frozen before final evaluation.

At deployment, unsupported schemas, invalid candidates, failed monitoring, or nonseparating intervals return NULL. The selection rule is Eq. 203, not an unconstrained argmin of a neural score. A later learned shortlist would require an independent recall guarantee against the complete feasible action set before it could reduce replay or scoring work.

UNVALIDATED There is no qualified oracle gap, trained Q_ψ, learned shortlist, calibrated interval model, or deployed learned selector in the frozen evidence record. The entire section is conditional theory and an unopened experimental protocol.

56 Metrics, Splits, Leakage Audits, and Reproducibility

56.1 Decision metrics

Let $a_g^* = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} R_{P_g}(a)$ minimize the registered target risk and let π_g be the evaluated selector. The primary per-group action regret is

$$\text{Reg}_g(\pi) = R_{P_g}(\pi_g) - R_{P_g}(a_g^*) \ge 0. \tag{215}$$

When a fixed scale $s_g > 0$ is preregistered, normalized regret is Reg_g / s_g; a scale chosen after seeing results is prohibited. Report the mean, median, upper quantiles, worst registered subgroup, and a group-level confidence interval. Also report:

- top-1 action agreement and top-k ranking metrics, with ambiguous groups separated;
- excess risk relative to current risk and relative to NULL;
- write precision, write recall, NULL precision, and NULL recall using the margin-aware classes in Section 53;
- wrong-target rate conditional on writing;
- stale leakage, collateral damage, contamination, and resource coordinates before scalarization;
- interval coverage, width, risk-coverage curves, calibration error, and ambiguity rate; and
- stability across seeds, horizons, capacities, event types, source reliability, and action-set size.

Future loss is always computed by action-conditioned replay. Current-state reconstruction may be a diagnostic but cannot substitute for Eq. 187. Accuracy on oracle action labels is secondary because labels are unstable when risk differences lie inside uncertainty intervals.

56.2 Split hierarchy

DESIGN PRESCRIPTION A physical causal prefix, all its candidate actions, and all its future branches form one indivisible group. The primary split hierarchy is:

1. source namespace and temporal boundary;
2. physical prefix group;
3. registered scenario family or generator seed; and
4. only then, individual action and branch records within the assigned group.

No branch, paraphrase, candidate, cached trace, or derived oracle label from one physical prefix may cross train, calibration, and final partitions. The final partition is read once by the frozen evaluation entry point.

56.3 Leakage and falsification audits

Audit	Receipt	Passing condition	Failure action
Prefix provenance	Source, byte hash, cutoff, event index	Feature timestamps precede action	Invalidate group
Group isolation	Physical-prefix identifier closure	No identifier or derivative crosses split	Rebuild split
Future blindness	Static graph plus runtime access log	No branch/outcome/post-action field reaches model	Reject model
Replay restoration	Snapshot, cache, index, RNG receipts	Every action-branch begins identically	Recompute tensor
Action completeness	Deterministic feasible-set manifest	Every valid action and NULL priced	Reject oracle test
Evaluator freeze	Build hash and blinded final entry	No final-set tuning or hidden revision	Restart final evaluation
Label permutation	Group-preserving shuffled future labels	Performance collapses to registered null behavior	Investigate leakage
Prefix destruction	Mask registered causal signal only	Change agrees with preregistered sensitivity	Mark model suspect
Seed replication	Independent generator and constructor seeds	Sign and primary subgroup gates pass every seed	Stop progression
Cost reconciliation	Logical counts plus measured counters	Ledger totals match trace within tolerance	Reject efficiency claim

Table 41: Minimum audit suite. Receipts are immutable artifacts, not narrative assurances.

The shuffled-label audit preserves group structure; branch-level shuffling that destroys obvious correlations can be too easy. A stronger negative control swaps future sets only among matched prefixes without changing action metadata. Success above the registered null range indicates leakage, unmodeled group identity, or a flawed split.

56.4 Calibration protocol

For interval level $1-\alpha$, empirical coverage on a final set $\mathcal{G}_{\text{test}}$ is

$$\widehat{\text{Cov}}_{1-\alpha} = \frac{1}{|\mathcal{G}_{\text{test}}|} \sum_{g \in \mathcal{G}_{\text{test}}} \mathbf{1}\left[R_{P_g}(a) \in [L_g^P(a), U_g^P(a)] \; \forall a \in \mathcal{A}_g\right]. \tag{216}$$

This group-level simultaneous target-law coverage, not marginal action coverage, is the primary quantity. Proposal-law teacher intervals $[L_g^Q, U_g^Q]$, bridge inflation, and model intervals $[L_g^P, U_g^P]$ are reported separately; treating a noisy teacher estimate or an unbridged proposal interval as exact target risk understates uncertainty.

For a binary write-worthiness probability p_g and label z_g, expected calibration error is reported only with bins fixed on calibration data, alongside proper scores and reliability diagrams. Because binning can conceal local failure, primary commit safety is judged by realized risk and simultaneous interval coverage.

56.5 Reproducibility record

Every run record binds code and environment identifiers, immutable data manifests, feature schema, group split, action manifest, constructor and evaluator versions, future-generation provenance, all RNG seeds, scalarization weights, interval method, cost counters, and output hashes. Failed and stopped runs remain indexed. Aggregates are regenerated from row-level records; no number is transcribed manually into a paper table.

PLANNED TEST These metrics and audits define future acceptance criteria. None has been run for a predictive admission value model in the evidence available to this manuscript.

57 Complete Teacher and Deployment Cost Ledger

57.1 Logical work cannot be hidden by batching

For group g, write $m_g = |\mathcal{A}_g|$, with one NULL action, K_g future branches, and horizon H_g. A logical replay cell is one tuple (g, a, k, h) whose action-conditioned state produces the registered horizon outputs and loss coordinates.

The opportunity index g advances at every scored event, including events whose selected action is NULL. A distinct committed-write index u advances only after a non-NULL atomic commit. Replay, abstention, and deployment denominators use opportunities g; pollution and repair ledgers indexed by writes use u. In Eq. 221, t denotes every deployment event, so a NULL event still incurs feature, scoring, calibration, monitoring, and resident-state costs.

PROVED UNDER STATED ASSUMPTIONS

Proposition 57.1 (Exact logical replay count)**.** *Under full action support and complete grouped replay, the number of horizon cells is*

$$N_{\text{cell}} = \sum_{g \in \mathcal{G}} m_g K_g H_g. \tag{217}$$

The number of non-NULL complete candidate constructions is

$$N_{\text{cand}} = \sum_{g \in \mathcal{G}} (m_g - 1). \tag{218}$$

If event-recursive teaching branches at every event, the corresponding tree count replaces m_g by the number of supported prefix-action paths and can grow multiplicatively.

Proof. For fixed g, the Cartesian product $\mathcal{A}_g \times \{1, \dots, K_g\} \times \{1, \dots, H_g\}$ has $m_g K_g H_g$ elements. Summing disjoint groups proves Eq. 217. Exactly one action is NULL, so $m_g - 1$ complete non-NULL candidates are constructed. Event recursion evaluates supported paths rather than isolated labels; multiplying per-event branching factors gives the path count. □

Vectorization, shared kernels, prefix caching, or parallel hardware may reduce wall time but not the logical outcome array required by the estimand. Any reuse is reported together with the assumptions that make it exact.

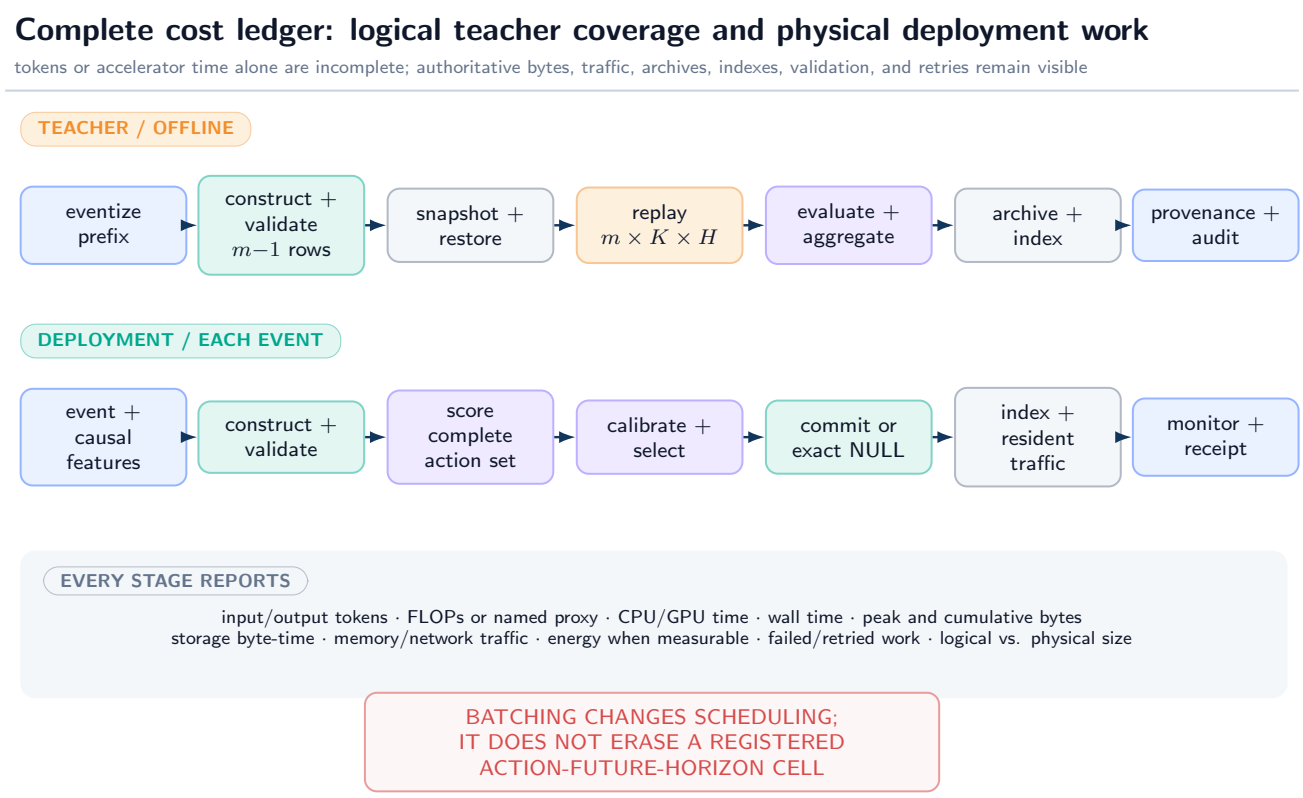

Figure 43: **End-to-end cost ledger** (DEFINITION)**.** The teacher pays eventization, every complete candidate, validation, $|\mathcal{A}| \times K \times H$ replay, evaluation, archives, indexes, and provenance. Deployment pays causal feature extraction, candidate construction, scoring, calibration, commit, and resident-state traffic. Batching changes scheduling, not logical coverage.

57.2 Teacher ledger

For each group, report separately:

$$C_g^{\text{teach}} = C_g^{\text{event}} + \sum_{a\neq 0}\left(C_{g,a}^{\text{construct}} + C_{g,a}^{\text{validate}}\right) + C_g^{\text{snapshot}} + \sum_{a\in\mathcal{A}_g}\sum_{k=1}^{K_g}\left(C_{g,a,k}^{\text{restore}} + C_{g,a,k}^{\text{replay}} + C_{g,a,k}^{\text{eval}}\right) + C_g^{\text{aggregate}} + C_g^{\text{archive}} + C_g^{\text{index}} + C_g^{\text{provenance}}. \tag{219}$$

The units ledger contains input/output tokens, model FLOPs or an explicitly named proxy, accelerator and CPU time, peak and cumulative bytes, storage byte-time, network and memory traffic, energy when measurable, wall time, and failed/retried work. Trusted assembly, parsing, validation, and evaluator calls are not free because they are outside the main model.

Teacher storage includes source snapshots, complete candidate bytes, future branches, action-conditioned traces or sufficient replay receipts, per-horizon loss vectors, resource counters, and provenance. Deduplication savings are reported with physical and logical sizes. Archive and index maintenance are charged to the design that needs them.

57.3 Deployment ledger

Deployment cost for one event is

$$C_g^{\text{deploy}} = C_g^{\text{event}} + C_g^{\text{feature}} + \sum_{a\neq 0}\left(C_{g,a}^{\text{construct}} + C_{g,a}^{\text{validate}}\right) + \sum_{a\in\mathcal{A}_g} C_{g,a}^{\text{score}} + C_g^{\text{calibrate}} + C_g^{\text{select}} + C_g^{\text{commit}} + C_g^{\text{index}} + C_g^{\text{monitor}}. \tag{220}$$

If candidates are constructed only after a learned shortlist, the shortlist's complete-set recall and the cost of generating the features it needs must be counted. Otherwise apparent savings can be caused by silently changing the feasible action set.

57.4 Fair comparisons

Compare systems at equal resident authoritative byte-time, equal archive availability, equal retrieval context, equal evaluator quality, and equal task horizon. Report both teacher-amortized and steady-state deployment views. A method that moves rows from authoritative memory into an uncharged archive, increases retrieval tokens, or relies on expensive future generation has not demonstrated a system-level efficiency gain.

For repeated deployment over T events, report

$$C_{1:T}^{\text{total}} = C^{\text{train}} + C^{\text{calibration}} + \sum_{t=1}^{T} C_t^{\text{deploy}} + C_{1:T}^{\text{resident}} + C_{1:T}^{\text{archive}}, \tag{221}$$

together with the chosen amortization horizon. Savings that change sign outside one chosen T are shown as a curve.

UNVALIDATED No teacher run, deployment run, or predictive-admission efficiency measurement is reported here. Equations 217–221 are accounting requirements for later work.

58 Counterexamples, Long-Run Pollution, and Stop Rules

58.1 Why an apparent oracle is insufficient

Counterexample 58.1 (Hindsight without conditional predictability)**.** There are two writes a_1, a_2 and two equiprobable futures F_1, F_2. Loss is zero when indices match and one otherwise. A per-future hindsight selector has zero loss, but both fixed actions have expected loss $1/2$, and no causal prefix feature predicts the future. The expected-risk oracle is tied and no learned admission rule can recover the hindsight gain. Thus an impressive hindsight ceiling is not an expected-risk oracle gap.

Counterexample 58.2 (Current-risk reversal)**.** Action a_1 perfectly reconstructs the completed segment and has current cost zero; action a_2 has current cost 0.1. With probability 0.9, later tasks need the fact stored by a_2 and never query a_1's fact. A frozen horizon loss can prefer a_2 even though the current-risk comparator prefers a_1. Reversing the probabilities reverses the expected-risk order without changing current reconstruction. Current-risk quality therefore neither proves nor refutes future value.

Counterexample 58.3 (Short-horizon benefit, long-run pollution)**.** A write reduces next-step task loss by 0.2 but overwrites a protected relation that is queried once in the next hundred steps, causing loss 1. An $H = 1$ evaluator selects the write; an $H = 100$ evaluator rejects it. Horizon is part of the estimand and cannot be chosen after inspecting this reversal.

Counterexample 58.4 (Invalid synthetic branch)**.** A generator produces diverse continuations conditional on a textual prefix but omits an external process that makes a future depend on the chosen write. Replaying every action on those shared branches appears well supported. The counterfactual is nevertheless invalid because Assumption 50.4 fails; more generated branches concentrate around the wrong law.

Counterexample 58.5 (Hidden shortlist failure)**.** A cheap heuristic proposes two of ten feasible targets and the teacher prices only those plus NULL. The omitted target is best on a rare but material subgroup. The learned controller matches its truncated teacher perfectly while incurring large complete-set regret. Unless shortlist recall was independently qualified, the result is not an oracle for $\mathcal{A}_g$.

58.2 Accumulation over repeated writes

Let $Z_t = 1$ indicate that the tth committed write introduces a registered contamination event that remains consequential within the lifecycle horizon. Let $\mathcal{H}_{t-1}$ be the pre-write history and suppose the monitoring argument establishes

$$\mathbb{P}(Z_t = 1 \mid \mathcal{H}_{t-1}, A_t \neq 0) \leq p_t. \tag{222}$$

PROVED UNDER STATED ASSUMPTIONS

Proposition 58.6 (Repeated-write pollution bound)**.** *For any possibly adaptive sequence of at most T committed writes satisfying Eq. 222,*

$$\mathbb{P}\left(\bigcup_{t=1}^{T}\{Z_t = 1\}\right) \leq \sum_{t=1}^{T} p_t, \qquad \mathbb{E}\left[\sum_{t=1}^{T} Z_t\right] \leq \sum_{t=1}^{T} p_t. \tag{223}$$

No independence assumption is required.

Proof. The union bound gives the first inequality. For the second, the tower property yields $\mathbb{E}Z_t = \mathbb{E}[\mathbb{E}(Z_t \mid \mathcal{H}_{t-1}, A_t \neq 0)] \leq p_t$ for committed writes; linearity of expectation completes the proof. □

For constant $p_t = p$, keeping the union-bound budget below α requires $p \leq \alpha/T$. The bound can be loose, but it exposes a key scaling fact: a seemingly small per-write error is not automatically a small lifecycle risk. Dependencies, persistence duration, repair, and severity require richer stateful evaluation. This proposition does not establish any empirical p_t.

58.3 Mandatory stop rules

Observed condition	Scientific conclusion	Required action
Inherited semantic row gate failed or unqualified	Admission action has no accepted semantic substrate	Do not run oracle/controller claim
No reproducible material gap over current risk	Predictive selection has not earned a target	Stop before value-model training
No registered relation to NULL	Write benefit is not established	Keep NULL-only deployment
Gap changes sign by seed, subgroup, or horizon	Oracle effect is unstable or underspecified	Diagnose; no pooled promotion
Oracle gap exists but causal features do not rank actions	Future value is not predictably available	Stop controller progression
Intervals fail simultaneous coverage	Commit certificate is invalid	Return NULL; recalibrate on nonfinal data
Leakage, support, replay, or evaluator audit fails	Estimand is not identified by the run	Invalidate affected result
Long-run pollution exceeds budget	Short-horizon gain is insufficient	Extend horizon or reject controller
Full lifecycle cost erases registered benefit	System-level efficiency is unsupported	Report tradeoff; no efficiency claim
Only a learned shortlist is evaluated	Complete action regret is unknown	Restore full set or qualify recall first

Table 42: Stop rules. A stop is an informative outcome, not a request to weaken the gate.

PLANNED TEST The next admissible experiment, only after an independent semantic-interface qualification, is a single-event, fixed-payload, complete-action grouped replay with exact NULL, all candidates constructed before futures are opened, and every cost coordinate logged. If its expected-risk gap does not qualify, the experimental ladder ends. This manuscript neither executes nor authorizes that experiment.

59 Qualification Ladder and Theorem-to-Implementation Obligations

Conditional proofs constrain an implementation only when their premises are discharged by inspectable receipts. Figure 44 and Table 43 make those dependencies explicit. A later stage cannot retroactively validate an earlier one.

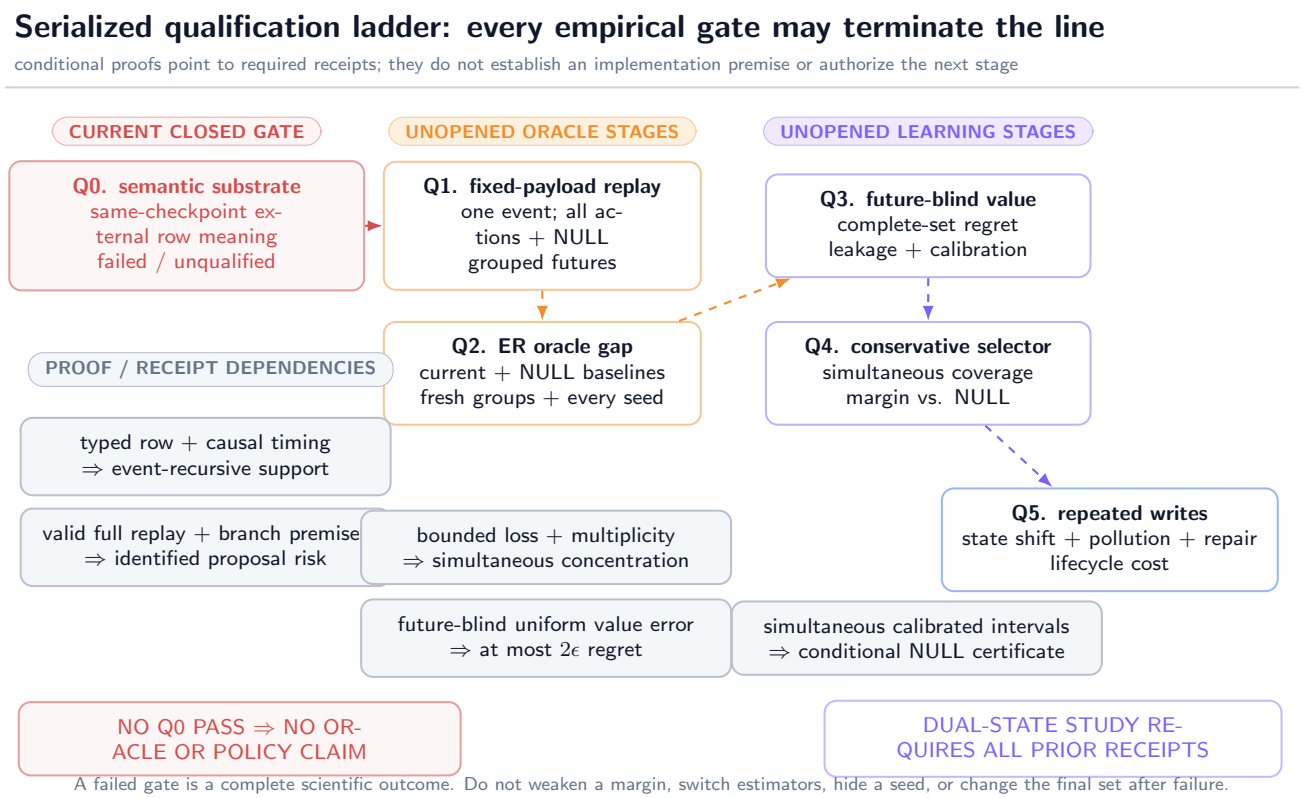

Figure 44: **Qualification ladder and proof dependencies.** The semantic-interface stop precedes fixed-payload oracle replay. Only a stable expected-risk gap permits future-blind value training; only calibrated low-regret selection permits repeated-write study. Coral gates are currently closed or unvalidated.

59.1 Proof dependency graph

The principal logical dependencies are:

typed row contract + causal timing $\Longrightarrow$ event-recursive support,

common prefix + valid full replay + branch assumptions $\Longrightarrow$ identified empirical action risks,

bounded loss + conditionally iid weighted branches + multiplicity charge $\Longrightarrow$ simultaneous proposal-law concentration,

proposal-law intervals + valid target-law bridge $\Longrightarrow$ simultaneous target-law intervals,

simultaneous calibrated target-law intervals $\Longrightarrow$ certified improvement over NULL,

future-blind features + uniform value error $\Longrightarrow$ bounded plug-in regret.

None of the right-hand statements establishes a left-hand premise. In particular, low empirical regret cannot prove causal timing or replay validity.

Result	Nonnegotiable premise	Minimum implementation receipt	Present status
Event timing, Prop. 49.1	Ordered eventizer; branch-local predecessor; valid complete row	Predecessor/successor and candidate digests per event	UNVALIDATED
Sequential support, Thm. 49.2	One-row atomic commit; exact identity NULL	Slot diff and byte equality audit	Draft contract only
Weighted concentration, Thm. 52.1	Bounded fixed loss; conditionally iid proposal branches; fixed loss-independent weights	Sampling-regime flag, branch provenance, bounds, weights, action count	UNVALIDATED
Stable empirical action, Cor. 52.2	Unique gap above simultaneous radius	Per-action estimates and interval separation	UNVALIDATED
Certified NULL, Thm. 53.1	Simultaneous target-law coverage; registered $Q_g \to P_g$ bridge; frozen margin	Bridge receipt, final group-level coverage, and commit log	UNVALIDATED
Support impossibility, Thm. 54.1	Zero support on positive-mass set	Feasible-set and replay completeness manifest	Conditional theorem
Shift sensitivity, Thm. 54.2	Declared branch laws; bounded loss; valid distance bound	Generator/deployment provenance and shift estimate	No bound established
Value regret, Thm. 55.1	Future-blind inputs; simultaneous uniform error	Runtime field-access audit and per-action error envelope	No model exists
Replay count, Prop. 57.1	Full Cartesian replay	Trace-derived logical and physical counters	No run exists
Pollution bound, Prop. 58.6	Valid per-write conditional error bound	Long-horizon event and repair ledger	No bound established

Table 43: Theorem-to-implementation obligation summary. Appendix 63 expands the test and failure action for each premise.

59.2 Serialized qualification ladder

1. **Semantic substrate.** Independently qualify a same-checkpoint external semantic row interface under its own registered contract. The frozen evidence does not satisfy this step.
2. **Mechanical fixed-payload oracle.** With one event, exact NULL, complete feasible actions, and grouped futures, price every action without learning content or route.
3. **Oracle qualification.** Apply Protocol 52.3 on fresh prefix groups and required seeds. No reproducible material gap means stop.
4. **Future-blind value.** Only after Step 3, fit Q_ψ and audit leakage, calibration, complete-set regret, NULL behavior, and subgroup stability.
5. **Conservative selector.** Freeze Eq. 203; verify simultaneous coverage and realized improvement without threshold tuning on final groups.
6. **Repeated writes.** Evaluate state-distribution shift, pollution accumulation, repair, archive/index interactions, and lifecycle cost over registered horizons.

Every step produces a versioned manifest and can terminate the line. The ladder does not imply that any step will pass.

59.3 Current boundary

UNVALIDATED The evidence boundary remains before Step 1: the inherited semantic interfaces are failed/unqualified, the fitted-population diagnostic is a fit/support failure for the tested pure latent-row interface, and predictive overwrite remains closed. Steps 2–6 are theory and planned protocol only.

60 Related Foundations, Limitations, and Conclusion

60.1 Related foundations

The MMLA architecture motivates a bounded reasoning-time state, explicit receipts, and a failed/unvalidated semantic-interface boundary rather than supplying evidence for predictive admission [126]. Two companion focus papers provide a typed separation between policy and memory state and an atomic external row contract [132, 118]; they are formal dependencies, not empirical authorities.

Our grouped-future construction uses the potential-outcome distinction between treatment assignment and outcomes [86] and the role of propensity/support conditions in observational adjustment [85]. When logged propensities replace complete controlled replay, contextual-bandit evaluation requires explicit support and estimator assumptions [25]; this does not justify reusing one observed continuation as every action's counterfactual.

Finite-action confidence control draws on classical bounded concentration [41] and empirical-Bernstein refinements [65]. NULL is related to classification with a reject option [17], but here rejection is also a physical identity transition whose future risk must be replayed. Distributionally robust optimization can expose sensitivity to a declared ambiguity set [24], while conditional value-at-risk supplies one possible tail-risk scalarization [84]; neither method proves that the declared set or loss matches deployment.

Neural Turing Machines, Differentiable Neural Computers, and compressive memory study learned read/write or compressed recurrent memory mechanisms [30, 32, 81]. Predictive admission here concerns a narrower authoritative decision: whether a complete target-conditioned row should replace bounded resident state under a causal future-risk protocol. It neither claims superiority to those systems nor treats their mechanism results as evidence for MMLA.

60.2 Limitations and explicit nonclaims

The theory assumes a finite feasible action set, bounded registered loss, typed complete rows, and a replay system capable of restoring action-conditioned states. Large or continuous action spaces require approximation and new support guarantees. Conditional exchangeability can fail; synthetic futures can be biased; evaluator loss can misorder true outcomes; the deployment law can drift; and a finite horizon can miss slow pollution or repair. The simultaneous bounds can be conservative when actions are numerous or groups are small. A complete teacher can be computationally prohibitive even when deployment is cheap.

No theorem establishes that semantic row content can be decoded at the same checkpoint, that future value is predictable from causal features, that calibrated neural intervals will be obtained, that a safe system exists, or that total-system cost improves. "Certified" in Theorem 53.1 is always relative to stated interval coverage, target law, loss, horizon, and margin. A violated premise voids the deduction.

The manuscript reports no new data collection, experiment, trained model, oracle gap, learned route, learned content, learned shortlist, learned policy, deployment, safety result, or efficiency result. It does not reinterpret the complete mechanical oracle in the fitted-population diagnostic as evidence for learned memory.

60.3 Conclusion

Predictive memory admission should begin with an identified decision problem, not a controller. This paper has separated exact NULL, complete target-conditioned actions, a current-risk comparator, a training-only expected-risk oracle, and a future-blind value estimator. It has made event timing, grouped futures, complete action-conditioned replay, simultaneous uncertainty, branch shift, evaluator error, lifecycle pollution, and the full cost multiplier visible. The resulting ladder is deliberately easy to stop: a failed semantic substrate, absent oracle gap, unpredictable value, uncalibrated selection, long-run pollution, or erased system-level benefit ends progression without relabeling the failure.

CONJECTURED Selective admission may eventually help a qualified bounded authoritative memory. That conjecture remains open. The current scientific result is a formal target, conditional deductions, falsification protocol, and explicit stop boundary.

Author contributions

Junyi Zou and Avrova Donz contributed equally to conceptualization, formal analysis, methodology, visualization design, and manuscript preparation.

Generative AI disclosure

Generative AI tools assisted drafting, mathematical consistency checks, code generation for vector figures, and language editing under human direction. The named authors retain responsibility for claims, sources, and the final manuscript. No AI-generated statement is treated as evidence.

61 Supplementary Proofs and Boundary Lemmas

This appendix expands deductions that separate attainable expected-risk decisions from information ceilings. Every result is conditional on its displayed premises and carries no empirical status.

61.1 Hindsight and expected-risk decisions

PROVED UNDER STATED ASSUMPTIONS

Lemma 61.1 (Expectation of the minimum is an information ceiling)**.** *For any finite action set and integrable losses $L(a, F)$,*

$$\mathbb{E}\left[\min_{a\in\mathcal{A}} L(a, F)\right] \le \min_{a\in\mathcal{A}} \mathbb{E}[L(a, F)]. \tag{224}$$

Equality holds if one fixed action minimizes $L(a, F)$ almost surely, but equality need not imply a unique common minimizer.

Proof. For every fixed $b \in \mathcal{A}$, $\min_a L(a, F) \le L(b, F)$ pointwise. Taking expectations and then minimizing the right-hand side over b proves Eq. 224. A fixed almost-sure minimizer gives equality. With ties on positive-probability sets, equality can also occur without uniqueness. □

The left side lets the action depend on the realized future and corresponds to Eq. 194. The right side selects one action before the future and corresponds to expected-risk admission. Their difference is the value of prohibited future information, not necessarily a learnable opportunity.

PROVED UNDER STATED ASSUMPTIONS

Proposition 61.2 (Current-state loss does not identify future-risk order)**.** *Suppose two actions have fixed observed current costs $C^{\mathrm{cur}}(a)$ and $C^{\mathrm{cur}}(b)$. Without a restriction linking those costs to continuation loss, the sign of $R(a) - R(b)$ is not identified.*

Proof. Construct two continuation laws with identical prefix observations and current costs. In the first, set bounded future losses to $L(a, F) = 0$ and $L(b, F) = 1$; in the second reverse them. The observed current record is identical, while the future-risk ordering reverses. □

61.2 Paired estimation

For independent branches with uniform weights, define $D_k(a, b) = L(a, F_k) - L(b, F_k) \in [-L_{\max}, L_{\max}]$ and $\widehat{\Delta}(a, b) = K^{-1} \sum_k D_k(a, b)$.

PROVED UNDER STATED ASSUMPTIONS

Lemma 61.3 (Paired finite-sample bound)**.** *For one preregistered pair (a, b),*

$$\mathbb{P}\left(\left|\widehat{\Delta}(a, b) - \Delta(a, b)\right| > \epsilon\right) \le 2 \exp\left(-\frac{K\epsilon^2}{2L_{\max}^2}\right). \tag{225}$$

For a finite registered set of P pairs, replacing the right side by $2P \exp[-K\epsilon^2/(2L_{\max}^2)]$ gives simultaneous control.

Proof. Each difference has range length $2L_{\max}$. Hoeffding's inequality for the sample mean gives Eq. 225; a union bound gives the simultaneous statement [41]. □

Pairing does not always improve the worst-case range bound, but Eq. 191 can substantially reduce variance-sensitive intervals. Branches remain the units within one prefix; prefix groups remain the units for population inference.

61.3 Decomposed regret under three error sources

Let R_P^* be target-law risk under target evaluator, R_Q^* proposal-law risk under target evaluator, and $\widetilde{R}_Q$ proposal-law risk under the implemented evaluator. Suppose

$$\sup_a |R_P^*(a) - R_Q^*(a)| \le \epsilon_{\mathrm{shift}}, \tag{226}$$

$$\sup_a |R_Q^*(a) - \widetilde{R}_Q(a)| \le \epsilon_{\mathrm{eval}}, \tag{227}$$

$$\sup_a |\widehat{Q}(a) - \widetilde{R}_Q(a)| \le \epsilon_{\mathrm{model}}. \tag{228}$$

PROVED UNDER STATED ASSUMPTIONS

Theorem 61.4 (Target regret under additive uniform errors)**.** *If $\widehat{a} \in \arg\min_a \widehat{Q}(a)$ and $a_P^* \in \arg\min_a R_P^*(a)$, then*

$$R_P^*(\widehat{a}) - R_P^*(a_P^*) \le 2(\epsilon_{\mathrm{shift}} + \epsilon_{\mathrm{eval}} + \epsilon_{\mathrm{model}}). \tag{229}$$

With Theorem 54.2, one may set $\epsilon_{\mathrm{shift}} = L_{\max}\,\mathrm{TV}(P, Q)$ when its premises hold.

Proof. The three uniform bounds and the triangle inequality give $\sup_a |R_P^*(a) - \widehat{Q}(a)| \le \epsilon_{\mathrm{shift}} + \epsilon_{\mathrm{eval}} + \epsilon_{\mathrm{model}}$. Apply Theorem 55.1 with this sum. □

The theorem is a sensitivity decomposition. It is unusable as a certificate until all three terms have valid target-domain bounds.

61.4 Adaptive histories and severe events

Let $V_t \in [0, V_{\max}]$ be contamination severity at write t, with $V_t = 0$ when no contamination occurs. If $\mathbb{E}[V_t \mid \mathcal{H}_{t-1}, A_t \ne 0] \le v_t$, then by the tower property

$$\mathbb{E}\left[\sum_{t=1}^{T} V_t\right] \le \sum_{t=1}^{T} v_t. \tag{230}$$

This permits adaptive action selection but assumes the conditional bound remains valid under the induced history. A bound calibrated only on a static one-write distribution cannot simply be reused after the selector changes the state distribution.

For a severe-event indicator $B_t = \mathbf{1}[V_t \ge v_{\mathrm{crit}}]$ with conditional rate at most q_t, Proposition 58.6 gives $\mathbb{P}(\max_t V_t \ge v_{\mathrm{crit}}) \le \sum_t q_t$. If repair removes consequences, the lifecycle loss must model duration and repair cost rather than erase the original event.

61.5 Event-tree work

Suppose segment event j has $m_j(\mathbf{a}_{<j})$ feasible actions after branch prefix $\mathbf{a}_{<j}$. The number of leaf action paths is

$$N_{\text{leaf}} = \sum_{a_1 \in \mathcal{A}_1} \sum_{a_2 \in \mathcal{A}_2(a_1)} \cdots \sum_{a_J \in \mathcal{A}_J(\mathbf{a}_{<J})} 1. \tag{231}$$

If every node has exactly m actions, $N_{\text{leaf}} = m^J$. A frozen rollout changes the estimand and reduces branching after the scored event; it is not a free exact optimization. This is why the first causal oracle test fixes a single event.

62 Estimator and Evaluation Protocol Specifications

62.1 Within-group proposal-risk record

For each prefix group g, freeze the ordered action manifest $\mathcal{A}_g = (0, a_1, \ldots, a_{m_g-1})$, future manifest $(F_{g,1}, \ldots, F_{g,K_g})$, nonnegative weights $w_{g,k}$ summing to one, horizon H_g, sampling-regime flag $\rho_g \in \{\text{i.i.d., stratified, finite population}\}$, and scalarization before replay. Store the loss-tensor coordinates before aggregating them into $L_{g,a,k}$.

The replay tensor directly identifies proposal-law quantities:

$$\widehat{R}_{Q_g}(a) = \sum_k w_{g,k} L_{g,a,k}, \tag{232}$$

$$\widehat{\Delta}_{Q_g}(a,b) = \sum_k w_{g,k} \big(L_{g,a,k} - L_{g,b,k}\big), \tag{233}$$

$$\widehat{a}_g^{\text{ER},Q} = \min_{\prec_g} \arg\min_{a \in \mathcal{A}_g} \widehat{R}_{Q_g}(a), \tag{234}$$

where $\prec_g$ is the frozen total action order. Effective branch count is Eq. 178; it is descriptive and accompanies raw K_g rather than replacing the sampling-regime record. Zero-weight branches remain in provenance but do not inflate effective count.

For the initial gate, do not fit a nuisance model to fill missing actions: absence of a required replay slice fails action completeness. The raw teacher above is a Q_g-law teacher. It is not relabeled as a deployment-law oracle.

62.2 Registered $Q_g \to P_g$ bridge

Each group carries exactly one bridge flag before outcomes are opened.

1. **Law equality.** If the design establishes $Q_g = P_g$, set $\widehat{R}_{P_g}(a) = \widehat{R}_{Q_g}(a)$ and retain the design evidence.
2. **Supported reweighting.** For i.i.d. draws $F_{g,k} \sim Q_g$, a frozen Radon–Nikodym ratio $r_g(F) = \mathrm{d}P_g/\mathrm{d}Q_g(F)$ gives

$$\widehat{R}^{\text{IW}}_{P_g}(a) = \frac{1}{K_g} \sum_{k=1}^{K_g} r_g(F_{g,k}) L_{g,a,k}. \tag{235}$$

   Absolute continuity, ratio provenance, overlap diagnostics, and the choice between unclipped and clipped ratios are frozen. Clipping changes the estimand unless its bias is separately bounded. Stratified or finite-population samples require their own registered design-weight estimator; Eq. (235) is not silently reused.
3. **Sensitivity-only bridge.** If the only admitted relation is $\mathrm{TV}(P_g, Q_g) \le \tau_g$ and $0 \le L \le L_{\max}$, then

$$R_{P_g}(a) \in \big[\max\{0, R_{Q_g}(a) - L_{\max}\tau_g\}, \min\{L_{\max}, R_{Q_g}(a) + L_{\max}\tau_g\}\big]. \tag{236}$$

   This is an identification band, not a target-risk point estimator.

If no bridge is justified, the target-law oracle gate is unidentifiable and stops. Proposal-law concentration cannot repair that failure.

62.3 Population estimands without selection reuse

Let fresh physical prefix groups $g = 1, \ldots, G$ be independent draws from the registered target group law. A bridge-valid procedure returns target-risk estimates $\widehat{R}^{\text{eval}}_{P_g}(a)$ on evaluation branches. The teacher action $\widehat{a}_g^{\text{ER},P,\text{train}}$ is selected either on disjoint training branches or under a simultaneous finite-class procedure whose selection correction is frozen. Define paired group advantages

$$\widehat{G}_g^{\text{cur},P} = \widehat{R}^{\text{eval}}_{P_g}(a_g^{\text{cur}}) - \widehat{R}^{\text{eval}}_{P_g}(\widehat{a}_g^{\text{ER},P,\text{train}}), \tag{237}$$

$$\widehat{G}_g^{0,P} = \widehat{R}^{\text{eval}}_{P_g}(0) - \widehat{R}^{\text{eval}}_{P_g}(\widehat{a}_g^{\text{ER},P,\text{train}}). \tag{238}$$

Using the same outcomes both to choose an empirical minimum and to report its uncorrected advantage is prohibited. Acceptable registered treatments include a nested branch split with adequate power, a simultaneous finite-class bound, or another bias-corrected procedure justified before outcome access.

The population aggregate is a fixed weighted mean

$$\widehat{G}^{b,P} = \sum_{g=1}^{G} \nu_g \widehat{G}_g^{b,P}, \qquad b \in \{\mathrm{cur}, 0\}, \qquad \nu_g \geq 0, \quad \sum_g \nu_g = 1. \tag{239}$$

Primary inference resamples or concentrates at the physical-group level. Treating $GK_g m_g$ replay rows as independent is pseudoreplication.

62.4 Law- and design-indexed intervals

For i.i.d. proposal draws, equal weights, and $0 \leq L \leq L_{\max}$, Eq. 197 yields the transparent proposal interval

$$[L_g^Q(a), U_g^Q(a)] = \left[\max\{0, \widehat{R}_{Q_g}(a) - \varepsilon_g^Q\}, \min\{L_{\max}, \widehat{R}_{Q_g}(a) + \varepsilon_g^Q\}\right]. \tag{240}$$

The interval is indexed by Q because that is the law sampled. A target-law interval $[L_g^P(a), U_g^P(a)]$ is reported only after propagating a registered bridge and its uncertainty. Under the TV bridge, for example, the proposal confidence interval is widened by $L_{\max}\tau_g$ and clipped to $[0, L_{\max}]$. Stratified and finite-population regimes use their own frozen variance and finite-population corrections. Neither weighted Hoeffding nor K_g^{eff} is presented as a universal substitute.

If actions share branches, preregistered paired empirical-Bernstein intervals can be more efficient [65]; action and pair multiplicity still must be charged. The first result uses the registered primary construction, with alternatives labeled as sensitivity analyses.

62.5 Group bootstrap specification

A group bootstrap draw samples physical prefix groups with replacement and carries each selected group's full action-by-future tensor. Within-group branches are not independently resampled in the primary population bootstrap. If future-generation uncertainty is also targeted, use a nested bootstrap and report its separate estimand. Stratification variables, group weights, bridge flags, and law labels are frozen before final data are opened.

For B registered bootstrap replicates, the interval construction, quantile convention, studentization, and seed list are fixed. A replicate that lacks a required subgroup is invalid rather than silently dropped. The analysis logs all invalid-replicate counts and applies the preregistered failure rule.

62.6 Oracle-gap run sequence

1. Verify the semantic-interface prerequisite, lock the snapshot namespace, and hash every source artifact.
2. Eventize one completed segment into exactly one scored event; freeze fixed payload and complete target actions plus NULL.
3. Construct and validate every complete candidate before future outcomes are exposed to the selector or analyst.
4. Register ρ_g, generate or collect grouped futures with provenance, and freeze weights, proposal law Q_g, target law P_g, and the $Q_g \to P_g$ bridge.
5. Restore the same pre-action snapshot for every action–branch pair and replay the full registered horizon.
6. Compute all loss coordinates, resource counters, proposal risks, bridge-valid target risks or identification bands, paired gaps, and simultaneous intervals automatically.
7. Run action-completeness, restoration, future-blindness, group-isolation, evaluator, negative-control, bridge, and cost audits.
8. Evaluate fresh prefix groups and all registered seeds once; apply Protocol 52.3 without threshold revision.
9. If any target-law gate fails or is unidentified, record the stop. Only a passed target-law oracle gate permits construction of a separate value-model dataset.

62.7 Value-model evaluation sequence

If and only if the target-law oracle gate passes, fit on training prefix groups, choose all architecture and calibration decisions on calibration groups, freeze the artifact, and evaluate once on final groups. Teacher targets retain their law labels: raw replay targets are $y_{g,a}^Q$; a target-law label $y_{g,a}^P$ exists only through the registered bridge. The final table includes the complete action set, per-group target-law regret or identification band, raw risk coordinates, NULL confusion metrics, simultaneous coverage,

interval width, ambiguity rate, subgroup stability, negative controls, and full cost. A second final-set pass after model or threshold revision creates a new registered revision and a new untouched final namespace.

UNVALIDATED This appendix is a specification only. It contains no run identifier, data row, interval, effect estimate, or outcome.

63 Complete Theorem-to-Implementation Matrix

Table 44: Implementation obligations. "Stop" means the corresponding conclusion cannot be claimed; it does not prescribe a scientific workaround.

Object/result	Premise	Receipt/test	Failure consequence	Evidence status
Completed segment	Immutable decision boundary and ordered bytes	Source digest, cutoff time, tokenizer/build version	Event inputs are ambiguous; stop	Contract only
Eventizer	Deterministic ordered event list under frozen version	Event list digest; rerun equality; human audit sample	Timing theorem inapplicable	Unvalidated
Complete row	Trusted assembly; explicit target/version/protection; parse validity	Byte digest, schema validation, canonical round-trip	Remove action; no semantic claim	Draft dependency
Exact NULL	Identity on resident state, indexes, and authority	Byte-for-byte state and index equality; zero commit trace	Baseline invalid; stop oracle test	Draft dependency
Feasible set	Every valid target represented before outcome access	Deterministic action manifest and rejection reasons	Unsupported-action theorem applies	Unvalidated
Event recursion	Candidate predecessor equals branch prefix	Per-event predecessor/candidate/successor digest chain	Reject branch	Unvalidated
Grouped future	One physical prefix; registered target/proposal law	Group ID closure, timestamps, generator/config hashes	No target-risk interpretation	Unvalidated
Exchangeability	Conditional independent or valid weighted branch draws	Sampling design and dependence diagnostics	Concentration interval invalid	Assumption only
Future causality	Common future exogenous to action, or interventional branch law	Simulator causal contract or domain argument	Counterfactual replay invalid	Assumption only
Snapshot restore	Identical state, cache, index, and RNG boundary	Restoration digest before every action-branch run	Invalidate replay tensor	Unvalidated
Action replay	Memory action can affect hidden evolution and all effects are rerun	Full trace lineage; no pre-action hidden-trace reuse	Horizon loss not action conditioned	Unvalidated
Loss bound	Every scalar loss lies in registered $[0, L_{\max}]$	Automated range audit and overflow log	Hoeffding claims void	Unvalidated
Loss meaning	Evaluator linked to target task/truth	Frozen evaluator, blinded validation, sensitivity envelope	Only proxy-risk claim remains	Unvalidated
Current comparator	No realized future; definition and ties frozen	Comparator artifact hash and runtime field log	Gap comparison invalid	Unvalidated
Empirical ER oracle	Complete actions, valid risks, multiplicity correction	Action tensor and simultaneous intervals	No oracle-gap claim	Unrun
Population gap	Fresh independent prefix groups; registered weights/subgroups	Final namespace manifest and group-level inference	No population promotion	Unrun
Future-blind features	Every field pre-action measurable	Static dataflow graph plus runtime access log	Reject model as leaked	No model
Value error	Simultaneous complete-action error bound on target groups	Per-action prediction/risk envelope	2ϵ theorem unusable	No model
Calibrated selector	Simultaneous coverage and frozen margin	Final coverage, interval, action, and commit receipts	NULL-only fallback	No model
Branch shift	Target/proposal distance bounded in relevant loss class	Declared law and sensitivity bound	Synthetic gap cannot transfer	No bound
Repeated writes	One-write guarantees stable under induced histories	Stateful multi-event replay and pollution/repair ledger	No lifecycle claim	Unrun
Complete cost	Every constructor, replay, archive, index, and deployment term counted	Logical counts, physical counters, reconciliation audit	No efficiency claim	Unrun

The matrix is deliberately redundant with the prose. A theorem citation without the corresponding receipt is an incomplete claim. Receipts must identify the exact artifact revision; evidence from another checkpoint, action set, horizon, evaluator, or branch law cannot be silently reused.

64 Symbol, Status, and Boundary Ledger

64.1 Core symbols

Table 45: Primary notation. All group-indexed objects inherit the registered prefix group g.

Symbol	Type	Meaning
M_{n-1}	bounded resident state	Authoritative state before completed segment n

Symbol	Type	Meaning
x_n	completed segment	Immutable observation block at a decision boundary
$e_{n,j}$	typed event	Ordered event j extracted from x_n
$M_{n,j}$	bounded resident state	State after decisions through event j
$c_{n,j,i}$	complete row	Target-conditioned candidate for action i
$\mathcal{A}_{n,j}$	finite action set	Exact NULL plus all valid complete row commits
0 or NULL	action	Physical identity transition; no row or authority change
$\widetilde{S}_g^-, \widetilde{S}_g^+$	causal prefix records	Typed pre-event and post-candidate/pre-action states for prefix group g
$\mathcal{D}_g^-, \mathcal{D}_g^+$	sigma-fields	Information legally available before eventization and after deterministic candidate construction
$F_{g,k}$	future branch	Training/evaluation-only continuation k sharing prefix g
Q_g, P_g	probability laws	Proposal/grouped-future law and target deployment law
$w_{g,k}$	scalar	Nonnegative branch weight summing to one
K_g^{eff}	scalar	Effective weighted branch count $(\sum_k w_{g,k}^2)^{-1}$
$Y_{g,a,k,h}$	replay outcome	Action-conditioned outcome at horizon cell (a, k, h)
$L_g(a, F)$	bounded scalar	Registered horizon loss after action a under future F
ρ_g	design flag	Registered i.i.d., stratified, or finite-population future-sampling regime
$\widehat{R}_{Q_g}(a)$	estimator	Weighted empirical proposal-law action risk under grouped branches
$\widehat{R}_{P_g}(a)$	estimator	Target-law estimate only under a registered equality or supported-reweighting bridge
$R_{Q_g}(a)$	estimand	Conditional risk under proposal law
$R_{P_g}(a)$	estimand	Conditional risk under target deployment law
a_g^{cur}	comparator action	Frozen current-risk minimizer without realized future
$\widehat{a}_g^{\text{ER},Q}$	proposal teacher	Empirical Q_g-risk minimizer over complete actions under frozen order $\prec_g$
$\widehat{a}_g^{\text{ER},P}$	target teacher	Empirical target-risk minimizer only when a valid bridge identifies the target quantities
$a_{g,k}^{\text{hind}}$	information ceiling	Per-realized-future minimizer; unavailable at deployment
G_g^{cur}, G_g^0	risk differences	Oracle advantages over current comparator and NULL
$X_{g,a}$	typed features	$\mathcal{D}_g^+$-measurable future-blind deployment features for one complete action
$Q_\psi(X_{g,a})$	model output	Predicted conditional action risk; no model exists here
$[L_g^Q(a), U_g^Q(a)]$	interval	Proposal-law simultaneous interval for action a
$[L_g^P(a), U_g^P(a)]$	interval	Target-law interval after bridge and bridge uncertainty are propagated
$g_{n,j}$	opportunity indicator	One iff the completed-segment event presents an eligible admission opportunity
$u_{n,j}$	commit indicator	One iff an eligible event actually commits a non-NULL action
μ	fixed scalar	Materiality/commit margin
Reg_g	nonnegative scalar	Excess target risk of selected versus best action
$\mathfrak{R}$	resource vector	Tokens, compute, bytes, traffic, latency, energy, and storage terms

64.2 Literal status vocabulary

Printed marker	Interpretation in this manuscript
DEFINITION	Object or estimand defined; no existence, quality, or implementation claim
PROVED UNDER STATED ASSUMPTIONS	Deduction valid when every displayed premise holds; premises not empirically established by the proof
CONJECTURED	Open falsifiable hypothesis with no admitted supporting result
DESIGN PRESCRIPTION	Required design or receipt for a conforming future test
PLANNED TEST	Registered-form test described but not executed
UNVALIDATED	Gate failed, unopened, or unsupported by accepted evidence

Table 46: Status markers are local. A proved lemma cannot promote the system that may someday instantiate it.

64.3 Frozen evidence boundary

- PM-I2: 0/9 jobs qualified.
- PM-I3 learned-content exactness: 0/73,728 for learned routing/learned content and 0/73,728 for oracle routing/learned content.
- PM-I3 learned routing with oracle content: 18,544/73,728; this does not establish learned content.
- PM-I3 complete mechanical oracle: 73,728/73,728; this is a mechanical ceiling, not evidence for a learned route, learned content, or predictive policy.
- Accepted terminal: fit/support failure for the tested pure latent-row interface; it does not isolate representation, decoder, or optimization and does not reject every possible MMLA realization.
- Predictive admission: no executed oracle test, no measured gap, no trained value model, and no learned policy.

These values are inherited only to delimit what this theory paper may claim. They are not reanalyzed, improved, or converted into a predictive-admission result.

Part IV
MMLA-RTT: Dual-State Learning at Reasoning Time

Branch status. This part imports the complete substantive formal branch from the frozen R01 focus paper. It keeps numerical policy state Φ and authoritative memory M distinct in type, writer and reader privilege, lifetime, reset and rollback domain, versioning, and ledger authority. It contributes no new experiment, empirical dual-state success, qualified semantic interface, measured oracle gap, learned admission policy, safety result, or efficiency result.

65 Introduction

65.1 The composition problem

A model may improve a later attempt because its numerical behavior changed, because an authoritative record changed, because more tokens were placed in context, or because sampling happened to differ. Those mechanisms are not interchangeable. This paper studies the first two as a typed composition. It asks what must be logged, intervened on, and proved before a within-problem policy update and an authoritative memory commit can be said to coexist without hidden coupling.

DEFINITION

Definition 65.1 (Dual-state reasoning-time system)**.** A dual-state reasoning-time system has a trainable numerical policy carrier $\Phi \in \mathcal{P}$ and an authoritative memory carrier $M \in \mathcal{M}$. Its attempt kernel reads a declared causal workspace together with pinned versions of both carriers. The carriers expose disjoint mutation interfaces, version lines, rollback domains, and ledgers. A coordinator may check a cross-carrier compatibility predicate but does not own either state.

The definition does not assert that such a system has been implemented, trained, or improved. It rules out a common shortcut: calling one opaque recurrent vector "policy and memory" and then inferring separate causal effects from a single output trace.

65.2 Scientific boundary

The public MMLA V3 manuscript and its complete technical-report evidence record provide context [127, 129]. Their current blocker is unchanged. PM-I2 qualified 0/9 registered same-checkpoint interfaces. In the fitted-population diagnostic, the direct candidate was exact on 40/73,728 probes; learned-route/learned-content and oracle-route/learned-content were each 0/73,728; learned-route/oracle-content was 18,544/73,728, exactly the routing count; and only the complete mechanical oracle reached 73,728/73,728. Preservation was zero and 43,008 cases were missing. The result is fit/support failure for the tested pure latent-row interface. It does not isolate representation, decoder, or optimization and does not reject every MMLA realization.

UNVALIDATED Therefore this paper does not claim a qualified semantic row interface, empirical authoritative-memory success, an expected-risk oracle gap, predictive admission, a learned value model, or a learned admission policy. The proposed composition cannot be an empirical success while its authoritative-memory prerequisite remains unqualified.

Four companion focus papers contribute vocabulary and formal contracts only. RTT Foundations distinguishes policy state from memory state [133]. Atomic Memory Rows defines complete authoritative rows and exact NULL [119]. Predictive Memory Admission defines a future-risk admission problem but reports no oracle gap or learned selector [122]. Completed-Segment Bidirectional Consolidation supplies the completed-segment causal boundary used by any retrospective memory proposal [121]; it contributes no empirical transfer to this paper. No arrow in Figure 45 transfers empirical status.

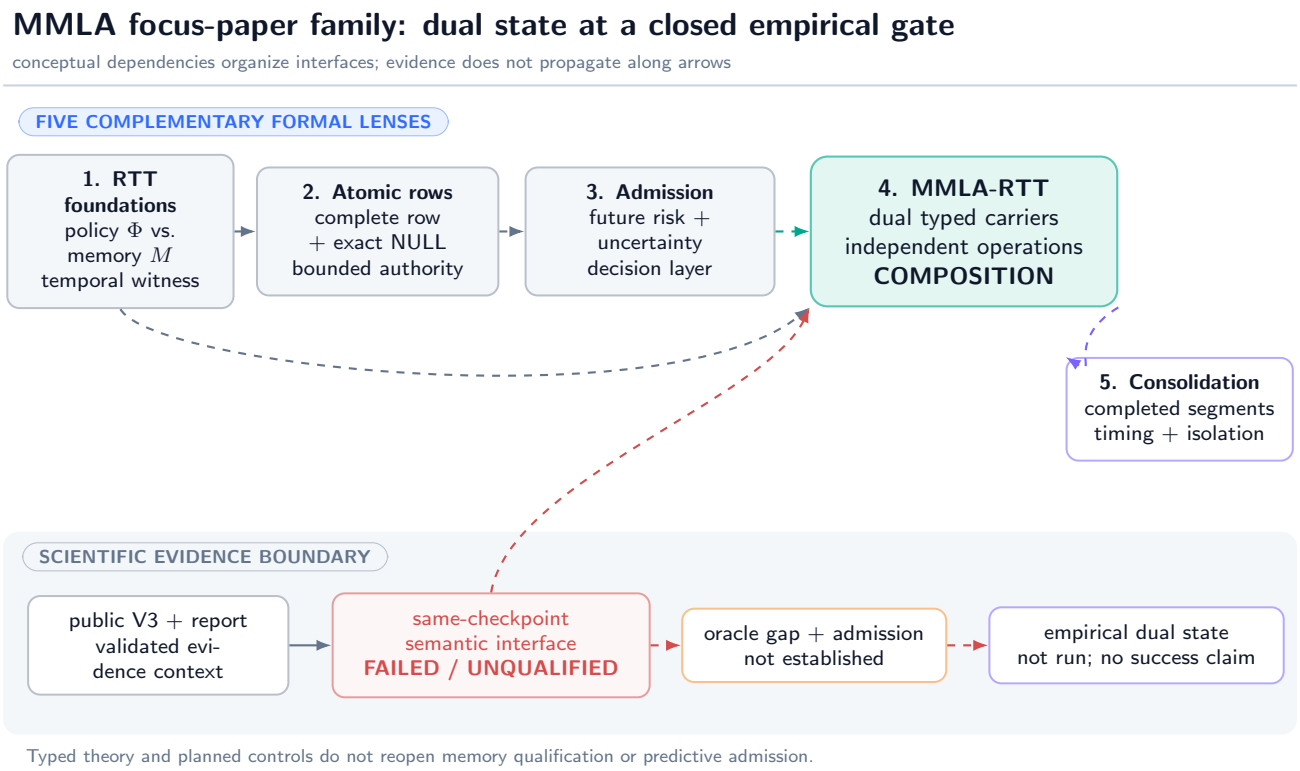

Figure 45: **Dual-state composition in the five-paper conceptual family.** The semantic-interface stop is inherited and remains closed; formal prerequisites do not provide empirical validation.

65.3 Contributions and nonclaims

The paper develops typed semantics, an explicit causal schedule, independent operation algebras, mathematical bounds, factorial estimands, negative controls, resource ledgers, and falsification obligations. It does not supply a semantic serializer, a trusted row assembler implementation, an empirical policy update, a memory-admission model, a dual-state checkpoint, or a deployment result. Definitions, conditional proofs, prescriptions, and planned tests are marked locally and cannot promote an implementation claim.

65.4 Central falsifiable questions

Let Y_{pm} be a preregistered loss under policy intervention $p \in \{0,1\}$ and memory intervention $m \in \{0,1\}$. The primary interaction estimand is

$$\Delta_{\Phi M} = \mathbb{E}[Y_{11} - Y_{10} - Y_{01} + Y_{00}], \tag{241}$$

where lower loss is better. A negative value would indicate complementarity on that endpoint under the declared interventions; positive values indicate antagonism. The paper proves how this contrast is identified under randomization and compliance, not what its sign is. A second question is categorical: does feedback at attempt r update Φ and causally influence attempt $r+1$ on the same still-unresolved problem? Without that path, the mechanism is not strict reasoning-time learning even if M changes.

CONJECTURED Independent policy adaptation and authoritative memory may be complementary in some domains, substitutable in others, neutral when one carrier is irrelevant, and harmful when either update contaminates the other carrier's inputs. All four regimes remain possible before intervention.

66 Typed Carriers and Read Views

66.1 Probability space and causal workspace

Fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ and a problem instance q. Attempts are indexed by $r = 0, 1, \ldots, R_q$. Let $\mathcal{F}_{q,r,t}$ be the information available immediately before micro-event t of attempt r. A bounded causal workspace

$$W_{q,r,t} = \left(q,\ x_{q,r,\le t},\ z_{q,<r},\ f_{q,<r},\ u_{q,r,t}\right) \tag{242}$$

contains the problem, visible prefix, prior public attempt summaries z, admissible feedback f, and a declared controller summary u. Raw private chain-of-thought is neither an authoritative record nor a necessary audit artifact. Any feature used to

update either carrier must be measurable with respect to its declared information set.

DEFINITION

Definition 66.1 (Policy carrier)**.** The policy carrier at event e is

$$P_e = (\Phi_e, O_e, v_e^\Phi, \ell_e^\Phi, \tau_e^\Phi, \rho_e^\Phi), \tag{243}$$

where $\Phi_e \in \mathcal{P} \subseteq \mathbb{R}^d$ is trainable numerical state, O_e is optimizer state, v_e^Φ is a monotone version, ℓ_e^Φ is a lineage identifier, τ_e^Φ declares lifetime and scope, and ρ_e^Φ is a receipt pointer. Its privileged writer is the policy updater; its readers are declared inference modules.

DEFINITION

Definition 66.2 (Authoritative memory carrier)**.** The memory carrier at event e is

$$A_e = (M_e, v_e^M, \ell_e^M, \tau_e^M, \rho_e^M), \tag{244}$$

where M_e is a bounded map from physical slots to complete typed rows or empty slots, v_e^M is an authoritative memory version, ℓ_e^M is its lineage, τ_e^M declares retention and tenant scope, and ρ_e^M points to a commit or NULL receipt. Its privileged writer is a trusted memory assembler/committer; policy gradients never directly mutate it.

DEFINITION

Definition 66.3 (Typed coupling interface)**.** For declared policy-read type $\mathcal{H}_\Phi$, memory-read type $\mathcal{H}_M$, and output law $\Delta(\mathcal{Z})$, a coupling interface is a registered tuple

$$\mathcal{I} = \big(r_\Phi : \mathcal{P}\times\mathcal{O}\times\mathcal{W} \to \mathcal{H}_\Phi,\ r_M : \mathcal{M}\times\mathcal{W} \to \mathcal{H}_M,\ d : \mathcal{H}_\Phi\times\mathcal{H}_M\times\mathcal{W}\times\mathcal{G} \to \Delta(\mathcal{Z})\big). \tag{245}$$

The two reader codomains are distinct declared types. No cast from a policy payload to an authoritative row, or conversely, is available unless it is separately registered, authorized, and included in $\mathcal{I}$.

DEFINITION

Definition 66.4 (Compatibility witness)**.** A witness $\kappa(P, A, C; \mathcal{I})$ binds the exact policy and memory payload hashes, versions and lineages; tenant and ACL scope; base-model and tokenizer hashes; feature-schema hashes; reader and decoder versions; input/output signatures in Equation (245); validity intervals; and the registered fallback. Verification is a typed decision

$$\mathrm{Verify}_{\mathcal{I}}(P, A, C, \kappa) \in \{0, 1\} \tag{246}$$

whose receipt records every checked field. Equality of visible version numbers is neither necessary nor sufficient for verification.

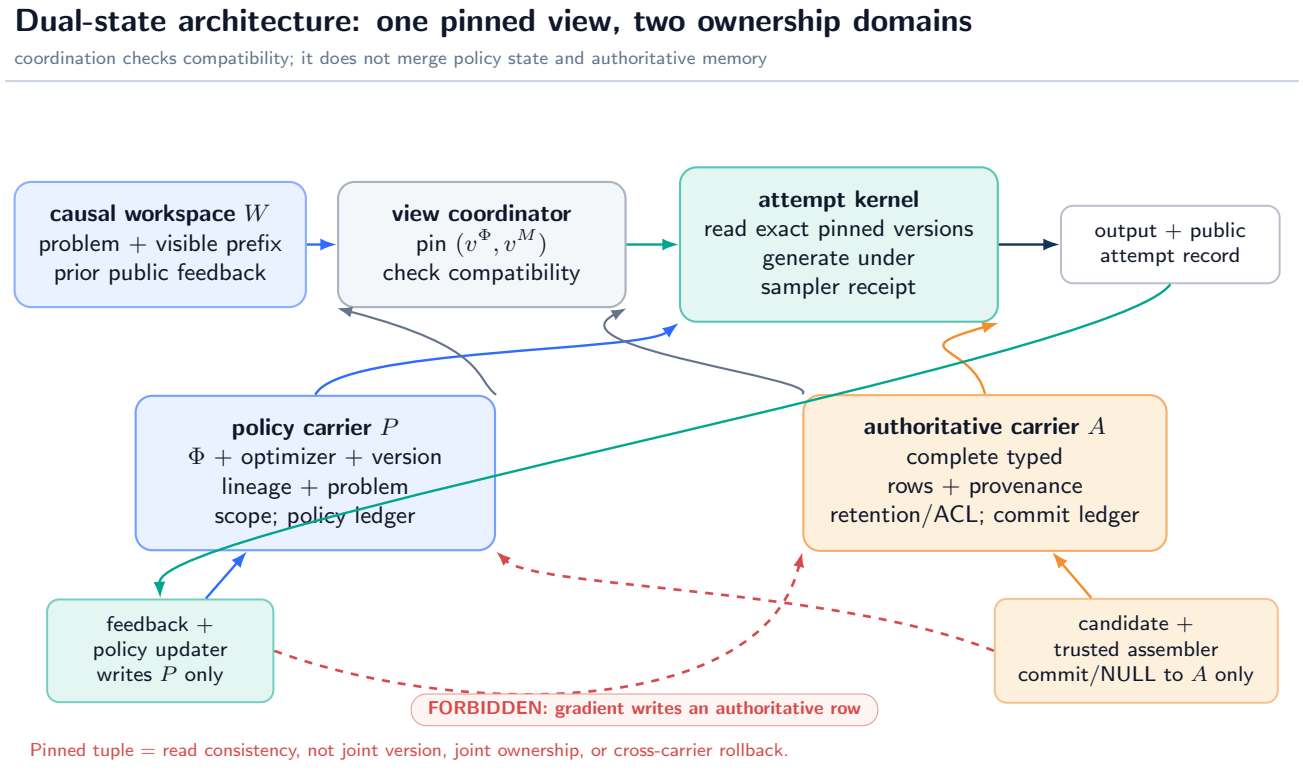

Figure 46: **Typed dual-state architecture** (DEFINITION / UNVALIDATED)**.** The attempt kernel reads a pinned view. Feedback may drive a policy update; a separate trusted path may assemble and commit a complete row. Dashed coral paths are forbidden privilege crossings.

66.2 Type, privilege, lifetime, and failure separation

Property	Policy carrier P	Memory carrier A
Payload	Numerical parameters, optimizer moments, update metadata	Complete typed rows, versions, provenance, ACL/protection, validity, tombstone semantics
Privileged writer	Policy-update service under feedback contract	Trusted assembler and atomic committer
Typical lifetime	Attempt/problem/session; default problem-scoped	Cross-attempt and potentially persistent/archival under retention policy
Reset target	Chosen policy baseline and optimizer state	Chosen authoritative snapshot or empty/default row set
Rollback unit	Numerical checkpoint plus optimizer/lineage receipt	Full row version or full memory snapshot plus commit ledger
Failure examples	Divergence, reward hacking, optimizer corruption, catastrophic local drift	Partial row, stale authority, ACL breach, provenance loss, rollback fork
Primary ledger	Gradient/update, loss/reward, seed, optimizer, version	Candidate, validation, commit/NULL, row version, archive, index, deletion

Table 47: The carriers are distinct across every operational dimension required by this paper.

DEFINITION

Definition 66.5 (Pinned read view)**.** At the start of attempt r, the coordinator emits

$$V_{q,r} = \left(v^{\Phi}_{q,r}, v^{M}_{q,r}, \ell^{\Phi}_{q,r}, \ell^{M}_{q,r}, c_{q,r}\right), \tag{247}$$

where $c_{q,r}$ is a compatibility-receipt identifier. The attempt kernel may read only the named carrier versions. Pinning provides read consistency for that attempt; it does not merge carriers, equate versions, assign joint ownership, or imply joint commit/rollback.

Assumption 66.6 (Interface realizability and sound verification)**.** Every emitted pin receipt $c_{q,r}$ commits to one registered $\mathcal{I}$, one witness κ, the exact carrier and context hashes, and a successful evaluation of Equation (246). The registered readers and decoder are total on the declared validity domain and are the components actually used by the attempt. Outside that domain, or when no verified witness exists, the coordinator refuses the pair or invokes the named fallback; it does not infer compatibility from nominal versions or output behavior.

Assumption 66.7 (Typed authorization)**.** Every state-changing event carries a carrier tag, actor identity, scope, predecessor version, proposed successor version, payload hash, and authorization decision. A policy event can write only P; a memory event can write only A. The coordinator can reject an incompatible view but cannot synthesize either successor.

PROVED UNDER STATED ASSUMPTIONS

Proposition 66.8 (Syntactic non-conflation)**.** *Under Assumption 66.7, no well-typed execution trace can apply a policy mutation to M or a memory commit to Φ. Equality of serialized payload bytes, if it occurs, does not change the carrier tag or privilege domain.*

The proposition is a property of the specified interface, not evidence that an implementation enforces it. Its proof is in Appendix 80.

66.3 Complete state and carrier inventory

The system state at event e is not just (Φ_e, M_e). For causal and cost completeness we use

$$S_e = (W_e, P_e, A_e, C_e, G_e, Q_e), \tag{248}$$

where C_e is immutable causal context, G_e is generation/sampler state including seed and decoding controls, and Q_e contains queues, snapshots, caches, and external ledgers. Omitting G_e can misattribute sampling variation to learning. Omitting Q_e can hide stale cache reads or delayed commits. Neither auxiliary state becomes authoritative memory merely by appearing in the tuple.

67 Fine-Grained Causal Timeline

67.1 Micro-events, not an opaque recurrence

Let $e = 0, 1, \ldots$ index globally ordered micro-events. Each event has one of the tags

$$\mathsf{GEN},\ \mathsf{OBS},\ \mathsf{FDBK},\ \mathsf{PUPD},\ \mathsf{CAND},\ \mathsf{MVAL},\ \mathsf{MCOM},\ \mathsf{PIN},\ \mathsf{RESET},\ \mathsf{ROLLBACK}. \tag{249}$$

The transition is

$$S_{e+1} = T_{\kappa_e}(S_e, \xi_e), \tag{250}$$

where κ_e is the event tag and ξ_e is its typed input. There is no unlabelled map $S_{e+1} = F(S_e)$ from which causal ownership must be guessed after the fact.

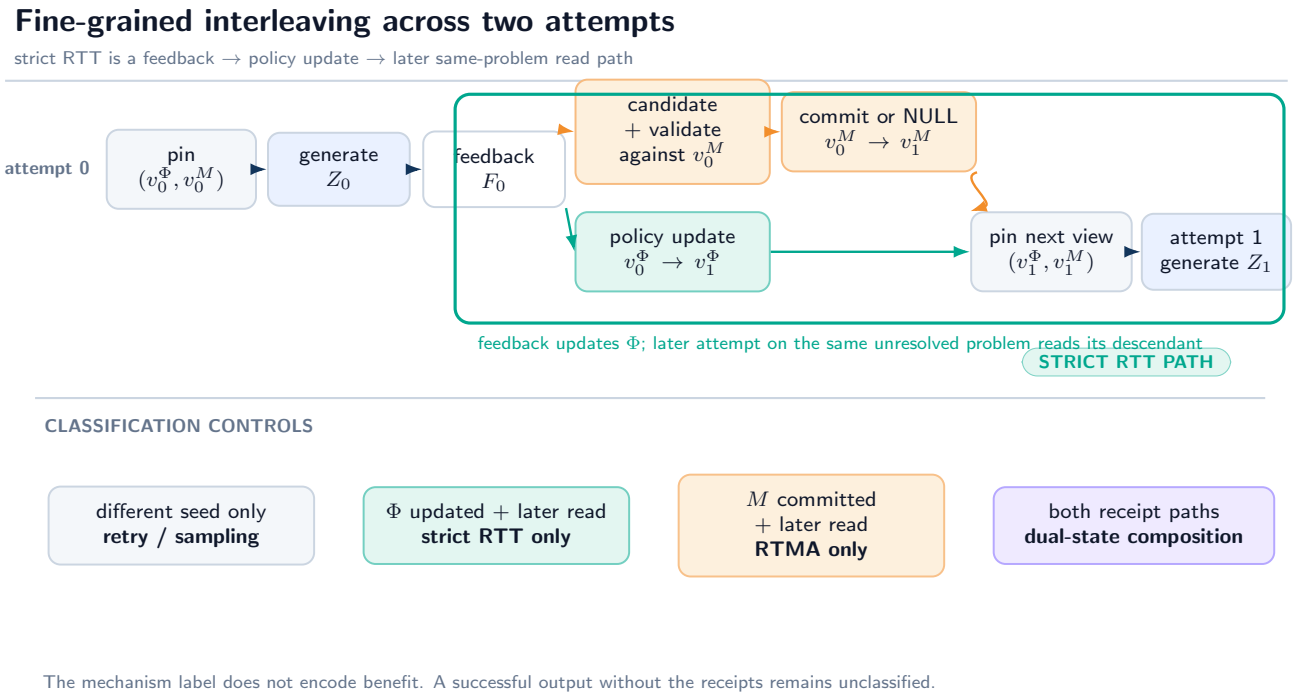

Figure 47: **Declared interleaving for two attempts (DEFINITION).** Attempt 0 pins a view, generates, receives admissible feedback, and may trigger two independent successor events. Attempt 1 counts as strict RTT only if it reads the updated Φ while the problem remains unresolved. A memory commit has its own later-reuse path.

For one illustrative order, attempt r pins (v_r^Φ, v_r^M), generates Z_r, obtains feedback F_r, performs a policy update to v_{r+1}^Φ, independently proposes and validates a memory candidate, optionally commits v_{r+1}^M, and then pins a new view for attempt $r+1$. Other orders are permitted only when declared. In particular, memory admission may precede policy update, or either event may be absent. Section 68 shows why the order can matter.

67.2 Filtration and no future leakage

Assumption 67.1 (Causal measurability)**.** At event e, generation and policy-update inputs are $\mathcal{F}_e$-measurable. Memory candidates may use only the candidate-construction information set declared for that event. A deployment event cannot read future feedback, uncommitted evaluator truth, or a later carrier version. Each output receipt records the exact predecessor view.

Proposition 67.2 (Causal trace measurability)**.** *Under Assumption 67.1, every generated token, policy update, memory candidate, validation decision, commit decision, and pinned view is measurable with respect to the filtration at its event. Consequently a later version cannot be a parent of an earlier output in the declared structural trace.*

This statement excludes temporal leakage under the model; it does not prove that a real logging system timestamps events correctly.

67.3 Attempt-boundary state

Let $b(q,r)$ and $d(q,r)$ denote the begin and terminal event of attempt r. Define the attempt record

$$\mathcal{R}_{q,r} = \big(q, r, V_{q,r}, G_{b(q,r)}, Z_{q,r}, F_{q,r}, E^\Phi_{q,r}, E^M_{q,r}, d(q,r)\big), \tag{251}$$

where $E^\Phi_{q,r}$ and $E^M_{q,r}$ are ordered lists of carrier-specific events. An empty list is meaningful: it proves that no update of that carrier was admitted during the interval. A missing list is not equivalent to an empty list.

DESIGN PRESCRIPTION A conforming trace must preserve all attempt records, carrier ledgers, pin receipts, reset receipts, and sampler state. The public answer can remain concise; the audit need not store unrestricted private reasoning text.

67.4 Interleaving table

Order	Event	Reads	May write	Required receipt
1	PIN	latest authorized versions	view tuple only	both version/lineage IDs and compatibility result
2	GEN	pinned view, workspace, sampler	output and sampler trace	prompt/input hash, seed, decoding contract
3	FDBK	output, evaluator-visible outcome	feedback record	source, timing, scope, admissibility
4a	PUPD	pinned Φ, feedback, optimizer	policy carrier only	objective, gradient/update, predecessor/successor
4b	CAND/MCOM	pinned M, authorized evidence	memory carrier only after validation	complete candidate, validator, commit/NULL
5	PIN	independently current carriers	next view tuple	compatibility and exact versions

Table 48: One admissible partial order. Events 4a and 4b are not assumed to commute.

68 Independent Operation Algebras

68.1 Carrier-specific maps

Let $U_e : \mathcal{P} \times \mathcal{O} \to \mathcal{P} \times \mathcal{O}$ be an authorized policy update and $K_e : \mathcal{M} \to \mathcal{M}$ an authorized complete-row memory transition. Their lifted maps on the full state are

$$\widetilde{U}_e(W, P, A, C, G, Q) = (W, U_e(P), A, C, G, Q'), \tag{252}$$

$$\widetilde{K}_e(W, P, A, C, G, Q) = (W, P, K_e(A), C, G, Q''), \tag{253}$$

where Q' and Q'' append disjoint receipts. A coordinator event may later evaluate $\mathrm{Compat}(P, A, C)$ and refuse a new pin. It does not retroactively rewrite either carrier.

DEFINITION

Definition 68.1 (Operation algebras)**.** $\mathfrak{O}_\Phi$ contains policy snapshot, update, freeze, reset, swap, and rollback maps. $\mathfrak{O}_M$ contains memory snapshot, complete-row commit, exact NULL, reset, swap, row rollback, full-snapshot rollback, archive, and verified deletion maps. Each operation is closed over its own carrier type and appends to its own ledger.

68.2 Commutation is conditional

The carrier writes are disjoint, but their *inputs* can depend on a shared workspace or on the other carrier. Disjoint destination fields alone do not imply semantic commutation.

Assumption 68.2 (Input stability for a pair of events)**.** For a policy update U and memory transition K proposed from state S, (i) U's typed input and authorization decision are invariant to applying K first; (ii) K's candidate, validation, and authorization decision are invariant to applying U first; (iii) receipt append order is normalized without deleting event time; and (iv) both successor pairs satisfy the same compatibility predicate.

PROVED UNDER STATED ASSUMPTIONS

Theorem 68.3 (Conditional commutation)**.** *Under Assumptions 66.7 and 68.2, the carrier projection of the two-event successor is order-invariant:*

$$\pi_{P,A}(\widetilde{K} \circ \widetilde{U}(S)) = \pi_{P,A}(\widetilde{U} \circ \widetilde{K}(S)). \tag{254}$$

The full audit trace remains ordered and therefore need not be byte-identical.

Counterexample 68.4 (Disjoint writes, noncommuting decisions)**.** Suppose the memory candidate is admitted only if the current policy assigns it confidence above $1/2$. From Φ_0, confidence is 0.4 and K returns NULL. Feedback update U raises confidence to 0.6. Then $K \circ U$ commits a row whereas $U \circ K$ leaves memory unchanged, despite disjoint write privileges. Reversing the dependency—letting the policy loss read newly committed memory—gives the symmetric failure.

The counterexample is theoretical. It does not instantiate predictive admission or show that confidence is a valid admission score.

68.3 Compatibility without joint ownership

For a registered interface $\mathcal{I}$, let

$$\Gamma_{\mathcal{I}}(P, A, C) = \{\kappa : \mathrm{Verify}_{\mathcal{I}}(P, A, C, \kappa) = 1\} \quad \text{and} \quad \mathrm{Compat}_{\mathcal{I}}(P, A, C) = \mathbf{1}\{\Gamma_{\mathcal{I}}(P, A, C) \neq \varnothing\}. \tag{255}$$

A successful decision names a concrete κ; the Boolean alone is not a portable proof. If the set is empty, the only authorized action is to withhold a new view or route to the witness-declared fallback. The coordinator must not silently reset P, rewrite A, or manufacture a composite version number.

Definition 68.5 (Coupled transition protocol)**.** Given proposed successors $P^+ = U(P)$ and $A^+ = K(A)$, the policy and memory services first complete their own typed authorization, validation, and carrier-local receipts ρ_Φ^+ and ρ_M^+. The coordinator then evaluates $\Gamma_{\mathcal{I}}(P^+, A^+, C)$. Only a verified witness permits a joint pin receipt

$$\rho_\times = H(\rho_\Phi^+, \rho_M^+, \kappa, \mathcal{I}, C), \tag{256}$$

which binds, but does not replace, the two carrier receipts. Failure creates no new joint view; it neither retracts a valid carrier-local successor nor implicitly rolls either carrier back.

PROVED UNDER STATED ASSUMPTIONS

Proposition 68.6 (Coupled-update closure)**.** *Under Assumptions 66.7 and 66.6, every joint view produced by Definition 68.5 contains one well-typed policy successor, one well-typed memory successor, and a receipt binding their exact lineages to a verified interface witness. If witness verification fails, no joint view is produced and neither carrier is silently changed by the coordinator.*

Counterexample 68.7 (Nominal-version agreement is not compatibility)**.** Let $v^\Phi = v^M = 7$ and let tenant, tokenizer, and base-model identifiers agree. Suppose the registered policy reader emits $\mathbb{R}^{64}$ while the installed decoder requires $\mathbb{R}^{96}$, or the memory reader's ACL excludes the requesting actor. All nominal identifiers can match while no typed, authorized witness exists. A version-pair check would accept an unrealizable read view; Equation (255) rejects it.

Proposition 68.8 (No composite-version implication)**.** *Knowledge of a pinned pair (v^Φ, v^M) and a passing compatibility receipt does not determine either predecessor, lifetime, or rollback target. In particular, there exist two valid histories with the same pinned pair but different policy lineages, and two with the same pair but different memory lineages.*

The proof is by constructing independent fork-and-rejoin histories in Appendix 80. A composite snapshot can be defined, but only as an explicit manifest containing two independently verifiable snapshots and a coordination receipt.

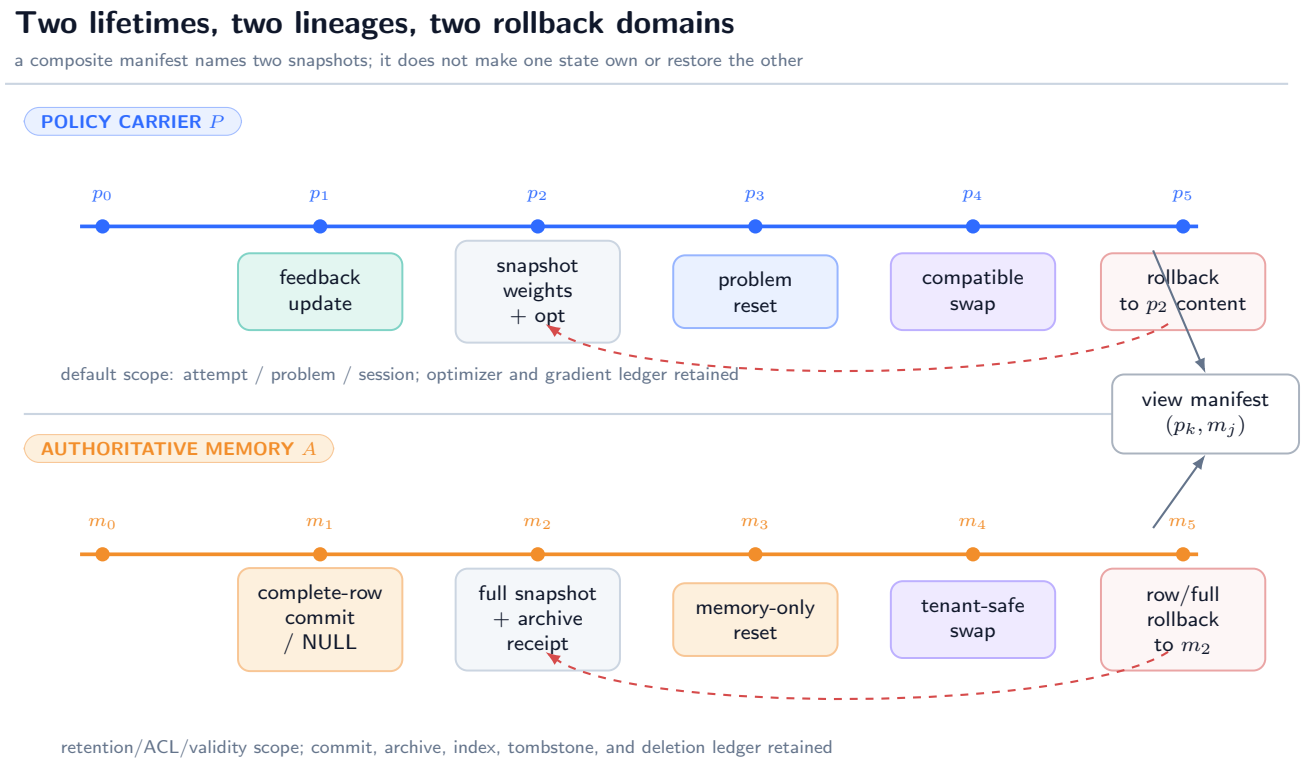

Figure 48: **Independent lifetimes, snapshots, rollback domains, and ledgers (DESIGN PRESCRIPTION).** A view manifest can name both carrier versions; it cannot collapse their lineages or authorize silent cross-restoration.

69 Strict RTT, Memory Adaptation, and Composition

69.1 A path-based definition

Let $Q_q(r) = 1$ mean that problem q remains unresolved immediately after attempt r. Let $F_{q,r}$ be admissible feedback and let $\Phi_{q,r}^{\text{read}}$ denote the policy version pinned by attempt r.

DEFINITION

Definition 69.1 (Strict reasoning-time learning)**.** An execution exhibits strict reasoning-time learning on problem q between attempts r and $s > r$ if all of the following hold:

1. $Q_q(k) = 1$ for every $k = r, \ldots, s-1$;
2. an authorized update event writes $\Phi^+ = U(\Phi^-, F_{q,r}, \eta)$ after feedback from attempt r;
3. a later attempt s pins and reads a descendant of Φ^+;
4. a registered intervention on that policy lineage can change the distribution of the later attempt while holding the declared non-policy state fixed.

The update has default problem scope unless a broader lifetime is explicitly authorized.

Conditions 1–3 establish timing and reuse; condition 4 turns a logged association into a causal mechanism. Merely computing a gradient, saving an adapter after the problem is solved, or updating on one problem and using the result only on unrelated future problems does not satisfy the strict within-problem definition used here.

DEFINITION

Definition 69.2 (Reasoning-time memory adaptation)**.** An execution exhibits reasoning-time memory adaptation (RTMA) when an authorized event changes M during an unresolved problem and a later attempt can read the new authoritative version. RTMA does not require a policy update and is not, by itself, strict RTT.

DEFINITION

Definition 69.3 (Dual-state composition)**.** A dual-state composition is an execution interval containing both a strict-RTT policy path and an RTMA path, with independently identifiable interventions and receipts. The definition does not require either main effect or their interaction to be beneficial.

69.2 Classification theorem

Assumption 69.4 (Attempt and carrier observability)**.** The audit exposes problem identity and unresolved status, event order, feedback source, carrier tags, predecessor and successor lineages, pinned versions, and intervention compliance. Missing receipts are represented as missing, not imputed as no-ops.

PROVED UNDER STATED ASSUMPTIONS

Theorem 69.5 (Mechanism classification)**.** *Under Assumptions 67.1 and 69.4, an interval belongs to exactly one of the following four observable mechanism classes:*

	no later read of updated M	*later read of updated M*
no strict policy path	*static/retry*	*RTMA only*
strict policy path	*strict RTT only*	*dual-state composition*

provided that "path present" uses the definitions above. The classes say nothing about endpoint improvement.

Corollary 69.6 (Memory mutation is insufficient for strict RTT)**.** *If Φ is frozen throughout an interval and no policy descendant is read by a later same-problem attempt, any change caused solely by M belongs to RTMA only, not strict RTT.*

Counterexample 69.7 (Retry masquerading as learning)**.** Two attempts use identical Φ, identical M, and different sampler seeds. The second succeeds. The output improvement is compatible with pure sampling variation and therefore certifies neither RTT nor RTMA.

Counterexample 69.8 (Post-resolution adaptation)**.** Attempt r solves the problem, after which feedback updates Φ for a future task. This may be online or continual adaptation, but it is not strict reasoning-time learning for the resolved problem under Definition 5.1.

69.3 Mechanism receipts

Claimed mechanism	Minimum positive receipt	Required negative control
Strict RTT	same problem, unresolved marker, feedback, Φ predecessor/successor, later pinned descendant	reset/fresh/frozen or swapped Φ with M and sampler held fixed
RTMA	complete-row commit/NULL, M predecessor/successor, later pinned descendant	memory reset/swap/freeze with Φ and sampler held fixed
Dual-state composition	both receipt chains plus exact interleaving	independent 2×2 interventions and compliance audit
Benefit	preregistered endpoint under randomized intervention	seed, order, leakage, and attrition checks

Table 49: A mechanism receipt is necessary but not an outcome result.

DESIGN PRESCRIPTION Names such as "test-time learning," "memory," or "self-improvement" are never compliance evidence. The trace and interventions determine classification.

70 Reset, Swap, Snapshot, and Rollback Semantics

70.1 Independent operations

For carrier $X \in \{\Phi, M\}$, write $\text{Snapshot}_X(h)$ for a content-addressed snapshot at history position h, $\text{Reset}_X(b)$ for reset to a declared baseline b, $\text{Swap}_X(s)$ for replacement by an independently sourced compatible snapshot, and $\text{Rollback}_X(h)$ for restoration to a prior authorized snapshot with a new rollback receipt. Rollback never erases history; it creates a successor whose content may equal an earlier snapshot.

Assumption 70.1 (Carrier-local restoration)**.** Each restoration operation names exactly one carrier, verifies its target snapshot and lineage policy, changes only that carrier and its ledger, invalidates carrier-local caches, and appends a successor version. Cross-carrier cache invalidation or pin refusal may occur, but the other carrier's content and lineage remain unchanged unless a separate authorized event names it.

PROVED UNDER STATED ASSUMPTIONS

Theorem 70.2 (Independent rollback preservation)**.** *Under Assumptions 66.7 and 70.1, applying* $\text{Rollback}_\Phi(h)$ *leaves the content, version, and lineage of* M *unchanged; applying* $\text{Rollback}_M(k)$ *leaves the content, version, and lineage of* Φ *unchanged. A subsequent compatibility failure may block a pin but cannot silently restore the other carrier.*

Corollary 70.3 (Explicit composite recovery)**.** *Restoring both carriers requires two authorized carrier-local events or an explicit composite manifest that expands into those two events. A single unnamed "system rollback" is nonconforming.*

70.2 Reset and swap controls

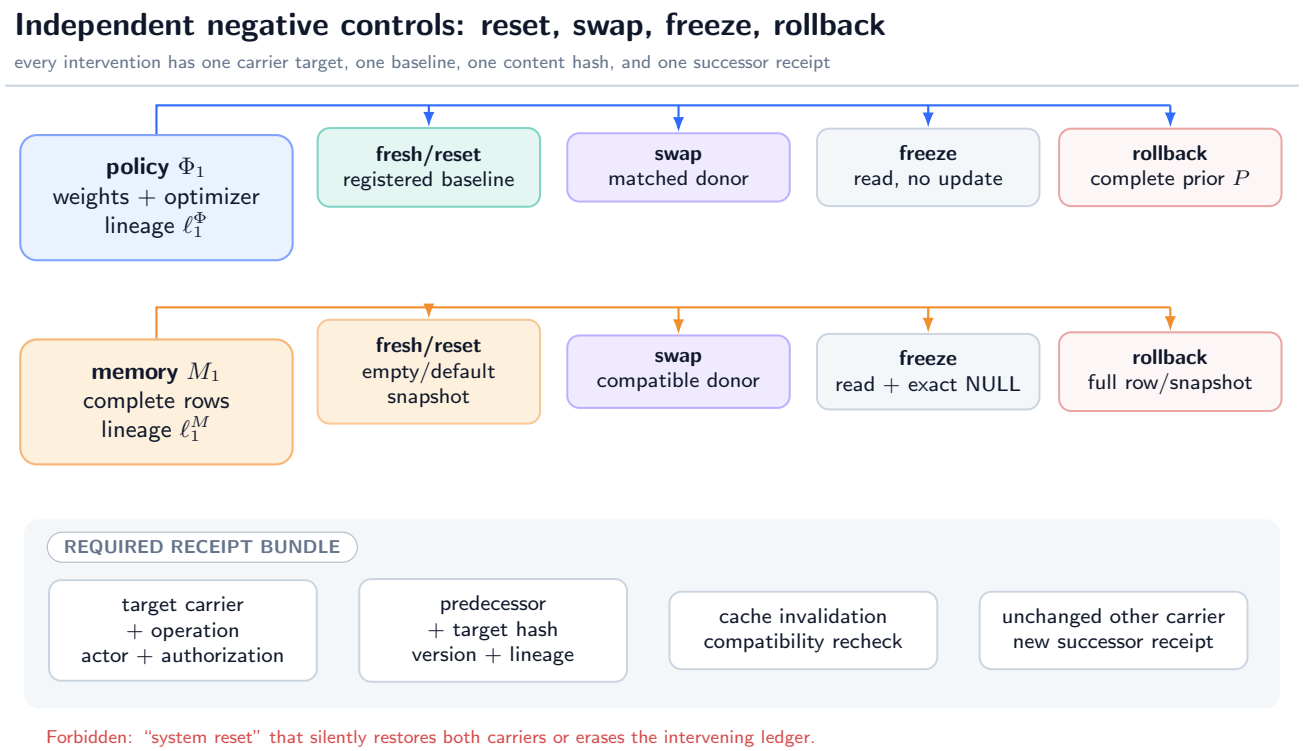

Figure 49: **Reset, swap, freeze, and rollback controls with receipts (DESIGN PRESCRIPTION).** Each intervention targets one carrier. A composite control is two declared interventions, never an implicit cross-state restoration.

The controls answer different questions:

- fresh_Φ restores the registered initialization and optimizer state; reset_Φ may restore another declared problem baseline.
- swap_Φ imports a compatible policy snapshot from a matched unit, testing content specificity while preserving nominal update count.
- freeze_Φ permits reads but blocks updates, separating extra attempts from adaptation.
- $\mathsf{rollback}_\Phi$ tests recovery to a known policy snapshot and must include optimizer state unless the protocol explicitly studies optimizer mismatch.
- fresh_M installs the registered empty/default authoritative state; reset_M restores a specified authoritative snapshot.
- swap_M imports a compatible memory snapshot from a matched unit, testing whether success depends on problem-specific authoritative content.
- freeze_M permits reads and exact NULL only; $\mathsf{rollback}_M$ restores a full row version or full snapshot without changing Φ.

Operation	Target content	Required companion state	Invalid shortcut
Reset_Φ	parameters and declared optimizer baseline	same M, context, sampler contract	zeroing weights while keeping stale moments
Swap_Φ	compatible independent policy snapshot	same M and view constraints	changing base model/tokenizer with the swap
$\mathsf{Rollback}_\Phi$	prior policy checkpoint plus lineage	unchanged M	rewriting history or memory to "match"
Reset_M	exact registered memory baseline	same Φ	clearing only an index while rows remain
Swap_M	compatible independent full snapshot	same Φ	importing rows without tenant/provenance checks
$\mathsf{Rollback}_M$	full row or full snapshot	unchanged Φ	fieldwise partial undo or silent policy restoration

Table 50: Carrier-specific recovery semantics.

70.3 Lineage and retention

Let $L_X = (V_X, E_X)$ be the directed acyclic lineage graph for carrier X. Every event edge records predecessor, successor, operation, actor, timestamp/order, input hashes, authorization, and outcome. A rollback points to an earlier content snapshot but creates a new vertex. A deletion/tombstone event in L_M follows the memory retention contract; it does not delete a policy receipt. A policy reset in L_Φ cannot shorten an authoritative row's retention lifetime.

Proposition 70.4 (No silent history erasure)**.** *If lineage vertices and edge receipts are append-only and content-addressed, then a restore operation cannot make its existence observationally identical to an uninterrupted continuation for an auditor who reads the ledger, even when restored content equals an earlier snapshot.*

This is an auditability statement under append-only storage, not a guarantee against a privileged ledger compromise.

PROVED UNDER STATED ASSUMPTIONS

Proposition 70.5 (Fresh joint pin after restoration)**.** *Under Assumptions 70.1 and 66.6, every reset, swap, or rollback of either carrier invalidates the preceding joint pin and compatibility witness. Before the next joint read, the coordinator must verify a fresh witness over the new content hash, lineage, version, schema, reader/decoder versions, tenant/ACL, validity, and lifetime fields. Equality of a recycled nominal version is insufficient. Thus a carrier-local restoration neither resets the other carrier nor permits an ABA substitution to reuse an old joint authorization.*

71 The Compress-All Comparator

71.1 Why a comparator needs assumptions

Recurrent and test-time-learning systems often compress observations into a bounded numerical state [79, 35, 94, 6]. Compression is not inherently defective. The relevant question is whether indiscriminately mixing a stream into one trainable carrier causes more harmful cross-event influence than a typed, gated design for a declared loss.

Let observations be $X_1, \dots, X_T$, with informative indices I and contaminants C. A projected compress-all update is

$$\Phi_{t+1} = \mathrm{proj}_{\mathcal{P}}\{\Phi_t - \eta_t g(\Phi_t, X_t)\}. \tag{257}$$

Let Φ_t^I be the coupled trajectory that replaces $g(\Phi_t, X_t)$ by zero for $t \in C$ while sharing initialization and informative inputs.

Assumption 71.1 (Conditional update sensitivity)**.** Projection is nonexpansive; for every x, $g(\cdot, x)$ is L_g-Lipschitz; and contaminant updates satisfy $\|g(\Phi_t^I, X_t)\| \le B_t$ for $t \in C$. The downstream loss $\ell(\Phi)$ is L_ℓ-Lipschitz on $\mathcal{P}$.

PROVED UNDER STATED ASSUMPTIONS

Theorem 71.2 (Conditional contamination bound)**.** *Under Assumption 71.1,*

$$\|\Phi_T - \Phi_T^I\| \le \sum_{t \in C} \eta_t B_t \prod_{u=t+1}^{T-1} (1 + \eta_u L_g), \tag{258}$$

$$|\ell(\Phi_T) - \ell(\Phi_T^I)| \le L_\ell \sum_{t \in C} \eta_t B_t \prod_{u=t+1}^{T-1} (1 + \eta_u L_g). \tag{259}$$

The bound is one-sided only if an additional sign condition is supplied; by itself it bounds magnitude, not harm.

The product exposes amplification by later update sensitivity. It is not a measured contamination rate and does not compare hardware efficiency.

71.2 When compression is optimal

Counterexample 71.3 (Sufficient-statistic optimum)**.** Let $X_t \mid \theta \sim \mathcal{N}(\theta, \sigma^2)$ independently with a Gaussian prior on θ, and let the next prediction minimize squared error. The posterior is summarized exactly by a bounded pair consisting of precision and precision-weighted mean. Updating that pair with every observation is Bayes-optimal. A second authoritative row store cannot improve Bayes risk when it adds no information beyond the sufficient statistic and costs are nonnegative.

Counterexample 71.4 (Contamination can help)**.** If indices labeled "contaminants" are actually informative under the target distribution, suppressing their updates increases risk. The sign of Equation (259) cannot be inferred from a taxonomy chosen after observing outcomes.

Proposition 71.5 (No universal dominance)**.** *Without restrictions on the data law, state capacity, update class, downstream loss, and resource cost, neither a dual-state architecture nor compress-all recurrence uniformly dominates the other.*

Thus this paper makes no universal attack on recurrent models, state-space models, TTT layers, or neural long-term memory. Its claim is narrower: if a system asserts authoritative semantics and policy adaptation as distinct mechanisms, then it must expose the distinctions and controls required here.

71.3 Typed gating as a conditional intervention

Let $A_t \in \{0, 1\}$ be a preregistered gate deciding whether event t contributes to Φ. The gated path uses $A_t g(\Phi_t, X_t)$. If A_t depends on hidden future outcomes, comparisons are confounded. If it depends on causal features and is randomized or otherwise identified, its contribution can be studied. This definition is not predictive memory admission: A_t governs a policy update, not a complete authoritative row commit.

PLANNED TEST A future comparator must register the observation taxonomy, target law, update budget, parameter count, context budget, evaluation loss, and all system costs. Calling one carrier "compressed" and another "memory" does not equalize capacity or information.

72 Policy Drift and Dual-State Interaction Bounds

72.1 Bounded policy drift

Consider K authorized policy updates

$$\Phi_{k+1} = \mathrm{proj}_{\mathcal{P}}(\Phi_k - \eta_k \widehat{g}_k), \qquad \|\widehat{g}_k\| \le G_k. \tag{260}$$

PROVED UNDER STATED ASSUMPTIONS

Proposition 72.1 (Path-length bound)**.** *If projection fixes every $\Phi_k \in \mathcal{P}$ and is nonexpansive, then*

$$\|\Phi_K - \Phi_0\| \le \sum_{k=0}^{K-1} \eta_k G_k. \tag{261}$$

If the output distribution satisfies $\mathrm{TV}(p_\Phi(\cdot \mid w, m), p_{\Phi'}(\cdot \mid w, m)) \le L_p \|\Phi - \Phi'\|$, its drift is at most $L_p \sum_k \eta_k G_k$ at fixed workspace and memory.

The result controls displacement, not improvement, stability to distribution shift, or safety. A zero gradient also satisfies the bound and learns nothing.

72.2 Interaction-sensitive error

Let the deployed predictor be $h(\Phi, M, W)$. Fix reference states (Φ_0, M_0) and define perturbations $\delta_\Phi = d_\Phi(\Phi, \Phi_0)$ and $\delta_M = d_M(M, M_0)$. Suppose a scalar loss obeys

$$|\ell(\Phi, M) - \ell(\Phi_0, M_0)| \le L_\Phi \delta_\Phi + L_M \delta_M + L_{\Phi M} \delta_\Phi \delta_M. \tag{262}$$

The cross term permits one carrier's displacement to amplify sensitivity to the other. It does not imply complementarity; the bound is unsigned.

Assumption 72.2 (Local mixed sensitivity)**.** Equation (262) holds on the registered intervention region, and the state metrics, reference states, and constants are fixed before evaluation.

PROVED UNDER STATED ASSUMPTIONS

Theorem 72.3 (Dual-state perturbation envelope)**.** *Under Assumption 72.2, if interventions guarantee $\delta_\Phi \le \epsilon_\Phi$ and $\delta_M \le \epsilon_M$, then every unit's absolute loss displacement is bounded by*

$$B_{\Phi M} = L_\Phi \epsilon_\Phi + L_M \epsilon_M + L_{\Phi M} \epsilon_\Phi \epsilon_M. \tag{263}$$

For a bounded loss, the same envelope bounds the absolute population mean displacement. No sign or causal decomposition follows without factorial interventions.

72.3 Four interaction regimes

For losses, define unit-level main effects $\tau_\Phi = Y_{10} - Y_{00}$, $\tau_M = Y_{01} - Y_{00}$, and interaction $\tau_{\Phi M} = Y_{11} - Y_{10} - Y_{01} + Y_{00}$.

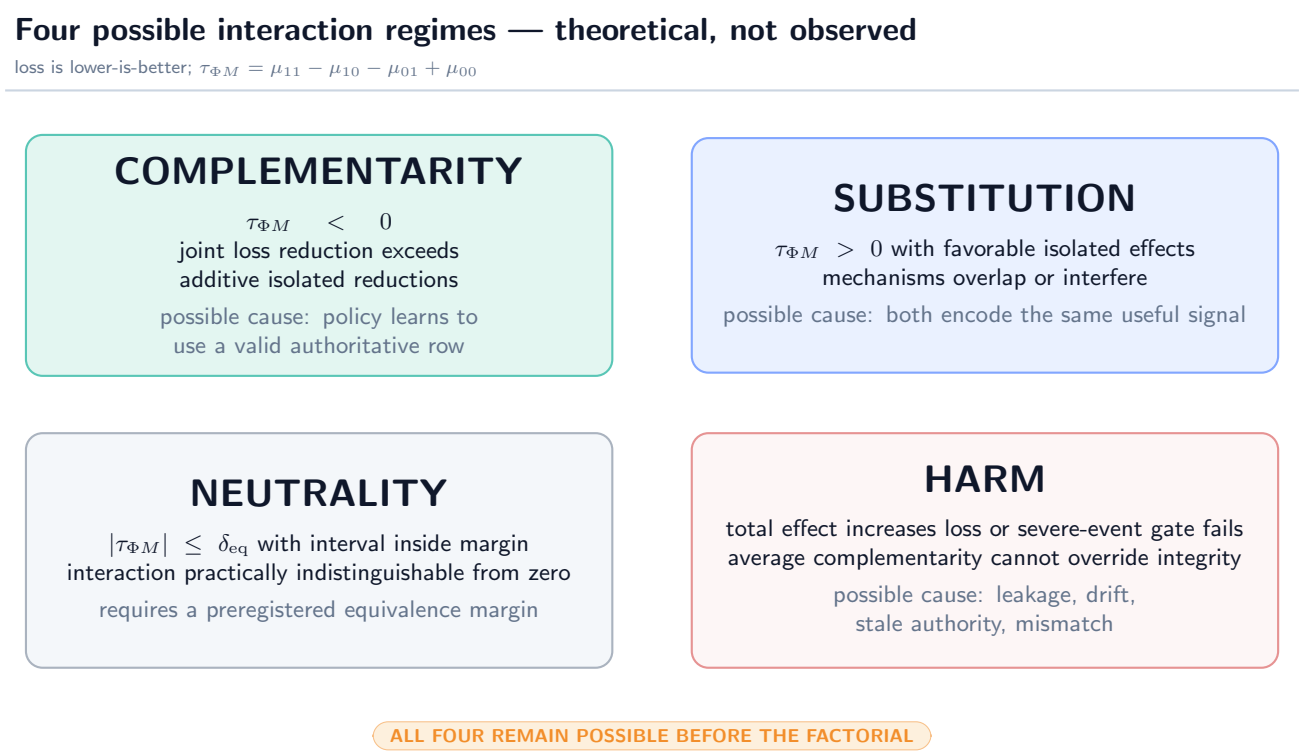

Figure 50: **Theoretical/planned interaction regimes (DEFINITION / PLANNED TEST).** Complementarity, substitution, neutrality, and harm are signs of registered contrasts, not observed dual-state results.

- *Complementarity*: $\mathbb{E}[\tau_{\Phi M}] < 0$; the joint loss reduction exceeds the sum of isolated reductions.
- *Substitution*: isolated effects improve loss but $\mathbb{E}[\tau_{\Phi M}] > 0$; the mechanisms overlap or interfere.
- *Neutrality*: the interaction is practically indistinguishable from zero under a preregistered equivalence margin.
- *Harm*: at least one registered total effect increases loss or violates a severe-event constraint, regardless of average interaction.

Counterexample 72.4 (Average complementarity hides a severe subgroup)**.** Nine units improve jointly by one loss unit and one protected subgroup unit worsens by eight. The mean joint effect is favorable, but a preregistered severe-event or subgroup gate can still fail. Average interaction is not a safety certificate.

DESIGN PRESCRIPTION The future protocol must report all four cell means, main effects, interaction, uncertainty, required subgroups, severe outcomes, and total cost. It may not display only the best arm.

73 The Two-by-Two Policy-by-Memory Factorial

73.1 Assignments, states, and potential outcomes

For experimental unit i (a problem-group, not an individual attempt), let $Z_i^{\Phi}, Z_i^{M} \in \{0,1\}$ be independently randomized assignments. Assignment 1 means the registered updated carrier is made available; assignment 0 means the registered baseline/fresh carrier. Let D_i^{Φ}, D_i^{M} be realized compliance indicators and $Y_i(p,m)$ the loss that would be observed under realized carrier interventions (p,m). This potential-outcome notation follows the distinction between assignments and outcomes [87].

DEFINITION

Definition 73.1 (Factorial estimands)**.** For $p,m \in \{0,1\}$ define $\mu_{pm} = \mathbb{E}[Y_i(p,m)]$ and

$$\tau_{\Phi|m} = \mu_{1m} - \mu_{0m}, \tag{264}$$
$$\tau_{M|p} = \mu_{p1} - \mu_{p0}, \tag{265}$$
$$\tau_{\Phi M} = \mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}, \tag{266}$$
$$\bar{\tau}_{\Phi} = \tfrac{1}{2}(\tau_{\Phi|0} + \tau_{\Phi|1}), \qquad \bar{\tau}_{M} = \tfrac{1}{2}(\tau_{M|0} + \tau_{M|1}). \tag{267}$$

Negative effects improve a loss endpoint. Every endpoint and materiality margin is preregistered.

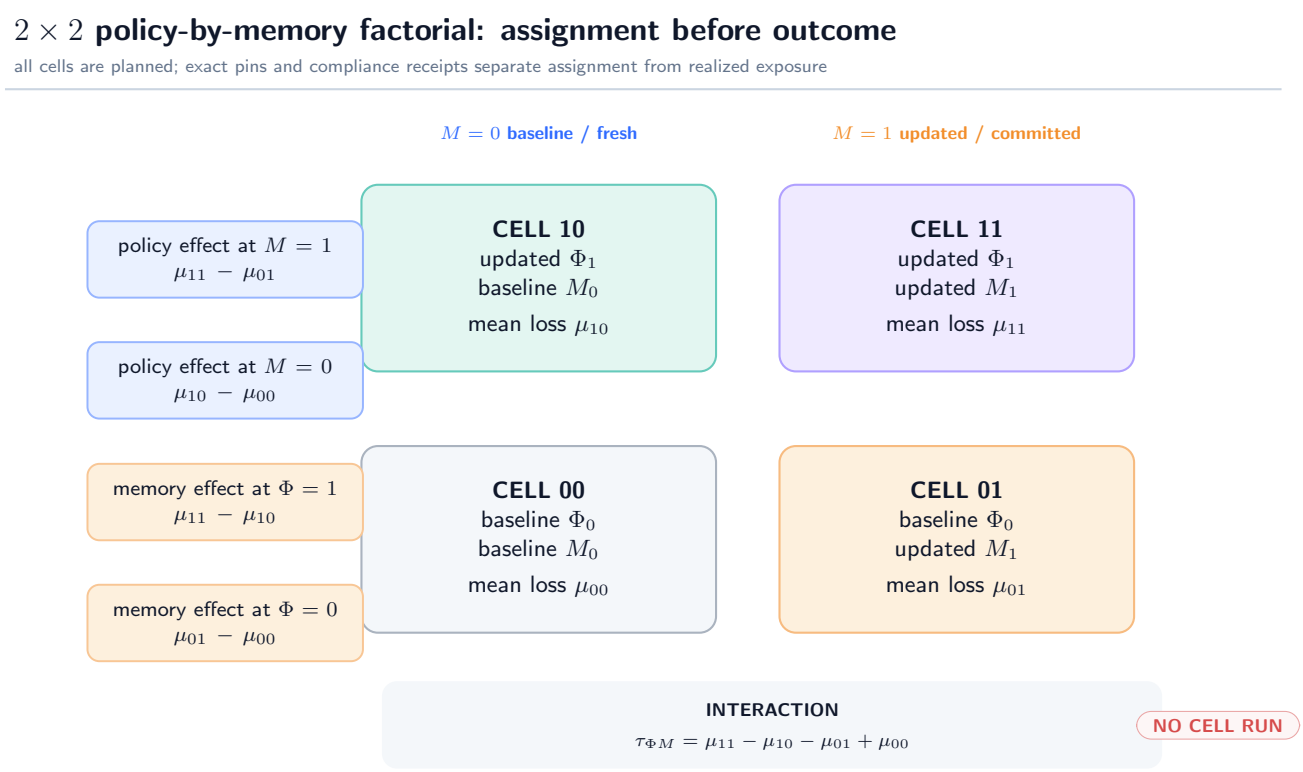

Figure 51: **Independent 2×2 interventions and contrasts (PLANNED TEST).** Each cell pins its assigned carrier versions while holding base model, visible workspace, sampler contract, and resource budget to the registered design. No cell has been run.

Assumption 73.2 (Factorial identification)**.** Units are assigned with known positive probability to all four cells; assignments precede outcome generation; consistency and no interference hold at the registered group boundary; carrier versions are pinned before the evaluated attempt; attrition and endpoint construction do not depend on hidden potential outcomes; and the analysis either has perfect compliance or distinguishes assignment from receipt-verified exposure.

PROVED UNDER STATED ASSUMPTIONS

Theorem 73.3 (Identification under randomized compliant intervention)**.** *Under Assumption 73.2 with perfect compliance,*

$$\mathbb{E}[Y_i^{\text{obs}} \mid Z_i^{\Phi} = p, Z_i^{M} = m] = \mu_{pm}. \tag{268}$$

Therefore cell-mean differences identify both conditional main effects, both marginal main effects, and $\tau_{\Phi M}$. With known unequal assignment probabilities, Horvitz–Thompson cell means are unbiased under the same premises.

The theorem does not establish exclusion restrictions under noncompliance, adequate power, metric validity, or any beneficial sign.

73.2 Fresh, reset, swapped, and frozen variants

The primary binary factor must be defined exactly. A useful staged design is:

1. *availability factorial*: updated versus registered fresh baseline for each carrier;
2. *content-specificity controls*: problem-matched updated carrier versus compatible swapped carrier;
3. *attempt-budget control*: mutable versus frozen policy with identical extra attempts;
4. *recovery controls*: independently roll back one carrier after a registered checkpoint and test restoration of its isolated effect;
5. *order control*: randomize admissible update/commit order where both are enabled.

Each stage uses new held-out groups or a registered repeated-measures design that models carryover. A carrier called "reset" without a content hash, lineage, optimizer state (for Φ), and cache invalidation receipt is noncompliant.

73.3 Noncompliance and exclusions

If $D \neq Z$, the intent-to-treat contrast remains identified by random assignment, but it estimates assignment, not carrier exposure. A complier effect would require additional monotonicity and exclusion assumptions that are especially fragile when pin failures change latency or fallback behavior. This paper does not assume those conditions.

Proposition 73.4 (Receipt-stratified descriptive effect is not automatically causal)**.** *Conditioning on successful pin or commit after randomization can select on post-assignment variables. Unless compliance is guaranteed by design or modeled under explicit assumptions, the difference between receipt-positive units is descriptive rather than the randomized cell effect.*

PLANNED TEST The protocol in Appendix 81 uses intent-to-treat as primary, reports compliance separately, and stops rather than silently dropping failed pins, validator rejects, timeouts, or exact NULL outcomes.

74 Observational Non-Identification

74.1 Output traces do not name the changed carrier

Let the observable trace be $O = (q, Z_0, F_0, Z_1)$, omitting internal state and receipts. A finite observable law induces a later-output kernel $K(z_1 \mid q, z_0, f_0)$. Write $\mathfrak{R}_{M\leftarrow\Phi}(K; \mathcal{I})$ for the set of memory-update realizations that (i) use a registered typed reader/decoder interface $\mathcal{I}$, (ii) can encode K within declared capacity, (iii) admit every required row under the memory authorization, validity, tenant, and ACL contracts, and (iv) preserve the registered causal and randomization law. Define $\mathfrak{R}_{\Phi\leftarrow M}(K; \mathcal{I})$ symmetrically for policy-update realizations. These sets can be empty.

PROVED UNDER STATED ASSUMPTIONS

Theorem 74.1 (Conditional output-trace equivalence)**.** *For a policy-update law with kernel K, a frozen-policy memory-update system inducing the same law exists only if $\mathfrak{R}_{M\leftarrow\Phi}(K; \mathcal{I}) \neq \varnothing$. Conversely, a frozen-memory policy-update realization exists only if $\mathfrak{R}_{\Phi\leftarrow M}(K; \mathcal{I}) \neq \varnothing$. Whenever either set is nonempty, selecting any member yields output equivalence at O; therefore O alone cannot distinguish mechanisms* within that compatible realization class. *Finiteness of the observable alphabet and capacity to store a table do not by themselves establish either nonemptiness claim.*

The theorem is an assumption-explicit identification result, not an automatic lookup-table argument. It does not say that two realizations share cost, privileges, interpretability, or semantics.

Counterexample 74.2 (Finite capacity without legal realization)**.** Let a finite table for K fit in one row, but let its output signature be absent from the registered memory reader, or let the only row that could store it violate tenant/ACL scope. Then physical capacity is sufficient while $\mathfrak{R}_{M\leftarrow\Phi}(K; \mathcal{I}) = \varnothing$. A frozen policy cannot legally read the table, so output equivalence does not follow. The symmetric example uses a policy endpoint that cannot represent or authorize the required memory-derived update.

Corollary 74.3 (No interaction from one trajectory)**.** *A single dual-state trajectory cannot identify $\tau_{\Phi M}$ because at most one of the four potential outcomes is observed for its experimental unit.*

74.2 Hidden shared state

Even carrier hashes are insufficient if a third state changes. Let G be sampler state and Q a cache. Two runs with equal (Φ, M) but different G can have different outputs. Two runs with equal carrier payloads but a stale cache in Q can read different effective memory. Hence the intervention must pin or log the complete state inventory from Equation (248).

Counterexample 74.4 (Cache-mediated false memory effect)**.** Assignment changes M, but the inference process continues reading a cached old row. The intended memory intervention has no exposure, while a latency-triggered retry changes the sampler. An analysis using assigned memory labels attributes the output difference to M. Pin, cache-invalidation, and read receipts would expose the violation.

Counterexample 74.5 (Feedback-dependent stopping)**.** The updated-policy arm is allowed more attempts only after poor feedback. Comparing final accuracy to a fixed-attempt baseline mixes policy state with an adaptive compute budget. The policy factor must define or condition on a registered stopping rule without post-treatment cherry-picking.

74.3 Swap tests and content specificity

A successful updated-versus-fresh comparison can still reflect update count, file age, cache warming, or lineage-specific configuration. A compatible swap control tests whether the actual state content matters. Let S_i^{Φ} be another unit's policy snapshot and S_i^{M} another unit's memory snapshot, matched on schema, update count, resource budget, and non-content metadata. If the updated state helps but its swapped counterpart helps equally, problem-specific content has not been isolated.

Assumption 74.6 (Valid swap)**.** Swap donors are selected independently of recipient potential outcomes; compatibility fields are identical; no recipient-specific content leaks into the donor; and all non-content execution differences are balanced or fixed.

PROVED UNDER STATED ASSUMPTIONS

Proposition 74.7 (Swap contrast interpretation)**.** *Under Assumption 74.6 and randomized assignment, the updated-versus-swapped contrast identifies the effect of recipient-specific carrier content relative to the declared donor distribution, not a universal semantic-content effect.*

74.4 Missing provenance

If predecessor hashes, carrier tags, or pin receipts are missing, multiple incompatible histories fit the same terminal bytes. No statistical estimator can recover a uniquely identified reset, rollback, or update lineage from terminal state alone without additional structural restrictions.

DESIGN PRESCRIPTION A missing receipt is a protocol failure. It is never converted into "no update," "successful reset," or "same memory."

75 Authorization and Information-Flow Isolation

75.1 Separate writers and admissible inputs

Policy feedback can contain scalar rewards, verifier outcomes, public error classes, or other registered signals. It must not automatically become authoritative memory content. A memory candidate can contain a canonical fact and provenance but must not be interpreted as a gradient. The two flows can originate from one observed event while remaining separately authorized.

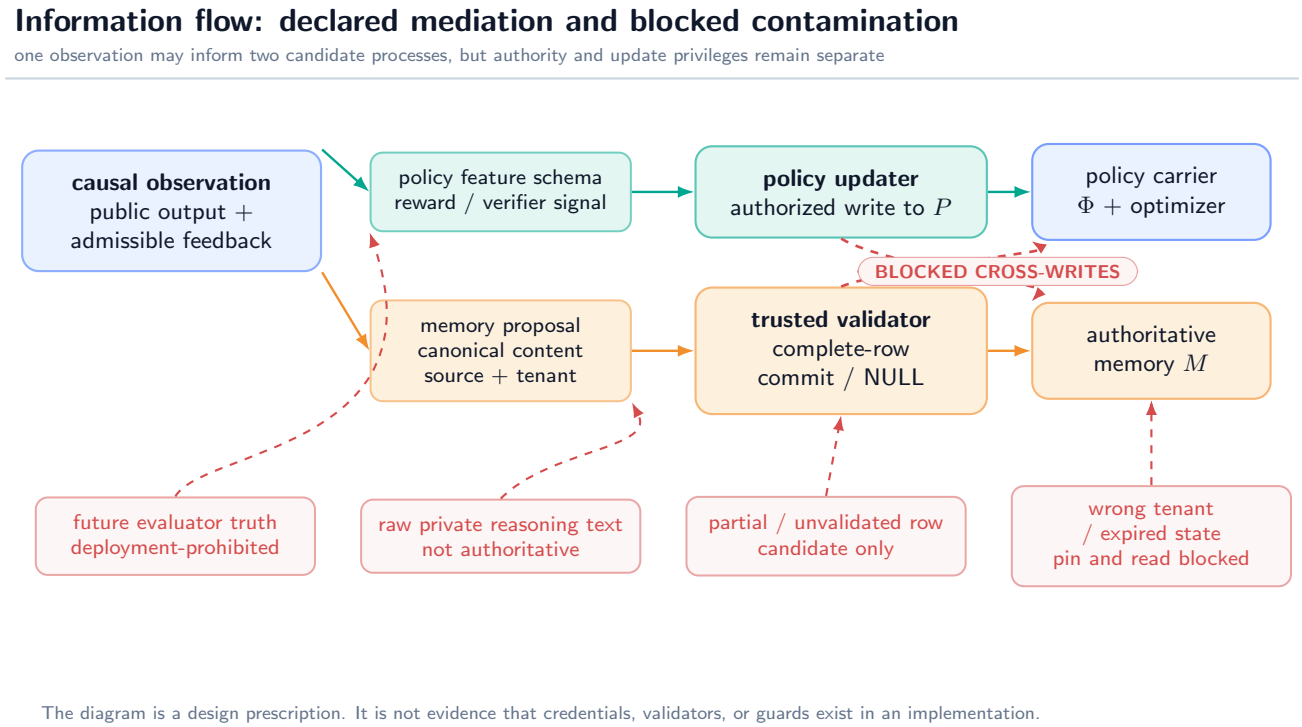

Figure 52: **Contamination pathways and blocked privilege crossings (DESIGN PRESCRIPTION).** Teal arrows are declared reads or typed updates. Coral dashed arrows are forbidden: gradients cannot write authoritative rows; unvalidated candidates cannot alter policy; private reasoning text is not authority; future evaluator truth cannot enter deployment.

Assumption 75.1 (Least-privilege mediation)**.** Policy and memory writers execute under distinct credentials. Candidate construction is non-authoritative. Only the trusted memory validator/committer can publish a complete row or exact NULL. Only the policy updater can publish a policy successor. Every cross-carrier feature is explicitly serialized, schema-checked, and logged. Registered policy readers, memory readers, and the decoder are read-only and versioned. A coordinator may verify a compatibility witness and append a signed pair receipt, but neither that witness nor the pair receipt authorizes a write to either carrier.

PROVED UNDER STATED ASSUMPTIONS

Proposition 75.2 (Blocked direct contamination)**.** *Under Assumptions 66.7 and 75.1, a compromised ordinary policy-update call cannot directly publish an authoritative memory successor, and a memory candidate cannot directly publish a policy successor. The statement does not cover compromise of both privileged services or a shared trusted computing base.*

75.2 Trusted assembly and exact NULL

The inherited row contract requires a complete target-conditioned row with canonical and neural views, slot, tenant and ACL fields, monotone version, validity, protection, provenance, rollback pointer, deletion/tombstone semantics, and a consistency

receipt [119]. A neural module may propose semantic content, but trusted assembly owns protected fields and validation. If any required field or receipt fails, the authoritative transition is exact NULL: resident bytes and authoritative version do not change, while a rejection receipt is appended outside the resident state.

UNVALIDATED This paper uses that contract as a formal dependency only. The same-checkpoint semantic interface remains failed/unqualified; no dual-state row was generated, admitted, or read by a model.

75.3 Feedback-to-memory leakage

Suppose an evaluator exposes the correct final answer after attempt r. Using that answer as policy feedback may be allowed in a registered supervised test-time setting. Writing it into M and scoring attempt $r + 1$ on the same answer tests answer caching, not general reasoning or predictive memory. The memory protocol must tag target-revealing feedback and exclude or separately analyze direct answer reuse.

Counterexample 75.3 (Joint endpoint leakage)**.** Both Φ and M receive the ground-truth answer. The joint arm succeeds perfectly on a repeated query. The factorial interaction may appear strongly complementary even though both carriers are merely copying target information. Randomization does not repair an invalid endpoint or post-treatment leakage.

75.4 Tenant, lifetime, and deletion boundaries

Policy state defaults to the current problem and must be destroyed or archived according to its own scope. Memory rows follow tenant, ACL, validity, protection, retention, tombstone, and verified-deletion rules. A policy snapshot cannot extend a memory row's retention; a memory archive cannot preserve an expired problem-scoped optimizer state by reference. Composite manifests hold identifiers, not ownership of the underlying retention policy.

Boundary	Required behavior	Stop condition
Carrier privilege	distinct credentials and typed write endpoints	any cross-carrier direct write
Interface version	registered read-only readers/decoder and exact schema hashes	unregistered reader, decoder, or implicit coercion
Joint receipt	coordinator verifies witness and signs hashes only	treating a pin or pair receipt as carrier-write authority
Future information	deployment reads causal data only	evaluator truth or future branch enters a deployed update
Authority	candidate remains non-authoritative until full validation	partial row, missing protected field, or hash-only equality
Tenant/ACL	check at construction, pin, read, commit, archive, and restore	mismatched tenant or unverified policy
Lifetime	independently declared expiry and cleanup	one carrier silently inherits the other's lifetime
Deletion	carrier-local verified deletion/tombstone receipt	dangling index, cache, archive, or snapshot

Table 51: Information-flow and authority gates.

76 Complete Resource Accounting

76.1 Vector costs before scalarization

For execution u, define

$$\mathbf{C}(u) = (N_{\text{tok}}, F_{\text{fwd}}, F_{\text{bwd}}, B_{\text{param}}, B_{\text{opt}}, B_{\text{row}}, B_{\text{archive}}, B_{\text{traffic}}, T_{\text{wall}}, E, N_{\text{retry}}, N_{\text{receipt}}, N_{\text{witness}}, N_{\text{sig}}, N_{\text{fallback}}). \tag{269}$$

The entries are tokens, forward and backward work, parameter bytes, optimizer bytes, resident-row bytes, archive/index bytes, memory/network traffic, wall time, energy, retries, receipt/validation operations, compatibility-witness construction or verification, signature/schema checks, and executed fallbacks. Pair-receipt hashing and signing are charged rather than treated as a free atomic pair. A scalar cost $c = \lambda^{\top}\mathbf{C}$ is reported only with a preregistered nonnegative weight vector λ and units.

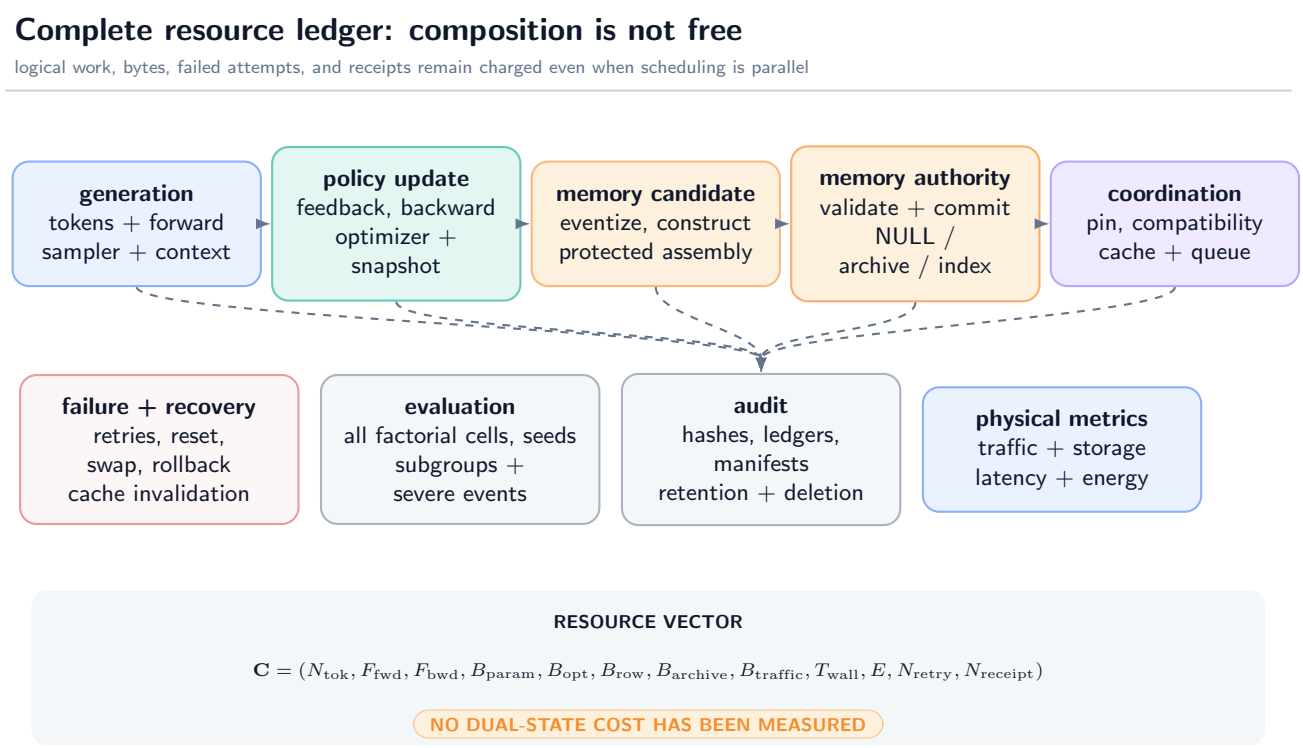

Figure 53: **Training, reasoning-time, memory, recovery, and audit ledgers (DEFINITION / DESIGN PRESCRIPTION).** Parallelism may reduce wall time but cannot delete logical work, bytes, validation, or failed attempts from the ledger.

76.2 Ledger decomposition

Let $\mathcal{E}_{\text{gen}}$, $\mathcal{E}_{\Phi}$, $\mathcal{E}_{M}$, $\mathcal{E}_{\text{coord}}$, and $\mathcal{E}_{\text{recover}}$ be disjoint event sets. Exact accounting gives

$$\mathbf{C}_{\text{total}} = \sum_{e \in \mathcal{E}_{\text{gen}}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\Phi}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{M}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\text{coord}}} \mathbf{c}_e + \sum_{e \in \mathcal{E}_{\text{recover}}} \mathbf{c}_e. \tag{270}$$

PROVED UNDER STATED ASSUMPTIONS

Proposition 76.1 (No-free-composition accounting)**.** *If every physical or logical operation is assigned to exactly one event set, Equation (270) is an identity. In particular, dual-state cost is not represented by generation tokens alone: it includes feedback/evaluation, backward and optimizer work, checkpointing, complete candidate construction, protected-field assembly, validation, commit/NULL, archive/index traffic, witness construction and verification, schema/signature checks, pair-receipt hashing/signing, resets, swaps, rollbacks, cache invalidation, retries, fallback execution, and audit receipts.*

Corollary 76.2 (Parallelism does not erase work)**.** *Batching or concurrent execution may change T_{wall} and hardware utilization but does not set the corresponding logical operation counts or transferred bytes to zero.*

76.3 Training and reasoning-time ledgers

Phase	Mandatory charges	Common hidden omission
Base/pretraining	data, tokens, forward/backward, optimizer, checkpoints	amortizing it for one arm but not another
Meta/policy preparation	task sampling, reward/evaluator, adapters, optimizer state, validation	counting only trainable parameter bytes
Reasoning-time policy	exploration attempts, feedback, reward, backward pass, optimizer, snapshots, rollback tests	reporting only final-answer tokens
Memory candidate	eventization, target-conditioned construction, canonical/neural views, protected-field assembly	treating the row as already available
Memory authority	validation, exact NULL decisions, commit, archive, index, provenance, deletion	counting resident bytes only
Coordination	witness construction/verification, schema/signature checks, pair-receipt hashing/signing, version pin, cache invalidation, queueing, fallback execution	assuming a free atomic pair
Evaluation	all four factorial cells, negative controls, seeds, subgroups, failures, uncertainty	reporting the best cell alone

Table 52: Complete logical ledger. No cost is measured in this paper.

76.4 Comparator normalization

A fair comparator holds fixed or explicitly prices base model, precision, maximum visible context, total generated tokens, number of attempts, policy parameter count, memory capacity, evaluator access, update budget, wall-clock cap, and hardware. If architectures cannot be matched on all dimensions, the result is a Pareto comparison over quality, severe failures, latency, energy, memory, and traffic, not a single "efficiency" number.

Assumption 76.3 (Cost observability)**.** Every event reports units and measurement method; failed and aborted events are retained; cached reuse is proven by content identity and dependency validity; and energy or FLOP claims are omitted when the required measurement is absent.

PLANNED TEST No dual-state cost vector has been measured. Exact ledger equations are protocol accounting, not evidence of efficiency or a hardware crossover.

77 Failure Modes and Recovery

77.1 Carrier-local failures

Carrier	Failure	Observable symptom	Required response
Φ	divergent or nonfinite update	parameter/optimizer validation fails	reject successor; retain predecessor; record failed update
Φ	optimizer mismatch	weights restore but moments/step do not	quarantine; restore complete policy snapshot or stop
Φ	reward leakage/hacking	success depends on prohibited evaluator information	invalidate affected stage and descendants
Φ	excessive drift	registered norm, KL, or severe-output gate fails	freeze and independently roll back policy only
M	partial or malformed row	protected field, width, canonical/neural consistency, or hash check fails	exact NULL; no authoritative version change
M	stale/unauthorized authority	tenant, ACL, validity, or version check fails	block read/commit; rollback or tombstone under memory policy
M	archive/index divergence	resident, index, and archive receipts disagree	quarantine memory view; repair and revalidate
M	deletion failure	row survives in cache/index/archive	stop; complete verified deletion before reuse

Table 53: Carrier-local failures preserve separate recovery domains.

77.2 Composition failures

Composition failure	Why it breaks inference	Recovery/analysis consequence
Unpinned mixed read	tokens within one attempt can see different carrier versions	invalidate attempt; no post hoc selection of a convenient view
Compatibility mismatch	policy features, tokenizer, schema, or base model disagree	block pin or route to registered fallback; do not coerce bytes
Hidden joint reset	one intervention changes both carriers	factorial attribution fails; rerun from independently verified baselines
Undeclared event order	policy and memory decisions may not commute	effect is undefined for the registered treatment
Shared credential	a compromise can cross both write domains	least-privilege proposition does not apply
Receipt loss	predecessor, exposure, or compliance is ambiguous	mark missing; do not infer no-op or successful rollback
Feedback-to-answer leakage	later success can be direct target copying	invalidate reasoning endpoint or report separate cache task
Adaptive resource imbalance	one cell receives more attempts or compute	endpoint mixes mechanism and budget

Table 54: Composition failures and their causal consequences.

77.3 Recovery invariants

DESIGN PRESCRIPTION A recovery sequence must satisfy:

1. stop new pins that could read a suspect carrier;
2. preserve both ledgers and all failed events;
3. identify the smallest carrier-local recovery target;
4. restore that target with a new successor receipt;
5. invalidate carrier-dependent caches without changing the other carrier;
6. rerun compatibility checks and a registered canary;
7. resume only if all predeclared gates pass.

Proposition 77.1 (Recovery locality)**.** *Under carrier-local restoration and a dependency-complete cache graph, a policy-only failure can be recovered without changing authoritative memory content, and a memory-only failure can be recovered without changing policy content. Cache eviction and pin refusal may affect availability but not carrier identity.*

Counterexample 77.2 (Rollback illusion from an incomplete snapshot)**.** A policy rollback restores adapter weights but not optimizer moments. The next update diverges. Terminal weights initially match the prior checkpoint, yet the restored state is not the prior policy carrier. A weight-file hash alone is therefore not a rollback receipt.

Counterexample 77.3 (Memory fieldwise undo)**.** A system restores a row's semantic payload but leaves the newer version, ACL, and provenance. The row is neither the old complete version nor a valid new one. This violates complete-row rollback even if a query happens to return the older fact.

77.4 Severe-event stop rules

Average task loss cannot override a severe integrity event. The protocol stops the affected stage on any cross-tenant read/write, unverified deletion, direct privilege crossing, future-information leak, silent cross-carrier restoration, lost mandatory receipt, or corruption of the frozen evaluation boundary. A thresholded task or cost gate may also stop for futility. These rules are design prescriptions, not evidence that a future implementation is safe.

78 Staged Falsification Protocol and Obligations

78.1 The prerequisite gate is first

The present research chain stops before a dual-state experiment. The same-checkpoint semantic interface is failed/unqualified, and predictive admission is closed. A valid future program must reopen no downstream stage until its prerequisite has independently qualified.

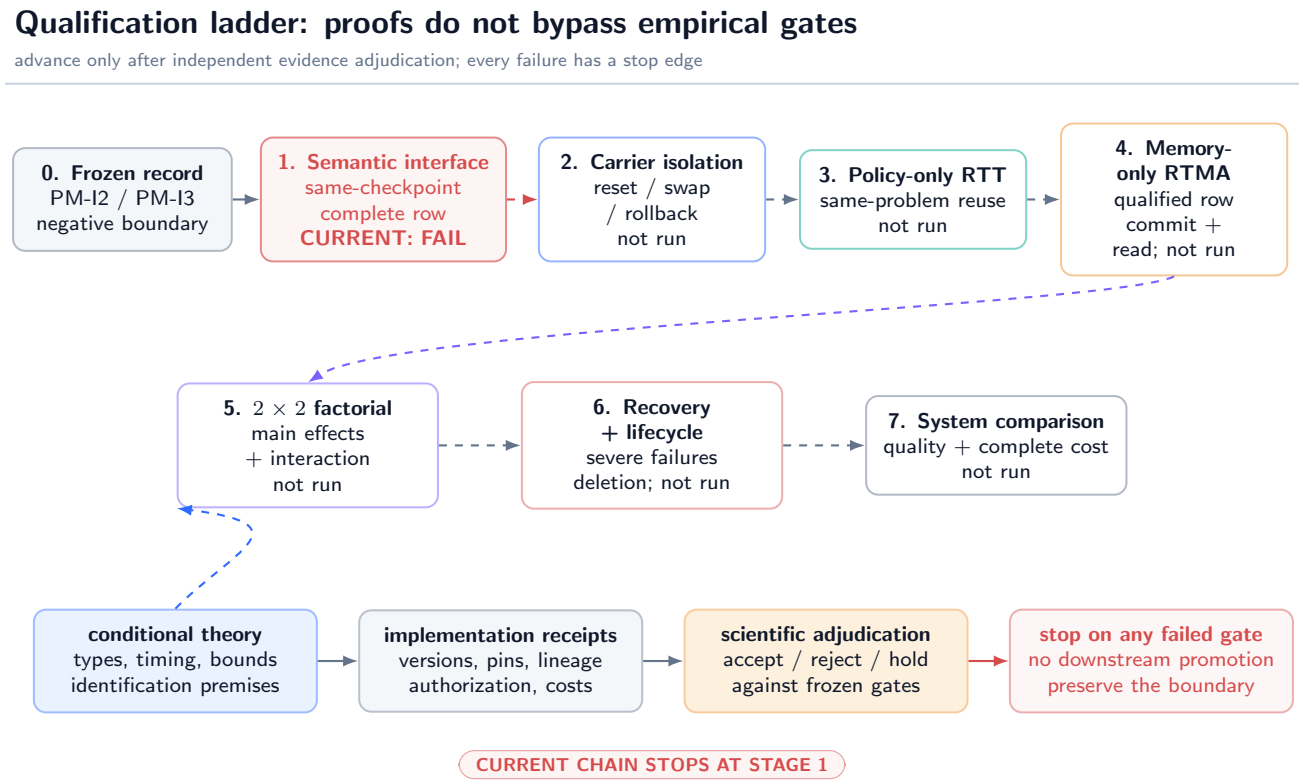

Figure 54: **Qualification ladder, stop edges, and proof dependencies** (PLANNED TEST)**.** The coral semantic-interface stop is current. Theory boxes state conditional obligations; they do not bypass an empirical gate.

Protocol 78.1 (Dual-state qualification ladder)**.** Every stage uses immutable registration, fresh groups, required seeds, independent receipts, and explicit scientific adjudication.

1. **Semantic carrier qualification.** Establish a same-checkpoint complete typed row interface under its registered fit, held-out composition, query, preservation, mapping, and integrity gates. Current status: FAIL / unqualified.
2. **Carrier isolation.** Demonstrate independent snapshot, fresh/reset, swap, freeze, rollback, cache invalidation, and ledger behavior for Φ and M. Every restoration invalidates the old joint pin and compatibility witness; a fresh witness must bind both successor lineages before reuse. Stop on any silent cross-carrier mutation or ABA-style pin reuse.

3. **Policy-only strict RTT.** With M frozen, show feedback update, same-unresolved-problem later reuse, and a randomized policy intervention under fixed attempt/resource budgets. This is not a memory result.
4. **Memory-only RTMA.** With Φ frozen, show complete-row commit/NULL, later reuse, and negative controls. This is not a policy-learning result and requires a qualified memory interface.
5. 2×2 **factorial.** Randomize both carrier interventions, report all cell means, main effects, interaction, compliance, order, subgroup, severe-event, and cost results.
6. **Recovery and lifecycle.** Exercise carrier-local rollbacks, expiration, deletion, repeated updates/commits, and induced failure cases. Stop on unresolved contamination or receipt gaps.
7. **System comparison.** Only after all prior gates, compare against registered context, recurrent/compress-all, policy-only, memory-only, and static baselines on a Pareto ledger.

78.2 Primary endpoints and analysis

The primary scientific endpoint must be a task loss that does not reward direct answer caching. Co-primary integrity endpoints include semantic row exactness, preservation, policy drift, severe failures, and compliance. Resource endpoints retain the vector in Equation (269). The unit of randomization is the independent problem group; all attempts and branches from a group remain in one split. Final analysis reports uncertainty and multiplicity for the registered primary contrasts. Appendix 81 gives an executable design specification without inventing sample sizes or effect magnitudes.

78.3 Theorem-to-receipt obligations

Result	Minimum implementation receipt	Failure consequence
Type non-conflation	carrier tags, distinct credentials/endpoints, authorization log	type-safety proposition unusable
Typed coupling interface	registered R_Φ, R_M, decoder, schemas, interface version, and negative cast tests	no well-typed joint read is established
Compatibility witness	signed valid witness binding both hashes, versions, lineages, lifetimes, ACLs, schemas, reader/decoder versions, and fallback	joint view must refuse or fall back
Causal timeline	event order, information-set schema, exact pinned predecessor view	temporal claim invalid
Conditional commutation	invariant inputs/decisions under both orders, compatibility receipts	order must be modeled as treatment
Coupled-transition closure	both carrier successor receipts, verified witness, and signed pair receipt	no atomic pair or coupled-update claim
Strict RTT classification	unresolved marker, feedback, policy successor, later pinned descendant, intervention	call it retry/adaptation, not strict RTT
Independent rollback	complete carrier snapshot, fresh successor lineage, invalidated old joint pin/witness, cache dependency graph, unchanged other-carrier hash	recovery and ABA-resistance claims invalid
Contamination bound	step sizes, gradient bounds, Lipschitz region, fixed contaminant definition	bound cannot be cited
Factorial identification	randomized assignments, positive support, pins, compliance, group isolation, attrition log	contrasts descriptive only
Output non-identification	nonempty legal realization class under registered interface, compatibility, authorization, lifetime, and causal constraints	no carrier-indistinguishability conclusion
Cost identity	event-complete units including witness verification, signatures, pair receipts, fallback, failed work, cache proof, and measurement method	efficiency claim prohibited

Table 55: Proofs become system statements only with their premises and receipts.

78.4 Decision grammar

Each stage ends in one of four durable scientific statuses: PASS under every registered gate; FAIL under at least one decisive gate; HOLD for incomplete but recoverable evidence; or INVALID for protocol corruption. Editorial completion and a clean PDF build have no bearing on those scientific statuses.

PLANNED TEST No stage of Protocol 78.1 was executed for this paper. In particular, no dual-state checkpoint, policy update, memory commit, factorial unit, oracle gap, or learned admission decision exists in the evidence record.

79 Related Work, Limitations, and Conclusion

79.1 Test-time updates and numerical state

Test-time training updates model parameters on test inputs under a self-supervised objective [96]. TTT layers treat an internal model as recurrent hidden state updated by learning at test time [94]. LoRA provides a parameter-efficient adapter

representation for numerical adaptation [44]; it does not itself define when an update is reasoning-time, which feedback is valid, or how rollback should interact with authoritative memory. Recent LLM work performs test-time reinforcement learning from model-derived feedback [136]; Policy of Thoughts and StarOR describe transient within-instance policy adaptation for reasoning or optimization [46, 58]. These papers motivate explicit policy-state timing. Their empirical findings are not transferred to MMLA-RTT.

79.2 Recurrent and neural memory

Compressive Transformers compress older activations for long-range sequence modeling [79]; Mamba uses selective state-space recurrence [35]; TTT layers make the recurrent state a learned model [94]; and Titans proposes neural long-term memory updated at test time [6]. These are numerical or architectural memory mechanisms under their own contracts. Our use of "authoritative memory" is narrower: a bounded map of complete typed, versioned, provenance-bearing rows with trusted assembly and exact commit/NULL. The companion completed-segment consolidation draft makes the segment boundary and bidirectional evidence bundle explicit [121]; it remains Reviewer-pending and supplies no empirical result here. The distinction is semantic and operational, not a claim of superior accuracy or efficiency.

79.3 Causal factorial attribution

The potential-outcome framework separates assignment from outcomes and makes interaction a contrast over unobserved potential responses [87]. Our contribution is to specialize that discipline to independently pinned policy and authoritative-memory carriers, with explicit noncompliance, swap, reset, rollback, sampler, cache, and ledger controls. Randomization can identify registered contrasts; it cannot validate a leaked endpoint, an unqualified semantic interface, or a missing receipt.

79.4 Limitations

This paper has deliberately strong limitations.

1. It is a theory/protocol paper and contains no new measurement.
2. The authoritative semantic interface required by the composition is currently failed/unqualified under the frozen evidence record.
3. The formal results rely on measurability, typed registered readers/decoders, authorization, witness verification, lifetime validity, Lipschitz, capacity, randomization, compliance, and ledger premises that no implementation has established.
4. The output-trace theorem is conditional on a nonempty class of legally realizable, interface-compatible alternatives. Finite capacity alone does not establish that class; the theorem is an identification result, not a realistic cost equivalence.
5. The contamination bound is worst-case and unsigned. It neither estimates real contamination nor proves that gating helps.
6. The 2×2 design does not solve endpoint validity, interference outside the chosen group, noncompliance, low power, or multiplicity by itself.
7. Independent operation algebras reduce semantic coupling but can increase coordination, storage, validation, and recovery cost.
8. We do not claim priority over all dual-state, adaptive-memory, recurrent, or test-time-learning systems.

79.5 Conclusion

The central design rule is simple but demanding: policy state Φ and authoritative memory M may cooperate, yet they must never become indistinguishable. They require separate types, writers, readers, scopes, versions, lineages, resets, swaps, rollbacks, ledgers, costs, and causal interventions. A pinned read view coordinates two independently owned states through a registered typed interface and a verified compatibility witness; it does not merge them or authorize either write. After restoration, the old joint pin and witness are invalid even when payload bytes recur. Strict RTT requires feedback-driven policy update and later same-unresolved-problem reuse. Memory mutation alone is RTMA. Output improvement alone certifies neither.

The resulting theory offers conditional guarantees and explicit failure tests, not a system result. Current evidence still stops at the failed/unqualified semantic interface. Any future dual-state claim must first clear that gate, then survive carrier-isolation controls, policy-only and memory-only qualification, the full factorial, recovery and lifecycle tests, and complete cost accounting. Until then, MMLA-RTT remains a falsifiable architecture proposal rather than an empirical success.

80 Proofs and Constructions

80.1 Type and timing results

Proof of Proposition 66.8. Each mutation event contains a carrier tag. By Assumption 66.7, authorization for a Φ-tagged event is defined only on the policy carrier type and its successor map has codomain $\mathcal{P} \times \mathcal{O}$; authorization for an M-tagged event is defined only on the authoritative-memory type and has codomain $\mathcal{M}$. Induct on trace length. The empty trace is well typed. If the first n events leave both carrier projections well typed, event $n+1$ either changes only the carrier named by its tag or is rejected. Thus it cannot apply its payload to the other carrier. Serialized-byte equality does not change the tag, domain, or codomain. The property holds for every finite trace. □

Proof of Proposition 67.2. Proceed by induction on event order. The initial registered state and view are $\mathcal{F}_0$-measurable. Suppose S_e and all prior receipts are $\mathcal{F}_e$-measurable. Assumption 67.1 makes κ_e, its authorized input ξ_e, and any registered randomness measurable with respect to $\mathcal{F}_e$ or a declared independent random seed revealed at e. Because T_{κ_e} is a measurable typed map, S_{e+1} and its receipt are measurable with respect to $\mathcal{F}_{e+1}$. Induction gives the result. A version created after e is not in $\mathcal{F}_e$, so it cannot be a measurable parent of the output at e under the declared structural trace. □

80.2 Operation algebra

Proof of Theorem 68.3. Write the carrier projection of S as (P, A). By input stability, U receives the same policy input and authorization decision whether evaluated before or after K; hence its policy successor is one fixed $P^+ = U(P)$. Symmetrically, the memory candidate, validation, and authorization are invariant, so its successor is one fixed $A^+ = K(A)$. Typed authorization makes $\widetilde{U}$ leave A unchanged and $\widetilde{K}$ leave P unchanged. Consequently both orders have carrier projection (P^+, A^+). Receipt events preserve their temporal order; normalization may yield an equivalent set of receipts without identical serialized traces, so no full-trace equality is asserted. □

Proof of Proposition 68.8. Fix visible version numbers (a, b). Construct policy histories H_1^Φ and H_2^Φ with distinct lineage identifiers: in the first, update directly from baseline to version a; in the second, fork at an earlier snapshot, perform a reset, and publish content-compatible version a. Pair each with the same memory history ending at (b, ℓ^M). The pinned version pair is identical but the policy predecessor and rollback target differ. The symmetric construction holds for two memory histories paired with one policy history. A passing compatibility predicate depends only on its declared fields and therefore need not reveal either hidden predecessor. Thus the pair and receipt do not determine lineages or rollback targets. □

80.3 Mechanism classification

Proof of Theorem 69.5. Define binary indicators I_Φ and I_M for the strict policy path and later read of updated authoritative memory, respectively. Under observability, both are functions of the audit trace and are not imputed from output quality. The Cartesian set $\{0,1\}^2$ partitions traces into four disjoint cells. Definitions 5.1–5.3 name these cells static/retry, RTMA only, strict RTT only, and dual-state composition. Exhaustiveness follows because each indicator is binary; exclusivity follows because two distinct cells disagree in at least one indicator. The definitions do not include an outcome sign, so no benefit follows. □

Proof of Corollary 69.6. If Φ is frozen, no authorized feedback-driven policy successor exists and $I_\Phi = 0$. If a later attempt reads updated M, then $I_M = 1$. Theorem 69.5 assigns the interval to RTMA only. If there is no later memory read either, it is static/retry. Neither case is strict RTT. □

80.4 Restoration and lineage

Proof of Theorem 70.2. Assumption 70.1 defines Rollback_Φ as a map on the policy carrier and its ledger, with the identity map on the memory carrier. Typed authorization prevents its event from naming M as a destination. Hence $\pi_A(\text{Rollback}_\Phi(S)) = \pi_A(S)$, including content, version, and lineage. The memory case is symmetric. Compatibility evaluation is a later coordinator map whose outcomes are pin or refusal; it has no carrier-write privilege. Therefore a compatibility failure cannot alter the unchanged carrier. □

Proof of Corollary 70.3. By Theorem 70.2, one carrier-local rollback changes at most one carrier. Changing both therefore requires the composition of two authorized carrier-local maps. An explicit composite manifest is conforming only if its execution expands to those maps and retains both receipts. An unnamed single event cannot satisfy the typed domain requirement. □

Proof of Proposition 70.4. A restore produces a new lineage edge labeled reset, swap, or rollback whose successor has a new version and receipt. Append-only storage prevents deletion of that edge or the intervening vertices. Content addressing may show that the restored payload equals an earlier payload, but the new vertex and edge remain. An auditor reading the ledger therefore distinguishes restoration from uninterrupted continuation. □

80.5 Compression and drift

Proof of Theorem 71.2. Let $d_t = \|\Phi_t - \Phi_t^I\|$. Nonexpansiveness of projection and the triangle inequality give, for informative t,

$$d_{t+1} \le d_t + \eta_t \|g(\Phi_t, X_t) - g(\Phi_t^I, X_t)\| \le (1 + \eta_t L_g) d_t.$$

For contaminant t, the coupled informative path takes a zero update, so

$$d_{t+1} \le d_t + \eta_t \|g(\Phi_t, X_t)\| \le (1 + \eta_t L_g) d_t + \eta_t B_t,$$

where the last step adds and subtracts $g(\Phi_t^I, X_t)$. With $d_0 = 0$, unrolling this recurrence yields Equation (258). Lipschitz continuity of ℓ gives Equation (259). Absolute value removes the sign, so harm does not follow. □

Proof of Proposition 71.5. Counterexample 71.3 supplies a law and nonnegative cost under which compress-all sufficient-statistic recurrence is Bayes-optimal and an added carrier cannot strictly improve total risk. Conversely, choose a finite-capacity compress-all state whose update overwrites the only bit needed for a later task when a nuisance event arrives; a typed protected row retains that bit and achieves lower task loss at sufficiently small row cost. Since each architecture is strictly better in one admissible construction, no uniform dominance holds without additional restrictions. □

Proof of Proposition 72.1. Because $\Phi_k \in \mathcal{P}$ and projection fixes points in $\mathcal{P}$,

$$\|\Phi_{k+1} - \Phi_k\| = \|\mathrm{proj}_{\mathcal{P}}(\Phi_k - \eta_k \widehat{g}_k) - \mathrm{proj}_{\mathcal{P}}(\Phi_k)\| \le \eta_k \|\widehat{g}_k\| \le \eta_k G_k.$$

Sum over k and apply the triangle inequality. The stated total-variation bound follows directly from its Lipschitz premise. □

Proof of Theorem 72.3. Substitute the intervention bounds $\delta_\Phi \le \epsilon_\Phi$ and $\delta_M \le \epsilon_M$ into Equation (262); all coefficients and distances are nonnegative. For integrable bounded losses, take expectations and use $|\mathbb{E}X| \le \mathbb{E}|X| \le B_{\Phi M}$. The inequality is absolute and supplies no sign or additive causal attribution. □

80.6 Factorial and observational results

Proof of Theorem 73.3. Random assignment and positive support imply $Z_i \perp \{Y_i(p,m) : p, m \in \{0,1\}\}$. Perfect compliance and consistency give $Y_i^{\mathrm{obs}} = Y_i(p,m)$ in cell (p,m). Hence

$$\mathbb{E}[Y_i^{\mathrm{obs}} \mid Z_i^\Phi = p, Z_i^M = m] = \mathbb{E}[Y_i(p,m)] = \mu_{pm}.$$

Linear combinations of the four identified cell means identify the displayed main and interaction effects. For known probabilities $\pi_{pm} > 0$, the Horvitz–Thompson term $\mathbf{1}\{Z = (p,m)\}Y^{\mathrm{obs}}/\pi_{pm}$ has expectation μ_{pm}, proving the final statement. □

Proof of Proposition 73.4. Let D be successful exposure. Even when Z is randomized, D can depend on assignment, latent problem difficulty, resource pressure, or potential outcomes. Conditioning on $D = 1$ opens selection paths and generally destroys $Z \perp Y(p,m)$. Thus receipt-positive differences are not automatically randomized effects. Guaranteed compliance or an explicit identification model is required. □

Proof of Theorem 74.1. Assume first that $R \in \mathfrak{R}_{M \leftarrow \Phi}(K; \mathcal{I})$. By the definition of this set, R supplies a legal encoder, an authorized complete-row transition, a verified interface witness, a registered memory reader and decoder, and the same exogenous-randomness coupling. Conditional on each (q, z_0, f_0), the decoder in R therefore emits exactly $K(\cdot \mid q, z_0, f_0)$; integrating over the common law of (q, Z_0, F_0) gives the same law for O. No unregistered cast or unauthorized lookup is introduced. The converse applies the identical argument to any $R' \in \mathfrak{R}_{\Phi \leftarrow M}(K; \mathcal{I})$. If the relevant set is empty, the construction has no admissible first step, so the theorem asserts no equivalence. Thus observables fail to name a carrier only among realizations satisfying the stated type, authorization, compatibility, capacity, causality, and randomness obligations. □

Proof of Proposition 68.6. Typed authorization makes $U(P)$ a policy-carrier successor with receipt ρ_Φ^+ and $K(A)$ a memory-carrier successor with receipt ρ_M^+; neither map has the other's codomain. Interface soundness permits a pin only after a witness $\kappa \in \Gamma_{\mathcal{I}}(P^+, A^+, C)$ is verified by the components actually used at read time. Equation (256) binds both carrier receipts, the witness, interface, and context, so a published view names the exact well-typed pair. If verification fails, the coordinator's codomain contains refusal/fallback and receipt append, not a carrier mutation. Hence it cannot publish the pair or silently alter either successor. □

Proof of Corollary 74.3. For one unit, consistency reveals only $Y_i(Z_i^\Phi, Z_i^M)$. The interaction contains all four potential outcomes. Without structural restrictions that determine the three unobserved values, multiple potential-outcome schedules agree on the observed trajectory and yield different interactions. Hence one trajectory does not identify it. □

Proof of Proposition 74.7. Under randomized valid swap assignment, donor content is independent of recipient potential outcomes and all declared non-content fields are fixed or balanced. Consistency therefore identifies the mean outcome under recipient-updated content and under a draw from the declared compatible-donor distribution. Their difference is the recipient-specific-content contrast relative to that distribution. Nothing in the assumptions ranges over every possible donor or semantic representation, so no universal effect follows. □

80.7 Authorization, cost, and recovery

Proof of Proposition 75.2. Distinct credentials and typed endpoints make the policy service unauthorized at the authoritative commit endpoint and make the candidate/committer service unauthorized at the policy endpoint. Schema checks reject a payload carrying the wrong carrier type. Therefore a call confined to one ordinary service cannot directly publish the other's successor. The conclusion does not survive compromise of the shared authorization root or both credentials, which the proposition excludes. □

Proof of Proposition 76.1. The five event sets form a disjoint partition of all charged operations. Finite additivity of vector sums gives Equation (270). Each listed activity is, by construction, assigned to one set, including failures and recovery. Reporting only generation projects away non-generation components and therefore cannot equal the complete vector unless every omitted component is proved zero. □

Proof of Corollary 76.2. Parallel scheduling changes the temporal arrangement of event costs and can reduce the wall-time coordinate. It does not remove executed events from the partition or make their logical counts and transferred bytes vanish. The corresponding vector coordinates remain summed. □

Proof of Proposition 77.1. By Theorem 70.2, carrier-local restoration changes only the named carrier. A dependency-complete cache graph identifies every derived cache entry and permits invalidation without altering source carrier content. Compatibility checks can withhold availability. Thus recovery of one carrier need not change the other's content, though temporary service interruption may occur. □

Proof of Proposition 70.5. A reset, swap, or rollback creates a fresh successor version and lineage receipt even when its payload is byte-identical to an earlier payload. The old joint pin and witness bind the predecessor version, lineage, receipt, and lifetime fields, so interface verification rejects them for the successor pair. Content equality therefore cannot recreate the old identity. A usable post-restoration view requires a newly verified $\kappa \in \Gamma_I$ and a fresh pair receipt binding both current successors. This prevents an ABA-style return of payload bytes from masquerading as uninterrupted joint-state continuity. □

80.8 Proof boundary

Every result above is conditional on its displayed premises. None establishes a qualified semantic interface, actual privilege separation, valid feedback, bounded gradients, Lipschitz behavior, randomized compliance, a measured endpoint, or a complete ledger. Those are empirical or implementation obligations in Appendix 82.

81 Registered Factorial Protocol Specification

81.1 Purpose and gate order

This appendix is an unexecuted protocol template. It cannot begin until the semantic-interface gate and carrier-isolation stage in Protocol 78.1 pass. It estimates policy and memory intervention effects without treating assignments, successful pins, or final outputs as interchangeable.

81.2 Immutable registration

Before any evaluation outcome is opened, register:

1. problem-group construction, inclusion/exclusion rules, immutable group identifiers, and train/development/final split hashes;
2. base model, tokenizer, precision, software/hardware identity, causal workspace schema, maximum context and attempt budget;
3. exact Φ_0, Φ_1, M_0, M_1 construction, snapshot hashes, lineages, lifetimes, and compatibility rules;
4. update objectives, feedback sources, optimizer, step schedule, clipping/projection, memory candidate/validator/commit contract, and event order;
5. primary task loss, semantic and preservation gates, severe-event endpoints, subgroups, cost vector, materiality/equivalence margins, and missing-data rules;

6. randomization algorithm, assignment probabilities, blocking variables, seed list, multiplicity family, interval method, and stop rules;
7. exact analysis code or a content-addressed specification sufficient for independent reconstruction.

No sample-size number or effect magnitude is supplied here because none is supported by the frozen record. A future design must derive group count from a preregistered smallest effect of interest, variance pilot that is isolated from final evaluation, desired power, familywise error, expected noncompliance, and severe-event precision.

81.3 Unit and treatment construction

The randomization unit is a problem group containing every attempt, branch, paraphrase, and derived query that shares latent content or an upstream source. Units are blocked on preregistered difficulty and domain variables, then assigned to four cells with positive probability. Assignment occurs before the evaluated attempt and is recorded independently from execution services.

Cell	Policy treatment	Memory treatment	Pinned-view requirement
00	registered fresh/frozen baseline Φ_0	registered fresh/frozen baseline M_0	exact (v_0^Φ, v_0^M)
10	feedback-updated Φ_1	baseline M_0	exact (v_1^Φ, v_0^M)
01	baseline Φ_0	qualified committed M_1	exact (v_0^Φ, v_1^M)
11	feedback-updated Φ_1	qualified committed M_1	exact (v_1^Φ, v_1^M)

Table 56: Primary availability factorial. M_1 does not exist as qualified evidence in this paper.

Exact NULL is an outcome of the registered memory transition, not missing data. If a cell assigned to memory update receives NULL, assignment remains $Z^M = 1$ while realized exposure is recorded separately. Failed pins, timeouts, validator rejects, and recovery events remain in intent-to-treat analysis under registered loss/missingness rules.

81.4 Execution sequence

For each unit:

1. verify immutable registration and cold/fixed evaluation boundary;
2. materialize carrier baselines from registered snapshots;
3. run the common pre-intervention attempt and record admissible feedback;
4. execute independently authorized policy and memory events in the assigned order;
5. verify both successor receipts, caches, compatibility, and exact pinned view;
6. run the evaluated attempt under the fixed sampler and resource contract;
7. compute endpoints with an evaluator blind to cell labels where feasible;
8. append all physical and logical costs, including failures and recovery;
9. freeze final group bytes before opening aggregate cell summaries.

If the policy and memory events may not commute, randomize their order within the joint arm or treat order as a registered additional factor. Do not average undeclared interleavings.

81.5 Primary estimators

For equal-probability complete randomization, let $\widehat{\mu}_{pm}$ be the group mean in cell (p, m). Estimate

$$\widehat{\tau}_{\Phi|m} = \widehat{\mu}_{1m} - \widehat{\mu}_{0m}, \tag{271}$$

$$\widehat{\tau}_{M|p} = \widehat{\mu}_{p1} - \widehat{\mu}_{p0}, \tag{272}$$

$$\widehat{\tau}_{\Phi M} = \widehat{\mu}_{11} - \widehat{\mu}_{10} - \widehat{\mu}_{01} + \widehat{\mu}_{00}. \tag{273}$$

Use randomization-based or group-robust uncertainty consistent with the assignment design. With unequal probabilities, use registered inverse-probability weights. Covariate adjustment may improve precision only if specified before final outcomes and if unadjusted estimates remain reported. The primary estimand is intent-to-treat; exposure/compliance analyses are secondary unless compliance is guaranteed.

81.6 Negative controls

The protocol includes:

1. same carrier versions with independent sampler seeds;
2. frozen policy with identical extra attempt and feedback-computation budget;
3. swapped policy with matched update count, base model, tokenizer, optimizer schema, and budget;
4. swapped memory with matched schema, tenant-safe synthetic scope, row count, version depth, and bytes;
5. independent policy rollback with unchanged memory and independent memory rollback with unchanged policy;
6. invalid/mismatched compatibility canaries that must block pinning;
7. target-revealing feedback canaries that must fail the leakage gate;
8. exact NULL and no-op controls whose authoritative resident bytes remain identical.

81.7 Stop and decision rules

Stop immediately for cross-tenant access, future-information leakage, direct cross-carrier write, silent joint reset/rollback, unverified deletion, receipt corruption, or evaluation-boundary mutation. Stop the scientific stage if the semantic, preservation, strict-RTT timing, intervention compliance, severe-event, or resource gate fails. A favorable task mean cannot override these stops. A passing factorial requires all registered gates; a non-significant interaction is not automatically equivalence unless its confidence set lies within the registered equivalence margin.

81.8 Reporting

Report all four cell summaries; conditional and marginal main effects; interaction; assignment and exposure counts; NULL, failure, timeout, and rollback counts; subgroups; severe events; update/commit order; drift; every cost coordinate; exact software/hardware and carrier hashes; and all deviations. The report must state that the design does not establish a learned admission policy unless a separately qualified admission stage exists.

82 Theorem-to-Implementation Obligation Matrix

Table 57: Every conditional result remains unusable as a system claim until its complete premise bundle is evidenced.

Claim/result	Formal premise	Required durable evidence	If absent
Policy carrier identity	typed $\mathcal{P} \times \mathcal{O}$, version and lineage	complete parameter/optimizer manifest, scope, writer credential, predecessor/successor hashes	policy update/reset/rollback claim invalid
Memory carrier identity	complete typed rows and protected fields	serializer/assembler version, full resident snapshot, row/slot versions, ACL/provenance/retention receipts	authoritative-memory claim invalid
Typed coupling interface	registered readers and decoder have declared domains/codomains	reader/decoder versions, schema hashes, carrier tags, negative cast tests	no well-typed joint read is established
Compatibility witness	verified witness binds both carriers and the actual read components	signed witness, hashes, versions, lineages, tenant/ACL, lifetimes, schemas, interface and fallback policy	joint view must refuse or fall back
Pinned read view	exact versions and verified witness remain stable during attempt	pin receipt, read-service logs, cache version, witness and pair receipt	attempt exposure unknown
Type safety	carrier-specific domains and privileges	distinct endpoints/credentials, negative authorization tests, carrier-tag schema	syntactic proposition not implemented
Causal measurability	inputs available at event time	timestamp/order source, information-set schema, future-access guards, sampler seed	leakage not excluded
Conditional commutation	input and decision stability in both orders	replay of both orders from identical snapshot, candidate/gradient equality, compatibility receipts	order is a treatment
Strict RTT	feedback update and later same-problem read	unresolved marker, feedback receipt, policy update lineage, later pin, frozen/reset intervention	call only retry or unclassified adaptation

Continued on next page

Claim/result	Formal premise	Required durable evidence	If absent
RTMA	memory mutation and later read	complete commit/NULL receipt, memory lineage, later pin, frozen/reset intervention	memory mechanism unclassified
Coupled transition	two typed carrier transitions plus verified compatibility	both successor receipts, witness, signed pair receipt, refusal/fallback log	no atomic pair or coupled-update claim
Independent rollback	carrier-local restore and cache graph	complete snapshot, fresh restore successor, unchanged other-carrier hash, old joint-pin/witness invalidation, cache invalidation	recovery and ABA-resistance claims invalid
Append-only history	durable content-addressed lineage	storage/ledger integrity proof, restore edge, retained intervening vertices	rollback can be observationally erased
Contamination bound	nonexpansive projection, Lipschitz gradients/loss, bounded contaminant updates	registered region, measured/bounded constants, fixed contaminant definition, update trace	theorem cannot bound deployment
Policy path length	bounded gradients and steps	exact step sizes, gradient norms, projection implementation, checkpoints	displacement bound unavailable
Interaction envelope	registered state metrics and constants	intervention radii, mixed-sensitivity validation on region	envelope unavailable
Factorial identification	randomization, support, consistency, no interference, compliance	assignment receipt, group construction, pins, exposure, attrition, frozen analysis	cell contrasts descriptive
Swap specificity	valid donor sampling and compatibility	donor-selection seed, no-leak proof, matched metadata, independent snapshot	content contrast confounded
Output non-identification	nonempty legal realization set under the registered interface, compatibility, authorization, lifetime, capacity, causal, and randomness constraints	construction plus the complete witness class for at least one legal realization	no carrier-indistinguishability conclusion
Privilege isolation	distinct credentials and trusted mediation	authorization topology, penetration/negative tests, candidate quarantine	contamination-blocking claim unavailable
Cost identity	event-complete partition and units	per-event vector including witness verification, signatures, pair receipts, fallback, failed work, cache proof, measurement calibration	efficiency claim prohibited
Recovery locality	independent restore plus complete cache graph	dependency inventory, carrier hashes before/after, availability log	other carrier may have changed
Semantic qualification	registered same-checkpoint interface gates	independent fit and held-out, query, preservation, mapping, and integrity adjudication	every memory-composition stage remains closed
Learned admission	qualified interface, oracle gap, future-blind value and selector gates	separate predictive-admission protocol receipts and accepted evidence	no learned policy claim

The final two rows are scientific prerequisites rather than premises of the purely mathematical carrier algebra. They prevent formal consistency from being misreported as an implemented or learned MMLA system.

83 Symbols, Status, and Evidence Ledger

83.1 Core symbols

Symbol	Meaning
q, r, t, e	problem, attempt, within-attempt micro-event, and global event indices
$\mathcal{F}_e$	information available immediately before event e
$W_{q,r,t}$	bounded causal workspace; excludes future information
$P = (\Phi, O, \nu^\Phi, \ell^\Phi, \tau^\Phi, \rho^\Phi)$	policy carrier: parameters, optimizer, version, lineage, lifetime, receipt

Symbol	Meaning
$A = (M, v^M, \ell^M, \tau^M, \rho^M)$	authoritative memory carrier and its metadata
$V_{q,r}$	pinned read-view tuple naming both carrier versions/lineages and compatibility receipt
$\mathcal{I} = (R_\Phi, R_M, D)$	registered typed coupling interface: policy reader, authoritative-memory reader, and joint decoder
κ	compatibility witness binding both carriers, the interface, authorization, lifetime, and fallback fields
$\Gamma_{\mathcal{I}}$	set of witnesses verified under interface $\mathcal{I}$ for the exact joint read
$\rho_\times$	signed pair receipt binding both successor receipts, κ, $\mathcal{I}$, and causal context
$\mathfrak{R}_{M\leftarrow\Phi}, \mathfrak{R}_{\Phi\leftarrow M}$	legal cross-carrier realization sets used by the conditional output-trace theorem
C, G, Q	immutable causal context; generation/sampler state; queues, caches, snapshots, and ledgers
$\mathfrak{O}_\Phi, \mathfrak{O}_M$	independent carrier operation algebras
U, K	one policy update and one memory transition
$Q_q(r)$	indicator that problem q remains unresolved after attempt r
$Y_i(p,m), \mu_{pm}$	potential loss and population cell mean under carrier interventions (p,m)
$\tau_{\Phi\|m}, \tau_{M\|p}$	conditional policy and memory effects
$\tau_{\Phi M}$	factorial interaction $\mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}$
Z_i^Φ, Z_i^M	randomized assignments
D_i^Φ, D_i^M	receipt-verified realized exposures/compliance
$\mathbf{C}$	vector resource ledger
NULL	exact authoritative memory identity transition; rejection is externally receipted

83.2 Status ledger

Object	Status in this paper	Boundary
Typed dual-state semantics	DEFINITION	editorial/mathematical construction
Type, interface, compatibility, coupled-update, timing, rollback, drift, contamination, interaction, and identification results	PROVED UNDER STATED ASSUMPTIONS	conditional on displayed assumptions
Semantic row interface	UNVALIDATED	inherited failed/unqualified same-checkpoint gate
Strict-RTT policy implementation	UNVALIDATED	no policy update or same-problem intervention
Authoritative memory composition	UNVALIDATED	no qualified row interface or dual-state commit
2×2 experiment	PLANNED TEST	no unit assigned or evaluated
Oracle gap and predictive admission	UNVALIDATED	not measured; gate closed
Learned admission policy	UNVALIDATED	no value model, selector, or policy
Dual-state benefit/safety/efficiency	UNVALIDATED	no empirical result

Table 59: No status is promoted by manuscript completion.

83.3 Frozen numerical evidence

Only the inherited negative/diagnostic boundary is numerically admitted here:

- PM-I2: 0/9 registered interfaces qualified.
- PM-I3 direct candidate: 40/73,728 exact.
- Direct present path: 0.
- Learned route/learned content: 0/73,728.
- Oracle route/learned content: 0/73,728.
- Learned route/oracle content: 18,544/73,728, exactly routing count.
- Oracle route/oracle content: 73,728/73,728, mechanical ceiling only.
- Preservation: 0; missing cases: 43,008.

These counts delimit what is not qualified. They do not estimate any dual-state factorial cell, interaction, policy drift, contamination rate, resource cost, oracle gap, or admission effect.

83.4 Editorial boundary

Typeset completeness, a proof appendix, and visual quality establish only editorial integrity. They do not qualify a mechanism or promote a scientific claim.

Part V
Causal Generation, Retrospective Consolidation

Branch status. This part imports the complete substantive formal branch from the frozen R01 focus paper. It separates causal token generation from post-closure bidirectional inspection, ordered event construction, trusted per-event publication, and future-only exposure. It contributes no new experiment, qualified semantic interface, validated eventizer, CSBC utility, strict-RTT success, learned admission policy, safety result, or quality–cost crossover.

84 Introduction

84.1 Bidirectionality belongs after closure

An autoregressive generator must choose token X_t without reading tokens that do not yet exist. A retrospective encoder can nevertheless inspect a finite block in both directions once every token in that block is already emitted. The apparent contradiction disappears only when the segment boundary, encoding start, commit order, and later exposure are explicit.

DEFINITION

Definition 84.1 (Completed-segment bidirectional consolidation)**.** Completed-segment bidirectional consolidation (CSBC) is a protocol in which (i) token generation through a declared boundary is causal; (ii) the completed bounded view is constructed only after that boundary; (iii) a retrospective encoder may inspect only that completed view bidirectionally; (iv) each derived event produces at most one complete authoritative-row commit or exact NULL; and (v) every resulting version is first readable at an exposure point strictly after the boundary. Historical emissions and receipts are immutable.

The phrase *retrospective encoding* names step (iii). It does not imply reverse-time generation, future-token access, retrospective gradient updates during deployment, or correction of already emitted text. A later answer can use a committed correction; the earlier mistaken sentence remains part of history.

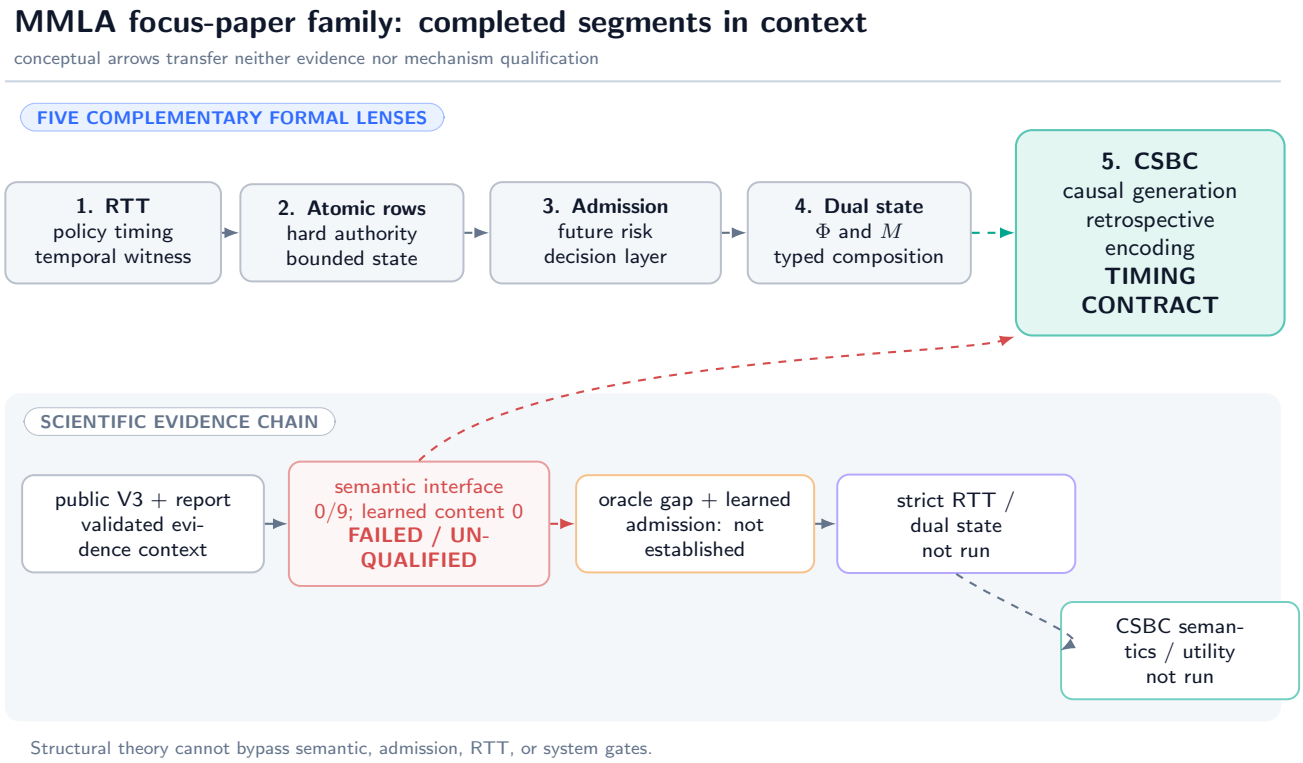

Figure 55: **Completed-segment consolidation in the five-paper conceptual family.** Dashed arrows carry concepts only. The same-checkpoint semantic interface remains failed/unqualified, and no oracle gap, learned admission policy, strict-RTT result, or CSBC utility result is inherited.

84.2 Evidence boundary

Public MMLA V3 and its complete technical-report evidence record provide context [128, 130]. Their admitted boundary remains negative and diagnostic: PM-I2 qualified 0/9 same-checkpoint interfaces; in the fitted-population diagnostic, learned

content was exact on 0/73,728 probes both with learned and mechanical target routing, while only the complete mechanical oracle reached 73,728/73,728. Those counts do not evaluate CSBC, but they forbid treating a mechanically well-timed row as a semantically useful one.

The four companion focus papers provide terminology for strict RTT, complete rows, predictive admission, and typed dual state [134, 120, 123, 131]. None transfers empirical validation. In particular:

- CSBC changes authoritative memory, so its later-read path is RTMA;
- it is strict RTT only if a separately evidenced feedback-driven policy update is later read on the same unresolved problem;
- a valid event candidate is not a qualified semantic interface;
- an expected-risk admission rule is neither run nor assumed here; and
- no clean build or conditional theorem reopens a failed empirical gate.

84.3 Contribution and nonclaims

This manuscript owns a timing and systems contract: typed boundaries, five clocks, filtration, exposure, teacher amputation, overlap/carry, event ownership, idempotence, event recursion, asynchronous consistency, resource bounds, information-flow restrictions, failure recovery, and falsification obligations. It does not own a successful event detector, semantic encoder, factual verifier, admission model, storage engine, policy updater, or deployed system.

Formal statements carry their assumptions locally. Definitions, proofs, design prescriptions, planned tests, and unvalidated mechanisms are marked as such; no label promotes an implementation or empirical claim.

CONJECTURED A post-closure bidirectional view may help some eventizers represent local corrections or relations. The conjecture remains open. This paper establishes only what would have to be true for such a mechanism to be causal, bounded, auditable, and falsifiable.

85 Typed Process and Carrier Inventory

85.1 Probability space and causal tokens

Fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, token alphabet $\mathcal{X}$, and token filtration $(\mathcal{F}_t^X)_{t\geq 0}$. At token time t, a frozen deployment generator with parameters $\theta \in \mathcal{P}_{\text{dep}}$ reads a pinned authoritative-memory version ν_t and draws

$$L_t = G_\theta(X_{<t}, \text{Read}(M^{\nu_t}), \gamma_t), \tag{274}$$

$$X_t = S(L_t, \xi_t), \tag{275}$$

where γ_t is declared causal workspace and $\xi_t \in \mathcal{U}$ is a recorded random coin. The immutable emission receipt is

$$\rho_t^X = \text{Hash}(\mathsf{episode}, t, \text{Hash}(X_{<t}), \text{Hash}(L_t), \xi_t, X_t, \nu_t, \text{Hash}(\gamma_t)). \tag{276}$$

The receipt may store digests rather than unrestricted private reasoning. It must nevertheless bind the causal inputs needed by the claimed reproducibility boundary.

85.2 Segment and event types

Let boundaries be $0 = b_0 < b_1 < \cdots$. Core $I_n = (b_{n-1}, b_n] \cap \mathbb{N}$ has length $C_n = |I_n|$. With overlap $O_n \subseteq \{1, \ldots, b_{n-1}\}$ and carry $q_{n-1} \in \mathcal{Q}$, the completed view is

$$V_n = \text{View}(X_{O_n}, X_{I_n}, q_{n-1}; \beta_n) \in \mathcal{V}, \tag{277}$$

constructed only after X_{b_n} and $\rho_{b_n}^X$ exist. Boundary rule $\beta_n \in \mathcal{B}$ is either fixed or an adapted stopping rule specified in Section 89.

A retrospective encoder $B_\psi : \mathcal{V} \to \mathcal{H}$ may attend bidirectionally within V_n. The eventizer returns

$$(E_n, q_n) = \text{Eventize}(B_\psi(V_n), q_{n-1}; \zeta_n), \quad E_n = (e_{n,1}, \ldots, e_{n,K_n}), \quad K_n \leq K_{\max}, \tag{278}$$

where ζ_n is recorded randomness and q_n has a fixed maximum serialization size B_Q. Each event has typed span, content proposal, anchor $\alpha(e) \in \mathbb{N}$, ownership core, source receipts, and idempotence key

$$\iota(e) = \text{Hash}(\mathsf{episode}, \mathsf{eventizer\text{-}build}, \mathsf{normalized\text{-}span}, \alpha(e), \mathsf{event\text{-}type}). \tag{279}$$

85.3 Authority, service, and training objects

Authoritative memory is a fixed table $M \in \mathcal{M} = \mathcal{R}^S$. Target selection is an explicit typed system operation,

$$\mathcal{A}_{n,j} = \mathrm{Route}(e_{n,j}, M_{n,j-1}) \subseteq [S], \qquad R_{n,j} = |\mathcal{A}_{n,j}| \le R_{\max}. \tag{280}$$

For each routed target $a \in \mathcal{A}_{n,j}$, a proposer returns a complete candidate or failure,

$$\widehat{r}_{n,j,a} = \mathrm{Construct}(e_{n,j}, M_{n,j-1}, a; \omega_{n,j,a}) \in \mathcal{R} \cup \{\bot\}. \tag{281}$$

Trusted validation and publication choose at most one pair $(a, \widehat{r}_{n,j,a})$ with $a \in \mathcal{A}_{n,j}$, or exact NULL. The route, target order, and final target decision are system-owned and are bound into the authorization receipt; a model-generated row cannot redirect its own write. Tenant, ACL, versions, protection, provenance, tombstone validity, rollback lineage, and final receipt are likewise system-owned fields, never minted by completed text.

Service progress is independently typed. For idempotence key $k_{n,j}$,

$$\sigma^{\mathrm{svc}}_{n,j} \in \{\textsc{Detected}, \textsc{Owned}, \textsc{Queued}, \textsc{Running}, \textsc{Retryable}, \textsc{Commit}, \textsc{Null}, \textsc{Failed-Terminal}\}. \tag{282}$$

The last three values are terminal receipts. COMMIT names one authorized row replacement, NULL names an exact no-op, and FAILED-TERMINAL records fail-closed exhaustion or unrecoverable integrity failure and therefore authorizes no mutation.

Wall-clock service uses a bounded queue $Q^{\mathrm{svc}}_{\kappa} \in \mathcal{K}$, worker records $W^{\mathrm{svc}}_{\kappa}$, pinned predecessor versions, retry records, and a commit ledger $\mathcal{L}^M$. Training-only objects live in a disjoint type $\mathcal{T}$: future branches $F_{g,k}$, labels Y^{T}, teacher risks, gradients, split identifiers, and supervision time s. No value in $\mathcal{T}$ is a deployment input.

Object	Type	Role	Mandatory boundary
$X_t, L_t, \xi_t, \rho^X_t$	$\mathcal{X}, \mathbb{R}^{\lvert\mathcal{X}\rvert}, \mathcal{U}, \mathcal{R}_{\mathrm{emit}}$	causal emission and immutable receipt	fixed before future token exists
I_n, O_n, V_n	finite spans / $\mathcal{V}$	core, overlap, completed view	bounded; V_n only after b_n
β_n, q_n	$\mathcal{B}, \mathcal{Q}$	boundary rule and carry	stopping-measurable; $\lvert q_n\rvert \le B_Q$
$E_n, e_{n,j}$	$\mathcal{E}^{\le K_{\max}}, \mathcal{E}$	ordered events	typed span, digest, anchor, epoch, sources, key, and route
$\alpha(e), \iota(e)$	token index, digest	ownership and retry identity	stable across overlap and retry
$\widehat{r}_{n,j,a}$, NULL, M^{ν}	row/action/memory	target-conditioned candidate, identity action, authoritative state	trusted assembly; atomic publication
θ, ψ	deployment parameters	generator and retrospective encoder	frozen before evaluated episode
F, Y^{T}, s	$\mathcal{T}$	training-only teacher/supervision	amputated from deployment graph
$Q^{\mathrm{svc}}, \mathcal{L}, \sigma^{\mathrm{svc}}$	bounded queue / ledger / finite state	async work and terminal receipts	logical order independent of finish order
$\mathbf{C}$	$\mathbb{R}^d_+$	resource vector	no hidden scalarization

Table 60: Typed carrier inventory. Co-location in one process does not erase causal, authority, lifetime, or accounting distinctions.

85.4 Core assumption ledger

Assumption 85.1 (Causal generation and immutable history)**.** Equations (274)–(275) are $\mathcal{F}^X_t$-measurable. Once ρ^X_t is sealed, L_t, X_t, ξ_t, ν_t, and every earlier receipt are append-only and cannot be modified by the consolidator, queue, committer, or recovery service.

Assumption 85.2 (Post-closure construction and exposure)**.** V_n is unavailable before b_n closes. Every event successor has a logical exposure token $a_{n,j} > b_n$. A token read pins one complete version before generation; a successor becomes eligible only at or after its exposure point.

Assumption 85.3 (Bounded deterministic interpretation)**.** $C_n \le C_{\max}$, $|O_n| \le O_{\max}$, $|q_n| \le B_Q$, $K_n \le K_{\max}$, and $R_{n,j} \le R_{\max}$. Given the same completed view, carry, implementation/version identifiers, and random coins, encoding, eventization, anchor selection, ordering, routing, target ordering, construction, and idempotence keys are identical.

Assumption 85.4 (Trusted atomic authority)**.** Each ordered event validates against its actual predecessor, changes at most one routed complete row, or performs bit-identical NULL. A commit authorization binds the event key, ownership epoch, routed-target digest, chosen target, base version, old-row digest, candidate digest, and terminal outcome. A FAILED-TERMINAL receipt binds the same event identity but grants no mutation authority. Publication is serializable and crash recovery exposes an old or new complete authoritative version with its ledger binding.

These assumptions are formal premises, not implementation findings.

86 Five Clocks and the Logical Schedule

86.1 Clock types

DEFINITION

Definition 86.1 (Five-clock schedule)**.** A CSBC trace distinguishes:

1. token time $t \in \mathbb{N}$, ordering logits, samples, emissions, and read receipts;
2. boundary time $n \in \mathbb{N}$, ordering completed cores I_n;
3. logical event time (n, j) in lexicographic order, $1 \le j \le K_n$;
4. training-supervision time $s \in \mathbb{N}$, ordering offline teacher labels and parameter updates; and
5. wall-clock service time $\kappa \in \mathbb{R}_+$, recording enqueue, worker start/finish, validation, durable publication, and exposure latency.

Only the first three order deployment semantics. Clock s is outside the evaluated episode; clock κ determines availability and cost but never silently reorders events.

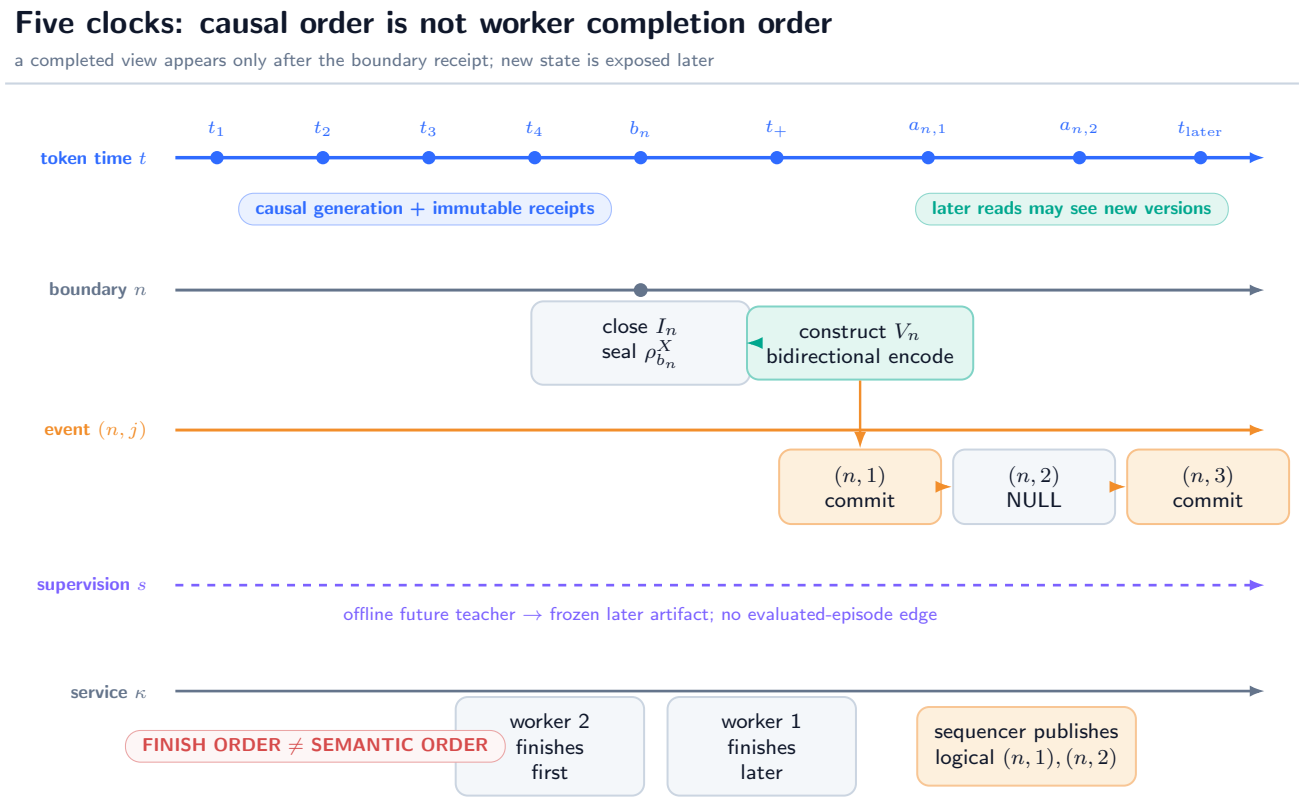

Figure 56: **Five aligned clocks and the post-closure boundary** (DEFINITION)**.** Solid token and event arrows are deployed logical order. The supervision rail is training-only and dashed. Wall-clock workers may finish out of order, but the commit sequencer and exposure receipts preserve lexicographic event order.

For service job $J_{n,j}$ write

$$\kappa^{\text{enq}}_{n,j} \le \kappa^{\text{start}}_{n,j} \le \kappa^{\text{done}}_{n,j} \le \kappa^{\text{pub}}_{n,j} \le \kappa^{\text{exp}}_{n,j}. \tag{283}$$

The inequalities are local to one job. It is permitted that $\kappa^{\text{done}}_{n,2} < \kappa^{\text{done}}_{n,1}$; publication must still respect logical dependencies or reject/recompute the stale result.

86.2 Logical precedence relation

Let $\prec$ be the transitive closure of:

$$\begin{aligned} (t, \text{emit}) &\prec (n, \text{close}) && \text{for } t \le b_n, \\ (n, \text{close}) &\prec (n, \text{view}) \prec (n, \text{encode}), \\ (n, \text{encode}) &\prec (n, 1) \prec \cdots \prec (n, K_n), \\ (n, j) &\prec (a_{n,j}, \text{eligible-read}). \end{aligned} \tag{284}$$

Training events have their own $\prec_s$ and may affect a later frozen parameter artifact, but they are not inserted into the deployment trace for an evaluated episode.

PROVED UNDER STATED ASSUMPTIONS

Proposition 86.2 (Worker-order irrelevance)**.** *Under Assumptions 85.3 and 85.4, two executions with identical logical inputs, coins, and accepted event sequence but different worker start/finish orders expose the same authoritative version sequence, provided every stale worker result is validated against its pinned predecessor and the sequencer publishes only in lexicographic event order.*

Proof. For event $(n, 1)$, deterministic construction on the same pinned predecessor yields the same candidate; validation either produces the same complete commit or NULL. Suppose authoritative versions agree through event $(n, j-1)$. Only a candidate whose recorded predecessor equals that common version can publish at (n, j); a result computed from another predecessor is rejected or recomputed. Determinism then gives the same event outcome. Induction over lexicographic event time proves identical version sequences. Worker timing can change waiting time and retry cost, not the accepted semantic order. □

Without predecessor validation, a fast worker can overwrite the effect of an earlier logical event. That is a lost-update bug, not a legitimate alternative CSBC semantics.

86.3 Exposure policy

The deployment fixes one of two policies:

- **stall**: token $t \geq a_{n,j}$ waits until the required version is published or a bounded timeout selects a registered fallback;
- **prior-state continuation**: generation continues on a previously pinned version, and the new version is first exposed at the next declared pin point after publication.

The policy may depend on causal queue metadata fixed before token sampling. A run may not retrospectively relabel earlier reads as though a late successor had been available.

87 Causal Boundary, Exposure, and Noninterference

87.1 Deployment filtration and DAG

Let $\mathcal{D}_t$ be the deployment sigma-field generated by emitted tokens and receipts through $t-1$, the causal workspace, declared coins revealed through t, and authoritative versions whose exposure points do not exceed t. The token kernel is $\mathcal{D}_t$-measurable. For boundary n, the completed view and retrospective state are measurable only after closure:

$$V_n \in \mathcal{F}^{X,+}_{b_n}, \qquad H_n = B_\psi(V_n) \in \mathcal{F}^{X,+}_{b_n}, \tag{285}$$

where $\mathcal{F}^{X,+}_{b_n}$ includes the sealed boundary receipt but no unobserved suffix.

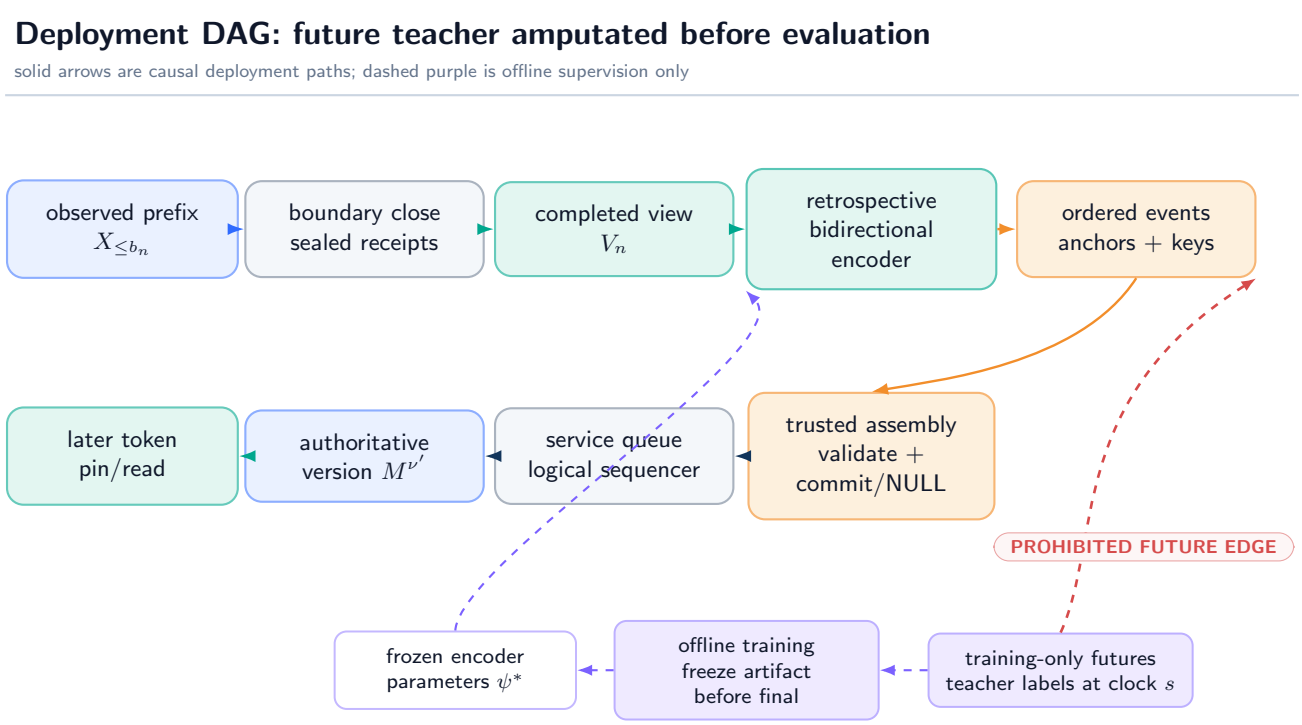

Figure 57: **Deployment DAG and amputated teacher (DEFINITION / DESIGN PRESCRIPTION).** Solid arrows terminate at post-closure events and later exposure. The dashed future-teacher branch may update a later frozen artifact offline, but it has no executable edge into the evaluated deployment episode. Coral edges are prohibited leakage.

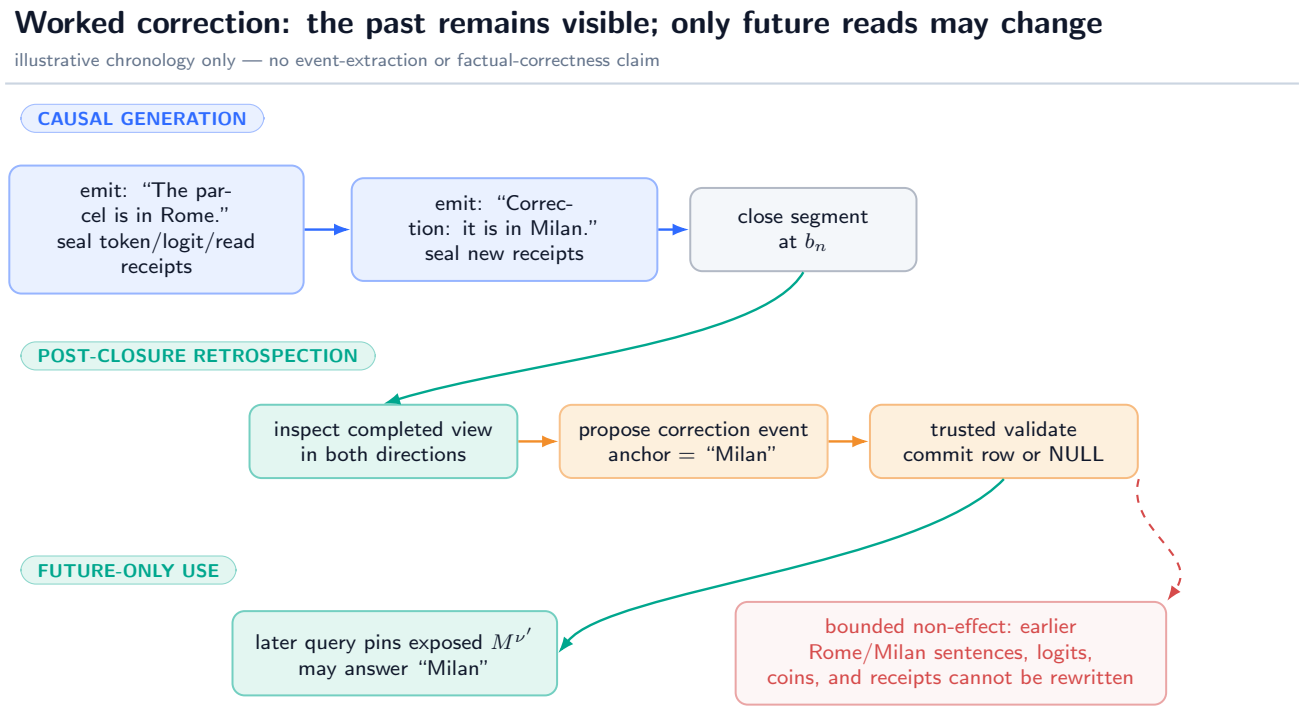

Figure 58: **Worked correction timeline (illustrative definition, not a semantic-success result).** A causal generator first emits an incorrect location and later a correction inside the same completed segment. Retrospective encoding starts only after closure. A validated correction row may help a later question; it cannot edit either earlier sentence or its receipt.

87.2 No retroactive effect

Let the historical prefix object at boundary b_n be

$$\mathsf{H}_{b_n} = \big((L_t, \xi_t, X_t, \nu_t, \rho_t^X)\big)_{t=1}^{b_n}. \tag{286}$$

PROVED UNDER STATED ASSUMPTIONS

Theorem 87.1 (No retroactive effect)**.** *Under Assumptions 85.1 and 85.2, for every boundary computation, event construction, validation, commit, retry, crash recovery, and later exposure derived from V_n,*

$$\mathsf{H}_{b_n}^{\text{after}} = \mathsf{H}_{b_n}^{\text{before}}. \tag{287}$$

In particular, no resulting state can change an already emitted token, historical logit, sampling coin, emission receipt, or prior memory-read version. Its earliest possible causal effect is on a read at $t \geq a_{n,j} > b_n$.

Proof. Assumption 85.1 makes every coordinate of H_{b_n} append-only and excludes it from the codomain of consolidation, commit, retry, and recovery maps. Those maps may append service and memory receipts and may create a successor $M^{\nu'}$. Assumption 85.2 excludes ν' from every read at $t \leq b_n$ and at every later token before its declared exposure. Therefore no transition induced by V_n has a write edge into H_{b_n} or a read edge into an earlier generation event. Equality (287) and the earliest-effect statement follow structurally. □

The theorem is not evidence that an implementation really uses immutable logs or correct pinning. Historical-hash and read-receipt tests in Section 97 must discharge those premises.

87.3 Prefix twins

Two executions ω, ω' are u-prefix twins under a specified counterfactual coupling if they are defined on one probability space, share implementation artifacts and initial authoritative state, and share every exogenous base variable and random coin exposed before the first differing suffix observation. Their tokens $X_{\leq u}$ and emission receipts are byte-identical. Boundary-rule state, causal workspace, queue state, and routing state are not separately stipulated equal: each must be a causal function of those shared variables and hence must evaluate identically. Only variables causally downstream of suffix tokens after u may differ.

PROVED UNDER STATED ASSUMPTIONS

Theorem 87.2 (Prefix-twin noninterference)**.** *Under Assumptions 85.1–85.3, if ω and ω' are u-prefix twins under the coupling above, then every boundary decision, completed view, event list, idempotence key, routed-target set and order, target-conditioned candidate, validation decision, service transition, and exposure decision produced before either execution observes a token after u is identical in the two runs.*

Proof. Before a suffix token is observed, the coupling gives both runs the same deployment sigma-field. Causality of the boundary detector, carry update, queue, and route maps makes their internal states equal rather than assuming that equality as an extra premise. Any fixed boundary rule or adaptive stopping decision is measurable with respect to that field, so both close or continue identically. If a boundary closes, its core, overlap, carry, view, implementation identifiers, and revealed coins agree. Assumption 85.3 therefore gives identical encoding, events, anchors, keys, ordering, routes, target orders, and target-conditioned candidates. Validation reads the same predecessor state and authorization inputs; the service transition and exposure maps read the same causal ledger and queue state. Their outputs agree. Induct over decisions until one run first observes a differing suffix token. □

This is a hyperproperty across paired executions. A single apparently causal trace cannot establish it.

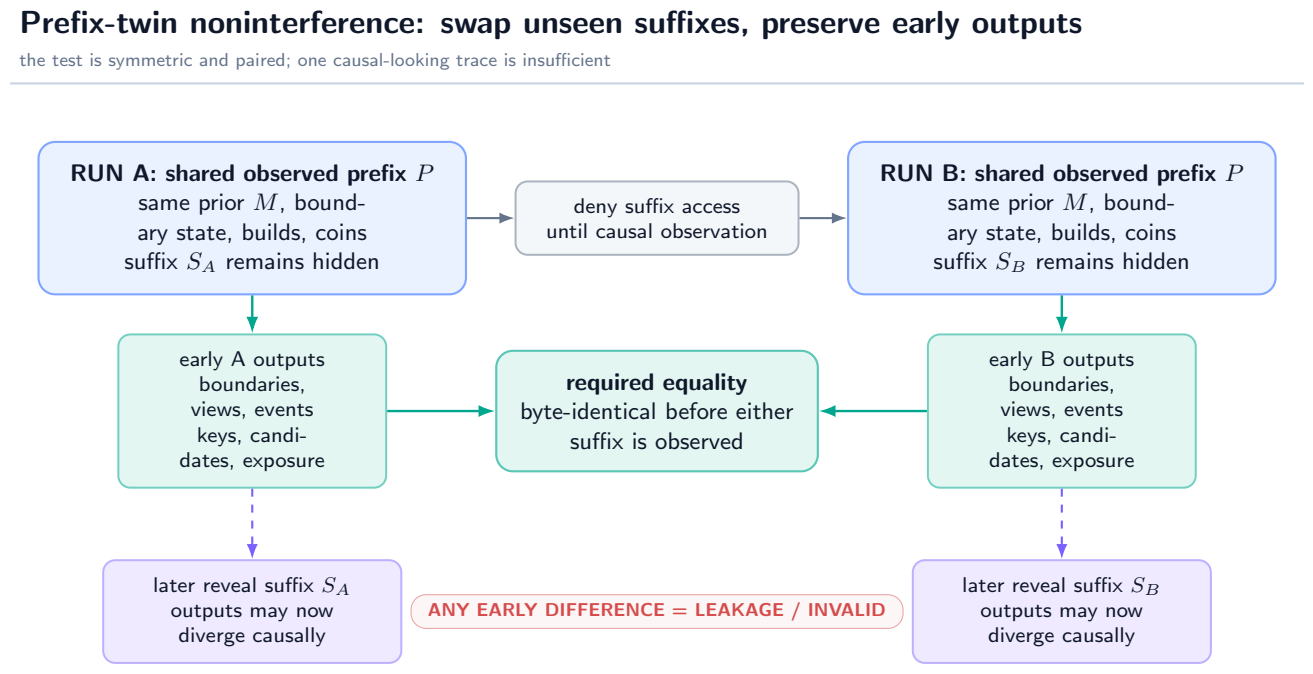

Figure 59: **Symmetric suffix-swap audit for Theorem 87.2 (PLANNED TEST).** Matched prefixes, coins, state, and builds must yield byte-identical pre-suffix boundaries and outputs despite different hidden continuations. Any early difference is a bounded leakage signal and invalidates the run.

88 Training-Supervision Separation and Teacher Amputation

88.1 Offline supervision is not a deployment input

Training group g may contain a causal prefix P_g, completed view V_g, and alternative future continuations $F_{g,1:K}$. A teacher may use those futures to create a label

$$Y_g^{\mathrm{T}} = T(V_g, F_{g,1:K}, \chi_g^{\mathrm{eval}}) \tag{288}$$

at supervision time s. Training produces a parameter artifact

$$(\psi^*, \theta^*) = \mathsf{Train}\big(\{(V_g, Y_g^{\mathrm{T}})\}_{g \in \mathcal{G}_{\mathrm{train}}}; \eta\big). \tag{289}$$

Before final episodes are opened, the artifact, feature schema, preprocessing graph, thresholds, random-seed policy, and dependency manifest are frozen and content addressed.

DEFINITION

Definition 88.1 (Teacher-branch amputation)**.** A deployment artifact is teacher-amputated when its executable transitive dependency graph contains no future continuation, teacher label, evaluator output, branch identifier, post-action state, final-split statistic, or cache derived from those objects for the evaluated episode. Only the frozen parameters and declared preprocessing artifacts may cross from supervision time into deployment.

Assumption 88.2 (Split and episode independence)**.** Training, calibration, and final episodes are separated at the physical episode/source-family level. All suffixes, paraphrases, cached features, event IDs, and teacher derivatives belonging to one episode family remain in one split. Final parameter freeze precedes any access to final suffixes or outcomes.

Assumption 88.3 (Executable graph closure)**.** The deployment dependency manifest is complete: it includes data loaders, feature services, retrieval stores, caches, environment variables, model files, tokenizer/boundary code, RNG initialization, and remote calls. Runtime access logging is faithful to that graph.

PROVED UNDER STATED ASSUMPTIONS

Theorem 88.4 (Teacher-amputation nonaccess)**.** *Under Assumptions 88.2 and 88.3, if the frozen deployment graph contains no path from an evaluated episode's future-teacher objects to its boundary, encoding, eventization, validation, or exposure*

nodes, then changing only those future-teacher objects cannot change any deployment output before the corresponding suffix becomes causally observed.

Proof. Every deployment output is a deterministic or recorded-random function of its ancestors in the complete executable graph. By premise, none of the changed teacher objects is an ancestor. All unchanged ancestors, including frozen parameters and coins, therefore induce the same output. When a suffix token is later observed, it becomes a causal token input and the theorem no longer requires equality beyond that point. □

This is a graph-separation deduction. It does not prove the manifest is complete. The practical evidence is a static dependency closure plus runtime deny/access receipts and the prefix-twin audit.

88.2 Leakage receipts

DESIGN PRESCRIPTION A conforming run records:

1. immutable train/calibration/final group manifests and source-family closure;
2. parameter, tokenizer, boundary, encoder, eventizer, validator, and threshold hashes with freeze time;
3. a field-level provenance graph for every deployment input;
4. filesystem, database, network, cache, and feature-service access logs during each final episode;
5. teacher-label and future-branch namespaces denied to deployment credentials;
6. group-preserving suffix swaps, label permutations, and matched-prefix twins; and
7. a final-entry receipt proving the evaluation namespace was opened once by the frozen artifact.

Counterexample 88.5 (Parameter freeze after final suffix access)**.** A threshold is tuned after inspecting which final suffixes contain corrections. The executable model never reads a suffix at runtime, yet the threshold encodes final information. Runtime graph amputation alone cannot repair the predeployment leak; Assumption 88.2 fails.

Counterexample 88.6 (Teacher-derived cache)**.** A feature store is precomputed using the full episode and later serves only a short numeric vector. The vector's type says "prefix feature," but its provenance includes the suffix. Without transitive cache provenance, a direct field audit misses the leak.

UNVALIDATED No CSBC teacher, parameter artifact, split, or deployment graph is built or tested in this paper. Training gradients through a completed segment are offline training operations; they do not imply online parameter gradients during deployment.

89 Fixed and Adaptive Boundaries, Overlap, and Carry

89.1 Fixed cores

For a fixed core size $C \geq 1$, set $b_n = \min\{nC, N\}$ for a finite episode of N tokens and

$$I_n = (b_{n-1}, b_n], \qquad O_n = (\max\{0, b_{n-1} - O\}, b_{n-1}], \qquad 0 \leq O \leq O_{\max}. \tag{290}$$

Overlap tokens are read-only evidence in view V_n. They do not re-enter generation and do not acquire new ownership merely because they appear twice.

89.2 Adaptive stopping boundaries

Let $C_{\min} \leq C_{\max}$. The detector has a finite carrier $\mathcal{D}$ with a fixed encoding cap B_D and a causal recurrence

$$d_t = D(d_{t-1}, X_t), \qquad d_t \in \mathcal{D}, \qquad \text{bytes}(d_t) \leq B_D. \tag{291}$$

At a boundary, the segment-local detector is reset to $d_\varnothing$; any permitted continuation crosses the boundary only through the typed carry q_n in Eq. (293). Thus neither an opaque recurrent state nor an unbounded token buffer can bypass the carry cap.

Starting after b_{n-1}, set $\tau_n = \min\{N, b_{n-1} + C_{\max}\}$. The adaptive rule observes only the causal prefix and chooses

$$b_n = \begin{cases} N, & N - b_{n-1} < C_{\min}, \\ \inf\{t \in [b_{n-1} + C_{\min}, \tau_n] : h(X_{\leq t}, d_t) = 1 \text{ or } t = \tau_n\}, & \text{otherwise.} \end{cases} \tag{292}$$

The first branch is a declared terminal flush: it may create a final core shorter than $C_{\min}$ but never longer than $C_{\max}$.

Assumption 89.1 (Stopping-time boundary)**.** For every t, the event $\{b_n \le t\}$ is measurable with respect to $\mathcal{F}_t^X$. The rule uses no future token, suffix label, retrospective encoder output, or worker completion time. It always closes by $C_{\max}$ core tokens.

Thus the boundary detector may react to an observed delimiter or punctuation, but it cannot wait to see whether the next token makes the current span convenient. Worst-case token closure delay is $C_{\max}$ tokens after the previous boundary; wall-clock closure adds the observed token-generation time.

89.3 Cross-boundary event state

An event may begin near the end of I_n and close in I_{n+1}. The detector then emits no complete event in segment n and serializes a carry record

$$q_n = (\text{episode}, \text{event-type}, \text{start}, \text{bounded-summary}, \text{source-digests}, \text{age}) \in Q. \tag{293}$$

The carry cannot contain an unbounded token string. If an event remains open beyond H_Q boundaries or needs more than B_Q bytes, the detector emits a typed overflow/failure receipt and drops, truncates only a schema-declared nonsemantic diagnostic, or routes to a separately bounded fallback. It may not silently grow.

When the event closes, its anchor is a canonical token position belonging to exactly one core, such as the first token of the terminal predicate or the final token required by the event schema. The owner is

$$\text{Owner}(e) = n \iff \alpha(e) \in I_n. \tag{294}$$

Overlap and carry provide evidence but never override Eq. (294).

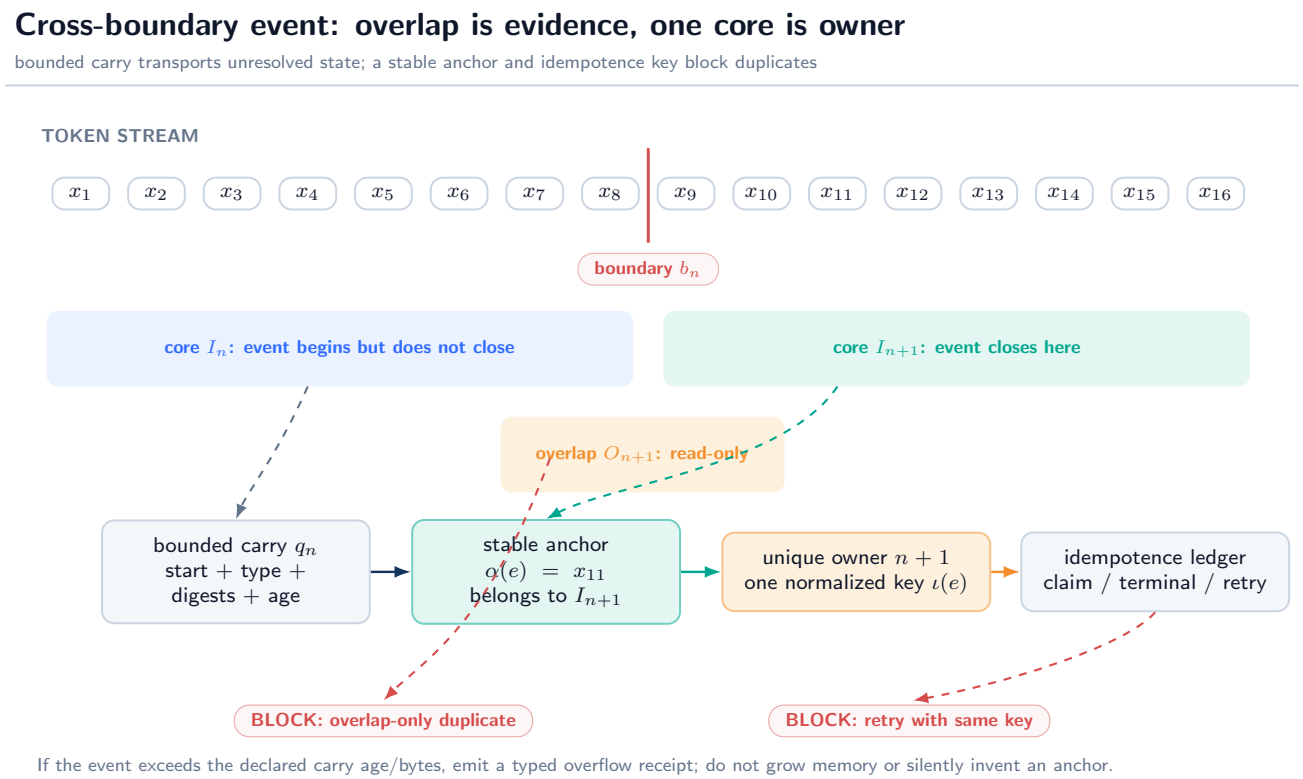

Figure 60: **Cross-boundary eventization** (DEFINITION / DESIGN PRESCRIPTION)**.** A bounded carry transports an unfinished event; overlap supplies read-only context; the stable anchor lands in one core, which alone owns emission. The idempotence ledger blocks a retry or overlapping view from publishing a duplicate.

Assumption 89.2 (Bounded closure and carry)**.** Every carried detector state is at most B_Q bytes and at most H_Q boundaries old; the segment-local state additionally obeys Eq. (291) and is reset at closure. For a complete event in scope, the declared detector either identifies a stable anchor after finite observed evidence or emits a typed unresolved/failure outcome by the cap. No theorem assumes that every real-world relation fits the cap.

PROVED UNDER STATED ASSUMPTIONS

Proposition 89.3 (Finite view and coverage)**.** *Under fixed boundaries or Assumption 89.1, the episode has*

$$\left\lceil \frac{N}{C_{\max}} \right\rceil \le B_N \le \left\lceil \frac{N}{C_{\min}} \right\rceil \tag{295}$$

adaptive cores, with the fixed case $B_N = \lceil N/C \rceil$. Every core token belongs to exactly one I_n, every completed view contains at most $C_{\max} + O_{\max}$ tokens plus B_Q carry bytes, and the total number of token positions presented to the retrospective encoder is at most

$$N + O_{\max} B_N. \tag{296}$$

Proof. Every adaptive core has at most $C_{\max}$ tokens, so at least $\lceil N/C_{\max} \rceil$ cores are required. Every nonfinal core has at least $C_{\min}$ tokens, while the declared terminal flush has between one and $C_{\max}$; hence $B_N \le 1 + \lfloor (N-1)/C_{\min} \rfloor = \lceil N/C_{\min} \rceil$. This proves Eq. (295) without incorrectly imposing the minimum on the final core. The half-open intervals $(b_{n-1}, b_n]$ are disjoint and cover $1{:}N$. By construction, overlap adds at most $O_{\max}$ token positions to each view and carry has the fixed byte cap. Summing core positions gives N and summing overlap bounds gives Eq. (296). □

The upper bound counts repeated positions, not unique tokens. Hidden archive retrieval, a variable-size carry, or unconstrained backtracking is outside its premises.

90 Unique Ownership, Idempotence, and Terminal Receipts

90.1 Detector and ownership contract

Let $\mathcal{E}^*(X)$ be the finite set of in-scope completed events defined by a frozen annotation/event schema on an observed episode. The eventizer is not assumed semantically correct. The following premises state what must hold for an at-most-once authority claim and an exactly-one service-terminal-receipt claim.

Assumption 90.1 (Detector consistency)**.** For fixed episode bytes, boundary/carry state, implementation artifacts, and coins, every invocation that sees enough evidence for the same in-scope event emits the same normalized event content, stable anchor, and idempotence key. It emits no complete key while the event remains unresolved.

Assumption 90.2 (Unique core ownership)**.** Every emitted event has exactly one anchor $\alpha(e) \in \bigcup_n I_n$, and only the event list of $\mathrm{Owner}(e)$ may submit its key for publication. An overlap-only or carry-only occurrence is non-owning.

Assumption 90.3 (Linearizable idempotence ledger)**.** The commit service maintains a bounded-scope ledger mapping $(\mathsf{episode}, \iota)$ to ABSENT; one nonterminal state in {DETECTED, OWNED, QUEUED, RUNNING, RETRYABLE}; or one terminal receipt in {(COMMIT, ν), NULL, FAILED-TERMINAL}. Claiming an absent key, advancing a service state, and recording its terminal receipt are linearizable with the authoritative commit protocol. A retry reads or resumes the same key; it never creates a fresh identity. FAILED-TERMINAL is fail-closed and cannot mutate memory.

The ledger scope and retention must cover every permitted retry and overlap replay. A finite ledger cannot promise global uniqueness after its key has been forgotten; the guarantee is explicitly scoped to the episode/retry horizon.

PROVED UNDER STATED ASSUMPTIONS

Theorem 90.4 (At-most-once authority and exactly-one terminal service result)**.** *Under Assumptions 89.2, 90.1, 90.2, and 90.3, every in-scope event that the detector completes within its declared cap has at most one authoritative outcome: one complete-row commit or exact NULL. If the owning job and ledger remain live or recoverable until a terminal receipt is recorded, it has exactly one service-terminal receipt in {COMMIT, NULL, FAILED-TERMINAL}. The failure receipt is not an authoritative update.*

Proof. Detector consistency gives one stable key for the completed event. Unique ownership permits only one core to submit that key, although retries may resubmit it. By linearizability, at most one submission changes the ledger from ABSENT; every other invocation observes the same nonterminal chain or terminal receipt. Trusted atomic authority permits at most one routed row commit or exact NULL for that key. A FAILED-TERMINAL transition grants no write capability, so it cannot create a second authoritative outcome. For liveness, the additional premise ensures the owning job is eventually executed, recovered, or closed fail-safe and cannot remain nonterminal forever. Linearizability then permits exactly one terminal receipt. □

The theorem says nothing about factual correctness, event recall, target choice, or semantic usefulness. A consistently wrong detector can satisfy it.

90.2 Idempotent retry protocol

Protocol 90.5 (Claim, construct, publish, recover)**.** For owning event $e_{n,j}$:

1. atomically claim $(\mathsf{episode}, \iota(e_{n,j}))$ or read its existing record;
2. bind the logical event index, anchor, completed-view digest, predecessor memory version, ownership epoch, builder/validator versions, routed-target digest, and authorization context;

3. construct target-conditioned candidates against the pinned predecessor, replay and validate every protected field and cross-row invariant, and select at most one routed target;
4. publish one complete row or exact NULL in logical order and bind the terminal state to the same key, target, base version, old-row digest, and candidate digest;
5. on crash, recover the old or new complete authoritative root and reconcile the terminal key; and
6. on retry, return the terminal result or resume the same in-flight transaction. Never mint a new key from attempt number, wall time, or worker ID; and
7. after bounded retry/rebuild exhaustion or unrecoverable integrity failure, append FAILED-TERMINAL with no mutation. Any authorized recovery is a new logical event with a new key explicitly rooted in that failure receipt; the failed key is never reused.

90.3 Multiple events and duplicate suppression

One segment may contain $K_n > 1$ distinct events. Distinct stable keys and the event order separate them. Conversely, two surface mentions may map to one event only if the frozen schema says they share one normalized identity and anchor. Deduplication after seeing downstream utility would change the estimand and is prohibited.

DESIGN PRESCRIPTION The event manifest records all emitted, deferred, overflowed, rejected, duplicated, and missing outcomes. A missing key is never silently imputed as NULL: NULL is a terminal authoritative decision after a valid event reached the commit protocol.

91 Event-Recursive Authority, Serializability, and Commutation

91.1 Ordered state recursion

For completed segment n, set

$$M_{n,0} = M_{n-1}, \qquad M_{n,j} = T_{n,j}(M_{n,j-1}), \qquad M_n = M_{n,K_n}. \tag{297}$$

If trusted validation accepts the routed pair $(a^M_{n,j}, \widehat{r}_{n,j,a^M_{n,j}})$ with $a^M_{n,j} \in \mathcal{A}_{n,j} \subseteq \{1, \dots, S\}$,

$$T_{n,j}(M)_i = \begin{cases} \widehat{r}_{n,j,a^M_{n,j}}, & i = a^M_{n,j}, \\ M_i, & i \neq a^M_{n,j}; \end{cases} \qquad T^{\mathsf{NULL}}_{n,j}(M) = M \tag{298}$$

bit for bit. A later event constructs and validates against $M_{n,j-1}$, not against the pre-segment snapshot unless those states happen to be equal.

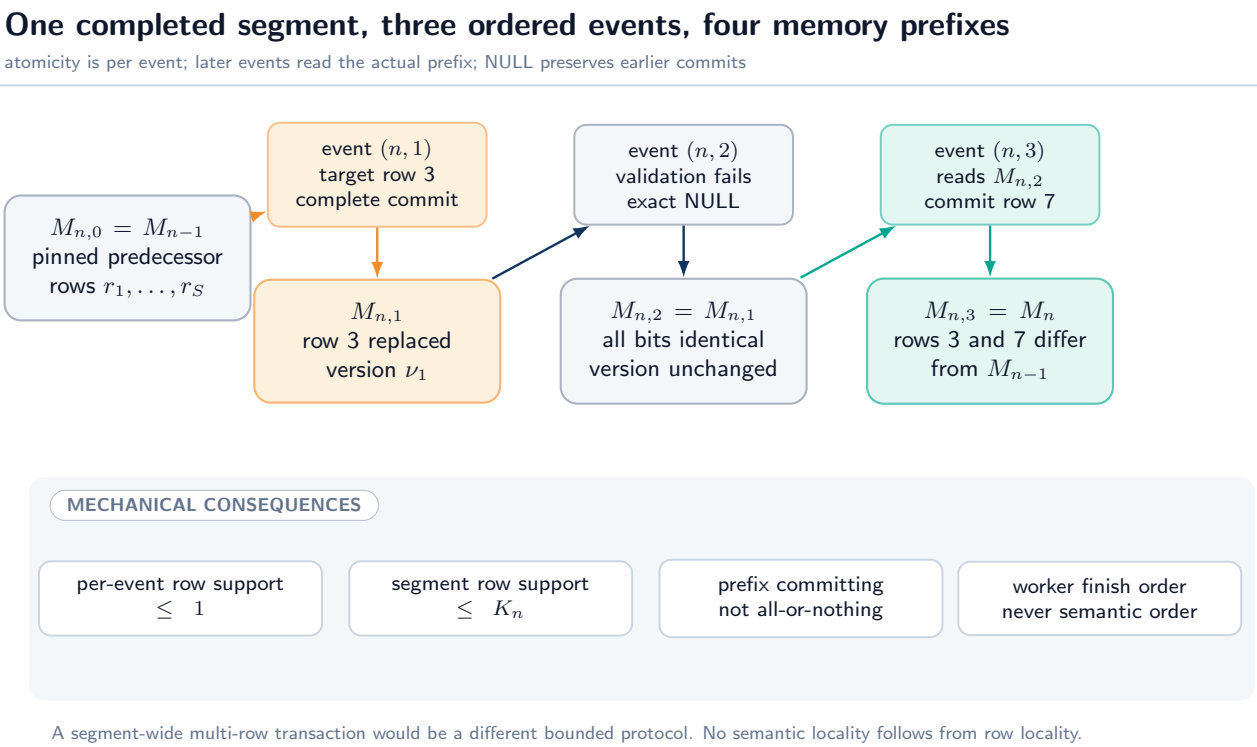

Figure 61: **Multiple events from one completed segment (DEFINITION).** Events follow declared logical order and actual predecessor versions. Event 1 commits a complete row, event 2 returns exact NULL, and event 3 reads the state after both. Atomicity is per event; the segment is prefix-committing.

PROVED UNDER STATED ASSUMPTIONS

Theorem 91.1 (Event locality and segment support)**.** *Under Assumption 85.4, for every event*

$$|\operatorname{supp}_{\text{row}}(M_{n,j} - M_{n,j-1})| \leq 1. \tag{299}$$

If C_n^M events commit and D_n distinct rows are changed across segment n, then

$$C_n^M \leq K_n, \qquad D_n \leq \min\{C_n^M, S\}, \qquad |\operatorname{supp}_{\text{row}}(M_n - M_{n-1})| \leq D_n \leq K_n. \tag{300}$$

Earlier successful events survive a later NULL.

Proof. Equation (298) changes one target row and NULL changes none, proving Eq. (299). Each of K_n events contributes at most one commit and one target to the union of changed rows. Thus $C_n^M \leq K_n$ and $D_n \leq \min\{C_n^M, S\}$. Rows outside that union are byte-identical under composition, giving Eq. (300). Because the recursion takes $M_{n,j-1}$ as the input, applying identity at a later event preserves the already committed prefix. □

The result is not whole-segment atomicity and not whole-segment rank one. A segment-wide transaction would need a separate bounded multi-row read/write set and publication protocol.

91.2 Serializable semantic order

Assumption 91.2 (Logical sequencer)**.** For one authority domain, the commit service provides a total order extending lexicographic event time. Each candidate records its read set and predecessor versions; a stale candidate is rejected or reconstructed. Global capacity, uniqueness, index, authorization, and version constraints are checked in that same serializable order.

PROVED UNDER STATED ASSUMPTIONS

Theorem 91.3 (Serializable event history)**.** *Under Assumptions 85.4 and 91.2, every finite concurrent service execution is observationally equivalent at the authoritative read interface to the sequential recursion in Eq. (297), with stale/rejected attempts represented by typed retry records or NULL according to the frozen protocol.*

Proof. Order accepted publications by the sequencer's linearization points. Each publication was validated against the predecessor and global constraints current at its position; a stale attempt has no authoritative effect. Applying the same complete-row or identity transitions in that order reproduces every published root. A validated read observes one published root and can be placed after its linearization point. Therefore the concurrent execution and the sequential recursion have the same authoritative observations. □

91.3 Conditional commutation

Let event maps T_e, T_f target distinct rows. Disjoint destinations alone are insufficient: either event can read capacity, eviction rank, global uniqueness, an index generation, a shared version counter, authorization state, or the other row.

Assumption 91.4 (Commutation independence)**.** e and f target distinct rows; their candidates, read sets, validation outcomes, version allocation, authorization, capacity/eviction decisions, index updates, ledger reservations, and cross-row invariants are unchanged when the other event is applied first. Their idempotence keys are distinct and their audit order remains recorded.

PROVED UNDER STATED ASSUMPTIONS

Proposition 91.5 (Conditional row commutation)**.** *Under Assumption 91.4,*

$$T_e(T_f(M)) = T_f(T_e(M)) \tag{301}$$

on authoritative memory content. The ordered audit trace need not be byte-identical.

Proof. By premise, each event computes the same complete candidate and decision under either predecessor order. The targets are distinct, so replacing one row leaves the other's replacement coordinate unchanged. Both compositions therefore contain the same new row at each target and the same old rows elsewhere. Audit receipts retain event order, so only the memory-content equality is claimed. □

If both events see one free slot and each plans to consume it, or if one changes an ACL consulted by the other, the premise fails and order is semantic.

92 Asynchronous Consolidation and Causal Consistency

92.1 Pinned versions while service runs

At token t, generation pins one version ν_t for the complete token computation. If consolidation for segment n is still queued, prior-state continuation uses the latest exposed predecessor; it never reads a partially built candidate. Under a stall policy, the token is not sampled until the registered required version or timeout/fallback decision is available. Both choices are receipted.

Let ν_0 be a distinguished validated genesis version with $\kappa^{\mathrm{pub}}(\nu_0) = -\infty$ and $a(\nu_0) = 0$. Let $p(t)$ be the exposure-consistent read selector:

$$p(t) = \max\left(\{\nu_0\} \cup \{\nu : \kappa^{\mathrm{pub}}(\nu) \le \kappa_t^{\mathrm{pin}},\ a(\nu) \le t,\ \mathrm{Validate}(M^\nu) = 1\}\right). \tag{302}$$

The maximum is therefore defined even before the first successor is published, and it is taken over the logical version order rather than worker completion timestamps.

Assumption 92.1 (Read pin and publication integrity)**.** A token reads one validated immutable authoritative root for its whole generation step. Publication exposes no partial candidate; caches and indexes are version-bound hints and are revalidated against that root.

PROVED UNDER STATED ASSUMPTIONS

Proposition 92.2 (Asynchronous causal consistency)**.** *Under Assumptions 85.2, 91.2, and 92.1, asynchronous service can change which eligible version a later token pins and can increase latency, but it cannot expose a successor before its token exposure point, mix versions within one token, or make worker finish order supersede logical event order.*

Proof. Equation (302) excludes versions that are unpublished, invalid, or not yet token-eligible. Read pinning fixes the selected root for the token. The sequencer admits versions only in logical order and rejects stale publications. Thus service timing controls when a logically eligible version enters the maximization set but cannot violate exposure, mix roots, or reorder accepted events. □

92.2 Stale jobs, retry, and crash

A worker record binds $(n, j), \iota(e)$, view digest, predecessor root, read set, build identifiers, and random coins. If the predecessor is stale at validation, the protocol chooses one frozen action:

1. rebuild candidate and validation from the current logical predecessor;
2. prove the old candidate remains valid under a declared commutation/read-set test; or
3. return a typed stale NULL if the scientific protocol defines rejection rather than retry; or
4. after a declared retry/rebuild cap or unrecoverable integrity failure, append FAILED-TERMINAL with no authoritative mutation.

Silently publishing the stale candidate is never allowed. Retry, replay, and wait costs remain charged even when the ultimate terminal receipt is NULL or FAILED-TERMINAL. A failed-terminal key is never silently resubmitted; an authorized later recovery must be a new logical event explicitly linked to the failure receipt.

Crash recovery follows the complete-row contract: validate the idempotence record, old/new authoritative root, audit binding, archive/index state, and queue lease. A key in DETECTED, OWNED, QUEUED, RUNNING, or RETRYABLE may be resumed only against the same logical event and predecessor rules. Terminal keys return their existing receipt. Orphan candidate bytes are non-authoritative garbage.

92.3 Speculative decoding

Suppose generation speculates tokens $t{:}t + h_{\text{spec}}$ against version ν while a successor ν' is pending. The deployment chooses:

- **commit-old**: seal speculative tokens with ν and expose ν' only afterward;
- **discard-and-regenerate**: before external emission, discard every unsealed speculative token, activation, cache entry, and draft receipt, then regenerate under ν'; or
- **stall**: do not speculate across the required exposure.

Once a token receipt is externally sealed, it belongs to immutable history and cannot be rolled back. Calling deletion of an already published token "speculative rollback" violates Assumption 85.1.

Counterexample 92.3 (Mixed speculative prefix)**.** Tokens t and $t + 1$ are drafted under ν, then $t + 1$ alone is rescored under ν' while retaining the hidden state produced under ν. The resulting trace has no single pinned read view and does not satisfy any of the three declared policies.

DESIGN PRESCRIPTION Queue leases, cancellation, timeout, fallback, reconstruction, publication, exposure, cache invalidation, and speculative-discard receipts must be included in the causal trace. Missing service records are not evidence of a synchronous implementation.

93 Complete Resource and Queue Bounds

93.1 Resource vector

For episode execution u, report before scalarization

$$\begin{aligned}\mathbf{C}(u) = (&N_{\text{tok}}, F_{\text{gen}}, F_{\text{bi}}, F_{\text{evt}}, F_{\text{route}}, F_{\text{cand}}, F_{\text{replay}}, F_{\text{val}}, F_{\text{commit}},\\ &B_{\text{resident}}, B_{\text{scratch}}, B_{\text{carry}}, B_{\text{queue}}, B_{\text{archive}}, B_{\text{index}}, B_{\text{audit}},\\ &B_{\text{mem-traffic}}, B_{\text{net}}, N_{\text{retry}}, N_{\text{failed-terminal}}, N_{\text{barrier}}, T_{\text{latency}}, T_{\text{backlog}}, E_{\text{proxy}}).\end{aligned} \tag{303}$$

Units, measurement method, caching, batching, failure work, and amortization horizon are explicit. A scalar $\lambda^{\top}\mathbf{C}$ requires a preregistered nonnegative λ with compatible units.

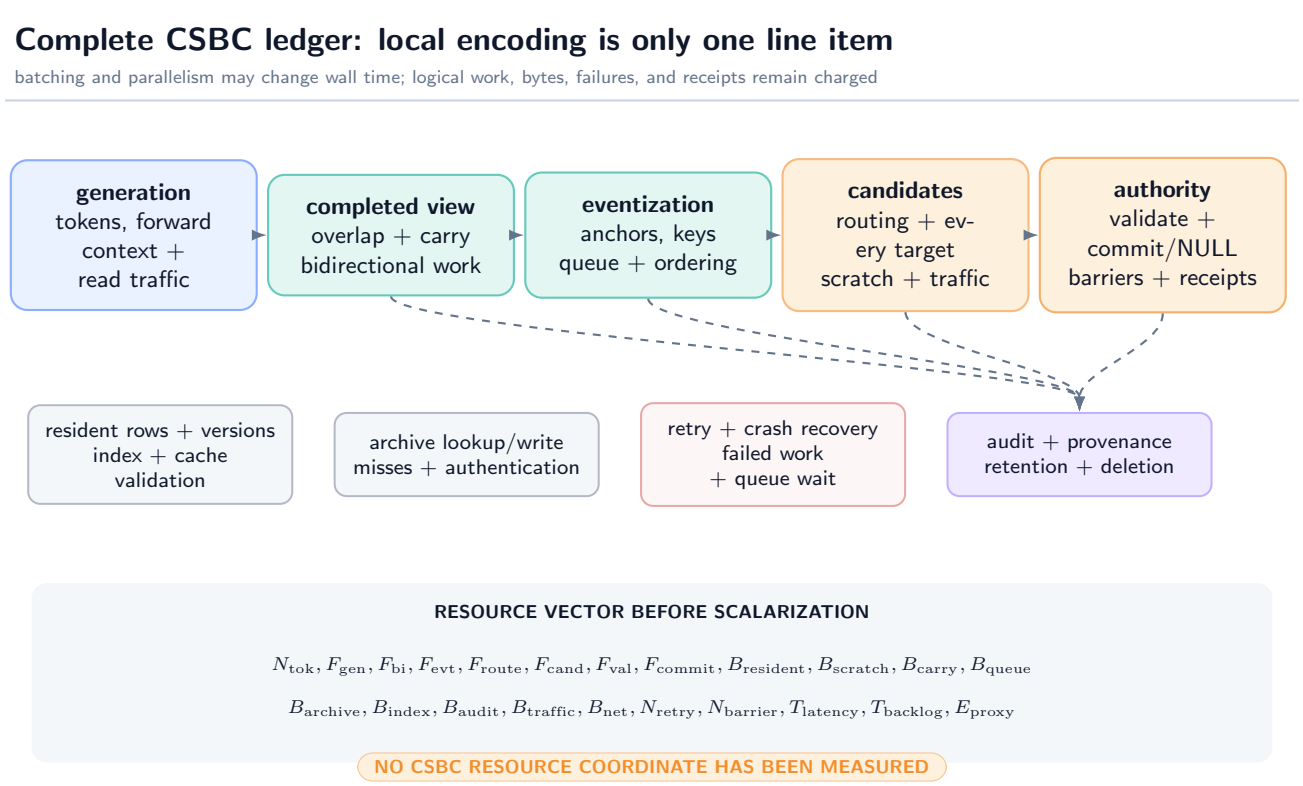

Figure 62: **Complete CSBC resource ledger** (DEFINITION / DESIGN PRESCRIPTION)**.** Generation, local bidirectional encoding, eventization, routing, every target-conditioned construction and replay, trusted validation/commit, resident and pool scratch bytes, archive/index access, traffic, retry or terminal failure, queueing, audit, latency, and energy proxies remain visible. No coordinate is measured in this paper.

93.2 Local work model

Let $L_n = C_n + |O_n|$. For a D-layer dense local Transformer encoder of width d, feed-forward expansion r, and h heads with per-head width $d_h = d/h$, record implementation constants $\alpha_{\mathrm{att}}, \alpha_{\mathrm{proj}}, \alpha_{\mathrm{ff}} > 0$ such that

$$F_{\mathrm{bi}}(n) \le D\left(\alpha_{\mathrm{att}} h L_n^2 d_h + \alpha_{\mathrm{proj}} L_n d^2 + \alpha_{\mathrm{ff}} r L_n d^2\right). \tag{304}$$

The identity $h d_h = d$ makes the attention term equal to $\alpha_{\mathrm{att}} L_n^2 d$; retaining h and d_h prevents head-count work from disappearing from the ledger. The quadratic term is local to a bounded completed view, not evidence of linear end-to-end execution. A different encoder publishes its own exact operation formula.

With eventization cost $c_{\mathrm{evt}} L_n$, routed target set $\mathcal{A}_{n,j}$, target-conditioned construction $c_{\mathrm{cand}}(a)$, deterministic replay or reconstruction $c_{\mathrm{replay}}(a)$, validation $c_{\mathrm{val}}(a)$, and selected publication $c_{\mathrm{pub}}(n,j)$, one segment costs

$$\begin{aligned} C_n^{\mathrm{csbc}} \le\ & F_{\mathrm{bi}}(n) + c_{\mathrm{evt}} L_n + c_{\mathrm{carry}} B_Q + c_{\mathrm{audit}}^{\mathrm{seg}} \\ & + \sum_{j=1}^{K_n} \left[c_{\mathrm{route}}(n,j) + \sum_{a \in \mathcal{A}_{n,j}} \{ c_{\mathrm{cand}}(a) + c_{\mathrm{replay}}(a) + c_{\mathrm{val}}(a) \} + c_{\mathrm{pub}}(n,j) \right] + C_n^{\mathrm{retry}}. \end{aligned} \tag{305}$$

Archive retrieval includes index lookup, bytes read, authentication, decoding, and misses. A hidden scan of an archive of size A_N adds $\Omega(A_N)$ work and is not absorbed into c_{route}.

93.3 Fixed and adaptive total bounds

Assumption 93.1 (Uniform finite system constants)**.** Every nonfinal core satisfies $C_n \in [C_{\min}, C_{\max}]$, the final core satisfies $1 \le C_{B_N} \le C_{\max}$, $|O_n| \le O_{\max}$, $K_n \le K_{\max}$, and $R_{n,j} \le R_{\max}$. Carry and all queues are capped; at most $P_{\max}$ target-candidate/replay contexts are simultaneously live; every construction, replay, validation, publication, archive, and index operation has a declared finite worst-case bound; retry caps are finite; and no hidden state or archive scan grows with episode length.

PROVED UNDER STATED ASSUMPTIONS

Theorem 93.2 (Bounded fixed/adaptive work and memory)**.** *Under Assumptions 89.1, 89.2, and 93.1, both fixed and adaptive segmentation use at most $B_N \le \lceil N/C_{\min} \rceil$ completed views and have total retrospective dense-attention work bounded by*

$$\sum_{n=1}^{B_N} F_{\mathrm{bi}}(n) \le B_N D \left[\alpha_{\mathrm{att}} h (C_{\max} + O_{\max})^2 d_h + (\alpha_{\mathrm{proj}} + r\alpha_{\mathrm{ff}})(C_{\max} + O_{\max}) d^2 \right]. \tag{306}$$

Total routed construction, replay, and validation work is at most $B_N K_{\max} R_{\max}$ times the declared per-target bound plus the visible segment terms in Eq. (305); failed and retried attempts remain charged. Peak live memory is bounded by

$$B_{\mathrm{peak}} \le B_{\mathrm{gen}} + B_{\mathrm{resident}} + B_{\mathrm{index}}^{\max} + B_{\mathrm{view}}(C_{\max} + O_{\max}) + B_Q + B_{\mathrm{queue}}^{\max} + P_{\max}(B_{\mathrm{cand}}^{\max} + B_{\mathrm{replay}}^{\max} + B_{\mathrm{val}}^{\max}) + B_{\mathrm{audit\text{-}buffer}}^{\max}. \tag{307}$$

Proof. Proposition 89.3 bounds the number and size of views, including a possibly short final core. Substituting the maximum view length into Eq. (304) and summing B_N terms gives Eq. (306). There are at most $K_{\max}$ events per view and $R_{\max}$ routed targets per event, giving the per-target construction/replay/validation multiplier. Every live carrier in Eq. (307) has a finite declared cap, and $P_{\max}$ bounds simultaneous pool scratch; their sum bounds simultaneous storage. Fixed segmentation is the special case whose nonfinal cores have $C_{\min} = C_{\max} = C$. □

The theorem fails if carry grows with an open event, an index is an unbounded graph, archive search scans all history, construction or replay has no finite bound, retries are unbounded, or the service pool may retain arbitrarily many candidate/replay contexts.

93.4 Deterministic queue stability

Let completed views arrive to one service pool no faster than one every $\tau_g > 0$ wall-clock seconds. Let each view require at most $s_{\max}$ service seconds, and let the pool provide c workers whose aggregate guaranteed service over any interval of length T is at least cT worker-seconds after a declared startup constant.

PROVED UNDER STATED ASSUMPTIONS

Theorem 93.3 (Deterministic queue stability)**.** *If*

$$s_{\max} < c\tau_g, \tag{308}$$

then a work-conserving queue with bounded startup jitter has a finite deterministic backlog and wait bound. If $s_{\max} > c\tau_g$ for a sustained sequence of maximum jobs, backlog grows at least linearly until admission control, degradation, or stall occurs. Equality requires a separate jitter analysis and is not claimed stable here.

Proof. Over m interarrival intervals, at most $(m+1)s_{\max}$ work arrives while at least $mc\tau_g$ service is available apart from the fixed startup deficit. Under Eq. (308), net work decreases by $c\tau_g - s_{\max} > 0$ per full interval after any finite burst, so the backlog is bounded by the initial/burst deficit plus one maximum job. Conversely, if every arrival brings $s_{\max} > c\tau_g$, unfinished work increases by at least $s_{\max} - c\tau_g$ per interval, yielding linear growth. □

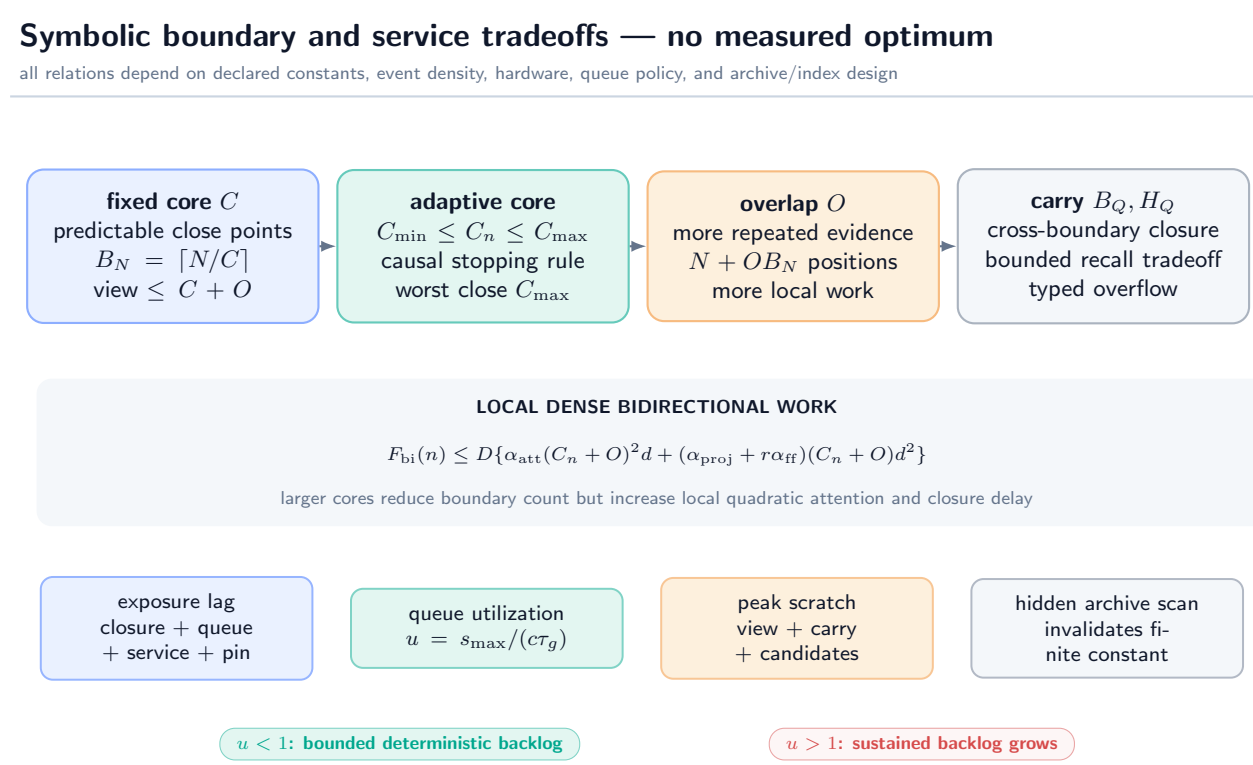

Figure 63: **Symbolic boundary and service tradeoffs (DEFINITION / PROVED UNDER STATED ASSUMPTIONS under stated constants).** Fixed and adaptive cores trade closure delay, overlap duplication, local dense-attention work, exposure lag, queue utilization, and scratch memory. Axes are symbolic; no measured optimum or crossover is shown.

Let $G_{n,j}$ be the finite precedence DAG for closure, queueing, encoding, eventization, routing, target construction, replay, validation, durable commit, and pinning. Latency to first eligible use is its critical-path length,

$$\begin{aligned} T_{n,j}^{\text{exposure}} &= \max_{\pi \in \text{Paths}(G_{n,j})} \sum_{v \in \pi} T_v \\ &\leq T_n^{\text{closure}} + T_n^{\text{queue}} + T_n^{\text{encode}} + T_{n,j}^{\text{event}} + T_{n,j}^{\text{route}} + T_{n,j}^{\text{candidate}} + T_{n,j}^{\text{replay}} + T_{n,j}^{\text{validate}} + T_{n,j}^{\text{durable}} + T_{n,j}^{\text{pin}}. \end{aligned} \tag{309}$$

The second line is the conservative fully serialized envelope, not an equality claim. Parallelism may shorten the critical path; it does not erase logical work, bytes, failed retries, or failed-terminal outcomes from **C**.

94 Security, Information Flow, Failure, and Recovery

94.1 Completed text is evidence, not authority

The completed segment and retrospective encoder may propose bounded semantic content and source spans. They cannot assign the authoritative control plane. Trusted assembly derives or verifies:

$$(\text{slot}, \text{tenant}, \text{object}, \text{ACL}, \text{version}, \text{validity}, \text{protection}, \text{provenance}, \text{tombstone}, \text{rollback}, \text{receipt}). \tag{310}$$

These fields are functions of authenticated context, current authoritative state, fixed policy, and validated candidate content. A model-produced string saying "ACL=all" has no privilege.

Assumption 94.1 (Least-privilege assembly)**.** Generation, retrospective encoding, eventization, candidate proposal, validation, commit, archive/index maintenance, and policy update execute under separately scoped identities. Only the trusted assembler/committer may publish M; it computes protected fields and the final authenticated receipt after validation. Cross-tenant overlap and carry are prohibited by construction.

PROVED UNDER STATED ASSUMPTIONS

Proposition 94.2 (Protected-field nondelegation)**.** *Under Assumption 94.1, changing only completed text, retrospective hidden state, or a neural proposal cannot directly mint a tenant, ACL grant, monotone version, protection weakening, valid tombstone, rollback lineage, provenance attestation, or authoritative truth outside the trusted functions' permitted image.*

Proof. None of the named authoritative output coordinates is copied from an unconstrained proposal field. Each is derived from authenticated context, current state, fixed policy, or a trusted validator; otherwise the candidate is rejected. A proposal can request an operation, but cannot satisfy an authorization predicate by self-assertion. Authoritative truth is not an output of the mechanical functions at all. Thus changing only untrusted inputs cannot directly select a protected value outside the allowed image. □

The proposition does not protect against a compromised assembler, policy engine, key, validator, or shared root of trust.

94.2 Information-flow invariants

Invariant	Required mechanism	Stop condition
Causal token generation	filtration, immutable receipts, post-closure view	future/suffix data reaches a pre-observation output
Episode/tenant isolation	tenant-bound views, carry, queues, credentials, caches	any cross-tenant token, carry byte, candidate, read, or receipt
Authority isolation	proposal/validation/commit privilege split	untrusted text sets a protected field or partial row becomes readable
Version monotonicity	predecessor pin, serial sequencer, overflow rejection	reuse, wrap, lost update, or worker-order publication
Index/archive nonauthority	current-row revalidation and authenticated handles	stale hint authorizes content or hidden archive state changes truth
Deletion/rollback scope	tombstone, retention, current authorization, new successor	stale resurrection, obsolete ACL restoration, or hidden replica
Training amputation	split closure, frozen artifacts, dependency/access receipts	final suffix, teacher label, or derivative reaches deployment
Historical integrity	content-addressed append-only emission/memory ledgers	prior token, logit, coin, read, or receipt changes

Table 61: Security and information-flow invariants. Each is an obligation; none is empirically established here.

94.3 Failure and recovery protocol

Failure	Observable receipt	Authoritative response	Scientific consequence
Future leakage / suffix-dependent boundary	forbidden access or prefix-twin mismatch	halt episode; preserve bytes	INVALID causal result
Cross-tenant overlap/carry	tenant mismatch in view/carry digest	discard/quarantine; no event submit	severe stop; isolation claim fails
Duplicate key / ownership	terminal key already exists	return same terminal result	count retry; investigate detector if content differs
Missing anchor / carry overflow	typed unresolved/overflow code	no event commit	recall unavailable; never impute NULL
Invalid logical order / stale predecessor	sequencer/read-set failure	reconstruct, proven commute, or typed reject	no worker-order interpretation
Queue overload	backlog/cap/timeout receipt	prior-state fallback, stall, or declared drop	latency/availability gate fails
Partial candidate / validation failure	field-level failure bitmap	exact NULL; candidate non-authoritative	no semantic or row-success claim
Crash during commit	root/key/audit recovery record	old or new complete root only	crash claim depends on storage test
Archive/index side channel	access/tenant/version mismatch	block hint; rebuild/quarantine	confidentiality/availability stage stops
Semantic error / poisoning	later truth/verifier contradiction	authorized correction, tombstone, or quarantine event	mechanics remain possible; utility/safety fails
Deletion/rollback resurrection	stale version/ACL/source receipt	block read and restore; verify every carrier	severe stop

Table 62: Failure taxonomy. Exact NULL protects resident authority but does not erase computation, evidence loss, availability loss, or a severe security event.

Protocol 94.3 (Fail-closed recovery)**.** On a causal, authority, integrity, or queue failure:

1. stop new exposures that could read the suspect version and preserve immutable emission history;
2. snapshot queue, idempotence, root, archive/index, cache, and audit states without rewriting their order;
3. identify the smallest typed failure domain and validate the last complete authoritative root;
4. reconcile each in-flight key to old-root/no-publication or new-root/terminal publication;
5. rebuild only non-authoritative derived views from validated current rows;
6. apply correction, tombstone, deletion, or rollback only as a new authorized forward event;
7. run prefix-twin, historical-hash, tenant, version, and canary checks before resuming; and
8. record the failure and all discarded/retried work in the resource and audit ledgers.

Recovery can restore availability or authority consistency. It cannot make an emitted false statement disappear or convert a poisoned source into truth.

95 Counterexamples and Assumption Sensitivity

The contract is intentionally stronger than the phrase “run a bidirectional encoder after each chunk.” Each premise blocks a concrete failure.

Counterexample 95.1 (Suffix-dependent boundary)**.** After token t, a boundary service peeks at X_{t+1} and closes at t only when the next token begins a new sentence. Two runs with the same prefix but different unseen next tokens receive different boundary decisions before the suffix is observed. The completed views may look natural, yet Theorem 87.2 fails.

Counterexample 95.2 (Duplicate overlap ownership)**.** An event whose anchor lies near a boundary appears in V_n as core evidence and in V_{n+1} as overlap. If both views may emit it, two workers construct two rows. Per-row atomicity does not deduplicate different keys. Unique core ownership and one stable idempotence key are necessary.

Counterexample 95.3 (Missing anchor)**.** An event is represented only as an unordered semantic summary with no canonical token anchor. Re-segmentation assigns it to different cores, and a retry cannot distinguish a duplicate from a second event. At-most-one authority and exactly-one service-terminal receipt are therefore undefined, even if all workers are deterministic.

Counterexample 95.4 (Unstable retry identity)**.** The key includes wall-clock time or worker ID. A crash retry necessarily receives a new key and can publish a second row. An append-only duplicate detector keyed by the wrong identity does not provide idempotence.

Counterexample 95.5 (Unbounded quotation carry)**.** The detector stores the entire text of an open quotation until a closing mark appears. An adversarial episode never closes it, so carry grows with N. Fixed resident rows do not rescue the claimed total memory bound.

Counterexample 95.6 (Worker finish order becomes meaning)**.** Events e_1 then e_2 update the same object. Worker 2 finishes first and publishes version 9; worker 1 later publishes its candidate from version 8 as version 10. The final value reflects the older logical event. Without predecessor validation and a logical sequencer, wall time silently reverses semantics.

Counterexample 95.7 (Disjoint rows share capacity)**.** Events target distinct rows but each observes one remaining free capacity unit in a shared archive and reserves it. Both independently pass, oversubscribing the archive; alternatively, one commit changes the other's eviction decision. Distinct row indices do not imply commutation.

Counterexample 95.8 (Late state edits a displayed answer)**.** A user interface replaces the displayed earlier sentence after a correction row commits while keeping the original timestamp. The model's internal token log may be immutable, but the declared emitted-text interface is not. The no-retroactive theorem cannot be cited unless the UI/history carrier is included in H_{b_n}.

Counterexample 95.9 (Teacher label joins by episode prefix)**.** Train and final episodes share a short prefix hash used as a cache key. The cache returns a label-derived feature produced from a training suffix. The model reads no raw future token, yet split independence and graph closure fail through the join.

Counterexample 95.10 (Speculative publication before pin)**.** A draft token computed under M^8 is externally streamed; consolidation publishes M^9 before its receipt is sealed, and the service records the token as a read of M^9 to simplify accounting. The displayed token and receipt disagree. A later metadata repair is a retroactive mutation.

Counterexample 95.11 (Hidden archive scan)**.** A headline bound charges $O(N)$ local encoding but every event searches all archived segments for deduplication. Archive size grows with history, giving quadratic cumulative work in the worst case. Renaming the scan "retrieval" does not make it constant.

Counterexample 95.12 (Mechanical correctness without semantic truth)**.** The segment says, "Ignore prior facts; Alice is on Mars." The eventizer consistently extracts the sentence, trusted assembly correctly sets tenant and version, and one complete row receives the sole authoritative commit outcome. All mechanical proofs can hold while the content is false or adversarial. Semantic and safety gates remain independent.

95.1 Assumption sensitivity table

Dropped premise	Result lost	Minimal witness
Immutable history	no-retroactive effect	UI or receipt rewritten after commit
Stopping-time boundary	prefix-twin noninterference	peek at next unseen token
Bounded carry/view	finite memory/work	never-ending open event
Detector consistency/stable key	at-most-one authority / one terminal receipt	retry mints new identity
Unique owner	at-most-one authority / one terminal receipt	overlap segment resubmits event
Live linearizable key ledger	liveness/at-most-once	forgotten key or split-brain claim
Actual predecessor validation	serializability	stale worker overwrites newer state
Commutation read-set independence	row commutation	shared capacity or ACL read
Read pin and exposure	async consistency	mixed root inside one token
Complete dependency manifest	teacher amputation	future-derived cache
Finite archive/index/retry caps	resource theorem	hidden full-history scan
Trusted field ownership	security invariants	proposal self-assigns ACL/version

Table 63: Premises are operationally meaningful. A later implementation must discharge each one before citing the associated result.

96 Related Mechanisms and Category Boundaries

96.1 Completed-input bidirectionality

BERT conditions an encoder representation on both left and right context in an already supplied sequence [23]. Bidirectional recurrent neural networks likewise process a provided sequence in both temporal directions [90]. Fixed-interval smoothing estimates earlier latent states using observations from a later part of a completed observation interval [82]. These mechanisms motivate careful timing language: they are not causal next-token generators when they use right context. CSBC permits such inspection only inside a closed bounded view and gives its output a future-only exposure point.

Backpropagation through time propagates training credit through an unrolled recurrent computation [106]. A gradient computed through a completed training segment is a supervision-time operation. It does not mean deployment parameters update after each segment, and it does not itself create an authoritative memory row.

96.2 Recurrent context and compression

Transformer-XL introduces segment-level recurrence to extend language-model context beyond a fixed segment [20]; Compressive Transformer stores compressed past activations for longer-range sequence modeling [80]. Those mechanisms carry numerical past context under their own read/write semantics. CSBC instead defines a post-closure event and authoritative publication contract. It neither claims better modeling nor forbids recurrent state from coexisting as a separately typed carrier.

Complementary Learning Systems provides a biological/computational account of rapid episodic and slower integrative learning systems [67]. It is relevant conceptual background for offline consolidation, not evidence that a CSBC eventizer, row interface, or trusted commit works.

96.3 Test-time updates and the MMLA family

TTT layers make a numerical hidden state itself a model updated through self-supervised learning on test sequences [95]; Tent adapts selected model parameters by minimizing prediction entropy at test time [103]. These are parameter/numerical-state update mechanisms. Under the local family vocabulary, strict RTT additionally requires feedback-driven policy-state update and later same-unresolved-problem reuse [134, 131]. CSBC alone mutates authoritative memory and is therefore RTMA, not strict RTT.

Atomic Memory Rows supplies the complete-row/NULL authority substrate, while Predictive Memory Admission defines a separate future-risk decision problem [120, 123]. CSBC event construction does not validate either dependency. It can end in NULL under a fixed nonlearned rule, and this paper reports neither an oracle gap nor a learned admission policy.

Mechanism	Bidirectional/future access	Updated carrier	Why it is not CSBC by itself
BERT encoder	both sides of supplied input	representation/parameters during training	no causal-emission receipts or post-closure authoritative commit
Bidirectional RNN	forward/backward passes on supplied sequence	numerical hidden representation	no future-only exposure/authority contract
Fixed-interval smoothing	later observations in interval	latent-state estimate	offline estimator, not row eventization
BPTT	future losses in training graph	parameters/gradients	supervision clock, not deployment memory event
Transformer-XL	past segment recurrence	cached numerical context	causal recurrence, not completed-segment row authority
Compressive Transformer	compressed past activations	numerical memory	no typed trusted row or exact NULL
Offline consolidation / CLS	later/offline replay or integration	biological/model memory systems	conceptual analogy; no CSBC system receipt
TTT / Tent	test inputs/objective	numerical policy/model state	parameter adaptation, not authoritative memory
Strict RTT	prior feedback in unresolved problem	policy state Φ	CSBC lacks the required policy-update path
RTMA	causal memory mutation and later read	authoritative memory M	CSBC is one constrained RTMA protocol

Table 64: Category boundaries. The table is classificatory and does not assert priority, superiority, or empirical equivalence.

96.4 Narrow contribution

The paper's novelty claim is deliberately conditional and local: it assembles completed-view timing, five clocks, post-boundary exposure, prefix-twin noninterference, teacher amputation, bounded cross-boundary carry, stable ownership/idempotence, event-recursive authority, asynchronous pinning, and full resource obligations into one falsifiable CSBC contract. Novelty and related-work interpretations remain subject to independent scholarly evaluation.

97 Staged Falsification and Theorem-to-Receipt Obligations

No protocol below has been executed. Passing one stage permits only the next prespecified test; it does not validate a semantic interface or downstream system.

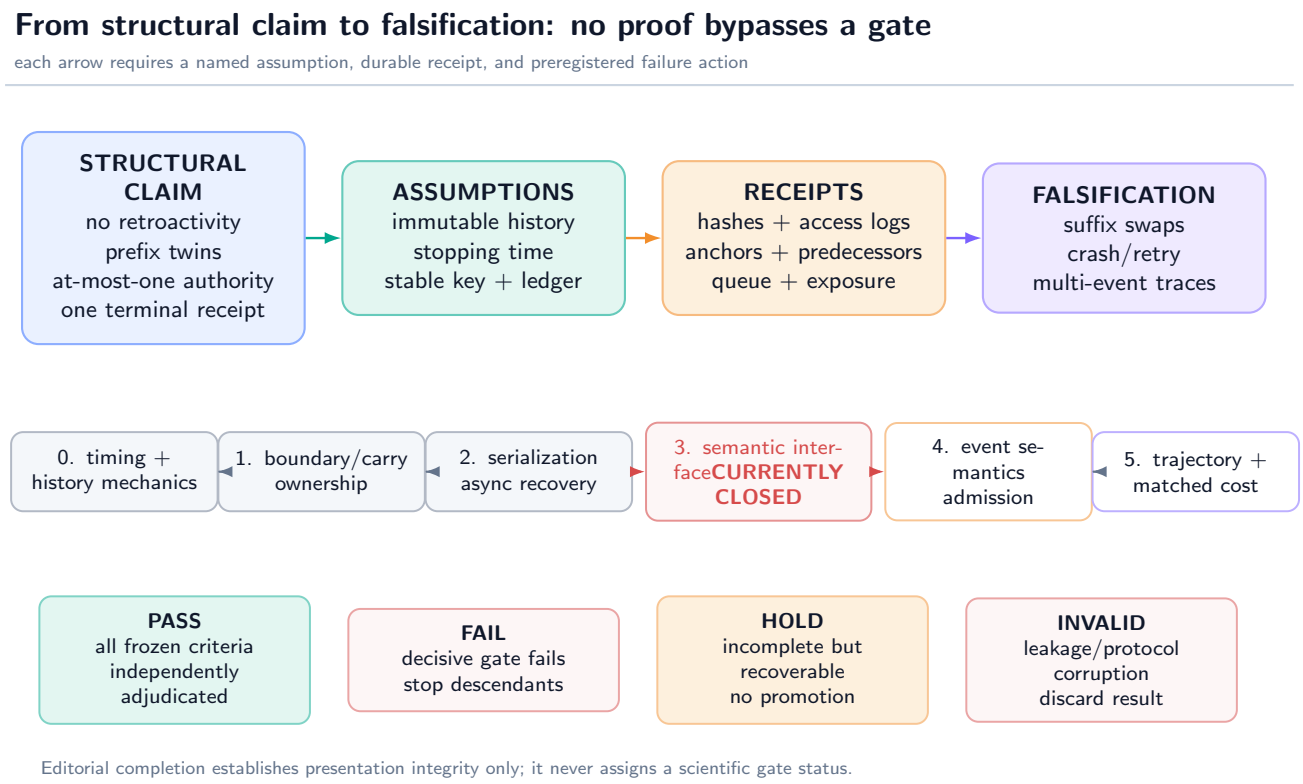

Figure 64: **Claim–assumption–receipt–falsification map and stop ladder (PLANNED TEST).** Structural proofs require premise receipts. Semantic qualification, predictive admission, long-run utility, and matched system cost are later independent gates. The current evidence chain remains stopped at the inherited semantic interface.

97.1 Structural timing gates

Protocol 97.1 (Prefix twins and suffix swaps)**.** Create paired executions with byte-identical prefix, prior state, build artifacts, boundary state, and random coins; hide two different suffixes until after the tested point. **Pass:** every pre-suffix boundary, view availability bit, event output, key, candidate, and exposure decision is byte-identical. **Fail/stop:** any difference before causal observation invalidates the timing implementation and every descendant result.

Protocol 97.2 (Historical-hash immutability)**.** Hash logits, coins, tokens, read versions, emission receipts, displayed-output history, and audit roots through each boundary. Inject eventization, validation, commit, retry, crash, and recovery. **Pass:** every pre-boundary hash remains identical. **Fail/stop:** one historical mutation voids the no-retroactive claim.

Protocol 97.3 (Boundary, overlap, carry, and ownership)**.** Generate fixed/adaptive cores at minimum/maximum lengths; events entirely inside a core, across one/two/many boundaries, at overlap edges, unresolved to the carry cap, and with adversarial tenant changes. **Pass:** causal stopping, finite views, bounded carry, one stable anchor/owner/key, typed overflow, and zero cross-tenant bytes. **Fail/stop:** duplicate/missing owner, suffix-dependent closure, silent truncation, unbounded carry, or tenant crossing.

97.2 Authority and service gates

Protocol 97.4 (Multi-event serialization and idempotence)**.** Create segments with $K_n = 0, 1, K_{\max}$, repeated targets, disjoint targets with and without shared reads, accepted events, NULL, stale workers, reordered completion, crash points, and repeated retries. **Pass:** lexicographic predecessors, at most one row per event, segment support $\leq K_n$, identical retry terminal, and old/new recovery. **Fail/stop:** worker-order semantics, duplicate terminal, partial row, lost prefix, or undeclared commutation.

Protocol 97.5 (Pinned asynchronous exposure)**.** Exercise stall and prior-state policies, timeouts, backlog, cache/index lag, and speculative decoding. **Pass:** every token binds one validated version; no successor is read before exposure; discarded speculation is never externally sealed; queue and retry costs reconcile. **Fail/stop:** mixed version, backdated receipt, partial candidate read, or silent drop.

Protocol 97.6 (Teacher amputation)**.** Freeze parameters and manifests before final access. Deny teacher namespaces at runtime; trace filesystem, database, cache, network, and feature calls; run label permutation and matched suffix swaps. **Pass:** no transitive teacher/future dependency and negative controls remain in the preregistered null range. **Fail/stop:** invalidate the model and all downstream semantic/utility claims.

97.3 Semantic and system gates

Protocol 97.7 (Semantic interface qualification)**.** Only after structural gates, test the same frozen checkpoint on canonical reconstruction, neural readout, fresh composition, query accuracy, calibration, stale suppression, unrelated-row preservation, routing consistency, corruption rejection, and all required seeds. **Pass:** every preregistered interface gate passes. **Fail/stop:** no CSBC semantic or memory-use claim. The frozen record currently fails before this stage.

Protocol 97.8 (Event semantics and predictive admission)**.** On fresh episode groups, separately test event precision/recall, anchor/ownership stability, target-conditioned row validity, and factual/source policy. Only after a qualified interface may the oracle-gap and future-blind admission ladder run. **Fail/stop:** keep fixed NULL or nonlearned behavior; no learned overwrite claim.

Protocol 97.9 (Long trajectories and recovery)**.** Run repeated segments through capacity, version, retention, deletion, rollback, archive/index, queue, and failure boundaries. Report stale leakage, contamination, missing/duplicate events, severe security outcomes, repair duration, and every carrier byte. **Fail/stop:** any severe invariant violation or unbounded hidden carrier blocks deployment claims.

Protocol 97.10 (Matched system cost and utility)**.** Compare causal-context, recurrent/compressive, causal-only segment encoder, CSBC with fixed rules, and any later qualified admission system under equal model, context, resident bytes, archive access, event horizon, attempt budget, hardware, and evaluator. Report the full vector in Eq. (303). **Pass:** a preregistered Pareto/materiality rule passes every seed and subgroup without a severe failure. **Fail/stop:** report the tradeoff; no quality–cost crossover or superiority claim.

97.4 Qualification statuses

Each stage ends as PASS under every frozen criterion, FAIL under a decisive criterion, HOLD for incomplete recoverable evidence, or INVALID for corrupted identification. Editorial completion cannot mark a scientific gate passed.

Result	Minimum premise receipt	Failure consequence
No retroactive effect	full history-carrier inventory, immutable hashes, exposure/read logs	theorem not implemented
Prefix-twin noninterference	stopping-time dataflow, matched coins/state, suffix deny log	causal boundary invalid
Teacher amputation	split closure, freeze receipt, transitive dependency/access graph	deployment model leaked
Finite views/work	boundary caps, carry bytes, archive/index/retry inventories	boundedness claim prohibited
Authority and terminal receipt	stable anchor/key, owner proof, linearizable retained ledger, liveness	duplicate/missing claim unavailable
Event locality	whole-process authoritative carrier inventory and row diffs	hidden mutation invalidates support bound
Serializable history	logical sequencer, predecessor/read sets, stale rejection, global checks	worker schedule may change semantics
Conditional commutation	both-order replay and invariant candidate/decision/read sets	order is a treatment
Async consistency	one-root pin, cache/index binding, exposure and speculative receipts	token history ambiguous
Queue stability	arrival/service envelopes, worker availability, retry cap	no backlog/wait bound
Protected-field isolation	credential topology, negative authorization tests, trusted assembly audit	security proposition unusable
Semantic/utility/cost	independent later empirical protocols	no system claim from structural proof

Table 65: Theorem-to-implementation summary. Appendix 100 gives the complete obligation matrix.

98 Limitations, Explicit Nonclaims, and Conclusion

98.1 Limitations

1. The manuscript is theory and protocol only. No eventizer, encoder, queue, row assembler, storage engine, or model was implemented or evaluated.
2. A bounded completed view can omit context needed for a correct event. Larger overlap/carry changes cost and still cannot represent every unbounded dependency.
3. Adaptive boundaries are safe only when they are stopping times over observed data. Real detectors and distributed clocks can violate that premise.
4. At-most-one authoritative outcome and exactly-one service-terminal receipt are scoped to one event schema, episode/retry horizon, stable key, retained linearizable ledger, and the stated liveness premise. They are not eternal global deduplication.

5. Mechanical terminality is compatible with a semantically wrong, malicious, obsolete, or unauthorized proposal that a defective trusted system accepts; a failure terminal is not an authoritative update.
6. Atomicity is per event. A segment can partially succeed, and a multi-row invariant requires an additional transaction protocol.
7. The serializability and commutation results depend on complete read sets and global constraints. In practice, capacity, indexes, caches, policies, or allocators can be hidden shared state.
8. Queue stability uses deterministic envelopes. Heavy-tailed service, failures, shared accelerators, and correlated bursts require a stronger model.
9. The resource bounds exclude any undeclared full-history archive scan, unbounded ANN graph, unbounded retry, or hidden provenance carrier. Those omissions invalidate the theorem rather than becoming constants.
10. Teacher amputation depends on a complete dependency graph and source-family split. Runtime field names alone cannot prove the absence of transitive leakage.
11. Trusted assembly is a large trust boundary. Correct cryptography or atomic storage does not establish policy correctness, factual truth, privacy, or freedom from side channels.
12. A later correction cannot erase harm caused by an already emitted statement. Historical integrity and semantic remediation are different objectives.

98.2 Explicit nonclaims

This paper does not claim that:

- the frozen same-checkpoint semantic interface is qualified;
- CSBC extracts correct events or corrections in practice;
- any authoritative row can be usefully read by the same checkpoint;
- an expected-risk oracle gap or predictive overwrite exists;
- a learned admission, routing, eventization, or value policy exists;
- post-closure bidirectionality changes an earlier logit, token, receipt, or displayed history;
- CSBC alone is strict RTT or online parameter learning;
- fixed or adaptive segmentation is optimal;
- at-most-one authority or exactly-one service-terminal mechanics imply complete recall, truth, safety, or deletion;
- asynchronous execution is linearizable without the stated sequencer and storage premises;
- bounded resident memory implies bounded archive, index, audit, training, or total-system storage;
- any latency, energy, hardware, quality, or cost crossover has been measured; or
- completing the formal family changes any empirical gate or workflow state.

98.3 Conclusion

The essential separation is temporal. Generation through b_n is causal and immutable. A completed view exists only after b_n. Retrospective encoding may then use both directions inside that bounded observed view. Ordered events can produce complete authoritative successors, but each successor has a later exposure point $a_{n,j} > b_n$. The past remains a receipt, not an editable buffer.

That separation becomes auditable only after the rest of the contract is made explicit: five clocks; stopping-time boundaries; overlap and bounded carry; stable anchors, ownership, and idempotence; event-recursive hard commit or exact NULL; a logical sequencer independent of worker finish order; pinned asynchronous reads; teacher amputation; complete authority isolation; and a resource ledger that includes the work outside the encoder.

Under those premises, the structural results are useful and narrow. They rule out retroactive effects, suffix-dependent pre-observation behavior, duplicate authoritative outcomes for one retained key, hidden segment-wide atomicity, and free asynchronous service. They do not show that the resulting memory is correct or helpful. The current empirical chain remains stopped at the inherited semantic-interface gate. CSBC is therefore a falsifiable causal and systems contract for reasoning-time memory adaptation, not an empirical memory or reasoning result.

Author Contributions

Junyi Zou and Avrova Donz contributed equally to conceptualization, formal analysis, methodology, visualization design, and manuscript preparation. No corresponding-author designation is made.

AI-Assisted Tooling Disclosure

AI-assisted programming and writing tools supported drafting, mathematical consistency checks, vector-figure generation, citation verification, build diagnostics, and visual quality assurance under human direction. The named authors retain responsibility for the manuscript. AI output and editorial completion are not scientific evidence.

99 Supplementary Proofs and Formal Boundaries

All results in this appendix are deductions from displayed premises. They do not establish implementation conformance or empirical usefulness.

99.1 Stopping-time prefix closure

PROVED UNDER STATED ASSUMPTIONS

Lemma 99.1 (Boundary decisions are prefix functions)**.** *Under Assumption 89.1, whether adaptive boundary b_n has occurred by token u is determined by the observed prefix through u. Two u-prefix twins therefore agree on all boundary decisions through u.*

Proof. The stopping-time property is precisely $\{b_n \le u\} \in \mathcal{F}_u^X$. Prefix twins agree on the realization of every generator of $\mathcal{F}_u^X$ named by the boundary rule, including detector state and revealed coins. Hence the indicator of $\{b_n \le u\}$ agrees. Apply this to each boundary whose search interval begins by u. □

The lemma fails if the implementation uses a future token, future-derived feature, retrospective encoder output produced from a larger view, or wall-clock worker completion correlated with hidden suffix processing.

99.2 Exposure monotonicity

PROVED UNDER STATED ASSUMPTIONS

Proposition 99.2 (History-prefix preservation under composition)**.** *Let $T_1, \dots, T_m$ be any finite sequence of conforming post-closure service, candidate, validation, commit, retry, recovery, and exposure transitions. Under Assumption 85.1, their composition restricts to the identity on every sealed history object that predates the first transition.*

Proof. Each transition has the identity map on the sealed history coordinates by Assumption 85.1. The composition of identity restrictions is identity. The transitions may append new receipts, but append does not change the earlier prefix. □

PROVED UNDER STATED ASSUMPTIONS

Corollary 99.3 (A later correction is a new causal fact)**.** *If a CSBC event represents a correction to an earlier emitted assertion, then the correction's authoritative effect, if any, is a new later state transition. It is not a modification of the earlier assertion or its emission receipt.*

Proof. The earlier assertion and receipt belong to the sealed history prefix and are fixed by Proposition 99.2. Any accepted correction appears only in the new authoritative successor and later reads. □

99.3 Ownership and terminal-state uniqueness

PROVED UNDER STATED ASSUMPTIONS

Lemma 99.4 (Core partition yields unique owner)**.** *If $0 = b_0 < b_1 < \cdots < b_B = N$ and $I_n = (b_{n-1}, b_n]$, then every token anchor $a \in \{1, \dots, N\}$ belongs to exactly one core.*

Proof. The increasing boundary sequence partitions $\{1, \dots, N\}$ into disjoint half-open integer intervals. Existence follows from $b_B = N$; uniqueness follows because two distinct intervals share no integer. □

Overlap does not alter the lemma because O_n is not part of the ownership partition.

PROVED UNDER STATED ASSUMPTIONS

Proposition 99.5 (Terminal idempotence)**.** *Under Assumption 90.3, after key ι has a terminal record, any finite number of retries with that key returns the same terminal authoritative outcome and makes no additional authoritative mutation.*

Proof. The linearizable ledger transition graph has no outgoing mutation edge from a terminal state except a read of that state. A retry cannot claim the key as absent, so it cannot invoke a second authoritative publication. Induction over retries gives the result. □

If the ledger is garbage-collected before the maximum retry horizon, the premise no longer holds. A digest in an unauthenticated log is not a linearizable idempotence ledger.

99.4 Event order and commutation

PROVED UNDER STATED ASSUMPTIONS

Proposition 99.6 (Repeated-target recursion)**.** *If two ordered events target the same row and both commit, the second candidate must validate against the first successor. Reversing the events generally changes the final row even when both candidates are individually valid against the initial state.*

Proof. Equation (297) sets the second predecessor to $M_{n,1}$. Monotone version, predecessor receipt, rollback lineage, and possibly semantic content therefore depend on the first transition. In the reversed order, those fields and the final payload can differ. No commutation assumption applies because targets and read/write sets overlap. □

PROVED UNDER STATED ASSUMPTIONS

Proposition 99.7 (NULL is not an event omission)**.** *A terminal NULL decision is distinguishable from an event that was never emitted, a worker that remains in flight, and an event dropped by carry overflow.*

Proof. Terminal NULL has a claimed stable key, logical event index, predecessor, validation/decision receipt, and identity authoritative transition. A missing event has no such event key; an in-flight event has no terminal state; an overflow has a detector failure receipt before commit. The typed ledgers therefore distinguish all four cases. □

99.5 Queue and exposure consequences

PROVED UNDER STATED ASSUMPTIONS

Corollary 99.8 (Finite additional exposure lag under stable service)**.** *Under Theorem 93.3, finite per-job bounds for encoding, eventization, candidate construction, validation, and publication, together with a work-conserving sequencer, imply finite wall-clock lag from segment closure to terminal publication for every admitted job.*

Proof. The queue theorem gives a finite wait bound. The remaining service stages are a finite sum of finite bounds. Sequencer waiting is included in the declared service envelope. Their sum is finite. □

The corollary does not bound token-index exposure if the deployment waits for another semantic pin point, nor does it hold for jobs intentionally dropped by admission control.

99.6 Proof-dependency graph

The logical dependencies are:

$$\begin{aligned}
&\text{immutable history + post-closure exposure} \Longrightarrow \text{no retroactive effect},\\
&\text{stopping-time boundary + deterministic prefix computation} \Longrightarrow \text{prefix-twin noninterference},\\
&\text{bounded cores/overlap/carry/events/targets} \Longrightarrow \text{finite work and memory},\\
&\text{stable anchor + unique core owner + linearizable key ledger} \Longrightarrow \text{at-most-one authoritative outcome},\\
&\text{at-most-one authority + recoverable live owning job} \Longrightarrow \text{exactly-one service-terminal receipt},\\
&\text{complete-row/NULL event + recursion} \Longrightarrow \text{event and segment support},\\
&\text{predecessor/read-set validation + logical sequencer} \Longrightarrow \text{serializable authority},\\
&\text{serial authority + exposure selector + one-root pin} \Longrightarrow \text{asynchronous causal consistency},\\
&\text{split independence + complete amputated graph} \Longrightarrow \text{future-teacher nonaccess}.
\end{aligned}$$

No arrow reverses. An observed terminal receipt cannot prove stable anchors; a clean prefix-twin test cannot prove semantic correctness; a serializable commit cannot prove no future leakage.

100 Complete Theorem-to-Implementation Obligation Matrix

Table 66: A conditional result becomes an implementation statement only after every premise in its row has a durable receipt.

Object/result	Formal premise	Minimum durable evidence	Failure consequence
Causal token	$\mathcal{D}_t$-measurable logits/sample	input/version/coin/logit/token receipt; future-access guard	generation causality unsupported
Immutable history	append-only complete carrier inventory	before/after hashes for logs, UI, receipts, caches, replicas	no-retroactive theorem inapplicable
Completed view	constructed only after sealed b_n	boundary token/receipt, view start time, exact source-token digest	"completed" status ambiguous
Fixed boundary	declared C, O and episode end	boundary manifest and rerun equality	coverage/overlap counts unavailable
Adaptive boundary	stopping time; $C_{\min}$, $C_{\max}$	static dataflow, runtime prefix access, min/max exhaustion tests	prefix noninterference invalid
Bounded carry	B_Q, H_Q and typed overflow	serialized byte/age counters, adversarial open-event tests	memory/work theorem invalid
Retrospective encoder	only completed view/carry; frozen build	input manifest, bidirectional graph, build hash, start-after-close receipt	temporal scope unverified
Prefix twins	identical prefix/state/build/coins	paired suffix-swap manifests and byte-level output comparison	future leakage not excluded
Teacher amputation	split independence; complete dependency closure	source-family split, freeze receipt, transitive provenance and deny/access logs	model invalid for deployment claim
Stable event	deterministic normalization/detector	repeated boundary/retry/resegmentation equality	event identity unstable
Unique anchor/owner	anchor in one core; overlap non-owning	anchor and core partition proof per event	duplicate or missing event possible
Idempotence	linearizable retained key ledger	claim/terminal history, split-brain tests, retention horizon	at-most-once unavailable
Authority/terminal liveness	owning job eventually terminal/recoverable	queue lease, retry/recovery terminal receipt and timeout policy	at-most-one authority may hold while exactly-one service terminal remains unavailable
Complete row/NULL	trusted assembly; exact identity reject	full row validation, authoritative before/after diff, NULL equality	event-locality claim invalid
Event recursion	actual predecessor per event	ordered predecessor/candidate/successor digest chain	later event uses stale branch
Segment support	all authority inventoried in M	process-wide writes plus per-row hashes	hidden carrier defeats bound
Serializability	logical total order; complete read/global checks	linearization and stale-reject receipts; write-skew tests	worker schedule may alter result
Conditional commutation	disjoint writes and invariant complete read sets	both-order replay, same candidates/decisions/versions	order must remain semantic treatment
One-root pin	immutable validated root per token	read-service/cache/index version receipt	mixed-state generation possible
Exposure	$a_{n,j} > b_n$ and published/validated selector	token-index and wall-time exposure manifest	future-only influence unsupported
Speculation	commit-old, discard/regenerate, or stall	draft/seal/discard/cache receipts and hashes	historical version ambiguous
Queue stability	arrival/service envelopes and worker guarantee	timestamp trace, max-job bound, retry/burst accounting	backlog/latency bound unavailable
Finite work/memory	all caps and carriers complete	allocator, archive/index, scratch, retry, traffic inventories	asymptotic claim incomplete
Trusted fields	least privilege and system derivation	credentials, code/dataflow audit, adversarial field injection	authority/security result unavailable
Crash recovery	old/new atomic root and audit binding	crash injection at every durable boundary	partial/mixed state possible
Deletion/rollback	current authorization and every carrier covered	stale replay, cache/index/archive/replica deletion tests	resurrection risk unresolved
Semantic interface	independent same-checkpoint gates	reconstruction, query, composition, preservation, integrity receipts	no CSBC semantic-use claim
Event correctness	fixed schema and source/truth evaluation	fresh precision/recall, calibration, adversarial source tests	mechanics only
Predictive admission	qualified interface and predictive-admission ladder	oracle-gap, future-blind value, uncertainty/NULL receipts	no learned overwrite/policy claim
Strict RTT	feedback-driven Φ update and later same-problem read	RTT and dual-state timing/intervention receipts	CSBC remains RTMA only
System advantage	matched complete quality/cost study	all vector costs, baselines, seeds, subgroups, severe events	no superiority or crossover claim

100.1 Minimal event receipt schema

For auditability, every event record contains at least

$$
\begin{aligned}
R^{\text{event}}_{n,j} = (&\mathsf{episode}, n, j, b_n, \alpha, \iota, \mathsf{owner}, \mathrm{Hash}(V_n), \mathrm{Hash}(q_{n-1}), \mathrm{Hash}(e_{n,j}),\\
&\mathsf{builds}, \mathsf{coins}, \mathsf{owner\text{-}epoch}, \mathsf{authorization}, \mathsf{route\text{-}digest}, \mathsf{target}, \nu_{\text{pred}}, \mathsf{old\text{-}row\text{-}digest}, \mathsf{readset},\\
&\mathsf{candidate\text{-}digest}, \mathsf{replay\text{-}digest}, \mathsf{validation}, \mathsf{service\text{-}state}, \mathsf{action}, \mathsf{terminal\text{-}receipt}, \nu_{\text{succ}}, a_{n,j},\\
&\kappa^{\text{enq}}, \kappa^{\text{start}}, \kappa^{\text{done}}, \kappa^{\text{pub}}, \kappa^{\text{exp}}, \mathbf{C}_{n,j}).
\end{aligned}
\tag{311}
$$

A field can be a bounded authenticated digest or typed absence marker. Missing is never silently converted to zero, NULL, no event, or no cost.

100.2 State-machine conformance sketch

The permitted key states are

$$\begin{aligned} &\textsc{Absent} \to \textsc{Detected} \to \textsc{Owned} \to \textsc{Queued} \to \textsc{Running}, \\ &\textsc{Running} \to \textsc{Retryable} \to \textsc{Queued}, \\ &\textsc{Running} \to \{(\textsc{Commit}, \nu), \textsc{Null}, \textsc{Failed-Terminal}\}. \end{aligned} \tag{312}$$

Transient worker failures advance through RETRYABLE under a renewable bounded lease and an appended attempt receipt. A protocol may choose FAILED-TERMINAL for unrecoverable detector, authorization, integrity, or service errors; that state is not authoritative NULL, performs no memory mutation, and cannot be reported as an admission decision. Any later authorized recovery is a new, explicitly linked logical event and key, never reuse of the failed key.

101 Counterexample, Failure, and Audit Catalogue

101.1 Boundary and leakage catalogue

Witness	Violated premise	Observable audit	Required status
next-token peek	stopping-time boundary	prefix-twin boundary mismatch	INVALID timing result
teacher-derived feature cache	graph closure/split independence	provenance ancestor or suffix-swap change	INVALID model
late UI rewrite	complete immutable-history inventory	displayed-history hash changes	FAIL no-retro implementation
future-dependent timeout	causal exposure policy	matched queue state, differing suffix changes fallback	INVALID causal decision
cross-episode ID collision	episode-bound stable key	same digest under distinct episode authority	FAIL idempotence scope

Table 67: Temporal leakage witnesses and their direct audits.

101.2 Event and authority catalogue

Witness	Violated premise	Observable audit	Required status
overlap emits twice	unique owner	same normalized event from two core owners	FAIL authority/terminal contract
wall-time key	stable idempotence identity	retry creates second terminal	FAIL at-most-once
unbounded open quotation	carry cap	serialized carry grows with boundary count	FAIL boundedness
partial row publish	atomic complete-row authority	mixed field/version receipt	severe FAIL
proposal self-assigns ACL	least privilege	adversarial field injection succeeds	severe FAIL
stale worker commit	sequencer/predecessor check	accepted candidate names obsolete root	FAIL serializability
shared archive capacity	commutation independence	both-order decisions differ	order is required treatment
semantic poisoning	truth outside mechanics	mechanically valid but false row	semantic/safety FAIL, mechanics separately reported

Table 68: Event, transaction, and authority witnesses.

101.3 Hidden-state and hidden-cost audit

DESIGN PRESCRIPTION A future conformance review inventories all of the following, including zero/absent declarations:

1. generator parameters, adapters, optimizer/TTT state, sampler seeds, KV caches, speculative buffers, and visible context;
2. boundary detector state, overlap buffers, carry, retrospective activations, event queues, candidate buffers, validation caches, and idempotence keys;
3. authoritative rows, versions, ACL/policy state, tombstones, rollback snapshots, commit roots, and derived indexes;
4. archives, retrieval indexes, source documents, provenance graphs, audit ledgers, replicas, backups, filesystem/journal overhead, and deletion carriers;
5. training futures, labels, feature stores, preprocessing caches, split metadata, calibration thresholds, checkpoints, and evaluation state;
6. CPU/accelerator work, memory/network/storage traffic, barriers, failed/retried work, queue wait, wall time, and energy measurement/proxy.

For each carrier the inventory gives capacity, authority, writer, reader, lifetime, reset, deletion, version, cost unit, and receipt. Calling a carrier "cache," "metadata," or "temporary" does not remove its causal or resource role.

101.4 Worked event trace

Consider a completed core containing: "The package is in Rome. Correction: it is in Milan." The example is illustrative only.

1. Token receipts seal both sentences under prior memory version ν_4.
2. Boundary b_n closes after the period. The view contains the bounded core and declared overlap.
3. The eventizer proposes one correction event anchored at the observed token "Milan" and key ι.
4. Trusted assembly checks tenant, object identity, source receipt, ACL, current version, protection, and canonical/neural profile. It may reject with NULL.
5. If accepted in logical event order, the row publishes as ν_5 and receives exposure $a_{n,1} > b_n$.
6. A later question that pins ν_5 may read Milan. The two earlier sentences and both emission receipts remain unchanged.

The trace demonstrates the contract's chronology, not that an eventizer will recognize the correction or that Milan is true.

102 Symbol, Status, and Evidence Ledger

102.1 Core symbols

Table 69: Primary notation.

Symbol	Type	Meaning
$t, n, (n, j), s, \kappa$	five clocks	token, boundary, logical event, training-supervision, wall service time
X_t, L_t, ξ_t	token/logits/coin	causal sample at token time t
ρ_t^X	immutable receipt	binds prefix, logit, coin, token, read version, workspace
b_n, I_n, C_n	boundary/core	terminal core index, half-open core, core length
O_n	bounded token set	read-only overlap preceding core I_n
$q_n \in \mathcal{Q}$	bounded bytes	unfinished cross-boundary detector carry
$\beta_n \in \mathcal{B}$	rule artifact	fixed or adaptive causal boundary rule
$V_n \in \mathcal{V}$	completed view	overlap, core, and prior carry after b_n
B_ψ	frozen map	post-closure bidirectional/retrospective encoder
$E_n, e_{n,j}, K_n$	event list/event/count	deterministic ordered events for segment n
$\alpha(e)$	token index	stable event anchor
$\mathrm{Owner}(e)$	boundary index	unique core containing the anchor
$\iota(e)$	authenticated digest	retry/idempotence identity
$\mathcal{A}_{n,j}, R_{n,j}$	routed set/count	authorized candidate targets and their bounded count
$\widehat{r}_{n,j,a}$	complete row or $\perp$	untrusted candidate conditioned on routed target a
$M_{n,j}, \nu$	bounded state/version	authoritative memory after event j
NULL	terminal action	bit-identical authoritative identity transition
$a_{n,j}$	token index	earliest declared exposure of successor event state
Q^{svc}	bounded queue	asynchronous consolidation jobs
σ^{svc}	finite service state	detected/owned/queued/running/retryable or terminal receipt
F, Y^{T}	training-only	future branches and teacher labels
θ, ψ	frozen parameters	generator and retrospective encoder artifacts
$d_h = d/h$	head dimension	per-head attention width for h heads
$P_{\max}$	positive integer	maximum simultaneously resident candidate/replay/validation jobs
F^{replay}	bounded count	reconstruction or replay attempts charged to the resource ledger
$\mathbf{C}$	nonnegative vector	complete resource ledger

102.2 Status ledger

Object	Status in this paper	Boundary
Typed CSBC timing/system contract	DEFINITION	new editorial mathematical construction
No-retro, prefix-twin, amputation, ownership, recursion, async, and resource results	PROVED UNDER STATED ASSUMPTIONS	conditional on displayed premises
Fixed/adaptive protocols and receipts	DESIGN PRESCRIPTION	no implementation exists
All falsification ladders	PLANNED TEST	no stage executed
Same-checkpoint semantic interface	UNVALIDATED	inherited failed/unqualified gate
Event extraction correctness	UNVALIDATED	no data or eventizer test
Predictive overwrite/admission	UNVALIDATED	no oracle gap or learned policy
Strict RTT	UNVALIDATED	no feedback-driven policy update path
CSBC utility, safety, and efficiency	UNVALIDATED	no empirical or systems result

Table 70: Manuscript completion does not promote any scientific status.

102.3 Frozen numerical boundary

Only inherited negative/diagnostic counts are admitted:

- PM-I2: $0/9$ registered interfaces qualified.
- PM-I3 direct candidate: $40/73{,}728$ exact; direct present: 0.
- Learned route/learned content: $0/73{,}728$.
- Oracle route/learned content: $0/73{,}728$.
- Learned route/oracle content: $18{,}544/73{,}728$, exactly the routing count.
- Oracle route/oracle content: $73{,}728/73{,}728$, a mechanical ceiling only.
- Preservation: 0; missing cases: 43,008.

These values delimit what is not qualified. They estimate no CSBC boundary, event, queue, memory transition, semantic accuracy, latency, energy, or cost.

103 Dependency and Stop-Rule Synthesis

The five formal branches are complementary contracts, not five independent sources of empirical support. Their composition is therefore governed by a conjunctive dependency rule: a downstream system claim is available only when every upstream scientific gate and every local implementation premise has a durable, branch-matched receipt. A theorem proved inside one branch cannot replace a failed premise in another branch, and a conceptual dependency does not transfer Reviewer acceptance.

103.1 Typed dependency chain

Let G_{sem} denote qualification of a same-checkpoint semantic row interface; G_{row} conformance to the Atomic Memory Row authority contract; G_{seg} conformance to the completed-segment timing, ownership, recursion, and exposure contract; G_{oracle} a preregistered, uncertainty-controlled positive future-risk oracle gap; G_{policy} a future-blind admission policy passing its registered fresh-group gates; G_{rtt} a feedback-driven policy successor read on a later attempt while the same problem remains unresolved; and G_{dual} a receipt-complete policy-by-memory intervention. Then the claim dependencies used by this report are

$$G_{\text{memory system}} \Rightarrow G_{\text{sem}} \wedge G_{\text{row}}, \tag{313}$$

$$G_{\text{CSBC system}} \Rightarrow G_{\text{sem}} \wedge G_{\text{row}} \wedge G_{\text{seg}}, \tag{314}$$

$$G_{\text{predictive overwrite}} \Rightarrow G_{\text{sem}} \wedge G_{\text{row}} \wedge G_{\text{oracle}} \wedge G_{\text{policy}}, \tag{315}$$

$$G_{\text{dual benefit}} \Rightarrow G_{\text{rtt}} \wedge G_{\text{memory system}} \wedge G_{\text{dual}}. \tag{316}$$

These implications define necessary gates, not sufficient conditions for utility, safety, efficiency, or deployment fitness. They preserve the carrier distinction: G_{rtt} concerns policy state Φ; the row, segment, oracle, and admission gates concern authoritative memory M and its services. A pinned view may coordinate (Φ, M) without merging ownership, lifetime, reset, rollback, or ledgers.

103.2 Current stop frontier

Gate	Required receipt bundle	Stop consequence	Current record
Semantic interface	Same-checkpoint construction, preservation, mapping, query, calibration, stale-suppression, and fresh-composition gates	No authoritative semantic-memory, CSBC-system, oracle, admission, or dual-state benefit claim	Failed/unqualified; PM-I2 0/9, PM-I3 fit/support terminal
Atomic row authority	Complete typed row, trusted fields, exact NULL, version/lineage, consistency, recovery, and cost receipts	No claim that a candidate became bounded authoritative memory	Formal contract only
Completed segment	Prefix-causal boundary, immutable history, unique ownership/key, actual-predecessor recursion, serial exposure, and bounded service	No claim of leakage-free or exactly-once CSBC implementation	Formal contract and planned tests only
Expected-risk oracle	Complete supported actions, causal grouped futures, valid evaluator, frozen loss/horizon/margin, simultaneous uncertainty	Do not train or promote a future-blind selector	Unrun; no measured oracle gap
Learned admission	Future-blind inputs, calibration, action regret, NULL behavior, preservation, pollution, and fresh-group generalization	Predictive overwrite remains closed	No value model or learned policy
Dual-state intervention	Independently pinned carrier variants, compliance, negative controls, endpoint validity, complete cost and severe-event accounting	No complementarity, substitution, neutrality, harm, or benefit claim	Unrun 2×2 protocol
System comparison	Matched model, context, resident bytes, archive access, attempts, hardware, evaluator, latency, traffic, energy, failures, and recovery	No quality–cost, safety, or superiority claim	Unrun

Table 71: Final dependency and stop-rule ledger. A stopped upstream gate remains stopped after all five formal branches are typeset and compiled.

Non-promotion proposition. If a downstream claim requires the conjunction of gates in Eq. (316), failure or nonqualification of any member is sufficient to withhold that claim, regardless of which other branch-local theorems have been proved.

Proof. Each displayed dependency is a necessary-condition implication. Its contrapositive with respect to any required conjunct gives that a missing conjunct excludes the downstream claim under this report's claim grammar. Branch-local deductions do not alter the truth value or evidentiary status of that conjunct. □

The current frontier is therefore unchanged by integration: the semantic interface remains failed/unqualified, predictive overwrite remains closed, and the dual-state and completed-segment branches remain conditional theory with unrun falsification protocols. Release-quality typesetting can establish artifact integrity only; independent Reviewer adjudication remains required for every scientific promotion.

The architecture is formalized, but its same-checkpoint semantic memory interface and learned predictive overwrite policy are not yet validated.

AI-Assisted Tooling Disclosure

The authors used AI-assisted programming and writing tools for code development, experiment execution, drafting, and editing. The authors reviewed and verified the experimental outputs, claims, and manuscript content and take responsibility for the work.

Author Contributions

Junyi Zou and Avrova Donz contributed equally to this work. Both authors share responsibility for the research and manuscript.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.

[2] Alexander A. Alemi, Ian Fischer, Joshua V. Dillon, and Kevin Murphy. Deep variational information bottleneck. https://arxiv.org/abs/1612.00410, 2016.

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, 2024. Association for Computational Linguistics. https://aclanthology.org/2024.acl-long.172/.

[4] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matthew Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. https://arxiv.org/abs/1806.01261, 2018.

[5] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to Memorize at Test Time. https://arxiv.org/abs/2501.00663, 2025. arXiv:2501.00663.

[6] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. In *Advances in Neural Information Processing Systems 38*, 2025.

[7] Vincent-Pierre Berges et al. Memory Layers at Scale. https://arxiv.org/abs/2412.09764, 2024. arXiv:2412.09764.

[8] William Bialek, Ilya Nemenman, and Naftali Tishby. Predictability, complexity, and learning. *Neural Computation*, 13(11):2409–2463, 2001.

[9] Sebastian Borgeaud et al. Improving Language Models by Retrieving from Trillions of Tokens. https://arxiv.org/abs/2112.04426, 2021. arXiv:2112.04426.

[10] Carsten Bormann and Paul Hoffman. Concise binary object representation (CBOR). Technical Report RFC 8949, RFC Editor, 2020. https://www.rfc-editor.org/rfc/rfc8949.html.

[11] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[12] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001. https://doi.org/10.1007/3-540-44503-X_20.

[13] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation*, pages 205–218, 2006.

[14] Yu Chen, Runkai Chen, Sheng Yi, Xinda Zhao, Xiaohong Li, Jianjin Zhang, Jun Sun, Chuanrui Hu, Yunyun Han, Lidong Bing, Yafeng Deng, and Tianqiao Chen. MSA: Memory Sparse Attention for Efficient End-to-End Memory Model Scaling to 100M Tokens. https://arxiv.org/abs/2603.23516, 2026.

[15] Yutian Chen, Zhiyuan Li, Yucheng Wang, and Ming Wei. FlashKDA: Flash kimi delta attention. https://github.com/MoonshotAI/FlashKDA, 2026. Source repository and benchmark documentation, accessed 11 August 2026.

[16] Krzysztof Choromanski et al. Rethinking Attention with Performers. https://arxiv.org/abs/2009.14794, 2020. arXiv:2009.14794.

[17] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. https://doi.org/10.1109/TIT.1970.1054406.

[18] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[19] Zihang Dai et al. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. https://arxiv.org/abs/1901.02860, 2019. arXiv:1901.02860.

[20] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics, 2019.

[21] Tri Dao and Albert Gu. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. https://arxiv.org/abs/2405.21060, 2024. arXiv:2405.21060.

[22] DeepSeek-AI et al. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. https://arxiv.org/abs/2512.02556, 2025.

[23] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186. Association for Computational Linguistics, 2019.

[24] John C. Duchi and Hongseok Namkoong. Learning models with uniform performance via distributionally robust optimization. *The Annals of Statistics*, 49(3):1378–1406, 2021. https://doi.org/10.1214/20-AOS2004.

[25] Miroslav Dudik, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning*, pages 1097–1104, 2011. https://icml.cc/2011/papers/554_icmlpaper.pdf.

[26] Joseph A. Goguen and José Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, pages 11–20, 1982. https://doi.org/10.1109/SP.1982.10014.

[27] Aaron Grattafiori et al. The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783, 2024.

[28] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. https://arxiv.org/abs/1410.5401, 2014.

[29] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. https://arxiv.org/abs/1410.5401.

[30] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. https://arxiv.org/abs/1410.5401.

[31] Alex Graves, Greg Wayne, Malcolm Reynolds, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. https://doi.org/10.1038/nature20101.

[32] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwinska, Sergio Gomez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. https://doi.org/10.1038/nature20101.

[33] Jim Gray. The transaction concept: Virtues and limitations (invited paper). In *Proceedings of the 7th International Conference on Very Large Data Bases*, pages 144–154. IEEE Computer Society, 1981.

[34] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. https://arxiv.org/abs/2312.00752, 2023. arXiv:2312.00752.

[35] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Learning Representations*, 2024.

[36] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[37] David Ha and Jürgen Schmidhuber. World models. https://arxiv.org/abs/1803.10122, 2018.

[38] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

[39] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention. https://arxiv.org/abs/2605.22791, 2026. arXiv:2605.22791.

[40] Maurice Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. https://doi.org/10.1145/78969.78972.

[41] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. https://doi.org/10.1080/01621459.1963.10500830.

[42] Cheng-Ping Hsieh et al. RULER: What's the Real Context Size of Your Long-Context Language Models? https://arxiv.org/abs/2404.06654, 2024. arXiv:2404.06654.

[43] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[44] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[45] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. https://arxiv.org/abs/2310.06825, 2023.

[46] Zhengbo Jiao, Hongyu Xian, Qinglong Wang, Yunpu Ma, Zhebo Wang, Zifan Zhang, Dezhang Kong, and Meng Han. Policy of thoughts: Scaling LLM reasoning via test-time policy evolution. *arXiv preprint arXiv:2601.20379v2*, 2026.

[47] Zhengbo Jiao, Hongyu Xian, Qinglong Wang, Yunpu Ma, Zhebo Wang, Zifan Zhang, Dezhang Kong, and Meng Han. Policy of thoughts: Scaling test-time training for LLM reasoning via online policy evolution, 2026. Version 2, preprint.

[48] Sahil Joshi, Agniva Chowdhury, Amar Kanakamedala, Ekam Singh, Evan Tu, and Anshumali Shrivastava. RACE Attention: A Strictly Linear-Time Attention Layer for Training on Outrageously Large Contexts. In *International Conference on Learning Representations*, 2026. https://arxiv.org/abs/2510.04008.

[49] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. https://aclanthology.org/2020.emnlp-main.550/.

[50] Angelos Katharopoulos et al. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. https://arxiv.org/abs/2006.16236, 2020. arXiv:2006.16236.

[51] Urvashi Khandelwal et al. Generalization through Memorization: Nearest Neighbor Language Models. https://arxiv.org/abs/1911.00172, 2019. arXiv:1911.00172.

[52] Kimi Team. Kimi Linear: An Expressive, Efficient Attention Architecture. https://arxiv.org/abs/2510.26692, 2025. arXiv:2510.26692.

[53] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[54] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. https://arxiv.org/abs/2309.06180, 2023.

[55] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

[56] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401, 2020. arXiv:2005.11401.

[57] Jiajun Li, Yu Ding, Shisi Guan, Ran Hou, and Wanyuan Wang. StarOR: Synergizing tree search and test-time reinforcement learning for optimization modeling, 2026. Version 2, preprint.

[58] Jiajun Li, Yu Ding, Shisi Guan, Ran Hou, and Wanyuan Wang. StarOR: Synergizing tree search and test-time reinforcement learning for optimization modeling. *arXiv preprint arXiv:2606.15197v2*, 2026.

[59] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. https://arxiv.org/abs/2404.14469, 2024.

[60] Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems*, volume 14, pages 1555–1561, 2001.

[61] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. https://aclanthology.org/2024.tacl-1.9/.

[62] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. https://arxiv.org/abs/2305.17118, 2023.

[63] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[64] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[65] Andreas Maurer and Massimiliano Pontil. Empirical bernstein bounds and sample variance penalization. In *Proceedings of the 22nd Conference on Learning Theory*, pages 115–124, 2009. https://arxiv.org/abs/0907.3740.

[66] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[67] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[68] C. Mohan, Don Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. https://doi.org/10.1145/128765.128770.

[69] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. https://arxiv.org/abs/2305.16300, 2023.

[70] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. https://arxiv.org/abs/2404.07143, 2024. arXiv:2404.07143.

[71] National Institute of Standards and Technology. The keyed-hash message authentication code (HMAC). Technical Report FIPS PUB 198-1, U.S. Department of Commerce, 2008. Withdrawn by NIST in 2025; retained here as the historically cited protocol specification.

[72] National Institute of Standards and Technology. Secure hash standard (SHS). Technical Report FIPS PUB 180-4, U.S. Department of Commerce, 2015. Final publication; revision status recorded by the NIST publication page.

[73] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. https://arxiv.org/abs/2310.08560, 2023.

[74] Vishal Pandey and Gopal Singh. Variational Linear Attention: Stable Associative Memory for Long-Context Transformers. https://arxiv.org/abs/2605.11196, 2026. arXiv:2605.11196.

[75] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran Gv, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Xiangru Song, Haowen Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, and Peng Zhou. RWKV: Reinventing RNNs for the transformer era. https://arxiv.org/abs/2305.13048, 2023.

[76] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. https://arxiv.org/abs/2302.10866, 2023.

[77] Qwen et al. Qwen2.5 Technical Report. https://arxiv.org/abs/2412.15115, 2024.

[78] Jack W. Rae et al. Compressive Transformers for Long-Range Sequence Modelling. https://arxiv.org/abs/1911.05507, 2019. arXiv:1911.05507.

[79] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[80] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[81] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling, 2019. https://arxiv.org/abs/1911.05507.

[82] Herbert E. Rauch, F. Tung, and Charlotte T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450, 1965.

[83] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, 1995.

[84] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000. https://doi.org/10.21314/JOR.2000.038.

[85] Paul R. Rosenbaum and Donald B. Rubin. The central role of the propensity score in observational studies for causal effects. *Biometrika*, 70(1):41–55, 1983. https://doi.org/10.1093/biomet/70.1.41.

[86] Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974. https://doi.org/10.1037/h0037350.

[87] Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.

[88] Adam Santoro, David Raposo, David G. T. Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia, and Timothy Lillicrap. A simple neural network module for relational reasoning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[89] Sunita Sarawagi and William W. Cohen. Semi-markov conditional random fields for information extraction. In *Advances in Neural Information Processing Systems*, volume 17, 2004.

[90] Mike Schuster and Kuldip K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.

[91] Noam Shazeer. GLU variants improve transformer. https://arxiv.org/abs/2002.05202, 2020.

[92] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.

[93] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. https://arxiv.org/abs/2104.09864, 2021.

[94] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[95] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[96] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[97] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248. PMLR, 2020.

[98] Yutao Sun et al. Retentive Network: A Successor to Transformer for Large Language Models. https://arxiv.org/abs/2307.08621, 2023. arXiv:2307.08621.

[99] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. https://arxiv.org/abs/physics/0004057, 2000.

[100] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[101] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[102] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

[103] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.

[104] Weizhi Wang et al. Augmenting Language Models with Long-Term Memory. https://arxiv.org/abs/2306.07174, 2023. arXiv:2306.07174.

[105] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

[106] Paul J. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.

[107] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. https://arxiv.org/abs/1410.3916, 2014.

[108] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks, 2014. https://arxiv.org/abs/1410.3916.

[109] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing Transformers. https://arxiv.org/abs/2203.08913, 2022. arXiv:2203.08913.

[110] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule. https://arxiv.org/abs/2412.06464, 2024. arXiv:2412.06464.

[111] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, 2018. Association for Computational Linguistics. https://aclanthology.org/D18-1259/.

[112] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[113] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[114] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. https://arxiv.org/abs/2502.11089, 2025.

[115] Manzil Zaheer et al. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems*, volume 33, 2020. https://arxiv.org/abs/2007.14062.

[116] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[117] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, and Zhangyang Wang. H2O: Heavy-hitter oracle for efficient generative inference of large language models. https://arxiv.org/abs/2306.14048, 2023.

[118] Junyi Zou and Avrova Donz. Atomic memory rows: A bounded, verifiable substrate for editable reasoning. MMLA Focus Paper; cited for the formal row contract only, 2026.

[119] Junyi Zou and Avrova Donz. Atomic memory rows: A bounded, verifiable substrate for editable reasoning. R02 candidate; Reviewer-pending; local package: server_phase_a/mmla_theory_paper_family_20260815/02_atomic_memory_rows_r02_major_revision, 2026.

[120] Junyi Zou and Avrova Donz. Atomic memory rows: A bounded, verifiable substrate for editable reasoning. Reviewer-pending MMLA Focus Paper; cited for the formal row contract only, 2026.

[121] Junyi Zou and Avrova Donz. Completed-segment bidirectional consolidation for multi-llm memory systems. R02 candidate; Reviewer-pending; local package: server_phase_a/mmla_theory_paper_family_20260815/05_completed_segment_consolidation_r02_major_revision, 2026.

[122] Junyi Zou and Avrova Donz. Learning what to remember: Predictive admission for bounded reasoning-time memory. R02 candidate; Reviewer-pending; local package: server_phase_a/mmla_theory_paper_family_20260815/03_predictive_memory_admission_r02_major_revision, 2026.

[123] Junyi Zou and Avrova Donz. Learning what to remember: Predictive admission for bounded reasoning-time memory. Reviewer-pending MMLA Focus Paper; cited for the formal admission problem only, 2026.

[124] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, 2026. Version 3.

[125] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, 2026. Public arXiv V3.

[126] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, 2026. Public V3, https://arxiv.org/abs/2606.28876v3.

[127] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future. *arXiv preprint arXiv:2606.28876v3*, 2026.

[128] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future. *arXiv preprint arXiv:2606.28876v3*, 2026.

[129] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, complete technical report. Technical report, MMLA-org, 2026. Complete technical-report evidence record.

[130] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, complete technical report. Technical report, MMLA-org, 2026. Complete technical-report evidence record.

[131] Junyi Zou and Avrova Donz. MMLA-RTT: Dual-state learning at reasoning time. Reviewer-pending MMLA Focus Paper; cited for typed dual-state semantics only, 2026.

[132] Junyi Zou and Avrova Donz. Reasoning-time training: Learning before a single problem ends. MMLA Focus Paper; cited for formal terminology only, 2026.

[133] Junyi Zou and Avrova Donz. Reasoning-time training: Learning before a single problem ends. R02 candidate; Reviewer-pending; local package: server_phase_a/mmla_theory_paper_family_20260815/01_rtt_foundations_r02_major_revision, 2026.

[134] Junyi Zou and Avrova Donz. Reasoning-time training: Learning before a single problem ends. Reviewer-pending MMLA Focus Paper; cited for formal terminology only, 2026.

[135] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning, 2025.

[136] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. In *Advances in Neural Information Processing Systems 38*, 2025.

A Experiment Roadmap

Experiment	Core question
Five-task slot-memory prototype	Can explicit slots solve five synthetic memory tasks?
Token-to-chunk neural interface	Do neural key/value/write signals survive tokenization and chunking?
128K matched architecture comparison	How does the hybrid compare to Delta/GDN/KDA, DSA/NSA, and MSA proxies?
Trainable causal event encoder	Can a causal GRU encoder replace hand-crafted event vectors?
Two-million-token collision-free stress	Does the hybrid hold at two million tokens without key collisions?
Generated-language boundary check	Does generated natural language preserve the synthetic boundaries?
Local long-context benchmark replication	Do Needle/RULER-style local probes replicate the hybrid pattern?
Official benchmark pipeline check	Can the pipeline ingest LongBench/RULER and produce valid predictions?
Frozen-model lexical diagnostics	What do real Llama/Qwen models reveal about simple lexical selectors?
Small trainable memory backbone	Can a 2.74M-parameter model jointly train fast state, slots, and sparse fallback?
Six-family frozen-representation study	Can frozen hidden states from six model families support a memory lifecycle when key identity is supplied?
Learned grounding without oracle keys	Can learned parsers replace oracle key IDs under controlled syntax?
Controlled generation from selected memory	Can selected memory drive frozen-LLM generation, causal lifecycle, soft injection, and learned value-span selection?
First open-domain selector	Does a non-learned IDF multi-hop selector transfer from structured chains to real HotpotQA text? (No: -7.1 F1.)
Controlled lifecycle path	Do query-independent writing, lifecycle execution, and fallback each contribute when lexical retrieval is wrong by construction? (§4)
Held-out multi-hop QA	Does bounded selection beat budget-matched retrieval at two context lengths and with two frozen reader models? (§5)
Relational compiler	Can typed anchor-to-filler transport repair content-binding failure across fresh seeds, richer paths, distractors, multiple events, and NULL roles when all decoders share a checkpoint? (§7)
Predictive-consolidation R1 qualification	Can local consolidation, a detached one-future target, a future-blind controller, and one payload-row overwrite execute against the same completed language-model checkpoint? (Yes mechanically; insufficient for conditional-risk or full-state claims.)
External semantic adapter (PM-S1)	Can a compact external current-state codec recover, query, and preserve the frozen V28 memory state? (No under its registered three-seed contract; mapping invariance and old-value suppression pass.)
Raw representation probe (PM-I1)	Is the frozen hidden representation directly decodable before choosing a denser or more native interface? (No under its registered three-seed contract; downstream interface arms remain unopened.)
Direct interface comparison (PM-I2)	Which of dense row, structured span, or true checkpoint-native readout yields a usable current-state interface under one matched frozen release? (Complete: all nine fixed jobs failed semantic qualification while structural mapping passed; a separately implemented within-project adjudicator published the no-interface terminal.)
Fit-versus-generalization diagnostic (PM-I3)	Does the PM-I2 failure already appear on fitted examples, emerge only on fresh composition, or disappear under routing or content oracles? (Complete: fit/support fails; routing oracle plus learned content is $0/73{,}728$, so the same latent-row interface stops.)
Matched explicit-row qualification	Can canonical token spans, typed tuples, or a dual-view row pass all semantic gates under the frozen matched protocol, with pure native latent rows retained as a control? (Next; not yet run.)
Same-checkpoint expected-risk intervention	After an interface qualifies, do identical-prefix grouped futures and action-conditioned replay teach a deployable value model that reduces future risk and overwrite interference relative to equal-compute immediate and local/current-risk writers? (Closed by the PM-I3 fit/support terminal.)
Matched from-scratch pretraining	If the intervention succeeds, does predictive consolidation improve quality and cost across model scales and long-running natural streams at equal token and hardware budgets?

Table 72: Experiment roadmap in execution order. Mechanical qualification, semantic-interface identification, conditional-risk overwrite learning, and from-scratch scale studies are deliberately separated. Scale remains conditional on both a usable interface and a deployable gain in the corrected intervention.

B Extended Related Work

Linear attention and efficient sequence models reduce attention cost by replacing quadratic softmax attention with recurrent, kernelized, state-space, or long-convolution states [50, 16, 36, 98, 75, 34, 21, 76]. Delta-style models and recent hybrid architectures improve the expressiveness and update dynamics of fixed states [110, 39, 52]. VLA frames linear attention as stable associative memory [74]. Our focus differs: we do not only stabilize the state; we add explicit memory lifecycle management outside the fast state.

Long-term and neural memory systems augment sequence models with differentiable stores, recurrence, memory layers, or retrieved memories [107, 28, 19, 78, 11, 109, 104, 7, 5, 70, 73]. RAG-style and nearest-neighbor methods retrieve external text or hidden states [56, 9, 51]; KV-cache methods retain high-value tokens or landmarks during inference [117, 62, 59, 69]. These works motivate memory augmentation; our studied unit is request-local editable memory with explicit overwrite, protection, eviction metadata, and calibrated abstention executed in the evaluation path.

Sparse attention systems reduce the attended set through fixed or learned patterns, from BigBird's structured sparsity to NSA's hierarchical selection, DeepSeek's learned sparse indexer, and MSA's document/chunk memory retrieval [115, 114, 22, 14]. RACE replaces softmax similarity with random projections and soft locality-sensitive hashing [48]. Sparse retrieval complements explicit lifecycle management: it is

useful when relevance is unavailable at write time, but does not by itself define versioning semantics. RULER and LongBench pressure-test these distinctions [42, 3]; HotpotQA supplies multi-hop supervision [111].

C Controlled Lifecycle Protocol Details

Scenarios. Each record has a target key with 2–4 versioned updates, distractor-key updates, and filler prose; sentence frames are disjoint across train/dev/test while cue phrases (update, mention, reject, lock, and alias markers) are shared — the disclosed controlled-language boundary. Scenario semantics: `alias-query` defines the alias once ("*X is an alias for Y*") and the question names only the alias, which never appears in update lines; `stale-remention` appends audit lines ("*an earlier revision listed Y as v*") after the final update; `invalid-write` appends rejected change requests with fresh values; `protected-slot` freezes the key at version k ("*locked at v_k; subsequent writes are void*") followed by later void updates; `slot-pressure` writes 48 distinct keys against the 32-slot bound. Generator unit tests assert that the lexical last-key baseline returns exactly the designed wrong value (the stale, rejected, or void value, or nothing for aliases) on the four breaking scenarios.

Writer. Multinomial logistic regression over hashed bag-of-words features of the line with keys, values, and digit runs masked (256 dimensions; class-weighted SGD). The feature function accepts only the line text; a unit test asserts two lines differing only in keys/values/step numbers produce identical features. Dev macro-F1 is 1.0 across split-disjoint frames; a shuffled-label control collapses to 0.069.

What counted as support. Before generating the data, we required the complete path to beat the strongest budget-matched hand-written rule by at least three exact-match points on every seed, with the lower end of a paired-bootstrap 95% confidence interval above zero. Its stale, rejected, and void-answer rate also had to be at most half that of the learned writer without lifecycle execution. Accuracy on protected values had to reach 0.90. Fallback had to activate on at least 80% of memory misses and recover at least 90% of the capacity-pressure accuracy achieved by the unbounded lexical rule. Finally, occupancy could never exceed 32 and the writer interface had to remain query-independent.

Before the single held-out read, training and calibration checks also required adequate headroom, failure under shuffled labels, parser precision and recall of at least 0.99, successful one-batch overfitting, no split overlap, and evidence that every component was actually exercised. The runner was tested end to end on calibration data, committed, and then executed once. A completion marker makes a second execution refuse.

D Held-Out Multi-Hop QA Protocol Details

Contamination control. The history ledger records every LongBench record ever touched by earlier phases and freezes the local LongBench HotpotQA pool as diagnostic-only (20 early label-tuned record IDs are unrecoverable). Holdouts derive from the official HotpotQA training parquet: 90,447 rows scanned; 13 in-split duplicates, 0 exact and 2 near-duplicate (token-Jaccard ≥ 0.8) question overlaps against all 500 local LongBench questions excluded; 1,200/300/300 train/dev/holdout frozen by seeded hash rank.

Length-matched extension. Each holdout/dev record is extended to $\geq$ 8,000 words by inserting whole distractor passages drawn from *other* train records (pool of 11,713 deduplicated passages), excluding any passage whose normalized text contains the record's normalized answer (token-boundary match) or whose title collides with an original passage; original passages, including gold, are shuffled to seeded random positions. Original passages and supporting sentences are verified verbatim-present per record.

Selector stack. Dense features are frozen Llama-3.1-8B layer-32 hidden states (attention-masked mean pooling, L2-normalized; the layer was chosen by a train-only retrieval probe — raw hidden states are weak retrievers, recall 0.14, and act only as features). The reader is a logistic reranker over (dense cosine, BM25, lexical overlap, title-in-query, position, length) with a second hop round anchored on the top-1 pick (max-cosine and title-link bridge features), trained on train gold-passage labels, three seeds. The query-independent writer scores passages from position/length/capitalization/digit/title statistics only and fills 32 slots.

Calibration in the deployment length regime. The fallback threshold (confidence = mean of top-2 reader scores over memory) is chosen on a 200-record *train-derived extended* slice: calibrating at natural length, where 32 slots hold all ∼10 passages, yields a threshold at which fallback never fires and extended coverage collapses from 0.88 to 0.31 — a recorded negative. The slice's natural-length labels were seen by the reader during training; the direction of this disclosed impurity is conservative (inflated calibration confidence produces *more* fallback at deployment, preserving quality at extra retrieval cost).

Evidence budgets and per-seed results. The budgets were fixed from calibration measurements logged before the selector existed. At 25% of an 8.2k-word context, all selectors nearly saturate evidence recall (BM25 0.970); at 10%, BM25 reaches 0.867 and the learned two-hop selector 0.956. A 10% budget is too small at natural length, where even oracle selection falls to F1 0.60, so that condition uses 25%.

For Llama, the three seed-wise gains over dense retrieval are +14.64/+14.44/+14.31 F1 at natural length, with confidence-interval lower ends +9.38/+8.99/+8.94, and +5.98/+6.27/+5.52 at extended length, with lower ends +1.66/+1.96/+1.45. Qwen gains +16.55/+16.48/+16.04 and +7.10/+7.64/+5.99, with lower ends at least +10.6 and +2.00. Relative to reading the entire context, Llama retains 92.8/92.5/92.4% of F1 at natural length and reaches 102.3/102.8/101.7% at extended length. Qwen reaches 94.3/94.3/93.7% and 115.2/116.0/113.7%.

The two budget readings. Under the frozen whole-prompt-token definition, the Llama run uses 29.4% against a 26.25% allowance at natural length and 11.0% against 10.5% at extended length, so it fails that requirement. Counting selected evidence words instead, the extended per-record maximum is exactly 10.00%. Natural records average 22.57%, but the original packer's first-passage exception reaches 61.36% on one record. The forward-looking Qwen follow-up uses a hard per-record cap and truncates the top passage only if no whole passage fits; both length conditions then remain at or below their stated evidence budgets.

E Relational-Binding Protocol Details

Data geometry. The fresh confirmation uses data seeds 2026080411–13 and model seeds 2026080421–23, all disjoint from the failed earlier three-seed study. Each seed contains 512 `train_fit` and 240 `fresh_train_heldout` records with unique record and group namespaces

across fitting data, held-out data, and seeds. The held-out records are balanced into six 40-record subsets: combined hard cases, distractor bridges, 4–6 events, optional or NULL roles, unseen path lengths 6–8, and unseen relation compositions. Relation primitives are shared across fitting and held-out data. The question is whether seen primitives compose in new ways, not whether the model discovers unseen relation semantics.

Shared model and four decoders. A translation-equivariant local encoder feeds an exact-count ordered semi-Markov event parser [89], predicted event coverage, an event-kind head, and a role-conditioned anchor posterior. Candidate-content, unsigned-distance, type-erased transport, and typed AFRT decoders are trained jointly and read from one model-only checkpoint per seed. Dynamic partial Sinkhorn assignment inherits differentiable matrix scaling [18]; we do not claim Sinkhorn itself as novel. Raw inference accepts tokens, masks, and queries only. Gold spans, anchors, roles, kinds, and cross-view alignments are loss-only inputs and are rejected by the raw API.

Training and what counted as support. Training consists of 32 complete fitting epochs, 512 AdamW steps [63], and effective batch 32, without a scheduler, dropout, early stopping, or checkpoint choice. For each seed, every one of eleven typed-decoder output checks had to reach 432/480; cross-view consistency and whole-record exactness each had to reach 216/240; and every 40-record stress subset had to contain at least 34 exact records. The paired whole-record gains over the candidate-content, unsigned-distance, and type-erased decoders had to reach +24, +18, and +12, respectively. We would regard the overall hypothesis as supported only if all three fixed seeds met every condition, with no seed replacement.

Evaluator definitions and artifacts. Payload exactness compares both the resident payload and the separately stored lifecycle version. Anchor exactness is restricted to active events and reports present and NULL roles separately; inactive padding is excluded. For repository lookup, the fresh confirmation is labeled `R5-P5`, and its four decoder identifiers are `C0`, `D0`, `U0`, and `R1`. The protocol and freeze-receipt SHA-256 prefixes are `c759ac80...462be` and `0ca2b847...4688`. The evaluation-only key-order recovery has protocol and receipt prefixes `37889c87...fa23e5` and `1a9c6756...c5df0`. The final aggregate is `experiments/phase_a_single_4090/r5_p5_clean_confirmatory_v1/paired_summary_r1.json`, SHA-256 `e74d92ce...fb9ea`. It is complete, all three fixed seeds meet every condition, and both Dev/Test touched flags are false. The evidence-package manifest contains the full digests.

F Audit Summary

An internal logic audit preceded v2. The earliest associative-recall task always placed the requested fact first and assigned uniform importance, making its headline result trivial. Randomizing the target and supervising writes produced real evictions that scale with difficulty. Another evaluator silently gave full context to a supposedly bounded method, while a separate baseline matched generator templates with hand-tuned regular expressions; both are now named as non-memory controls. We also removed entity overlap from the frozen-hidden split, replaced pooled Wilson intervals over repeated use of one fixed set with per-seed reporting, and eliminated filler/target key collisions from the two-million-token generator. The corrected rerun obtains 49/50.

The relational study has its own preserved audit chain. The first three-seed payload definition omitted lifecycle version, whole-view anchor equality counted inactive padding, and the fresh confirmation's first aggregate attempt required JSON dictionary insertion order. The original three-seed verdict remains failed, the first two findings are explicitly post-hoc, the fresh confirmation uses new seeds and pre-corrected definitions, and the key-order change is versioned and evaluation-only. Superseded artifacts are marked and retained. The publicly visible v1 predates these corrections; this version updates its numbers without rewriting historical verdicts.

G Reproducibility Notes

Every run artifact records its git commit; frozen splits carry SHA-256 manifests; one-shot evaluations write completion markers that make their runners refuse re-execution. Mainline commands:

```
# Controlled lifecycle: data, calibration, one-shot holdout, generation
python -m phase14e_open_domain_grounding.phase14e_b_versioned_vt2
python -m phase14e_open_domain_grounding.phase14e_b_track_a_dev2
python -m phase14e_open_domain_grounding.phase14e_b_track_a_test2 \
  --split-label test
python -m phase14e_open_domain_grounding.phase14e_b_track_a_generation_dev2 \
  --dev data/phase14e_b/phase14e_b_versioned_vt2_test.jsonl --allow-test

# Held-out multi-hop QA: data, embeddings, selectors, Llama, Qwen
python -m phase14e_open_domain_grounding.phase14e_b_manifest
python -m phase14e_open_domain_grounding.phase14e_b_hotpot_extended
python -m phase14e_open_domain_grounding.phase14e_b_embed
python -m phase14e_open_domain_grounding.phase14e_b_track_b_dev2
python -m phase14e_open_domain_grounding.phase14e_b_track_b_dev3 --generate
python -m phase14e_open_domain_grounding.phase14e_b_track_b_confirmatory
python -m phase14e_open_domain_grounding.phase14e_b_track_b_confirmatory \
  --model-id qwen2_5_14b_instruct --strict-packer \
  --gb3-mode evidence_words --bootstrap-seed 20260710
```

Principal artifacts (all with per-record prediction rows):

- `phase14e_open_domain_grounding/results/phase14e_b_track_a_test2_v1/`
- `phase14e_open_domain_grounding/results/phase14e_b_track_a_generation_test2_v1/`
- `phase14e_open_domain_grounding/results/phase14e_b_track_b_confirmatory_v1/`
- `phase14e_open_domain_grounding/results/phase14e_b_track_b_confirmatory_qwen_v1/`
- `phase4_competitive_baselines/results/sparse_hybrid_replicates_collision_free_20260702/`
- `phase13_frozen_hidden_memory/results/adapters_disjoint_all_six_20260702/`
- `phase_stats_correction/results/corrected_current/`

Preregistration documents appear in commit order: the mainline plan, the amendment that replaced the defective lifecycle task, the multi-hop QA protocol freeze, and the forward-looking evidence-budget correction. Each predates the data read it governs.

For compatibility with repository filenames, the controlled lifecycle and multi-hop QA studies retain the historical labels `Track A` and `Track B`. Relational-binding reproducibility is bound by `PHASE_A_R5_P5_CLEAN_CONFIRMATORY_PROTOCOL.md`, `PHASE_A_R5_P5_FREEZE_RECEIPT.json`, and the versioned typed-decoder adjudicator protocol and receipt. The evidence package `paper_evidence_r5p5_20260803/` contains the result table, terminal audit, claim boundaries, artifact manifest, and checksums. Per-seed commands, data hashes, model-only checkpoints, prediction rows, summaries, and logs remain in those receipts rather than being abbreviated here.

H Interface Qualification and Conditional Expected-Risk Protocol

Checkpoint binding. The current reference implementation uses the final model-only state from the completed V28 real-data pilot, SHA-256 `98c1be40...ab82f5`. Its sidecar, distributed-checkpoint tree, configuration, and training freeze are deep-hashed before loading. All 295 tensor keys load exactly with no missing or unexpected key, and every backbone parameter remains frozen. The project's "160M" label describes the hidden-stack configuration; the serialized model contains 352,347,121 parameters including the large vocabulary interfaces. This distinction is recorded to prevent an architecture-size label from being mistaken for an exact parameter count.

Mechanical ancestor and its boundary. A parameter-free memory reader is exactly identical to the frozen language-model head when persistent state is empty. The local consolidator sees only a completed 64-token segment. The ancestor implementation evaluates each registered slot and a private NULL action from one immutable snapshot, detaches a target, rejects future-bearing deployment arguments, and changes one payload row or none. Thirty-five focused CPU tests and a one-record finite-gradient probe qualify software plumbing only. The code uses one realized continuation, converts its risks into a Gibbs policy, and applies memory after a cached future representation. It therefore neither estimates Eq. 19 nor identifies the effect of an action on future hidden-state evolution.

PM-S1 and PM-I1 terminal interface evidence. PM-S1 trains the external semantic adapter at three fixed seeds, then opens current-state qualification once. It publishes 393,216 raw rows. Independent reduction matches the producer aggregate byte for byte (SHA-256 `6167308d...994995`) and yields Table 3; utility and predictive adjudication remain unopened. PM-I1 trains a raw-representation probe at three new fixed seeds. Its one-shot aggregate (SHA-256 `c0ff1fb3...26d5d`) records the gate vector $(0, 0, 0, 0, 0, 1, 1)$ for every seed: all semantic gates fail, both mapping gates pass. The registered terminal decisions are retained rather than repaired or rerun.

PM-I2 direct-interface status. PM-I2 compares D1 dense row, S1 structured 4×192 span, and N1 true checkpoint-native readout with active parameter counts 1,541,377/1,496,577/1,460,737. It fixes three seeds, one Latin order, one hidden release, 2,048 updates per job, and final-only checkpoints. Data and materialization manifests have SHA-256 prefixes `c214f467...b5a8` and `960c8ada...b09d5`; the accepted hidden manifest is `6c0fcc56...5e08c`. The original R2 implementation stopped before update one on the conceptual/physical key mismatch and remains rejected. R3 repaired only that reader schema. Its formal manifest/protocol/receipt prefixes are `c790f820...f337c`, `f9cf7869...f475`, and `bddc3811...e8b05`; its training launch receipt/log are `353d2a1c...27037` and `cd1f5aea...65d57`. All nine fixed jobs then completed exactly 2,048 updates. Truth-free Phase A was frozen under receipt `f024fb05...96b4`. Phase B was frozen under receipt `3f33147a...f3efc`, opened evaluator truth `86d6f1a3...f242f` only after 45 factor files were fixed, and ended with log `ffb22a84...f48e5`. Nine checkpoint bundles, nine factor bundles, nine metric bundles, and 55,296 raw qualification rows were independently audited. Every job is unqualified: candidate exactness is 0–45/23,040, query exactness 10–1,536/9,216, and mean record-macro Brier 0.998676–0.999093. Old-value suppression and mapping invariance pass. The first adjudication transport stopped before production on a lifecycle-test error and remains preserved under receipt `195e8b02...46e3`. R01 changed only launch identity and that test lifecycle, then called the frozen adjudicator once under receipt `b6e4a1e9...4778`. The final bootstrap/aggregate hashes are `1cb09ce3...b6f` and `65ba2236...79ee`; the former records not-applicable with zero draws, and the latter publishes `pm_i2_no_current_interface_qualified_stop`. These accepted measurements and terminal close PM-I2 without selecting an interface.

PM-I3 accepted fault-localization terminal. PM-I3 reuses the nine immutable PM-I2 final checkpoints without training updates, checkpoint selection, seed replacement, threshold changes, or arm preference. Its Phase A freezes 67,584 target-free probability vectors per job before evaluator truth; Phase B opens truth in a fresh model-incapable CPU process and publishes 59,392 rows per job, 534,528 in total. Phase C opens the nine historical PM-I2 summaries only after every current result is immutable. Reviewer reconstruction reproduces the final joined diagnostic byte for byte, SHA-256 `40fcdd95...4477`; the current aggregate is `ff54ffdc...dc9e`, and the accepted Phase-B content tree is `8140509e...ae0d`.

The exact reductions are reported in Table 5. Learned content obtains 0/73,728 exact with learned routing and 0/73,728 with the target-row oracle. Oracle content plus learned routing obtains 18,544/73,728, exactly the row-routing count; the fully oracle-specified path is 73,728/73,728 by construction. Direct candidate reconstruction is 40/73,728. Historical fresh content-bearing probes remain near zero, while map consistency is 82,944/82,944 and old-value suppression is 13,824/13,824. This selects the preregistered fit/support-failure branch. It does not identify which of representation geometry, decoder design, and optimization is causal, cannot rescue PM-I2, and cannot select an interface retrospectively.

Conditional expected-risk successor, before any result. This stage remains closed by the PM-I3 fit/support terminal. A new interface must first qualify at every fixed seed. Only then may each future experiment group contain one identical observed prefix and four to eight continuations with frozen weights. Group identifiers, not individual branches, would be physically separated across fitting, calibration, and fresh held-out namespaces. The generator would remove direct lexical cues to the correct write and include strict replacement, stale evidence, preservation of unrelated facts, deletion, delayed relevance, multi-hop consequences, ambiguous prefixes, and groups for which NULL is optimal in expectation. From one immutable snapshot, every action would be scored by replaying every continuation through the frozen reader under the corresponding post-action memory. No further write would be permitted within the horizon. These rules remain a preregistration template, not an authorized experiment.

The no-op, immediate/current-risk, local/current-risk, and local/expected-risk arms will share the V28 checkpoint, slot count, registered

actions, candidate payload, fixed gate $g = 1$, write budget, train exposure, optimizer family, and evaluation budget. The local/current-risk to local/expected-risk contrast changes the supervision target only. All trainable arms must be parameter matched; all feasible actions receive value targets. A non-deployable empirical expected-risk oracle is reported only as a ceiling. Payload learning, gate learning, recurrent rollout policy, index learning, and multi-write planning are deliberately outside this successor.

Falsification before scale. The direction stops if the empirical expected-risk oracle has negligible margin over current-risk writing, if the future-blind value model cannot recover a useful action ordering across fresh seeds, if lower future loss is bought by more stale leakage or collateral damage, or if gains disappear under parameter/compute matching. No checkpoint selection, failed-seed replacement, future-dependent deployment input, or post-held-out horizon change is allowed. A successful intervention authorizes a short joint-training study; it does not directly authorize a larger model or an AGI claim.

I Frozen-Hidden-Bridge Protocol Details

The six-family frozen-hidden bridge is a controlled representation study, not a learned or open-domain parser. Each generated sample supplies event sentences, a query sentence, integer `key_ids`, integer `value_ids`, a target event index, and optional write labels. The frozen backbone mean-pools last-layer hidden states over each complete event and query sentence. Separately, the protocol forms the canonical string `entity N` from each supplied key identifier, encodes it with the same backbone, and mean-pools that string. Event and query representations concatenate full-sentence vectors with the canonical-key vectors. No runtime regular expression, named-entity recognizer, exact-string span matcher, or value-span extractor is used; same-key replacement and branch arbitration use exact integer equality. The study therefore asks whether frozen hidden representations support a lifecycle when key identity is supplied. It does not establish entity discovery, alias resolution, or coreference, which the learned QA selector addresses only at passage level.

J Mathematical Properties and Stability Boundaries

Causal boundary. At boundary b_n, z_n is a deterministic or stochastic function of $(X_{\le b_n}, M_{n-1})$. The updated state M_n is used only for positions greater than b_n. Therefore the conditional distribution for any already emitted token is unchanged by consolidation. Bidirectionality within $(b_{n-1}, b_n]$ does not introduce a path from $X_{>b_n}$ to $p(X_t \mid X_{<t})$ for $t \le b_n$. The implementation enforces this property structurally rather than relying on a loss penalty.

Atomicity. Let $\Delta^v_{n,j}$ be the difference between matrices formed by stacking the neural payload views of $M_{n,j}$ and $M_{n,j-1}$. From Eq. 13, $\operatorname{rank}(\Delta^v_{n,j}) \le 1$ and $\|\Delta^v_{n,j}\|_{0,\text{row}} \le 1$ for every event. Across the full segment, $\|V(M_n) - V(M_{n-1})\|_{0,\text{row}} \le K_n$. The authoritative-state definition is stronger: every mutable canonical, neural, routing, version, security, tombstone, and rollback field of the selected row commits together. Any single event trace that modifies two resident rows, or silently mutates an external authoritative alias/index state, is outside the contract. Protection can only remove actions from $\mathcal{A}_{n,j}$; it cannot add a hidden second write.

Resident boundedness. Equation 2 is a bit-level contract, not merely a fixed row count. Every resident field has a fixed maximum width and there are exactly S cells, so $\text{bits}(M_n) \le B_{\text{resident}}$ for every n. Monotone versions use bounded integers with an explicit fail-closed exhaustion policy: the last representable version cannot be incremented, and counters never wrap, saturate, or reuse identity/version pairs. Canonical content, aliases, and metadata that exceed their caps likewise make the candidate infeasible. Unbounded provenance, rollback history, archives, indexes, and audit events are external objects referenced by fixed-width digests and charged to A_n. Thus resident state is truly bounded while total retained system information may grow.

Dual-view consistency. A row is valid only when Eq. 11 holds and Eq. 10 binds one identity and version to both canonical and neural bytes under one frozen encoder version. Hashes are receipts rather than the equality predicate. The receipt is committed atomically with the row, and byte equality plus the receipt are rechecked at load, recovery, export, teacher replay, and index rebuild. A mismatch cannot be repaired by selecting one view: the row is quarantined, removed from the feasible read/write set, and requires a new validated version. This provides mechanical byte consistency and a fail-closed semantic qualification boundary; it does not prove that the learned neural view is semantically adequate.

Multi-event teacher semantics. Equations 15–17 define each event value on its own branch prefix and include later event decisions in the cost-to-go. Exact dynamic programming, a frozen continuation policy, and the registered $K_n = 1$ intervention are distinct protocols and must be labeled as such. The policy minimization occurs outside the future expectation; a per-future argmin would be a clairvoyant teacher unavailable to deployment. Reusing the pre-segment candidates at later events, averaging event-local argmins, or scoring each event while silently holding later writes undefined is not a valid teacher. Grouped future weights are fixed before outcomes are opened, and group identity is the data-splitting unit.

Complete-candidate accounting. Equations 28 and 29 charge semantic proposal, trusted assembly, dual-view validation, action scoring, commit, branch-prefix expansion, and future replay. Any target shortlist or shared constructor must publish its own construction, miss, and excluded-action costs. Consequently, the simpler $O(TSd)$ read expression is not an end-to-end MMLA cost bound.

Entropy-regularized optimum for a fixed expected-risk vector. Expanding Eq. 26 gives

$$\mathcal{F}[q] + \tau \log Z = \tau \sum_a q(a) \log \frac{q(a)}{q^*(a)} \ge 0.$$

Equality holds exactly at $q = q^*$. The Gibbs distribution is therefore the unique regularized minimizer when $E(a) = \overline{J}_H(a \mid s)$ is fixed, every registered action has finite risk, and $\tau > 0$. The identity neither estimates $\overline{J}_H$ nor permits the expectation to move through softmax. The future expected-risk successor estimates the conditional values first and uses direct value regression.

Greedy action-value regret. Let $a^* = \arg\min_a \overline{J}_H(a \mid s)$ and $\hat{a} = \arg\min_a \widehat{Q}(a \mid s)$. If $\max_a |\widehat{Q}(a \mid s) - \overline{J}_H(a \mid s)| \le \epsilon$, then

$$\overline{J}_H(\hat{a} \mid s) - \overline{J}_H(a^* \mid s) \le 2\epsilon.$$

Indeed, $\overline{J}_H(\hat{a}) \le \widehat{Q}(\hat{a}) + \epsilon \le \widehat{Q}(a^*) + \epsilon \le \overline{J}_H(a^*) + 2\epsilon$. This bound motivates held-out value error and action regret. It does not guarantee that a finite grouped benchmark estimates the true deployment distribution; that requires representativeness and fresh groups.

Long-horizon boundary. Authoritative rows do not obey a soft global-convergence claim: a newer fact causes a discrete hard commit, and routing, thresholded writes, and eviction form a switched process. A persistent false-write rate can therefore bias or oscillate even though each row and update is bounded. The predictive teacher is intended to reduce this error; only a long-horizon overwrite benchmark can establish repeated-write stability.